LÖVE WIKI · PART 7 OF 9

Modules

506 pages

wiki pages 1551–2056

Contents of this part

love.audio
love.audio.getActiveEffects
love.audio.getActiveSourceCount
love.audio.getDistanceModel
love.audio.getDopplerScale
love.audio.getEffect
love.audio.getMaxSceneEffects
love.audio.getMaxSourceEffects
love.audio.getNumSources
love.audio.getOrientation
love.audio.getPlaybackDevice
love.audio.getPlaybackDevices
love.audio.getPosition
love.audio.getRecordingDevices
love.audio.getSourceCount
love.audio.getVelocity
love.audio.getVolume
love.audio.isEffectsSupported
love.audio.new QueueableSource
love.audio.new Source
love.audio.pause
love.audio.play
love.audio.resume
love.audio.rew ind
love.audio.setDistanceModel
love.audio.setDopplerScale
love.audio.setEffect
love.audio.setMixWithSystem
love.audio.setOrientation
love.audio.setPlaybackDevice
love.audio.setPosition
love.audio.setVelocity
love.audio.setVolume
love.audio.stop
love.audiodisconnected
love.body
love.bundle
love.data
love.data.compress
love.data.decode
love.data.decompress
love.data.encode
love.data.getPackedSize
love.data.hash
love.data.new ByteData
love.data.new DataView
love.data.pack
love.data.unpack
love.directorydropped
love.displayrotated
love.draw
love.dropbegan
love.dropcompleted
love.dropmoved
love.errhand
love.errorhandler
love.event
love.event.clear
love.event.poll
love.event.pump
love.event.push
love.event.quit
love.event.restart
love.event.w ait
love.exposed
love.filedropped
love.filesystem
love.filesystem.append
love.filesystem.areSymlinksEnabled
love.filesystem.createDirectory
love.filesystem.enumerate
love.filesystem.exists
love.filesystem.getAppdataDirectory
love.filesystem.getCRequirePath
love.filesystem.getDirectoryItems
love.filesystem.getFullCommonPath
love.filesystem.getIdentity
love.filesystem.getInfo
love.filesystem.getLastModified
love.filesystem.getRealDirectory
love.filesystem.getRequirePath
love.filesystem.getSaveDirectory
love.filesystem.getSize
love.filesystem.getSource
love.filesystem.getSourceBaseDirectory
love.filesystem.getUserDirectory
love.filesystem.getWorkingDirectory
love.filesystem.init
love.filesystem.isDirectory
love.filesystem.isFile
love.filesystem.isFused
love.filesystem.isSymlink
love.filesystem.lines
love.filesystem.load
love.filesystem.mkdir
love.filesystem.mount
love.filesystem.mountCommonPath
love.filesystem.mountFullPath
love.filesystem.new File
love.filesystem.new FileData
love.filesystem.openFile
love.filesystem.openNativeFile
love.filesystem.read
love.filesystem.remove
love.filesystem.setCRequirePath
love.filesystem.setIdentity
love.filesystem.setRequirePath
love.filesystem.setSource
love.filesystem.setSymlinksEnabled
love.filesystem.unmount
love.filesystem.unmountCommonPath
love.filesystem.unmountFullPath
love.filesystem.w rite
love.focus
love.font
love.font.new BMFontRasterizer
love.font.new FontData
love.font.new GlyphData
love.font.new ImageRasterizer
love.font.new Rasterizer
love.font.new TrueTypeRasterizer
love.gamepadaxis
love.gamepadpressed
love.gamepadreleased
love.getVersion
love.graphics
love.graphics.applyTransform
love.graphics.arc
love.graphics.captureScreenshot
love.graphics.checkMode
love.graphics.circle
love.graphics.clear
love.graphics.copyBuffer
love.graphics.discard
love.graphics.draw
love.graphics.draw Instanced
love.graphics.draw Layer
love.graphics.draw q
love.graphics.ellipse
love.graphics.flushBatch
love.graphics.getBackgroundColor
love.graphics.getBlendMode
love.graphics.getCanvas
love.graphics.getCanvasFormats
love.graphics.getCaption
love.graphics.getColor
love.graphics.getColorMask
love.graphics.getColorMode
love.graphics.getCompressedImageFormats
love.graphics.getDefaultFilter
love.graphics.getDefaultImageFilter
love.graphics.getDepthMode
love.graphics.getDimensions
love.graphics.getDPIScale
love.graphics.getFont
love.graphics.getFrontFaceWinding
love.graphics.getHeight
love.graphics.getImageFormats
love.graphics.getLineJoin
love.graphics.getLineStipple
love.graphics.getLineStyle
love.graphics.getLineWidth
love.graphics.getMaxImageSize
love.graphics.getMaxPointSize
love.graphics.getMeshCullMode
love.graphics.getMode
love.graphics.getModes
love.graphics.getPixelDimensions
love.graphics.getPixelEffect
love.graphics.getPixelHeight
love.graphics.getPixelWidth
love.graphics.getPointSize
love.graphics.getPointStyle
love.graphics.getRendererInfo
love.graphics.getScissor
love.graphics.getShader
love.graphics.getStackDepth
love.graphics.getStats
love.graphics.getStencilMode
love.graphics.getStencilState
love.graphics.getStencilTest
love.graphics.getSupported
love.graphics.getSystemLimit
love.graphics.getSystemLimits
love.graphics.getTextureFormats
love.graphics.getTextureTypes
love.graphics.getWidth
love.graphics.hasFocus
love.graphics.intersectScissor
love.graphics.inverseTransformPoint
love.graphics.isActive
love.graphics.isCreated
love.graphics.isGammaCorrect
love.graphics.isSupported
love.graphics.isWireframe

love.graphics.line
love.graphics.newArrayImage
love.graphics.newBuffer
love.graphics.newCanvas
love.graphics.newCubeImage
love.graphics.newFont
love.graphics.newFramebuffer
love.graphics.newImage
love.graphics.newImageFont
love.graphics.newMesh
love.graphics.newParticleSystem
love.graphics.newPixelEffect
love.graphics.newQuad
love.graphics.newScreenshot
love.graphics.newShader
love.graphics.newSpriteBatch
love.graphics.newStencil
love.graphics.newText
love.graphics.newTextBatch
love.graphics.newTexture
love.graphics.newVideo
love.graphics.newVolumeImage
love.graphics.origin
love.graphics.point
love.graphics.points
love.graphics.polygon
love.graphics.pop
love.graphics.present
love.graphics.print
love.graphics.printf
love.graphics.push
love.graphics.quad
love.graphics.readbackBuffer
love.graphics.readbackTexture
love.graphics.rectangle
love.graphics.replaceTransform
love.graphics.reset
love.graphics.rotate
love.graphics.scale
love.graphics.setBackgroundColor
love.graphics.setBlendMode
love.graphics.setBlendState
love.graphics.setCanvas
love.graphics.setCaption
love.graphics.setColor
love.graphics.setColorMask
love.graphics.setColorMode
love.graphics.setDefaultFilter
love.graphics.setDefaultImageFilter
love.graphics.setDepthMode
love.graphics.setFont
love.graphics.setFrontFaceWinding
love.graphics.setIcon
love.graphics.setInvertedStencil
love.graphics.setLine
love.graphics.setLineJoin
love.graphics.setLineStipple
love.graphics.setLineStyle
love.graphics.setLineWidth
love.graphics.setMeshCullMode
love.graphics.setMode
love.graphics.setNewFont
love.graphics.setPixelEffect
love.graphics.setPoint
love.graphics.setPointSize
love.graphics.setPointStyle
love.graphics.setProjection
love.graphics.setRenderTarget
love.graphics.setScissor
love.graphics.setShader
love.graphics.setStencil
love.graphics.setStencilState
love.graphics.setStencilTest
love.graphics.setWireframe
love.graphics.shear
love.graphics.stencil
love.graphics.toggleFullscreen
love.graphics.transformPoint
love.graphics.translate
love.graphics.triangle
love.graphics.validateShader
love.hasDeprecationOutput
love.image
love.image.isCompressed
love.image.newCompressedData
love.image.newEncodedImageData
love.image.newImageData
love.isVersionCompatible
love.joystick
love.joystick.close
love.joystick.getAxes
love.joystick.getAxis
love.joystick.getBall
love.joystick.getGamepadMappingString
love.joystick.getHat
love.joystick.getJoystickCount
love.joystick.getJoysticks
love.joystick.getName
love.joystick.getNumAxes
love.joystick.getNumBalls
love.joystick.getNumButtons
love.joystick.getNumHats
love.joystick.getNumJoysticks
love.joystick.isDown
love.joystick.isOpen
love.joystick.loadGamepadMappings
love.joystick.open
love.joystick.saveGamepadMappings
love.joystick.setGamepadMapping
love.joystickadded
love.joystickaxis
love.joystickhat
love.joystickpressed
love.joystickreleased
love.joystickremoved
love.joysticksensorupdated
love.keyboard
love.keyboard.getKeyFromScancode
love.keyboard.getKeyRepeat
love.keyboard.getScancodeFromKey
love.keyboard.hasKeyRepeat
love.keyboard.hasScreenKeyboard
love.keyboard.hasTextInput
love.keyboard.isDown
love.keyboard.isModifierActive
love.keyboard.isScancodeDown
love.keyboard.setKeyRepeat
love.keyboard.setTextInput
love.keypressed
love.keyreleased
love.load
love.localechanged
love.lowmemory
love.maker
love.math
love.math.colorFromBytes
love.math.colorToBytes
love.math.compress
love.math.decompress
love.math.gammaToLinear
love.math.getRandomSeed
love.math.getRandomState
love.math.isConvex
love.math.linearToGamma
love.math.newBezierCurve
love.math.newRandomGenerator
love.math.newTransform
love.math.noise
love.math.perlinNoise
love.math.random
love.math.randomNormal
love.math.setRandomSeed
love.math.setRandomState
love.math.simplexNoise
love.math.triangulate
love.mouse
love.mouse.getCursor
love.mouse.getGlobalPosition
love.mouse.getPosition
love.mouse.getRelativeMode
love.mouse.getSystemCursor
love.mouse.getX
love.mouse.getY
love.mouse.hasCursor
love.mouse.isCursorSupported
love.mouse.isDown
love.mouse.isGrabbed
love.mouse.isVisible
love.mouse.newCursor
love.mouse.setCursor
love.mouse.setGrab
love.mouse.setGrabbed
love.mouse.setPosition
love.mouse.setRelativeMode
love.mouse.setVisible
love.mouse.setX
love.mouse.setY
love.mousefocus
love.mousemoved
love.mousepressed
love.mousereleased
love.occluded
love.physics
love.physics.getDistance
love.physics.getMeter
love.physics.newBody
love.physics.newChainShape
love.physics.newCircleBody
love.physics.newCircleShape
love.physics.newDistanceJoint
love.physics.newEdgeBody
love.physics.newEdgeShape
love.physics.newFixture
love.physics.newFrictionJoint
love.physics.newGearJoint
love.physics.newMotorJoint
love.physics.newMouseJoint
love.physics.newPolygonBody
love.physics.newPolygonShape
love.physics.newPrismaticJoint
love.physics.newPulleyJoint
love.physics.newRectangleBody
love.physics.newRectangleShape
love.physics.newRevoluteJoint
love.physics.newRopeJoint
love.physics.newWeldJoint
love.physics.newWheelJoint

love.physics.new World
love.physics.setMeter
love.quit
love.releaseerrhand
love.resize
love.run
love.scene
love.screen
love.sensor
love.sensor.hasSensor
love.sensorupdated
love.setDeprecationOutput
love.sound
love.sound.new Decoder
love.sound.new SoundData
love.system
love.system.getClipboardText
love.system.getMemorySize
love.system.getOS
love.system.getPow erInfo
love.system.getPreferredLocales
love.system.getProcessorCount
love.system.hasBackgroundMusic
love.system.openURL
love.system.setClipboardText
love.system.vibrate
love.textedited
love.textinput
love.thread
love.thread.getChannel
love.thread.getThread
love.thread.getThreads
love.thread.new Channel
love.thread.new Thread
love.threaderror
love.timer
love.timer.getAverageDelta
love.timer.getDelta
love.timer.getFPS
love.timer.getMicroTime
love.timer.getTime
love.timer.sleep
love.timer.step
love.touch
love.touch.getDeviceType
love.touch.getPosition
love.touch.getPressure
love.touch.getTouches
love.touch.isMouse
love.touchmoved
love.touchpressed
love.touchreleased
love.update
love.video
love.video.new VideoStream
love.visible
love.w heelmoved
love.w indow
love.w indow.close
love.w indow.focus
love.w indow.fromPixels
love.w indow.getDesktopDimensions
love.w indow.getDimensions
love.w indow.getDisplayCount
love.w indow.getDisplayName
love.w indow.getDisplayOrientation
love.w indow.getDPIScale
love.w indow.getFullscreen
love.w indow.getFullscreenModes
love.w indow.getHeight
love.w indow.getIcon
love.w indow.getMode
love.w indow.getPixelScale
love.w indow.getPointer
love.w indow.getPosition
love.w indow.getSafeArea
love.w indow.getSystemTheme
love.w indow.getTitle
love.w indow.getVSync
love.w indow.getWidth
love.w indow.hasFocus
love.w indow.hasMouseFocus
love.w indow.isCreated
love.w indow.isDisplaySleepEnabled
love.w indow.isMaximized
love.w indow.isMinimized
love.w indow.isOccluded
love.w indow.isOpen
love.w indow.isVisible
love.w indow.maximize
love.w indow.minimize
love.w indow.requestAttention
love.w indow.restore
love.w indow.setDisplaySleepEnabled
love.w indow.setFullscreen
love.w indow.setIcon
love.w indow.setMode
love.w indow.setPosition
love.w indow.setTitle
love.w indow.setVSync
love.w indow.show FileDialog
love.w indow.show MessageBox
love.w indow.toPixels
love.w indow.updateMode

love.audio

love.audio · revision 29378 · last edited 2025-09-11

Provides of audio interface for playback/recording sound.

Types

RecordingDevice	Represents an audio input device capable of recording sounds.	Added since 11.0
Source	A Source represents audio you can play back.	

Functions

love.audio.getActiveEffects	Gets a list of the names of the currently enabled effects.	Added since 11.0	
love.audio.getActiveSourceCount	Gets the current number of simultaneously playing sources.	Added since 11.0	
love.audio.getDistanceModel	Returns the distance attenuation model.	Added since 0.8.0	
love.audio.getDopplerScale	Gets the global scale factor for doppler effects.	Added since 0.9.2	
love.audio.getEffect	Gets the settings associated with an effect.	Added since 11.0	
love.audio.getMaxSceneEffects	Gets the maximum number of active effects.	Added since 11.0	
love.audio.getMaxSourceEffects	Gets the maximum number of active Effects for each Source.	Added since 11.0	
love.audio.getNumSources	Gets the current number of simultaneously playing sources.		Removed in 0.9.0
love.audio.getOrientation	Returns the orientation of the listener.		
love.audio.getPosition	Returns the position of the listener.		
love.audio.getRecordingDevices	Gets a list of RecordingDevices on the system.	Added since 11.0	
love.audio.getSourceCount	Gets the current number of simultaneously playing sources.	Added since 0.9.0	Deprecated in 11.0
love.audio.getVelocity	Returns the velocity of the listener.		
love.audio.getVolume	Returns the master volume.		
love.audio.isEffectsSupported	Gets whether Effects are supported in the system.	Added since 11.0	
love.audio.newQueueableSource	Creates a new Source usable for real-time generated sound playback with Source:queue.	Added since 11.0	
love.audio.newSource	Creates a new Source from a file, SoundData, or Decoder.		
love.audio.pause	Pauses specific or all currently played Sources.		
love.audio.play	Plays specific Sources.		

love.audio.resume	Resumes all audio.	Removed in 11.0
love.audio.rewind	Rewinds all playing audio.	Removed in 11.0
love.audio.setDistanceModel	Sets the distance attenuation model.	Added since 0.8.0
love.audio.setDopplerScale	Sets a global scale factor for doppler effects.	Added since 0.9.2
love.audio.setEffect	Defines an effect that can be applied to a Source.	Added since 11.0
love.audio.setMixWithSystem	Sets whether the system should mix the audio with the system's audio.	Added since 11.0
love.audio.setOrientation	Sets the orientation of the listener.	
love.audio.setPosition	Sets the position of the listener.	
love.audio.setVelocity	Sets the velocity of the listener.	
love.audio.setVolume	Sets the master volume.	
love.audio.stop	Stops specific or all currently played sources.	

Enums

DistanceModel	The different distance models.	Added since 0.8.0
EffectType	Different types of audio effects.	Added since 11.0
EffectWaveform	Types of waveforms for **ringmodulator** effect.	Added since 11.0
SourceType	Types of audio sources.	
TimeUnit	Units that represent time.	Added since 0.8.0

See Also

- love
- Tutorial:Audio
- Audio Formats

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getActiveEffects

love.audio.getActiveEffects · revision 23929 · last edited 2026-05-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets a list of the names of the currently enabled effects.

Function

Synopsis

```
effects = love.audio.getActiveEffects()
```

Arguments

None.

Returns

`table effects`

The list of the names of the currently enabled effects.

See Also

- love.audio
- love.audio.setEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getActiveSourceCount

love.audio.getActiveSourceCount · revision 21301 · last edited 2026-05-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 11.0

It has been renamed from love.audio.getSourceCount.

Gets the current number of simultaneously playing sources.

Function

Synopsis

```
count = love.audio.getActiveSourceCount( )
```

Arguments

None.

Returns

`number count`

The current number of simultaneously playing sources.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getDistanceModel

love.audio.getDistanceModel · revision 16089 · last edited 2026-05-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Returns the distance attenuation model.

Function

Synopsis

```
model = love.audio.getDistanceModel( )
```

Arguments

None.

Returns

`DistanceModel model`

The current distance model. The default is 'inverseclamped'.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getDopplerScale

love.audio.getDopplerScale · revision 14660 · last edited 2026-05-09

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets the current global scale factor for velocity-based doppler effects.

Function

Synopsis

```
scale = love.audio.getDopplerScale( )
```

Arguments

None.

Returns

`number scale`

The current doppler scale factor.

See Also

- love.audio
- love.audio.setDopplerScale

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getEffect

love.audio.getEffect · revision 23233 · last edited 2026-05-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the settings associated with an effect.

Function

Synopsis

```
settings = love.audio.getEffect(name)
```

Arguments

`string name`

The name of the effect.

Returns

`table settings`

The settings associated with the effect.

See Also

- love.audio
- love.audio.setEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getMaxSceneEffects

love.audio.getMaxSceneEffects · revision 21919 · last edited 2026-05-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the maximum number of active effects supported by the system.

Function

Synopsis

```
maximum = love.audio.getMaxSceneEffects( )
```

Arguments

None.

Returns

`number maximum`

The maximum number of active effects.

See Also

- love.audio
- love.audio.getMaxSourceEffects
- love.audio.isEffectsSupported
- love.audio.setEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getMaxSourceEffects

love.audio.getMaxSourceEffects · revision 23249 · last edited 2026-05-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the maximum number of active Effects in a single Source object, that the system can support.

Function

Synopsis

```
maximum = love.audio.getMaxSourceEffects( )
```

Arguments

None.

Returns

`number maximum`

The maximum number of active Effects per Source.

Notes

This function return 0 for system that doesn't support audio effects.

See Also

- love.audio
- love.audio.isEffectsSupported
- love.audio.setEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getNumSources

love.audio.getNumSources · revision 11818 · last edited 2025-09-28

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

It has been renamed to love.audio.getSourceCount.

Gets the current number of simultaneously playing sources.

Function

Synopsis

```
numSources = love.audio.getNumSources( )
```

Arguments

None.

Returns

`number numSources`

The current number of simultaneously playing sources.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getOrientation

love.audio.getOrientation · revision 5736 · last edited 2026-05-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Returns the orientation of the listener.

Function

Synopsis

```
fx, fy, fz, ux, uy, uz = love.audio.getOrientation( )
```

Arguments

None.

Returns

number fx, fy, fz
: Forward vector of the listener orientation.

number ux, uy, uz
: Up vector of the listener orientation.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getPlaybackDevice

love.audio.getPlaybackDevice · revision 29148 · last edited 2026-01-07

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets the currently active playback device.

Function

Synopsis

```
device = love.audio.getPlaybackDevice( )
```

Arguments

None.

Returns

`string device`

The name of the current playback device.

See Also

- love.audio
- love.audio.getPlaybackDevices
- love.audio.setPlaybackDevice

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getPlaybackDevices

love.audio.getPlaybackDevices · revision 29147 · last edited 2024-04-13

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets a list of playback devices on the system.

The first device in the list is the user's default playback device.

Function

Synopsis

```
devices = love.audio.getPlaybackDevices( )
```

Arguments

None.

Returns

`table devices`

The list of connected playback device names as strings.

See Also

- love.audio
- love.audio.getPlaybackDevice
- love.audio.setPlaybackDevice

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getPosition

love.audio.getPosition · revision 13719 · last edited 2025-09-29

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns the position of the listener. Please note that positional audio only works for mono (i.e. non-stereo) sources.

Function

Synopsis

```
x, y, z = love.audio.getPosition( )
```

Arguments

None.

Returns

number x
: The X position of the listener.

number y
: The Y position of the listener.

number z
: The Z position of the listener.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getRecordingDevices

love.audio.getRecordingDevices · revision 29203 · last edited 2026-05-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets a list of RecordingDevices on the system.

The first device in the list is the user's default recording device. The list may be empty if there are no microphones connected to the system.

Audio recording is currently not supported on iOS.

Function

Synopsis

```
devices = love.audio.getRecordingDevices( )
```

Arguments

None.

Returns

`table devices`

The list of connected recording devices.

Notes

Audio recording for Android is supported since 11.3. However, it's not supported when APK from Play Store is used.

See Also

- love.audio
- RecordingDevice

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getSourceCount

love.audio.getSourceCount · revision 22122 · last edited 2025-09-28

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from love.audio.getNumSources.

Deprecated in LÖVE 11.0

It has been renamed to love.audio.getActiveSourceCount.

Gets the current number of simultaneously playing sources.

Function

Synopsis

```
numSources = love.audio.getSourceCount( )
```

Arguments

None.

Returns

`number numSources`

The current number of simultaneously playing sources.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getVelocity

love.audio.getVelocity · revision 5738 · last edited 2026-05-09

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns the velocity of the listener.

Function

Synopsis

```
x, y, z = love.audio.getVelocity( )
```

Arguments

None.

Returns

number x
: The X velocity of the listener.

number y
: The Y velocity of the listener.

number z
: The Z velocity of the listener.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.getVolume

love.audio.getVolume · revision 5739 · last edited 2026-05-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Returns the master volume.

Function

Synopsis

```
volume = love.audio.getVolume( )
```

Arguments

None.

Returns

`number volume`
: The current master volume

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.isEffectsSupported

love.audio.isEffectsSupported · revision 23258 · last edited 2026-05-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets whether audio effects are supported in the system.

Function

Synopsis

```
supported = love.audio.isEffectsSupported( )
```

Arguments

None.

Returns

`boolean supported`

True if effects are supported, false otherwise.

Notes

Older Linux distributions that ship with older OpenAL library may not support audio effects. Furthermore, iOS doesn't support audio effects at all.

See Also

- love.audio
- love.audio.getMaxSceneEffects
- love.audio.getMaxSourceEffects
- love.audio.setEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.newQueueableSource

love.audio.newQueueableSource · revision 28037 · last edited 2026-05-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Creates a new Source usable for real-time generated sound playback with Source:queue.

Queueable sources allow SoundData objects to be played seamlessly one after another, without the need to wait for an update cycle.


This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Synopsis

```
source = love.audio.newQueueableSource( samplerate, bitdepth, channels, buffercount )
```

Arguments

number samplerate
: Number of samples per second when playing.

number bitdepth
: Bits per sample (8 or 16).

number channels
: 1 for mono or 2 for stereo.

number buffercount (8)
: The number of buffers that can be queued up at any given time with Source:queue. Cannot be greater than 64. A sensible default (~8) is chosen if no value is specified.

Returns

Source source

The new Source usable with Source:queue.

Notes

The sample rate, bit depth, and channel count of any SoundData used with Source:queue must match the parameters given to this constructor.

See Also

- love.audio
- Source
- Source:queue
- SourceType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.newSource

love.audio.newSource · revision 27445 · last edited 2025-09-29

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
  - Load background music and play it
  - Load a sound effect and play it
  - Load SoundData and create a Source
  - Load Decoder and create a Source
- See Also
- Other Languages

Creates a new Source from a filepath, File, Decoder or SoundData. Sources created from SoundData are always static.



This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
source = love.audio.newSource( filename, type )
```

Arguments

`string filename`

The filepath to the audio file.

`SourceType type`

Streaming or static source.

Returns

`Source source`

A new Source that can play the specified audio.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
source = love.audio.newSource( file, type )
```

Arguments

`File file`

A File pointing to an audio file.

`SourceType type`

Streaming or static source.

Returns

`Source source`

A new Source that can play the specified audio.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
source = love.audio.newSource( decoder, type )
```

Arguments

`Decoder decoder`

The Decoder to create a Source from.

`SourceType type`

Streaming or static source.

Returns

`Source source`

A new Source that can play the specified audio.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
source = love.audio.newSource( data, type )
```

Arguments

FileData data
: The FileData to create a Source from.

SourceType type
: Streaming or static source.

Returns

Source source
: A new Source that can play the specified audio.

Function

Synopsis

```
source = love.audio.newSource( data )
```

Arguments

SoundData data
: The SoundData to create a Source from.

Returns

Source source
: A new Source that can play the specified audio. The SourceType of the returned audio is "static".

Function

Removed in LÖVE 11.0
This variant is not supported in that and later versions.

Synopsis

```
source = love.audio.newSource( filename, type )
```

Arguments

string filename
: The filepath to the audio file.

SourceType type ("stream")

Streaming or static source.

Returns

`Source source`
A new Source that can play the specified audio.

Function

Removed in LÖVE 11.0
This variant is not supported in that and later versions.

Synopsis

```
source = love.audio.newSource( file, type )
```

Arguments

`File file`
A File pointing to an audio file.
`SourceType type ("stream")`
Streaming or static source.

Returns

`Source source`
A new Source that can play the specified audio.

Function

Removed in LÖVE 11.0
This variant is not supported in that and later versions.

Synopsis

```
source = love.audio.newSource( decoder, type )
```

Arguments

`Decoder decoder`
The Decoder to create a Source from.
`SourceType type ("stream")`
Streaming or static source.

Returns

`Source source`

A new Source that can play the specified audio.

Function

Removed in LÖVE 11.0

This variant is not supported in that and later versions.

Synopsis

```
source = love.audio.newSource( data, type )
```

Arguments

`FileData data`

The FileData to create a Source from.

`SourceType type ("stream")`

Streaming or static source.

Returns

`Source source`

A new Source that can play the specified audio.

Notes

From 11.0 to 11.2, if **queue** is specified as SourceType for this specific constructor, it won't error, and getType will return **stream**; one should use love.audio.newQueueableSource for that specific source type instead. This was a bug fixed in 11.3.

Examples

Load background music and play it

```
bgm = love.audio.newSource("bgm.ogg", "stream")
love.audio.play(bgm)
```

Load a sound effect and play it

```
sfx = love.audio.newSource("sfx.wav", "static")
love.audio.play(sfx)
```

Load SoundData and create a Source

```
data = love.sound.newSoundData("sfx.wav")
sfx = love.audio.newSource(data)
```

Load Decoder and create a Source

```
decoder = love.sound.newDecoder("bgm.ogg")
bgm = love.audio.newSource(decoder, "stream")
```

See Also

- love.audio
- Source

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.pause

love.audio.pause · revision 22103 · last edited 2025-09-29

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Pauses specific or all currently played Sources.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Pauses all currently active Sources and returns them.

Synopsis

```
sources = love.audio.pause( )
```

Arguments

None.

Returns

`table Sources`

A table containing a list of Sources that were paused by this call.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Pauses the given Sources.

Synopsis

```
love.audio.pause( source, ... )
```

Arguments

`Source source`

The first Source to pause.

`Source ...`

Additional Sources to pause.

Returns

Nothing.

Function

Available since LÖVE 11.0
This variant is not supported in earlier versions.

Pauses the given Sources.

Synopsis

```
love.audio.pause( sources )
```

Arguments

`table sources`
A table containing a list of Sources to pause.

Returns

Nothing.

Function

Removed in LÖVE 11.0
This variant is not supported in that and later versions.

Pauses all currently active Sources.

Synopsis

```
love.audio.pause( )
```

Arguments

None.

Returns

Nothing.

Function

Removed in LÖVE 11.0
This variant is not supported in that and later versions.

Pauses a specific Source.

Synopsis

```
love.audio.pause( source )
```

Arguments

`Source source`
: The source on which to pause the playback.

Returns

Nothing.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.play

love.audio.play · revision 31596 · last edited 2026-05-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Plays specific Sources.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Starts playing one or multiple Sources simultaneously.

Synopsis

```
success = love.audio.play( source, ... )
```

Arguments

`Source source`
: The first Source to play.

`Source ...`
: Additional Sources to play.

Returns

`boolean success`
: Whether the Sources were able to successfully start playing.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Starts playing a list of Sources simultaneously.

Synopsis

```
success = love.audio.play( sources )
```

Arguments

`table sources`

Table containing a list of Sources to play.

Returns

`boolean success`
Whether the Sources were able to successfully start playing.

Function

Removed in LÖVE 0.9.1
This variant is not supported in that and later versions.

Plays the specified Source.

Synopsis

```
love.audio.play( source )
```

Arguments

`Source source`
The Source to play.

Returns

Nothing.

Function

Available since LÖVE 0.9.1 and removed in LÖVE 11.0
This variant is not supported in earlier or later versions.

Plays the specified Source.

Synopsis

```
success = love.audio.play( source )
```

Arguments

`Source source`
The Source to play.

Returns

`boolean success`
Whether the Sources were able to successfully start playing.

See Also

- love.audio
- Source:play

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.resume

love.audio.resume · revision 21363 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 11.0

Use love.audio.play instead.

Resumes all audio.

Function

Synopsis

```
love.audio.resume( )
```

Arguments

None.

Returns

Nothing.

Function

Synopsis

```
love.audio.resume( source )
```

Arguments

`Source source`
: The source on which to resume the playback.

Returns

Nothing.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.rewind

love.audio.rewind · revision 21848 · last edited 2025-09-29

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 11.0

Use Source:seek(0) instead.

Rewinds all playing audio.

Function

Synopsis

```
love.audio.rewind( )
```

Arguments

None.

Returns

Nothing.

Function

Synopsis

```
love.audio.rewind( source )
```

Arguments

`Source source`
: The source to rewind.

Returns

Nothing.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.setDistanceModel

love.audio.setDistanceModel · revision 8411 · last edited 2026-05-15

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Sets the distance attenuation model.

Function

Synopsis

```
love.audio.setDistanceModel( model )
```

Arguments

`DistanceModel model`

The new distance model.

Returns

Nothing.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.setDopplerScale

love.audio.setDopplerScale · revision 14659 · last edited 2026-05-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Sets a global scale factor for velocity-based doppler effects. The default scale value is 1.

Function

Synopsis

```
love.audio.setDopplerScale( scale )
```

Arguments

`number scale`
: The new doppler scale factor. The scale must be greater than 0.

Returns

Nothing.

See Also

- love.audio
- love.audio.getDopplerScale

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.setEffect

love.audio.setEffect · revision 30382 · last edited 2026-05-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
  - Play music with added reverb
  - Play music with distortion
  - Play effect while muting the dry sound
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Defines an effect that can be applied to a Source.

Not all systems support audio effects. Use love.audio.isEffectsSupported to check.

Function

Synopsis

```
success = love.audio.setEffect( name, settings )
```

Arguments

`string name`

The name of the effect.

`table settings`

The settings to use for this effect, with the following fields:

`EffectType type`

The type of effect to use.

`number volume`

The volume of the effect.

`number ...`

Effect-specific settings. See EffectType for available effects and their corresponding settings.

Returns

`boolean success`

Whether the effect was successfully created.

Function

Synopsis

```
success = love.audio.setEffect( name, enabled )
```

Arguments

string name

The name of the effect.

boolean enabled (true)

If false and the given effect name was previously set, disables the effect.

Returns

boolean success

Whether the effect was successfully disabled.

Notes

Audio produced by effects are added on top of the normal dry sound from Sources.

Examples

Play music with added reverb

```
love.audio.setEffect("myEffect", {type="reverb"})

local source = love.audio.newSource("music.ogg", "stream")
source:setEffect("myEffect")
source:play()
```

Play music with distortion

```
love.audio.setEffect("myEffect", {
    type = "distortion",
    gain = .5,
    edge = .25,
})
local source = love.audio.newSource("music.ogg", "stream")
source:setEffect("myEffect")
source:play()
```

Play effect while muting the dry sound

```
love.audio.setEffect("myEffect", {type="chorus", waveform="sine", rate=5, depth=.5})

local source = love.audio.newSource("music.ogg", "stream")
source:setEffect("myEffect")
```

```
source:setFilter{ volume=0, type="lowpass" } -- This volume only affects the dry sound.
source:play()
```

See Also

- love.audio
- love.audio.isEffectsSupported
- Source:setEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.setMixWithSystem

love.audio.setMixWithSystem · revision 22747 · last edited 2026-05-15

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Sets whether the system should mix the audio with the system's audio.

Function

Synopsis

```
success = love.audio.setMixWithSystem( mix )
```

Arguments

`boolean mix`

True to enable mixing, false to disable it.

Returns

`boolean success`

True if the change succeeded, false otherwise.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.setOrientation

love.audio.setOrientation · revision 5743 · last edited 2026-05-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Sets the orientation of the listener.

Function

Synopsis

```
love.audio.setOrientation( fx, fy, fz, ux, uy, uz )
```

Arguments

number fx, fy, fz
: Forward vector of the listener orientation.

number ux, uy, uz
: Up vector of the listener orientation.

Returns

Nothing.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.setPlaybackDevice

love.audio.setPlaybackDevice · revision 28061 · last edited 2024-04-13

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Change the audio device to specified device. Also used to reconnect audio device in case of device disconnection.

Function

Synopsis

```
success, message = love.audio.setPlaybackDevice( name )
```

Arguments

`string name (nil)`

Fully qualified device name, or nil to use system default.

Returns

`boolean success`

Is the function succeeded?

`string message`

Error message on failure (nil on succesful).

Notes

OpenAL-soft 1.22.0 or later is required for this to function. Officially LOVE-distributed binaries (with exception of iOS) ships with latest version of OpenAL-soft.

See Also

- love.audio
- love.audio.getPlaybackDevice
- love.audio.getPlaybackDevices

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.setPosition

love.audio.setPosition · revision 22051 · last edited 2026-05-15

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the position of the listener, which determines how sounds play.

Function

Synopsis

```
love.audio.setPosition( x, y, z )
```

Arguments

number x
: The x position of the listener.

number y
: The y position of the listener.

number z
: The z position of the listener.

Returns

Nothing.

See Also

- love.audio
- Source:setPosition

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.setVelocity

love.audio.setVelocity · revision 5745 · last edited 2025-09-29

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Sets the velocity of the listener.

Function

Synopsis

```
love.audio.setVelocity( x, y, z )
```

Arguments

number x
: The X velocity of the listener.

number y
: The Y velocity of the listener.

number z
: The Z velocity of the listener.

Returns

Nothing.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.setVolume

love.audio.setVolume · revision 14802 · last edited 2025-09-29

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Sets the master volume.

Function

Synopsis

```
love.audio.setVolume( volume )
```

Arguments

number volume
: 1.0 is max and 0.0 is off.

Returns

Nothing.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audio.stop

love.audio.stop · revision 31600 · last edited 2026-05-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Stops specific or all currently played sources.

Function

This function will stop all currently active sources.

Synopsis

```
love.audio.stop( )
```

Arguments

None.

Returns

Nothing.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Stops one or multiple Sources simultaneously.

Synopsis

```
love.audio.stop( source, ... )
```

Arguments

`Source source`
: The first Source to stop.

`Source ...`

Additional Sources to stop.

Returns

Nothing.

Function

Available since LÖVE 11.0
This variant is not supported in earlier versions.

Stops a list of Sources simultaneously.

Synopsis

```
love.audio.stop( sources )
```

Arguments

`table sources`
A table containing a list of Sources to stop.

Returns

Nothing.

Function

Removed in LÖVE 11.0
This variant is not supported in that and later versions.

This function will only stop the specified source.

Synopsis

```
love.audio.stop( source )
```

Arguments

`Source source`
The source on which to stop the playback.

Returns

Nothing.

See Also

- love.audio

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.audiodisconnected

love.audiodisconnected · revision 30562 · last edited 2026-01-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This callback is not supported in earlier versions.

Called when the active audio device is disconnected (e.g. physically unplugging headphones).

All audio are stopped **and** loses their playback position when this callback is called.

Function

Synopsis

```
reconnected = love.audiodisconnected( sources )
```

Arguments

`table sources`

List of sources that were playing.

Returns

`boolean reconnected`

Is audio re-connection attempt has been done?

Notes

If love.audio.setPlaybackDevice is called inside the callback, `true` must be returned to prevent LÖVE from re-connecting the audio device again.

See Also

- love
- love.audio.getPlaybackDevices
- love.audio.setPlaybackDevice

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.body

love.body · revision 25434 · last edited 2021-03-14

I didn't do it signed Leviskr361

love.bundle

love.bundle · revision 24498 · last edited 2023-07-05

- Original Thread (http://love2d.org/forums/viewtopic.php?f=5&t=1303)

love.bundle is a library for storing game data between sessions. Unlike the most used saving systems, you don't have to make your own converting algorithm for data to text and the opposite. Here all you have to do is putting your data in a created bundle, and save it. The library also features a function for setting a function to be called when the game exits, here the user can put stuff like saving their bundles. The bundle can store 4 types of values:

- string
- number
- boolean
- nil

Those 4 different types of values can easily be put into a created bundle or read from one with this:

```
-- Create a bundle with the given name/identifier
Bundle = love.bundle.newBundle("A Bundle Name")

-- Change the identity of the bundle
Bundle:setIdentity("Another Bundle Name")

-- Save the bundle to the saving directory
Bundle:save()

-- Load a saved bundle with that name if it exists
Bundle = love.bundle.loadBundle("Another Bundle Name")

-- Set aFunction to be called when someone exits the game
love.bundle.setExitFunction(aFunction)

-- The return value tells you whether the operation succeeded or not
success = Bundle:put("Name Of The Value", 42)

-- Return the value with that name, or nil if not found
var = Bundle:get("Name Of The Value")

-- Return the value with that name, or the 2nd argument if not found
var = Bundle:get("Name Of A Not Created Value", true)
```

To understand this more, you can download the documentation I have created which is just like the LÖVE Documentation one. It looks the same to make it easier to understand since you don't have to get used to a new type.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.data

love.data · revision 31496 · last edited 2026-04-20

Available since LÖVE 11.0

This module is not supported in earlier versions.

Provides functionality for creating and transforming data.

Types

ByteData	Data object containing arbitrary bytes in an contiguous memory.	Added since 11.0
CompressedData	Byte data compressed using a specific algorithm.	Added since 0.10.0
Data	The superclass of all data.	

Functions

love.data.compress	Compresses a string or data using a specific compression algorithm.	Added since 11.0
love.data.decode	Decode Data or a string from any of the EncodeFormats to Data or string.	Added since 11.0
love.data.decompress	Decompresses a CompressedData or previously compressed string or Data object.	Added since 11.0
love.data.encode	Encode Data or a string to a Data or string in one of the EncodeFormats.	Added since 11.0
love.data.getPackedSize	Gets the size in bytes that a given format used with love.data.pack will use.	Added since 11.0
love.data.hash	Compute message digest using specific hash algorithm.	Added since 11.0
love.data.newByteData	Creates a new Data object containing arbitrary bytes.	Added since 11.0
love.data.newDataView	Creates a new Data referencing a subsection of an existing Data object.	Added since 11.0
love.data.pack	Packs (serializes) simple Lua values.	Added since 11.0
love.data.unpack	Unpacks (deserializes) a byte-string or Data into simple Lua values.	Added since 11.0

Enums

CompressedDataFormat	Compressed data formats.	Added since 0.10.0
ContainerType	Return type of data-returning functions.	Added since 11.0
EncodeFormat	Encoding format used to encode or decode data.	Added since 11.0

HashFunction	Hash algorithm of hash function.	Added since 11.0

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.data.compress

love.data.compress · revision 21316 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

Deprecates love.math.compress.

Compresses a string or data using a specific compression algorithm.



This function, depending on the compression format and level, can be slow if called repeatedly, such as from love.update or love.draw. Some benchmarks are available here (https://github.com/Cyan4973/lz4#user-content-benchmarks).

Function

Synopsis

```
compressedData = love.data.compress( container, format, rawstring, level )
```

Arguments

`ContainerType container`
: What type to return the compressed data as.

`CompressedDataFormat format`
: The format to use when compressing the string.

`string rawstring`
: The raw (un-compressed) string to compress.

`number level (-1)`
: The level of compression to use, between 0 and 9. -1 indicates the default level. The meaning of this argument depends on the compression format being used.

Returns

`value compressedData`

CompressedData/string which contains the compressed version of rawstring.

Function

Synopsis

```
compressedData = love.data.compress( container, format, data, level )
```

Arguments

`ContainerType container`
What type to return the compressed data as.

`CompressedDataFormat format`
The format to use when compressing the data.

`Data data`
A Data object containing the raw (un-compressed) data to compress.

`number level (-1)`
The level of compression to use, between 0 and 9. -1 indicates the default level. The meaning of this argument depends on the compression format being used.

Returns

`value compressedData`
CompressedData/string which contains the compressed version of data.

See Also

- love.data
- love.data.decompress
- CompressedData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.data.decode

love.data.decode · revision 22619 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Decode Data or a string from any of the EncodeFormats to Data or string.

Function

Synopsis

```
decoded = love.data.decode( container, format, sourceString )
```

Arguments

`ContainerType container`
: What type to return the decoded data as.

`EncodeFormat format`
: The format of the input data.

`string sourceString`
: The raw (encoded) data to decode.

Returns

`value decoded`
: ByteData/string which contains the decoded version of source.

Function

Synopsis

```
decoded = love.data.decode( container, format, sourceData )
```

Arguments

`ContainerType container`
: What type to return the decoded data as.

`EncodeFormat format`
: The format of the input data.

`Data sourceData`

The raw (encoded) data to decode.

Returns

`value decoded`

ByteData/string which contains the decoded version of source.

See Also

- love.data
- love.data.encode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.data.decompress

love.data.decompress · revision 21319 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

Deprecates love.math.decompress.

Decompresses a CompressedData or previously compressed string or Data object.

Function

Synopsis

```
decompressedData = love.data.decompress( container, compressedData )
```

Arguments

`ContainerType container`
: What type to return the decompressed data as.

`CompressedData compressedData`
: The compressed data to decompress.

Returns

`value decompressedData`
: Data/string containing the raw decompressed data.

Function

Synopsis

```
decompressedData = love.data.decompress( container, format, compressedString )
```

Arguments

`ContainerType container`
: What type to return the decompressed data as.

`CompressedDataFormat format`
: The format that was used to compress the given string.

`string compressedString`

A string containing data previously compressed with love.data.compress.

Returns

value decompressedData
Data/string containing the raw decompressed data.

Function

Synopsis

```
decompressedData = love.data.decompress( container, format, data )
```

Arguments

ContainerType container
What type to return the decompressed data as.

CompressedDataFormat format
The format that was used to compress the given data.

Data data
A Data object containing data previously compressed with love.data.compress.

Returns

value decompressedData
Data/string containing the raw decompressed data.

See Also

- love.data
- love.data.compress

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.data.encode

love.data.encode · revision 22620 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Encode Data or a string to a Data or string in one of the EncodeFormats.

Function

Synopsis

```
encoded = love.data.encode( container, format, sourceString, linelength )
```

Arguments

`ContainerType container`
: What type to return the encoded data as.

`EncodeFormat format`
: The format of the output data.

`string sourceString`
: The raw data to encode.

`number linelength (0)`
: The maximum line length of the output. Only supported for base64, ignored if 0.

Returns

`value encoded`
: ByteData/string which contains the encoded version of source.

Function

Synopsis

```
encoded = love.data.encode( container, format, sourceData, linelength )
```

Arguments

`ContainerType container`
: What type to return the encoded data as.

`EncodeFormat format`
: The format of the output data.

`Data sourceData`

The raw data to encode.

`number linelength (0)`

The maximum line length of the output. Only supported for base64, ignored if 0.

Returns

`value encoded`

ByteData/string which contains the encoded version of source.

See Also

- love.data
- love.data.decode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.data.getPackedSize

love.data.getPackedSize · revision 21290 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the size in bytes that a given format used with love.data.pack will use.

This function behaves the same as Lua 5.3's string.packsize (https://www.lua.org/manual/5.3/manual.html#pdf-string.packsize).

Function

Synopsis

```
size = love.data.getPackedSize( format )
```

Arguments

`string format`

A string determining how the values are packed. Follows the rules of Lua 5.3's string.pack format strings (https://www.lua.org/manual/5.3/manual.html#6.4.2).

Returns

`number size`

The size in bytes that the packed data will use.

Notes

The format string cannot have the variable-length options 's' or 'z'.

See Also

- love.data
- love.data.pack
- love.data.unpack

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.data.hash

love.data.hash · revision 28142 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Compute the message digest of a string using a specified hash algorithm.

Function

Synopsis

```
rawdigest = love.data.hash( hashFunction, string )
```

Arguments

HashFunction hashFunction
: Hash algorithm to use.

string string
: String to hash.

Returns

string rawdigest
: Raw message digest string.

Function

Synopsis

```
rawdigest = love.data.hash( hashFunction, data )
```

Arguments

HashFunction hashFunction
: Hash algorithm to use.

Data data
: Data to hash.

Returns

string rawdigest

Raw message digest string.

Notes

LOVE before 11.3 had a bug (https://github.com/love2d/love/issues/1458) that gives wrong result for specific input length.

To return the hex string representation of the hash, use love.data.encode

```
hexDigestString = love.data.encode("string", "hex", love.data.hash(algo, data))
```

See Also

- love.data
- love.data.encode
- HashFunction

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.data.newByteData

love.data.newByteData · revision 25532 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Creates a new Data object containing arbitrary bytes.

Data:getPointer along with LuaJIT's FFI can be used to manipulate the contents of the ByteData object after it has been created.



This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Creates a new ByteData by copying the contents of the specified string.

Synopsis

```
bytedata = love.data.newByteData( datastring )
```

Arguments

`string datastring`

The byte string to copy.

Returns

`ByteData bytedata`

The new Data object.

Function

Creates a new ByteData by copying from an existing Data object.

Synopsis

```
bytedata = love.data.newByteData( data, offset, size )
```

Arguments

Data data
The existing Data object to copy.

number offset (0)
The offset of the subsection to copy, in bytes.

number size (data:getSize())
The size in bytes of the new Data object.

Returns

ByteData bytedata
The new Data object.

Function

Creates a new empty (zero-initialized) ByteData with the specific size.

Synopsis

```
bytedata = love.data.newByteData( size )
```

Arguments

number size
The size in bytes of the new Data object.

Returns

ByteData bytedata
The new Data object.

See Also

- love.data

- ByteData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.data.newDataView

love.data.newDataView · revision 28338 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Creates a new Data referencing a subsection of an existing Data object.



This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Synopsis

```
view = love.data.newDataView( data, offset, size )
```

Arguments

`Data data`

The Data object to reference.

`number offset`

The offset of the subsection to reference, in bytes.

`number size`

The size in bytes of the subsection to reference.

Returns

`Data view`

The new Data view.

Notes

Data:getString, Data:getPointer and Data:getFFIPointer will return the original Data object's contents, with the view's offset and size applied where applicable.

See Also

- love.data

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.data.pack

love.data.pack · revision 24128 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Packs (serializes) simple Lua values.

This function behaves the same as Lua 5.3's string.pack (https://w w w.lua.org/manual/5.3/manual.html#pdf-string.pack).

Function

Synopsis

```
data = love.data.pack( container, format, v1, ... )
```

Arguments

`ContainerType container`
: What type to return the encoded data as.

`string format`
: A string determining how the values are packed. Follows the rules of Lua 5.3's string.pack format strings (https://w w w.lua.org/manual/5.3/manual.html#6.4.2).

`value v1`
: The first value (number, boolean, or string) to serialize.

`value ...`
: Additional values to serialize.

Returns

`value data`
: Data/string which contains the serialized data.

Notes

Packing integers with values greater than 2^53 is not supported, as Lua 5.1 cannot represent those values in its number type.

See Also

- love.data
- love.data.unpack

- love.data.getPackedSize

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.data.unpack

love.data.unpack · revision 21289 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Unpacks (deserializes) a byte-string or Data into simple Lua values.

This function behaves the same as Lua 5.3's string.unpack (https://www.lua.org/manual/5.3/manual.html#pdf-string.unpack).

Function

Synopsis

```
v1, ..., index = love.data.unpack( format, datastring, pos )
```

Arguments

`string format`
: A string determining how the values were packed. Follows the rules of Lua 5.3's string.pack format strings (https://www.lua.org/manual/5.3/manual.html#6.4.2).

`string datastring`
: A string containing the packed (serialized) data.

`number pos (1)`
: Where to start reading in the string. Negative values can be used to read relative from the end of the string.

Returns

`value v1`
: The first value (number, boolean, or string) that was unpacked.

`value ...`
: Additional unpacked values.

`number index`
: The index of the first unread byte in the data string.

Function

Synopsis

```
v1, ..., index = love.data.unpack( format, data, pos )
```

Arguments

`string format`

A string determining how the values were packed. Follows the rules of Lua 5.3's string.pack format strings (https://www.lua.org/manual/5.3/manual.html#6.4.2).

Data data

A Data object containing the packed (serialized) data.

number pos (1)

1-based index indicating where to start reading in the Data. Negative values can be used to read relative from the end of the Data object.

Returns

value v1

The first value (number, boolean, or string) that was unpacked.

value ...

Additional unpacked values.

number index

The 1-based index of the first unread byte in the Data.

Notes

Unpacking integers with values greater than 2^52 is not supported, as Lua 5.1 cannot represent those values in its number type.

See Also

- love.data
- love.data.pack
- love.data.getPackedSize

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.directorydropped

love.directorydropped · revision 22508 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Callback function triggered when a directory is dragged and dropped onto the window.

Function

Synopsis

```
love.directorydropped( path )
```

Arguments

`string path`

The full platform-dependent path to the directory. It can be used as an argument to love.filesystem.mount, in order to gain read access to the directory with love.filesystem.

Returns

Nothing.

Notes

Paths passed into this callback are able to be used with love.filesystem.mount, which is the only way to get read access via love.filesystem to the dropped directory. love.filesystem.mount does not generally accept other full platform-dependent directory paths that haven't been dragged and dropped onto the window.

See Also

- love
- love.filedropped

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.displayrotated

love.displayrotated · revision 28152 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Workaround
- See Also
- Other Languages

Available since LÖVE 11.3

This callback is not supported in earlier versions.

Called when the device display orientation changed, for example, user rotated their phone 180 degrees.


Prior to 11.4, the orientation value was boolean instead of one of valid DisplayOrientation values! See below for workaround.

Function

Synopsis

```
love.displayrotated( index, orientation )
```

Arguments

`number index`
: The index of the display that changed orientation.

`DisplayOrientation orientation`
: The new orientation.

Returns

Nothing.

Workaround

Due to a bug in LOVE 11.3, the orientation value is boolean `true` instead. A workaround is as follows:

```
function love.displayrotated(index, orientation)
    orientation = love.window.getDisplayOrientation(index)
    -- The rest of your code goes here.
end
```

See Also

- love
- love.window.getDisplayOrientation

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.draw

love.draw · revision 29418 · last edited 2025-10-21

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Callback function used by the default love.run to draw on the screen every frame.

Function

Synopsis

```
love.draw( )
```

Arguments

None.

Returns

Nothing.

Examples

Draw an image that was loaded in love.load (putting love.graphics.newImage in love.draw would cause the image to be reloaded every frame, which would cause issues).

```
function love.load()
   hamster = love.graphics.newImage("hamster.png")
   x = 50
   y = 50
end
function love.draw()
   love.graphics.draw(hamster, x, y)
end
```

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.dropbegan

love.dropbegan · revision 29587 · last edited 2025-04-18

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Callback function triggered when a file or folder is first dragged onto the window, before the user drops it.

This can be used to change the cursor to something indicating a drag-and-drop operation, for example.

Function

Synopsis

```
love.dropbegan( )
```

Arguments

None.

Returns

Nothing.

See Also

- love
- love.dropmoved
- love.dropcompleted
- love.filedropped
- love.directorydropped

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.dropcompleted

love.dropcompleted · revision 29588 · last edited 2025-04-18

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Callback function triggered when a file or folder is done being dragged and dropped into the window.

Function

Synopsis

```
love.dropcompleted( x, y )
```

Arguments

number x
: The x coordinate of the final position of the drag-and-drop operation, within the window.

number y
: The y coordinate of the final position of the drag-and-drop operation, within the window.

Returns

Nothing.

See Also

- love
- love.dropbegan
- love.dropmoved
- love.filedropped
- love.directorydropped

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.dropmoved

love.dropmoved · revision 29589 · last edited 2025-04-18

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Callback function triggered when an in-progress file or folder drag-and-drop operation changes position within the window.

Function

Synopsis

```
love.dropmoved( x, y )
```

Arguments

number x
: The x coordinate of the new position of the drag-and-drop operation, within the window.

number y
: The y coordinate of the new position of the drag-and-drop operation, within the window.

Returns

Nothing.

See Also

- love
- love.dropbegan
- love.dropcompleted
- love.filedropped
- love.directorydropped

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.errhand

love.errhand · revision 24396 · last edited 2026-03-23

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - The default function used if you don't supply your own.
  - The default function used if you don't supply your own.
  - The default function used if you don't supply your own.
  - The default function used if you don't supply your own.
- See Also
- Other Languages

Deprecated in LÖVE 11.0

This function has been renamed to love.errorhandler.

The error handler, used to display error messages.

Function

Synopsis

```
love.errhand( msg )
```

Arguments

string msg

The error message.

Returns

Nothing.

Examples

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

The default function used if you don't supply your own.

```
local function error_printer(msg, layer)
    print((debug.traceback("Error: " .. tostring(msg), 1+(layer or 1)):gsub("\n[^\n]+$", "")))
end

function love.errhand(msg)
    msg = tostring(msg)

    error_printer(msg, 2)

    if not love.window or not love.graphics or not love.event then
        return
    end

    if not love.graphics.isCreated() or not love.window.isOpen() then
        local success, status = pcall(love.window.setMode, 800, 600)
        if not success or not status then
            return
```

```
    end
end

-- Reset state.
if love.mouse then
    love.mouse.setVisible(true)
    love.mouse.setGrabbed(false)
    love.mouse.setRelativeMode(false)
end
if love.joystick then
    -- Stop all joystick vibrations.
    for i,v in ipairs(love.joystick.getJoysticks()) do
        v:setVibration()
    end
end
if love.audio then love.audio.stop() end
love.graphics.reset()
local font = love.graphics.setNewFont(math.floor(love.window.toPixels(14)))

love.graphics.setBackgroundColor(89, 157, 220)
love.graphics.setColor(255, 255, 255, 255)

local trace = debug.traceback()

love.graphics.clear(love.graphics.getBackgroundColor())
love.graphics.origin()

local err = {}

table.insert(err, "Error\n")
table.insert(err, msg.."\n\n")

for l in string.gmatch(trace, "(.-)\n") do
    if not string.match(l, "boot.lua") then
        l = string.gsub(l, "stack traceback:", "Traceback\n")
        table.insert(err, l)
    end
end

local p = table.concat(err, "\n")

p = string.gsub(p, "\t", "")
p = string.gsub(p, "%[string \"(.-)\"%]", "%1")

local function draw()
    local pos = love.window.toPixels(70)
    love.graphics.clear(love.graphics.getBackgroundColor())
    love.graphics.printf(p, pos, pos, love.graphics.getWidth() - pos)
    love.graphics.present()
end

while true do
    love.event.pump()

    for e, a, b, c in love.event.poll() do
        if e == "quit" then
            return
        elseif e == "keypressed" and a == "escape" then
            return
        elseif e == "touchpressed" then
            local name = love.window.getTitle()
            if #name == 0 or name == "Untitled" then name = "Game" end
```

```
                local buttons = {"OK", "Cancel"}
                local pressed = love.window.showMessageBox("Quit "..name.."?", "", buttons)
                if pressed == 1 then
                    return
                end
            end
        end

        draw()

        if love.timer then
            love.timer.sleep(0.1)
        end
    end

end
```

Available since LÖVE 0.9.2 and removed in LÖVE 0.10.0

This variant is not supported in earlier or later versions.

The default function used if you don't supply your own.

```
local function error_printer(msg, layer)
    print((debug.traceback("Error: " .. tostring(msg), 1+(layer or 1)):gsub("\n[^\n]+$", "")))
end

function love.errhand(msg)
    msg = tostring(msg)

    error_printer(msg, 2)

    if not love.window or not love.graphics or not love.event then
        return
    end

    if not love.graphics.isCreated() or not love.window.isCreated() then
        local success, status = pcall(love.window.setMode, 800, 600)
        if not success or not status then
            return
        end
    end

    -- Reset state.
    if love.mouse then
        love.mouse.setVisible(true)
        love.mouse.setGrabbed(false)
    end
    if love.joystick then
        -- Stop all joystick vibrations.
        for i,v in ipairs(love.joystick.getJoysticks()) do
            v:setVibration()
        end
    end
    if love.audio then love.audio.stop() end
    love.graphics.reset()
    local font = love.graphics.setNewFont(math.floor(love.window.toPixels(14)))

    local sRGB = select(3, love.window.getMode()).srgb
    if sRGB and love.math then
        love.graphics.setBackgroundColor(love.math.gammaToLinear(89, 157, 220))
    else
```

```
        love.graphics.setBackgroundColor(89, 157, 220)
    end

    love.graphics.setColor(255, 255, 255, 255)

    local trace = debug.traceback()

    love.graphics.clear()
    love.graphics.origin()

    local err = {}

    table.insert(err, "Error\n")
    table.insert(err, msg.."\n\n")

    for l in string.gmatch(trace, "(.-)\n") do
        if not string.match(l, "boot.lua") then
            l = string.gsub(l, "stack traceback:", "Traceback\n")
            table.insert(err, l)
        end
    end

    local p = table.concat(err, "\n")

    p = string.gsub(p, "\t", "")
    p = string.gsub(p, "%[string \"(.-)\"%]", "%1")

    local function draw()
        local pos = love.window.toPixels(70)
        love.graphics.clear()
        love.graphics.printf(p, pos, pos, love.graphics.getWidth() - pos)
        love.graphics.present()
    end

    while true do
        love.event.pump()

        for e, a, b, c in love.event.poll() do
            if e == "quit" then
                return
            end
            if e == "keypressed" and a == "escape" then
                return
            end
        end

        draw()

        if love.timer then
            love.timer.sleep(0.1)
        end
    end
end
```

Available since LÖVE 0.9.0 and removed in LÖVE 0.9.2

This variant is not supported in earlier or later versions.

The default function used if you don't supply your own.

```
local function error_printer(msg, layer)
    print((debug.traceback("Error: " .. tostring(msg), 1+(layer or 1)):gsub("\n[^\n]+$", "")))
```

```lua
end

function love.errhand(msg)
    msg = tostring(msg)

    error_printer(msg, 2)

    if not love.window or not love.graphics or not love.event then
        return
    end

    if not love.graphics.isCreated() or not love.window.isCreated() then
        if not pcall(love.window.setMode, 800, 600) then
            return
        end
    end

    -- Reset state.
    if love.mouse then
        love.mouse.setVisible(true)
        love.mouse.setGrabbed(false)
    end
    if love.joystick then
        for i,v in ipairs(love.joystick.getJoysticks()) do
            v:setVibration() -- Stop all joystick vibrations.
        end
    end
    if love.audio then love.audio.stop() end
    love.graphics.reset()
    love.graphics.setBackgroundColor(89, 157, 220)
    local font = love.graphics.setNewFont(14)

    love.graphics.setColor(255, 255, 255, 255)

    local trace = debug.traceback()

    love.graphics.clear()
    love.graphics.origin()

    local err = {}

    table.insert(err, "Error\n")
    table.insert(err, msg.."\n\n")

    for l in string.gmatch(trace, "(.-)\n") do
        if not string.match(l, "boot.lua") then
            l = string.gsub(l, "stack traceback:", "Traceback\n")
            table.insert(err, l)
        end
    end

    local p = table.concat(err, "\n")

    p = string.gsub(p, "\t", "")
    p = string.gsub(p, "%[string \"(.-)\"%]", "%1")

    local function draw()
        love.graphics.clear()
        love.graphics.printf(p, 70, 70, love.graphics.getWidth() - 70)
        love.graphics.present()
    end
```

```
    while true do
        love.event.pump()

        for e, a, b, c in love.event.poll() do
            if e == "quit" then
                return
            end
            if e == "keypressed" and a == "escape" then
                return
            end
        end

        draw()

        if love.timer then
            love.timer.sleep(0.1)
        end
    end

end
```

Removed in LÖVE 0.9.0

This variant is not supported in that and later versions.

The default function used if you don't supply your own.

```
local function error_printer(msg, layer)
    print((debug.traceback("Error: " .. tostring(msg), 1+(layer or 1)):gsub("\n[^\n]+$", "")))
end

function love.errhand(msg)
    msg = tostring(msg)

    error_printer(msg, 2)

    if not love.graphics or not love.event or not love.graphics.isCreated() then
        return
    end

    -- Load.
    if love.audio then love.audio.stop() end
    love.graphics.reset()
    love.graphics.setBackgroundColor(89, 157, 220)
    local font = love.graphics.newFont(14)
    love.graphics.setFont(font)

    love.graphics.setColor(255, 255, 255, 255)

    local trace = debug.traceback()

    love.graphics.clear()

    local err = {}

    table.insert(err, "Error\n")
    table.insert(err, msg.."\n\n")

    for l in string.gmatch(trace, "(.-)\n") do
        if not string.match(l, "boot.lua") then
            l = string.gsub(l, "stack traceback:", "Traceback\n")
            table.insert(err, l)
```

```
        end
    end

    local p = table.concat(err, "\n")

    p = string.gsub(p, "\t", "")
    p = string.gsub(p, "%[string \"(.-)\"%]", "%1")

    local function draw()
        love.graphics.clear()
        love.graphics.printf(p, 70, 70, love.graphics.getWidth() - 70)
        love.graphics.present()
    end

    draw()

    local e, a, b, c
    while true do
        e, a, b, c = love.event.wait()

        if e == "quit" then
            return
        end
        if e == "keypressed" and a == "escape" then
            return
        end

        draw()

    end

end
```

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.errorhandler

love.errorhandler · revision 28570 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - The default function used if you don't supply your own.
  - The default function used if you don't supply your own.
- See Also
- Other Languages

Available since LÖVE 11.0

It has been renamed from love.errhand.

The error handler, used to display error messages.


Note that if there's an error in the error handler function, LÖVE will exit immediately!

Function

Synopsis

```
mainLoop = love.errorhandler( msg )
```

Arguments

string msg

The error message.

Returns

function mainLoop

Function which handles one frame, including events and rendering, when called. If this is nil then LÖVE exits immediately.

Examples

The default function used if you don't supply your own.

Available since LÖVE 11.4

This variant is not supported in earlier versions.

```
local utf8 = require("utf8")

local function error_printer(msg, layer)
    print((debug.traceback("Error: " .. tostring(msg), 1+(layer or 1)):gsub("\n[^\n]+$", "")))
end

function love.errorhandler(msg)
    msg = tostring(msg)

    error_printer(msg, 2)
```

```lua
if not love.window or not love.graphics or not love.event then
    return
end

if not love.graphics.isCreated() or not love.window.isOpen() then
    local success, status = pcall(love.window.setMode, 800, 600)
    if not success or not status then
        return
    end
end

-- Reset state.
if love.mouse then
    love.mouse.setVisible(true)
    love.mouse.setGrabbed(false)
    love.mouse.setRelativeMode(false)
    if love.mouse.isCursorSupported() then
        love.mouse.setCursor()
    end
end
if love.joystick then
    -- Stop all joystick vibrations.
    for i,v in ipairs(love.joystick.getJoysticks()) do
        v:setVibration()
    end
end
if love.audio then love.audio.stop() end

love.graphics.reset()
local font = love.graphics.setNewFont(14)

love.graphics.setColor(1, 1, 1)

local trace = debug.traceback()

love.graphics.origin()

local sanitizedmsg = {}
for char in msg:gmatch(utf8.charpattern) do
    table.insert(sanitizedmsg, char)
end
sanitizedmsg = table.concat(sanitizedmsg)

local err = {}

table.insert(err, "Error\n")
table.insert(err, sanitizedmsg)

if #sanitizedmsg ~= #msg then
    table.insert(err, "Invalid UTF-8 string in error message.")
end

table.insert(err, "\n")

for l in trace:gmatch("(.-)\n") do
    if not l:match("boot.lua") then
        l = l:gsub("stack traceback:", "Traceback\n")
        table.insert(err, l)
    end
end
```

```
    local p = table.concat(err, "\n")

    p = p:gsub("\t", "")
    p = p:gsub("%[string \"(.-)\"%]", "%1")

    local function draw()
        if not love.graphics.isActive() then return end
        local pos = 70
        love.graphics.clear(89/255, 157/255, 220/255)
        love.graphics.printf(p, pos, pos, love.graphics.getWidth() - pos)
        love.graphics.present()
    end

    local fullErrorText = p
    local function copyToClipboard()
        if not love.system then return end
        love.system.setClipboardText(fullErrorText)
        p = p .. "\nCopied to clipboard!"
    end

    if love.system then
        p = p .. "\n\nPress Ctrl+C or tap to copy this error"
    end

    return function()
        love.event.pump()

        for e, a, b, c in love.event.poll() do
            if e == "quit" then
                return 1
            elseif e == "keypressed" and a == "escape" then
                return 1
            elseif e == "keypressed" and a == "c" and love.keyboard.isDown("lctrl", "rctrl") then
                copyToClipboard()
            elseif e == "touchpressed" then
                local name = love.window.getTitle()
                if #name == 0 or name == "Untitled" then name = "Game" end
                local buttons = {"OK", "Cancel"}
                if love.system then
                    buttons[3] = "Copy to clipboard"
                end
                local pressed = love.window.showMessageBox("Quit "..name.."?", "", buttons)
                if pressed == 1 then
                    return 1
                elseif pressed == 3 then
                    copyToClipboard()
                end
            end
        end

        draw()

        if love.timer then
            love.timer.sleep(0.1)
        end
    end

end
```

Available since LÖVE 11.0 and removed in LÖVE 11.4

This variant is not supported in earlier or later versions.

The default function used if you don't supply your own.

```
local utf8 = require("utf8")

local function error_printer(msg, layer)
    print((debug.traceback("Error: " .. tostring(msg), 1+(layer or 1)):gsub("\n[^\n]+$", "")))
end

function love.errorhandler(msg)
    msg = tostring(msg)

    error_printer(msg, 2)

    if not love.window or not love.graphics or not love.event then
        return
    end

    if not love.graphics.isCreated() or not love.window.isOpen() then
        local success, status = pcall(love.window.setMode, 800, 600)
        if not success or not status then
            return
        end
    end

    -- Reset state.
    if love.mouse then
        love.mouse.setVisible(true)
        love.mouse.setGrabbed(false)
        love.mouse.setRelativeMode(false)
        if love.mouse.isCursorSupported() then
            love.mouse.setCursor()
        end
    end
    if love.joystick then
        -- Stop all joystick vibrations.
        for i,v in ipairs(love.joystick.getJoysticks()) do
            v:setVibration()
        end
    end
    if love.audio then love.audio.stop() end

    love.graphics.reset()
    local font = love.graphics.setNewFont(14)

    love.graphics.setColor(1, 1, 1, 1)

    local trace = debug.traceback()

    love.graphics.origin()

    local sanitizedmsg = {}
    for char in msg:gmatch(utf8.charpattern) do
        table.insert(sanitizedmsg, char)
    end
    sanitizedmsg = table.concat(sanitizedmsg)

    local err = {}

    table.insert(err, "Error\n")
    table.insert(err, sanitizedmsg)
```

```
if #sanitizedmsg ~= #msg then
    table.insert(err, "Invalid UTF-8 string in error message.")
end

table.insert(err, "\n")

for l in trace:gmatch("(.-)\n") do
    if not l:match("boot.lua") then
        l = l:gsub("stack traceback:", "Traceback\n")
        table.insert(err, l)
    end
end

local p = table.concat(err, "\n")

p = p:gsub("\t", "")
p = p:gsub("%[string \"(.-)\"%]", "%1")

local function draw()
    local pos = 70
    love.graphics.clear(89/255, 157/255, 220/255)
    love.graphics.printf(p, pos, pos, love.graphics.getWidth() - pos)
    love.graphics.present()
end

local fullErrorText = p
local function copyToClipboard()
    if not love.system then return end
    love.system.setClipboardText(fullErrorText)
    p = p .. "\nCopied to clipboard!"
    draw()
end

if love.system then
    p = p .. "\n\nPress Ctrl+C or tap to copy this error"
end

return function()
    love.event.pump()

    for e, a, b, c in love.event.poll() do
        if e == "quit" then
            return 1
        elseif e == "keypressed" and a == "escape" then
            return 1
        elseif e == "keypressed" and a == "c" and love.keyboard.isDown("lctrl", "rctrl") then
            copyToClipboard()
        elseif e == "touchpressed" then
            local name = love.window.getTitle()
            if #name == 0 or name == "Untitled" then name = "Game" end
            local buttons = {"OK", "Cancel"}
            if love.system then
                buttons[3] = "Copy to clipboard"
            end
            local pressed = love.window.showMessageBox("Quit "..name.."?", "", buttons)
            if pressed == 1 then
                return 1
            elseif pressed == 3 then
                copyToClipboard()
            end
        end
```

```
        end

        draw()

        if love.timer then
            love.timer.sleep(0.1)
        end
    end

end
```

See Also

- love
- love.run

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.event

love.event · revision 22253 · last edited 2025-09-28

Available since LÖVE 0.6.0

This module is not supported in earlier versions.

Manages events, like keypresses.

It is possible to define new events by appending the table `love.handlers`. Such functions can be invoked as usual, via love.event.push using the table index as an argument.

Functions

love.event.clear	Clears the event queue.	Added since 0.7.2
love.event.poll	Returns an iterator for messages in the event queue.	Added since 0.6.0
love.event.pump	Pump events into the event queue.	Added since 0.6.0
love.event.push	Adds an event to the event queue.	Added since 0.6.0
love.event.quit	Exits or restarts the LÖVE program.	Added since 0.8.0
love.event.wait	Like love.event.poll(), but blocks until there is an event in the queue.	Added since 0.6.0

Enums

Event	Arguments to love.event.push() and the like.	Added since 0.6.0

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.event.clear

love.event.clear · revision 22256 · last edited 2026-03-23

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.7.2

This function is not supported in earlier versions.

Clears the event queue.

Function

Synopsis

```
love.event.clear()
```

Arguments

None.

Returns

Nothing.

See Also

- love.event

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.event.poll

love.event.poll · revision 23997 · last edited 2026-03-23

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Checking for events in 0.10.0
  - Checking for events in 0.8.0
  - Checking for events in 0.7.2
- See Also
- Other Languages

Available since LÖVE 0.6.0

This function is not supported in earlier versions.

Returns an iterator for messages in the event queue.

Function

Synopsis

```
i = love.event.poll( )
```

Arguments

None.

Returns

function i

Iterator function usable in a for loop.

Examples

Checking for events in 0.10.0

```
for n, a, b, c, d, e, f in love.event.poll() do
    if n == "quit" then
        -- Quit!
    end
end
```

Checking for events in 0.8.0

```
for e, a, b, c, d in love.event.poll() do
    if e == "quit" then
        -- Quit!
    end
end
```

Checking for events in 0.7.2

```
for e, a, b, c in love.event.poll() do
    if e == "q" then
        -- Quit!
```

```
    end
end
```

See Also

- love.event
- love.event.wait

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.event.pump

love.event.pump · revision 29232 · last edited 2024-05-26

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.6.0

This function is not supported in earlier versions.

Pump events into the event queue.

This is a low-level function, and is usually not called by the user, but by `love.run`.

Note that this does need to be called for any OS to think your program is still running, and if you want to handle OS-generated events at all (think callbacks).



love.event.pump can only be called from the main thread, but afterwards, the rest of love.event can be used from any other thread.

Function

Synopsis

```
love.event.pump( )
```

Arguments

None.

Returns

Nothing.

See Also

- love.event.poll
- love.event.wait
- love.event

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.event.push

love.event.push · revision 25462 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 0.6.0

This function is not supported in earlier versions.

Adds an event to the event queue.

See Variant for the list of supported types for the arguments.

From 0.10.0 onwards, you may pass an arbitrary amount of arguments with this function, though the default callbacks don't ever use more than six.

Function

Synopsis

```
love.event.push( n, a, b, c, d, e, f, ... )
```

Arguments

`Event n`
: The name of the event.

`Variant a (nil)`
: First event argument.

`Variant b (nil)`
: Second event argument.

`Variant c (nil)`
: Third event argument.

Available since LÖVE 0.8.0

`Variant d (nil)`
: Fourth event argument.

Available since LÖVE 0.10.0

`Variant e (nil)`
: Fifth event argument.

`Variant f (nil)`
: Sixth event argument.

`Variant ... (nil)`

Further event arguments may follow.

Returns

Nothing.

Examples

Available since LÖVE 0.8.0

```
function love.keypressed(k)
    if k == 'escape' then
        love.event.push('quit') -- Quit the game.
    end
end
```

Available since LÖVE 0.6.0 and removed in LÖVE 0.7.2

```
function love.keypressed(k)
    if k == 'escape' then
        love.event.push('q') -- Quit the game.
    end
end
```

See Also

- love.event

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.event.quit

love.event.quit · revision 28587 · last edited 2026-04-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
- Synopsis
- Arguments
- Returns
- Example
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Adds the quit event to the queue.

The quit event is a signal for the event handler to close LÖVE. It's possible to abort the exit process with the love.quit callback.

Equivalent to `love.event.push("quit", exitstatus)`

Function

Synopsis

```
love.event.quit( exitstatus )
```

Arguments

Available since LÖVE 0.10.0

`number exitstatus (0)`
The program exit status to use when closing the application.

Returns

Nothing.

Notes

On iOS, programmatically exiting the app (this includes call to `os.exit`) is not recommended (https://developer.apple.com/library/archive/qa/qa1561/_index.html) and may result your app being rejected from App Store. Thus, calling this variant will actually perform restart (see below).

Function

Available since LÖVE 0.10.2

This variant is not supported in earlier versions.

Restarts the game without relaunching the executable. This cleanly shuts down the main Lua state instance and creates a brand new one.

Synopsis

```
love.event.quit( "restart" )
```

Arguments

string "restart"

Tells the default love.run to exit and restart the game without relaunching the executable.

Returns

Nothing.

Example

```
function love.keypressed(k)
   if k == 'escape' then
      love.event.quit()
   end
end
```

See Also

- love.event
- love.quit

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.event.restart

love.event.restart · revision 28287 · last edited 2025-12-01

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Restarts the game without relaunching the executable, by adding a quit event with a "restart" parameter to the queue. This cleanly shuts down the main Lua state instance and creates a brand new one.

Equivalent to `love.event.push("quit", "restart", restartarg)`.



If LÖVE is programmatically restarted, all active Lua threads must stop their work and the main thread must wait for them to finish, otherwise LOVE may fail to start again.

Function

Synopsis

```
love.event.restart( restartarg )
```

Arguments

Variant restartarg (nil)

A value which will appear in the love.restart table field after restarting. Can be a table containing multiple Lua values.

Returns

Nothing.

Example

```
local restartcount = tonumber(love.restart) or 0

function love.keypressed(k)
   if k == "r" then
      love.event.restart(restartcount + 1)
   end
end

function love.draw()
```

```
    love.graphics.print("Restarted " .. restartcount .. " times.", 8, 8)
end
```

See Also

- love.event
- love.restart
- love.event.quit

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.event.wait

love.event.wait · revision 25527 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 0.6.0

This function is not supported in earlier versions.

Like `love.event.poll()`, but blocks until there is an event in the queue.

Function

Synopsis

```
n, a, b, c, d, e, f, ... = love.event.wait( )
```

Arguments

None.

Returns

`Event n (nil)`
: The name of event, or nil if the event is unrecognized.

`Variant a`
: First event argument.

`Variant b`
: Second event argument.

`Variant c`
: Third event argument.

Available since LÖVE 0.8.0

`Variant d`
: Fourth event argument.

Available since LÖVE 0.10.0

`Variant e`
: Fifth event argument.

`Variant f`
: Sixth event argument.

`Variant ...`

Further event arguments may follow.

Examples

An example that replace `love.event.poll()` with this function.

```
function love.run()
    if love.load then love.load(love.arg.parseGameArguments(arg), arg) end

    -- Main loop time.
    return function()
        -- Process events.
        if love.event then
            local name, a,b,c,d,e,f = love.event.wait()
            if name then
                if name == "quit" then
                    if not love.quit or not love.quit() then
                        return a or 0
                    end
                end
                love.handlers[name](a,b,c,d,e,f)
            end
        end

        -- Call update and draw
        if love.update then love.update(0) end

        if love.graphics and love.graphics.isActive() then
            love.graphics.origin()
            love.graphics.clear(love.graphics.getBackgroundColor())

            if love.draw then love.draw() end

            love.graphics.present()
        end
    end
end
```

See Also

- love.event
- love.event.poll

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.exposed

love.exposed · revision 30608 · last edited 2026-02-13

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This callback is not supported in earlier versions.

Callback function triggered when the window was occluded, is uncovered.

This callback means that the window is no longer completely covered. This is not the same as a window being visible or not.

Function

Synopsis

```
love.exposed( )
```

Arguments

None.

Returns

Nothing.

See Also

- love
- love.window.isOccluded
- love.occluded
- love.visible

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filedropped

love.filedropped · revision 28150 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Callback function triggered when a file is dragged and dropped onto the window.

Function

Synopsis

```
love.filedropped( file )
```

Arguments

`DroppedFile file`

The unopened File object representing the file that was dropped.

Returns

Nothing.

Examples

Read and print the file, drag-and-dropped to love window using (File):read and (File):getFilename.

```
function love.filedropped(file)
    file:open("r")
    local data = file:read()
    print("Content of " .. file:getFilename() .. ' is')
    print(data)
    print("End of file")
end
```

If dropped file is a .png, use it to create a löve image using love.image and love.graphics

```
function love.filedropped(file)
    filename = file:getFilename()
    ext = filename:match("%.%w+$")

    if ext == ".png" then
        file:open("r")
        fileData = file:read("data")
        img = love.image.newImageData(fileData)
        img = love.graphics.newImage(img)
```

```
    end
end
```

Notes

File:open must be called on the file before reading from or writing to it. File:getFilename will return the full platform-dependent path to the file.

See Also

- love
- love.directorydropped

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem

love.filesystem · revision 29374 · last edited 2026-08-05

Provides an interface to the user's filesystem.

This module provides access to files in specific places:

- The root folder of the .love archive (or source directory)
- The root folder of the game's *save directory*.
- The folder *containing* the game's .love archive (or source directory), but only if specific conditions are met.

Each game is granted a single directory on the system where files can be saved through love.filesystem. This is the **only directory** where love.filesystem can write files. These directories will typically be found in something like:

OS	Path	Alternative	Notes
Windows	`C:\Users\user\AppData\Roaming\LOVE`	`%appdata%\LOVE\`	When fused, save directory will be created directly inside `AppData`, rather than as a sub-directory of `LOVE`.
macOS	`/Users/user/Library/Application Support/LOVE/`	-	-
Linux	`$XDG_DATA_HOME/love/`	`~/.local/share/love/`	-
Android	`/data/data/org.love2d.android/files/save/`	`/sdcard/Android/data/org.love2d.android/files/save/`	The alternative path is used when activating t.externalstorage config option. Neither path is accessible through device directly due to Android restrictions.

Files that are opened for write or append will always be created in the save directory. The same goes for other operations that involve writing to the filesystem, like createDirectory.

Files that are opened for read will be looked for in the save directory, and then in the .love archive (in that order). So if a file with a certain filename (and path) exist in both the .love archive and the save folder, the one in the save directory takes precedence.

Note: **All** paths are relative to the .love archive and save directory. (except for the get*Directory() calls)

It is recommended to set your game's identity first in your conf.lua. You can set it with love.filesystem.setIdentity as well.

Types

DroppedFile	Represents a file dropped from the window.	 Added since 0.10.0
File	Represents a file on the filesystem.	

FileData	Data representing the contents of a file.

Functions

love.filesystem.append	Append data to an existing file.	Added since 0.9.0	
love.filesystem.areSymlinksEnabled	Gets whether love.filesystem follows symbolic links.	Added since 0.9.2	
love.filesystem.createDirectory	Creates a directory.	Added since 0.9.0	
love.filesystem.enumerate	Returns all the files and subdirectories in the directory.	Added since 0.3.0	Removed in 0.9.0
love.filesystem.exists	Check whether a file or directory exists.		Deprecated in 11.0
love.filesystem.getAppdataDirectory	Returns the application data directory (could be the same as getUserDirectory)		
love.filesystem.getCRequirePath	Gets the filesystem paths that will be searched for c libraries when require (http://www.lua.org/manual/5.1/manual.html#pdf-require) is called.	Added since 11.0	
love.filesystem.getDirectoryItems	Returns all the files and subdirectories in the directory.	Added since 0.9.0	
love.filesystem.getIdentity	Gets the write directory name for your game.	Added since 0.9.0	
love.filesystem.getInfo	Gets information about the specified file or directory.	Added since 11.0	
love.filesystem.getLastModified	Gets the last modification time of a file.	Added since 0.7.1	Deprecated in 11.0
love.filesystem.getRealDirectory	Gets the absolute path of the directory containing a filepath.	Added since 0.9.2	
love.filesystem.getRequirePath	Gets the filesystem paths that will be searched when require (http://www.lua.org/manual/5.1/manual.html#pdf-require) is called.	Added since 0.10.0	
love.filesystem.getSaveDirectory	Gets the full path to the designated save directory.	Added since 0.5.0	
love.filesystem.getSize	Gets the size in bytes of a file.	Added since 0.9.0	Deprecated in 11.0
love.filesystem.getSource	Returns the full path to the .love file or directory.	Added since 0.9.0	
love.filesystem.getSourceBaseDirectory	Returns the full path to the directory containing the .love file.	Added since 0.9.0	
love.filesystem.getUserDirectory	Returns the path of the user's directory		

love.filesystem.getWorkingDirectory	Gets the current working directory.	Added since 0.5.0	
love.filesystem.init	Initializes love.filesystem. This is called internally, so should not be used explicitly.		
love.filesystem.isDirectory	Check whether something is a directory.		Deprecated in 11.0
love.filesystem.isFile	Check whether something is a file.		Deprecated in 11.0
love.filesystem.isFused	Gets whether the game is in fused mode or not.	Added since 0.9.0	
love.filesystem.isSymlink	Gets whether a filepath is actually a symbolic link.	Added since 0.9.2	Deprecated in 11.0
love.filesystem.lines	Iterate over the lines in a file.	Added since 0.5.0	
love.filesystem.load	Loads a Lua file (but does not run it).	Added since 0.5.0	
love.filesystem.mkdir	Creates a directory.		Removed in 0.9.0
love.filesystem.mount	Mounts a zip file or folder in the game's save directory for reading.	Added since 0.9.0	
love.filesystem.newFile	Creates a new File object.		
love.filesystem.newFileData	Creates a new FileData object from a file on disk, or from a string in memory.	Added since 0.7.0	
love.filesystem.read	Read the contents of a file.		
love.filesystem.remove	Removes a file (or directory).		
love.filesystem.setCRequirePath	Sets the filesystem paths that will be searched for c libraries when require (http://www.lua.org/manual/5.1/manual.html#pdf-require) is called.	Added since 11.0	
love.filesystem.setIdentity	Sets the write directory for your game.		
love.filesystem.setRequirePath	Sets the filesystem paths that will be searched when require (http://www.lua.org/manual/5.1/manual.html#pdf-require) is called.	Added since 0.10.0	
love.filesystem.setSource	Sets the source of the game, where the code is present. Used internally.		
love.filesystem.setSymlinksEnabled	Sets whether love.filesystem follows symbolic links.	Added since 0.9.2	
love.filesystem.unmount	Unmounts a zip file or folder previously mounted with love.filesystem.mount.	Added since 0.9.0	
love.filesystem.write	Write data to a file.		

Enums

FileDecoder	How to decode a given FileData.	Added since 0.7.0	Removed in 11.0
FileMode	The different modes you can open a File in.		

FileType	The type of a file.	Added since 11.0

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.append

love.filesystem.append · revision 12373 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Append data to an existing file.

Function

Synopsis

```
success, errormsg = love.filesystem.append( name, data, size )
```

Arguments

string name
: The name (and path) of the file.

string data
: The string data to append to the file.

number size (all)
: How many bytes to write.

Returns

boolean success
: True if the operation was successful, or nil if there was an error.

string errormsg
: The error message on failure.

Function

Synopsis

```
success, errormsg = love.filesystem.append( name, data, size )
```

Arguments

string name
: The name (and path) of the file.

Data data
: The Data object to append to the file.

number size (all)

How many bytes to write.

Returns

`boolean success`
: True if the operation was successful, or nil if there was an error.

`string errormsg`
: The error message on failure.

See Also

- love.filesystem
- love.filesystem.write

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.areSymlinksEnabled

love.filesystem.areSymlinksEnabled · revision 13756 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets whether love.filesystem follows symbolic links.

Function

Synopsis

```
enabled = love.filesystem.areSymlinksEnabled( )
```

Arguments

None.

Returns

`boolean enable`

Whether love.filesystem follows symbolic links.

See Also

- love.filesystem
- love.filesystem.setSymlinksEnabled
- love.filesystem.isSymlink

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.createDirectory

love.filesystem.createDirectory · revision 16116 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from love.filesystem.mkdir.

Recursively creates a directory.

When called with "a/b" it creates both "a" and "a/b", if they don't exist already.

Function

Synopsis

```
success = love.filesystem.createDirectory( name )
```

Arguments

`string name`

The directory to create.

Returns

`boolean success`

True if the directory was created, false if not.

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.enumerate

love.filesystem.enumerate · revision 28085 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Simple Example
  - Recursively find and display all files and folders in a folder and its subfolders.
- See Also
- Other Languages

Available since LÖVE 0.3.0 and removed in LÖVE 0.9.0

It has been renamed to love.filesystem.getDirectoryItems.

Returns a table with the names of files and subdirectories in the specified path. The table is not sorted in any way; the order is undefined.

If the path passed to the function exists in the game and the save directory, it will list the files and directories from both places.

Function

Synopsis

```
files = love.filesystem.enumerate( dir )
```

Arguments

`string dir`
: The directory.

Returns

`table files`
: A sequence with the names of all files and subdirectories as strings.

Examples

Simple Example

```
local dir = ""
--assuming that our path is full of lovely files (it should at least contain main.lua in this case)
local files = love.filesystem.enumerate(dir)
for k, file in ipairs(files) do
    print(k .. ". " .. file) --outputs something like "1. main.lua"
end
```

Recursively find and display all files and folders in a folder and its subfolders.

```
function love.load()
    filesString = recursiveEnumerate("", "")
end

-- This function will return a string filetree of all files
```

```
-- in the folder and files in all subfolders
function recursiveEnumerate(folder, fileTree)
    local lfs = love.filesystem
    local filesTable = lfs.enumerate(folder)
    for i,v in ipairs(filesTable) do
        local file = folder.."/"..v
        if lfs.isFile(file) then
            fileTree = fileTree.."\n"..file
        elseif lfs.isDirectory(file) then
            fileTree = fileTree.."\n"..file.." (DIR)"
            fileTree = recursiveEnumerate(file, fileTree)
        end
    end
    return fileTree
end

function love.draw()
    love.graphics.print(filesString, 0, 0)
end
```

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.exists

love.filesystem.exists · revision 29158 · last edited 2024-04-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Deprecated in LÖVE 11.0

This function is deprecated and is replaced by love.filesystem.getInfo.

Check whether a file or directory exists.

Function

Synopsis

```
exists = love.filesystem.exists( filename )
```

Arguments

`string filename`

The path to a potential file or directory.

Returns

`boolean exists`

True if there is a file or directory with the specified name. False otherwise.

Notes

This function is no longer deprecated in 12.0 as this function is cheaper than love.filesystem.getInfo if the additional info is not needed.

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.getAppdataDirectory

love.filesystem.getAppdataDirectory · revision 5771 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Returns the application data directory (could be the same as getUserDirectory)

Function

Synopsis

```
path = love.filesystem.getAppdataDirectory( )
```

Arguments

None.

Returns

`string path`
: The path of the application data directory

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.getCRequirePath

love.filesystem.getCRequirePath · revision 22646 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the filesystem paths that will be searched for c libraries when require (http://www.lua.org/manual/5.1/manual.html#pdf-require) is called.

The paths string returned by this function is a sequence of path templates separated by semicolons. The argument passed to *require* will be inserted in place of any question mark ("?") character in each template (after the dot characters in the argument passed to *require* are replaced by directory separators.) Additionally, any occurrence of a double question mark ("??") will be replaced by the name passed to require and the default library extension for the platform.

The paths are relative to the game's source and save directories, as well as any paths mounted with love.filesystem.mount.

Function

Synopsis

```
paths = love.filesystem.getCRequirePath( )
```

Arguments

None.

Returns

`string paths`

The paths that the *require* function will check for c libraries in love's filesystem.

Notes

The default paths string is `"??"`, which makes `require("cool")` try to load `cool.dll`, or `cool.so` depending on the platform.

See Also

- love.filesystem
- love.filesystem.setCRequirePath

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

love.filesystem.getDirectoryItems

love.filesystem.getDirectoryItems · revision 27989 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Simple Example
  - Recursively find and display all files and folders in a folder and its subfolders.
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from love.filesystem.enumerate.

Returns a table with the names of files and subdirectories in the specified path. The table is not sorted in any way; the order is undefined.

If the path passed to the function exists in the game and the save directory, it will list the files and directories from both places.

Function

Synopsis

```
files = love.filesystem.getDirectoryItems( dir )
```

Arguments

`string dir`
: The directory.

Returns

`table files`
: A sequence with the names of all files and subdirectories as strings.

Function

Available since LÖVE 0.9.1 and removed in LÖVE 0.10.0

This variant is not supported in earlier or later versions.

Synopsis

```
files = love.filesystem.getDirectoryItems( dir, callback )
```

Arguments

`string dir`
: The directory.

`function callback`

A function which is called for each file and folder in the directory. The filename is passed to the function as an argument.

Returns

table files

A sequence with the names of all files and subdirectories as strings.

Examples

Simple Example

```
local dir = ""
--assuming that our path is full of lovely files (it should at least contain main.lua in this case)
local files = love.filesystem.getDirectoryItems(dir)
for k, file in ipairs(files) do
    print(k .. ". " .. file) --outputs something like "1. main.lua"
end
```

Recursively find and display all files and folders in a folder and its subfolders.

```
function love.load()
    filesString = recursiveEnumerate("", "")
end

-- This function will return a string filetree of all files
-- in the folder and files in all subfolders
function recursiveEnumerate(folder, fileTree)
    local filesTable = love.filesystem.getDirectoryItems(folder)
    for i,v in ipairs(filesTable) do
        local file = folder.."/"..v
        local info = love.filesystem.getInfo(file)
        if info then
            if info.type == "file" then
                fileTree = fileTree.."\n"..file
            elseif info.type == "directory" then
                fileTree = fileTree.."\n"..file.." (DIR)"
                fileTree = recursiveEnumerate(file, fileTree)
            end
        end
    end
    return fileTree
end

function love.draw()
```

```
    love.graphics.print(filesString, 0, 0)
end
```

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.getFullCommonPath

love.filesystem.getFullCommonPath · revision 30599 · last edited 2026-02-04

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Retrieves a path to a directory in the active operating system without requiring exact knowledge of its whereabouts.

Function

Synopsis

```
outpath = love.filesystem.getFullCommonPath(commonpath)
if outpath ~= nil then
    -- outpath successfully found
end
```

Arguments

`CommonPath commonpath`

The type of common directory to be sought after.

Returns

`string outpath`

The resultant path to the common directory, if found. Otherwise, returns nil.

Notes

Desktop does not exist on Android devices, and attempting to retrieve it will fail.

See Also

- love.filesystem
- love.filesystem.mountCommonPath
- love.filesystem.unmountCommonPath

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.getIdentity

love.filesystem.getIdentity · revision 26121 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the write directory name for your game. Note that this only returns the name of the folder to store your files in, not the full path.

Function

Synopsis

```
name = love.filesystem.getIdentity( )
```

Arguments

None.

Returns

`string name`

The identity that is used as write directory.

See Also

- love.filesystem
- love.filesystem.setIdentity

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.getInfo

love.filesystem.getInfo · revision 21370 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function replaces love.filesystem.exists, isFile, isDirectory, isSymlink, getLastModified, and getSize.

Gets information about the specified file or directory.

Function

Synopsis

```
info = love.filesystem.getInfo( path, filtertype )
```

Arguments

`string path`
: The file or directory path to check.

`FileType filtertype (nil)`
: If supplied, this parameter causes getInfo to only return the info table if the item at the given path matches the specified file type.

Returns

`table info (nil)`
: A table containing information about the specified path, or nil if nothing exists at the path. The table contains the following fields:

  `FileType type`
  : The type of the object at the path (file, directory, symlink, etc.)

  `number size (nil)`
  : The size in bytes of the file, or nil if it can't be determined.

  `number modtime (nil)`

The file's last modification time in seconds since the unix epoch, or nil if it can't be determined.

Function

This variant accepts an existing table to fill in, instead of creating a new one.

Synopsis

```
info = love.filesystem.getInfo( path, info )
```

Arguments

string path
: The file or directory path to check.

table info
: A table which will be filled in with info about the specified path.

Returns

table info (nil)
: The table given as an argument, or nil if nothing exists at the path. The table will be filled in with the following fields:

  FileType type
  : The type of the object at the path (file, directory, symlink, etc.)

  number size (nil)
  : The size in bytes of the file, or nil if it can't be determined.

  number modtime (nil)
  : The file's last modification time in seconds since the unix epoch, or nil if it can't be determined.

Function

This variant only returns info if the item at the given path is the same file type as specified in the filtertype argument, and accepts an existing table to fill in, instead of creating a new one.

Synopsis

```
info = love.filesystem.getInfo( path, filtertype, info )
```

Arguments

string path
: The file or directory path to check.

FileType filtertype
: Causes getInfo to only return the info table if the item at the given path matches the specified file type.

table info
: A table which will be filled in with info about the specified path.

Returns

table info (nil)

The table given as an argument, or nil if nothing exists at the path. The table will be filled in with the following fields:

`FileType type`
: The type of the object at the path (file, directory, symlink, etc.)

`number size (nil)`
: The size in bytes of the file, or nil if it can't be determined.

`number modtime (nil)`
: The file's last modification time in seconds since the unix epoch, or nil if it can't be determined.

See Also

- love.filesystem
- love.filesystem.setSymlinksEnabled
- love.filesystem.areSymlinksEnabled

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.getLastModified

love.filesystem.getLastModified · revision 21335 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Getting the time of a file and printing it as a date.
- See Also
- Other Languages

Deprecated in LÖVE 11.0

This function is deprecated and is replaced by love.filesystem.getInfo.

Gets the last modification time of a file.

Function

Synopsis

```
modtime, errormsg = love.filesystem.getLastModified( filename )
```

Arguments

string filename
: The path and name to a file.

Returns

number modtime
: The last modification time in seconds since the unix epoch or nil on failure.

string errormsg
: The error message on failure.

Examples

Getting the time of a file and printing it as a date.

```
modtime, errormsg = love.filesystem.getLastModified("file.dat")

if modtime then
    print(os.date("%c", modtime)) -- "02/15/2011 12:32:11" (Depends on the system and locale)
end
```

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.getRealDirectory

love.filesystem.getRealDirectory · revision 14904 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets the platform-specific absolute path of the directory containing a filepath.

This can be used to determine whether a file is inside the save directory or the game's source .love.

Function

Synopsis

```
realdir = love.filesystem.getRealDirectory( filepath )
```

Arguments

`string filepath`

The filepath to get the directory of.

Returns

`string realdir`

The platform-specific full path of the directory containing the filepath.

Notes

This function returns the directory containing the given *file path*, rather than file. For example, if the file `screenshot1.png` exists in a directory called `screenshots` in the game's save directory, `love.filesystem.getRealDirectory("screenshots/screenshot1.png")` will return the same value as love.filesystem.getSaveDirectory.

Examples

```
-- Get all files in the "levels" folder.
-- There might be a "levels" folder in both the save directory and the game's source,
-- in which case this will get all files in both.
local filepaths = love.filesystem.getDirectoryItems("levels")

for i, filename in ipairs(filepaths) do
    -- For each filename, check whether it's in the save directory or not.
    local path = "levels/"..filename
    if love.filesystem.getRealDirectory(path) == love.filesystem.getSaveDirectory() then
        -- This file is in the save directory.
```

```
    end
end
```

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.getRequirePath

love.filesystem.getRequirePath · revision 15611 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the filesystem paths that will be searched when require (http://w w w.lua.org/manual/5.1/manual.html#pdf-require) is called.

The paths string returned by this function is a sequence of path templates separated by semicolons. The argument passed to *require* will be inserted in place of any question mark ("?") character in each template (after the dot characters in the argument passed to *require* are replaced by directory separators.)

The paths are relative to the game's source and save directories, as well as any paths mounted with love.filesystem.mount.

Function

Synopsis

```
paths = love.filesystem.getRequirePath( )
```

Arguments

None.

Returns

`string paths`

The paths that the *require* function will check in love's filesystem.

Notes

The default paths string is `"?.lua;?/init.lua"`, which makes `require("cool")` try to load `cool.lua` and then try `cool/init.lua` if cool.lua doesn't exist.

See Also

- love.filesystem
- love.filesystem.setRequirePath

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.getSaveDirectory

love.filesystem.getSaveDirectory · revision 22001 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.5.0

This function is not supported in earlier versions.

Gets the full path to the designated save directory. This can be useful if you want to use the standard io library (or something else) to read or write in the save directory.

Function

Synopsis

```
dir = love.filesystem.getSaveDirectory( )
```

Arguments

None.

Returns

`string dir`

The absolute path to the save directory.

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.getSize

love.filesystem.getSize · revision 21334 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Deprecated in LÖVE 11.0

This function is deprecated and is replaced by love.filesystem.getInfo.

Gets the size in bytes of a file.

Function

Synopsis

```
size, errormsg = love.filesystem.getSize( filename )
```

Arguments

`string filename`
: The path and name to a file.

Returns

`number size`
: The size in bytes of the file, or nil on failure.

`string errormsg (nil)`
: The error message on failure.

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.getSource

love.filesystem.getSource · revision 20486 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Returns the full path to the the .love file or directory. If the game is fused to the LÖVE executable, then the executable is returned.

Function

Synopsis

```
path = love.filesystem.getSource( )
```

Arguments

None.

Returns

`string path`

The full platform-dependent path of the .love file or directory.

See Also

- love.filesystem
- love.filesystem.isFused
- love.filesystem.getSourceBaseDirectory

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.getSourceBaseDirectory

love.filesystem.getSourceBaseDirectory · revision 17491 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - read files in the same folder as the game's .exe file
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Returns the full path to the directory containing the .love file. If the game is fused to the LÖVE executable, then the directory containing the executable is returned.

If love.filesystem.isFused is true, the path returned by this function can be passed to love.filesystem.mount, which will make the directory containing the main game (e.g. `C:\Program Files\coolgame\`) readable by love.filesystem.

Function

Synopsis

```
path = love.filesystem.getSourceBaseDirectory( )
```

Arguments

None.

Returns

`string path`

The full platform-dependent path of the directory containing the .love file.

Examples

read files in the same folder as the game's .exe file

```
if love.filesystem.isFused() then
    local dir = love.filesystem.getSourceBaseDirectory()
    local success = love.filesystem.mount(dir, "coolgame")

    if success then
        -- If the game is fused and it's located in C:\Program Files\mycoolgame\,
        -- then we can now load files from that path.
        coolimage = love.graphics.newImage("coolgame/coolimage.png")
    end
end

function love.draw()
    if coolimage then
        love.graphics.draw(coolimage, 0, 0)
```

```
    end
end
```

See Also

- love.filesystem
- love.filesystem.mount
- love.filesystem.isFused
- love.filesystem.getSource

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.getUserDirectory

love.filesystem.getUserDirectory · revision 5774 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Returns the path of the user's directory

Function

Synopsis

```
path = love.filesystem.getUserDirectory( )
```

Arguments

None.

Returns

`string path`
: The path of the user's directory

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.getWorkingDirectory

love.filesystem.getWorkingDirectory · revision 22000 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.5.0

This function is not supported in earlier versions.

Gets the current working directory.

Function

Synopsis

```
cwd = love.filesystem.getWorkingDirectory( )
```

Arguments

None.

Returns

`string cwd`

The current working directory.

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.init

love.filesystem.init · revision 31668 · last edited 2026-07-23

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Initializes love.filesystem. This is called internally, so should not be used explicitly.

Function

Synopsis

```
love.filesystem.init( appname )
```

Arguments

`string appname`
: The name of the application binary, typically `love`.

Returns

Nothing.

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.isDirectory

love.filesystem.isDirectory · revision 21338 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Deprecated in LÖVE 11.0

This function is deprecated and is replaced by love.filesystem.getInfo.

Check whether something is a directory.

Function

Synopsis

```
isDir = love.filesystem.isDirectory( filename )
```

Arguments

`string filename`

The path to a potential directory.

Returns

`boolean isDir`

True if there is a directory with the specified name. False otherwise.

See Also

- love.filesystem
- love.filesystem.isFile

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.isFile

love.filesystem.isFile · revision 21333 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Deprecated in LÖVE 11.0

This function is deprecated and is replaced by love.filesystem.getInfo.

Check whether something is a file.

Function

Synopsis

```
isFile = love.filesystem.isFile( filename )
```

Arguments

`string filename`

The path to a potential file.

Returns

`boolean isFile`

True if there is a file with the specified name. False otherwise.

See Also

- love.filesystem
- love.filesystem.isDirectory
- love.filesystem.exists

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.isFused

love.filesystem.isFused · revision 11077 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets whether the game is in fused mode or not.

If a game is in fused mode, its save directory will be directly in the Appdata directory instead of Appdata/LOVE/. The game will also be able to load C Lua dynamic libraries which are located in the save directory.

A game is in fused mode if the source .love has been fused to the executable (see Game Distribution), or if "--fused" has been given as a command-line argument when starting the game.

Function

Synopsis

```
fused = love.filesystem.isFused( )
```

Arguments

None

Returns

`boolean fused`

True if the game is in fused mode, false otherwise.

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.isSymlink

love.filesystem.isSymlink · revision 21336 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Deprecated in LÖVE 11.0

This function is deprecated and is replaced by love.filesystem.getInfo.

Gets whether a filepath is actually a symbolic link.

If symbolic links are not enabled (via love.filesystem.setSymlinksEnabled), this function will always return false.

Function

Synopsis

```
symlink = love.filesystem.isSymlink( path )
```

Arguments

`string path`
: The file or directory path to check.

Returns

`boolean symlink`
: True if the path is a symbolic link, false otherwise.

See Also

- love.filesystem
- love.filesystem.setSymlinksEnabled
- love.filesystem.areSymlinksEnabled

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.lines

love.filesystem.lines · revision 28083 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Load highscore list
  - Load comma separated values
- See Also
- Other Languages

Available since LÖVE 0.5.0

This function is not supported in earlier versions.

Iterate over the lines in a file.

Function

Synopsis

```
iterator = love.filesystem.lines( name )
```

Arguments

`string name`

The name (and path) of the file

Returns

`function iterator`

A function that iterates over all the lines in the file, returning the line with newlines stripped (if the line ends with `\r\n`, both are stripped independently of the OS)

Examples

Load highscore list

```
local highscores = {}
for line in love.filesystem.lines("highscores.txt") do
  table.insert(highscores, tonumber(line))
end
```

Load comma separated values

```
local function splitCsvLine(line)
    local values = {}

    for value in line:gmatch("[^,]+") do -- Note: We won't match empty values.
        -- Convert the value string to other Lua types in a "smart" way.
        if     tonumber(value)  then  table.insert(values, tonumber(value)) -- Number.
        elseif value == "true"  then  table.insert(values, true)            -- Boolean.
        elseif value == "false" then  table.insert(values, false)           -- Boolean.
        else                          table.insert(values, value)           -- String.
        end
```

```
    end

    return values
end

local function loadCsvFile(filename)
    local csv = {}
    for line in love.filesystem.lines(filename) do
        table.insert(csv, splitCsvLine(line))
    end
    return csv
end

--[[ cool.csv:
Foo,Bar
true,false,11.8
]]
local csv = loadCsvFile("cool.csv")
for row, values in ipairs(csv) do
    print("row="..row.." count="..#values.." values=", unpack(values))
end
```

See Also

- love.filesystem.read
- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.load

love.filesystem.load · revision 29154 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 0.5.0

This function is not supported in earlier versions.

Loads a Lua file (but does not run it).

This is equivalent to loadfile (https://www.lua.org/manual/5.1/manual.html#pdf-loadfile) except it operates on LÖVE filesystem paths.

Function

Synopsis

```
chunk, errormsg = love.filesystem.load( name, mode )
```

Arguments

`string name`

The name (and path) of the file.

Available since LÖVE 12.0

`LoadMode mode ("bt")`

Controls to only allow precompiled chunk, plain text, or both.

Returns

`function chunk`

The loaded chunk.

`string errormsg (nil)`

The error message if file could not be opened.

Examples

It is important to note that love.filesystem.load does **not** invoke the code, it just creates a function (a 'chunk') that will contain the contents of the file inside it. In order to execute the chunk, you have to put () behind it.

Also, it is worth noting that loaded files can return values. For example, the following file:

```
return 1+1, "foo"
```

Will return 2 and "foo", when called like this:

```
local chunk = love.filesystem.load("someFolder/myFile.lua") -- load the chunk
local result1, result2 = chunk() -- execute the chunk
```

```
print("Results: "..tostring(result1)..", "..tostring(result2)) -- prints 'Results: 2, foo'
```

This raises an error if there's a syntax error in the loaded file. If you want your game to continue if the file isn't valid (for example if you expect it to be written by users), you can protect the loading and calling of the file with **pcall** (https://www.lua.org/manual/5.1/manual.html#pdf-pcall):

```
-- success, valueOrErrormsg = runFile( name )
local function runFile(name)
    local ok, chunk, err = pcall(love.filesystem.load, name) -- load the chunk safely
    if not ok    then  return false, "Failed loading code: "..chunk  end
    if not chunk then  return false, "Failed reading file: "..err    end

    local ok, value = pcall(chunk) -- execute the chunk safely
    if not ok then  return false, "Failed calling chunk: "..tostring(value)  end

    return true, value -- success!
end

print(runFile("goodFile.lua"))
print(runFile("fileWithRuntimeError.lua"))
print(runFile("fileWithSyntaxError.lua"))
print(runFile("nonexistentFile.lua"))
```

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.mkdir

love.filesystem.mkdir · revision 11827 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

It has been renamed to love.filesystem.createDirectory.

Recursively creates a directory.

When called with "a/b" it creates both "a" and "a/b", if they don't exist already.

Function

Synopsis

```
ok = love.filesystem.mkdir( name )
```

Arguments

`string name`

The directory to create.

Returns

`boolean ok`

True if the directory was created, false if not.

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.mount

love.filesystem.mount · revision 29311 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Mount a zip file.
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Mounts a zip file or folder in the game's save directory for reading. It is also possible to mount love.filesystem.getSourceBaseDirectory if the game is in fused mode.

Function

Synopsis

```
success = love.filesystem.mount( archive, mountpoint, appendToPath )
```

Arguments

`string archive`

The folder or zip file in the game's save directory to mount.

`string mountpoint`

The new path the archive will be mounted to.

`boolean appendToPath (false)`

Whether the archive will be searched when reading a filepath before or after already-mounted archives. This includes the game's source and save directories.

Returns

`boolean success`

True if the archive was successfully mounted, false otherwise.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Mounts the contents of the given FileData in memory. The FileData's data must contain a zipped directory structure.

Synopsis

```
success = love.filesystem.mount( filedata, mountpoint, appendToPath )
```

Arguments

`FileData filedata`
: The FileData object in memory to mount.

`string mountpoint`
: The new path the archive will be mounted to.

`boolean appendToPath (false)`
: Whether the archive will be searched when reading a filepath before or after already-mounted archives. This includes the game's source and save directories.

Returns

`boolean success`
: True if the archive was successfully mounted, false otherwise.

Function

Available since LÖVE 11.0
This variant is not supported in earlier versions.

Mounts the contents of the given Data object in memory. The data must contain a zipped directory structure.

Synopsis

```
success = love.filesystem.mount( data, archivename, mountpoint, appendToPath )
```

Arguments

`Data data`
: The Data object in memory to mount.

`string archivename`
: The name to associate the mounted data with, for use with love.filesystem.unmount. Must be unique compared to other mounted data.

`string mountpoint`
: The new path the archive will be mounted to.

`boolean appendToPath (false)`
: Whether the archive will be searched when reading a filepath before or after already-mounted archives. This includes the game's source and save directories.

Returns

`boolean success`

True if the archive was successfully mounted, false otherwise.

Examples

Mount a zip file.

```
-- Assuming content.zip exists in the game's save directory and contains a file called 'myimage.png'.
love.filesystem.mount("content.zip", "content")

assert(love.filesystem.getInfo("content/myimage.png") ~= nil)
```

See Also

- love.filesystem
- love.filesystem.unmount
- love.filesystem.getSourceBaseDirectory
- love.directorydropped

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.mountCommonPath

love.filesystem.mountCommonPath · revision 30592 · last edited 2026-02-04

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Mounts to a platform-dependent path without exact knowledge of the path, using the CommonPath enum to denote the type the directory you would like to mount to.

Function

Synopsis

```
success = love.filesystem.mountCommonPath( commonpath, mountpoint, permission, appendToPath )
```

Arguments

`CommonPath commonpath`
: The type of common directory to be sought and mounted to.

`string mountpoint`
: Virtual filepath in love.filesystem where the resolved common directory is exposed.

`MountPermissions permission ("read")`
: The requested permissions for operating on files and folders in this path after mounting ("read", or "readwrite").

`boolean appendToPath (false)`
: Whether the archive will be searched when reading a filepath before or after already-mounted archives. This includes the game's source and save directories.

Returns

`boolean success`
: True if the archive was successfully mounted with the given path and permissions, false otherwise.

Notes

If your app is sandboxed, does not request explicit entitlement, and does not involve user selection, MacOS places some restrictions on write access for certain common path types (notably Documents and Desktop), unless a dialogue is accepted by the user. A user may restrict write access to certain common paths on Linux as well. SaveDir is the safest best for write access.

See Also

- love.filesystem
- love.filesystem.unmountCommonPath
- love.filesystem.getFullCommonPath

- love.filesystem.mountFullPath

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.mountFullPath

love.filesystem.mountFullPath · revision 29155 · last edited 2026-02-04

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Mounts a full platform-dependent path to a zip file or folder for reading or writing in love.filesystem.

Function

Synopsis

```
success = love.filesystem.mountFullPath( archive, mountpoint, permission, appendToPath )
```

Arguments

`string archive`
: The full platform-dependent path to a folder or zip file to mount.

`string mountpoint`
: The new path in love.filesystem the archive or the platform-dependent path will be mounted to.

`MountPermissions permission ("read")`
: The requested permissions for operating on files and folders in this path after mounting ("read", or "readwrite").

`boolean appendToPath (false)`
: Whether the archive will be searched when reading a filepath before or after already-mounted archives. This includes the game's source and save directories.

Returns

`boolean success`
: True if the archive was successfully mounted with the given path and permissions, false otherwise.

Notes

Most operating systems allow apps to read from more locations than they're allowed to write to. Therefore, this API will succeed in more situations when the "read" mount permission is used compared to "readwrite".

See Also

- love.filesystem
- love.filesystem.unmountFullPath
- love.filesystem.mount

- love.filesystem.mountCommonPath

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.newFile

love.filesystem.newFile · revision 14719 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Open a file and read everything
- See Also
- Other Languages

Creates a new File object. It needs to be opened before it can be accessed.

Function

Synopsis

```
file = love.filesystem.newFile( filename )
```

Arguments

`string filename`
: The filename of the file.

Returns

`File file`
: The new File object.

Notes

Please note that this function will not return any error message (e.g. if you use an invalid filename) because it just creates the File Object. You can still check if the file is valid by using File:open which returns a boolean and an error message if something goes wrong while opening the file.

Function

Available since LÖVE 0.9.0

This variant is not supported in earlier versions.

Creates a File object and opens it for reading, writing, or appending.

Synopsis

```
file, errorstr = love.filesystem.newFile( filename, mode )
```

Arguments

`string filename`
: The filename of the file.

`FileMode mode`

The mode to open the file in.

Returns

`File file`
The new File object, or nil if an error occurred.

`string errorstr`
The error string if an error occurred.

Examples

Open a file and read everything

```
file = love.filesystem.newFile("data.txt")
file:open("r")
data = file:read()
file:close()
-- use the data ...
```

See Also

- love.filesystem
- File

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.newFileData

love.filesystem.newFileData · revision 27332 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Creates a new FileData object from a file on disk, or from a string in memory.

Function

Creates a new FileData object from a string in memory.

Synopsis

```
data = love.filesystem.newFileData( contents, name )
```

Arguments

`string contents`
: The contents of the file in memory represented as a string.

`string name`
: The name of the file. The extension may be parsed and used by LÖVE when passing the FileData object into love.audio.newSource.

Returns

`FileData data`
: The new FileData.

Function

Available since LÖVE 11.4

This variant is not supported in earlier versions.

Creates a new FileData object from a Data object in memory.

Synopsis

```
data = love.filesystem.newFileData( originaldata, name )
```

Arguments

`Data originaldata`
: The Data object to copy into the new FileData object.

`string name`

The name of the file. The extension may be parsed and used by LÖVE when passing the FileData object into love.audio.newSource.

Returns

`FileData data`
The new FileData.

Function

Available since LÖVE 0.9.0
This variant is not supported in earlier versions.

Creates a new FileData from a file on the storage device.

Synopsis

```
data, err = love.filesystem.newFileData( filepath )
```

Arguments

`string filepath`
Path to the file.

Returns

`FileData data`
The new FileData, or nil if an error occurred.

`string err`
The error string, if an error occurred.

Function

Removed in LÖVE 11.0
The variant which decodes base64 data has been replaced by love.data.decode.

Synopsis

```
data = love.filesystem.newFileData( contents, name, decoder )
```

Arguments

`string contents`
The contents of the file.

`string name`
The name of the file.

`FileDecoder decoder`
The method to use when decoding the contents.

Returns

`FileData data`

Your new FileData.

See Also

- love.filesystem
- FileData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.openFile

love.filesystem.openFile · revision 28102 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
  - Open a file and read everything
- See Also
- Other Languages

Available since LÖVE 12.0

This function replaces love.filesystem.newFile.

Opens a new File object, which represents an existing or new file on disk.

Function

Synopsis

```
file, errorstr = love.filesystem.openFile( filename, mode )
```

Arguments

`string filename`
: The filename of the file.

`FileMode mode`
: The mode to open the file in.

Returns

`File file`
: The new File object, or nil if an error occurred.

`string errorstr`
: The error string if an error occurred.

Notes

love.filesystem.read, love.filesystem.write, and love.filesystem.append are more convenient functions to use in situations where a file is operated on once rather than multiple times.

Examples

Open a file and read everything

```
file = love.filesystem.openFile("data.txt", "r")
data = file:read()
```

```
file:close()
-- use the data ...
```

See Also

- love.filesystem
- File

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.openNativeFile

love.filesystem.openNativeFile · revision 29448 · last edited 2025-01-23

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Opens a new File object outside of love.filesystem paths.


This documents a function for a future version of LÖVE and may be changed before final release.

Function

Synopsis

```
file, errorstr = love.filesystem.openNativeFile( filename, mode )
```

Arguments

`string filename`
: The full platform-dependent path to the file.

`FileMode mode`
: The mode to open the file in.

Returns

`File file`
: The new File object, or nil if an error occurred.

`string errorstr`
: The error string if an error occurred.

Notes

Unlike love.filesystem.openFile which operates on restricted love.filesystem paths, this is more similar to Lua standard io.open.

See Also

- love.filesystem

- File

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.read

love.filesystem.read · revision 27959 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Returns (if error on reading)
- Function
- Synopsis
- Arguments
- Returns
- Returns (if error on reading)
- See Also
- Other Languages

Read the contents of a file.

Function

Synopsis

```
contents, size = love.filesystem.read( name, size )
```

Arguments

string name
: The name (and path) of the file.

number size (all)
: How many bytes to read.

Returns

string contents
: The file contents.

number size
: How many bytes have been read.

Returns (if error on reading)

nil contents
: returns nil as content.

string error
: returns an error message.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Reads the contents of a file into either a string or a FileData object.

Synopsis

```
contents, size = love.filesystem.read( container, name, size )
```

Arguments

`ContainerType container`
: What type to return the file's contents as.

`string name`
: The name (and path) of the file

`number size (all)`
: How many bytes to read

Returns

`value contents`
: FileData or string containing the file contents.

`number size`
: How many bytes have been read.

Returns (if error on reading)

`nil contents`
: returns nil as content.

`string error`
: returns an error message.

See Also

- love.filesystem.write
- love.filesystem.lines
- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.remove

love.filesystem.remove · revision 30152 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- See Also
- Other Languages

Removes a file or empty directory.

Function

Synopsis

```
success = love.filesystem.remove( name )
```

Arguments

string name
: The file or directory to remove.

Returns

boolean success
: True if the file/directory was removed, false otherwise.

Notes

- Like all filesystem operations, changes can only be made to the save directory. Removing files from the .love archive is not possible.
- If a directory is being removed, it must be empty before removal or else it will fail. Remove all files and folders in the directory beforehand.
- An opened File object prevents removal of the underlying file. Close the file to remove it.

Examples

Create a bunch of folders in the save folder and remove them and any file they may contain as soon as the game is quit.

```
function love.load()
    local dir = 'a'
    for _ = 1, 10 do
        dir = dir .. '/a'
    end
    love.filesystem.createDirectory( dir )
end

function love.quit()
    local function recursivelyDelete( item )
        if love.filesystem.getInfo( item , "directory" ) then
```

```
            for _, child in ipairs( love.filesystem.getDirectoryItems( item )) do
                recursivelyDelete( item .. '/' .. child )
                love.filesystem.remove( item .. '/' .. child )
            end
        elseif love.filesystem.getInfo( item ) then
            love.filesystem.remove( item )
        end
        love.filesystem.remove( item )
    end
    recursivelyDelete( 'a' )
end
```

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.setCRequirePath

love.filesystem.setCRequirePath · revision 21949 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Sets the filesystem paths that will be searched for c libraries when require (http://www.lua.org/manual/5.1/manual.html#pdf-require) is called.

The paths string returned by this function is a sequence of path templates separated by semicolons. The argument passed to *require* will be inserted in place of any question mark ("?") character in each template (after the dot characters in the argument passed to *require* are replaced by directory separators.) Additionally, any occurrence of a double question mark ("??") will be replaced by the name passed to require and the default library extension for the platform.

The paths are relative to the game's source and save directories, as well as any paths mounted with love.filesystem.mount.

Function

Synopsis

```
love.filesystem.setCRequirePath( paths )
```

Arguments

`string paths`

The paths that the *require* function will check in love's filesystem.

Returns

Nothing.

Notes

The default paths string is `"??"`, which makes `require("cool")` try to load `cool.dll`, or `cool.so` depending on the platform.

See Also

- love.filesystem
- love.filesystem.getCRequirePath

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

love.filesystem.setIdentity

love.filesystem.setIdentity · revision 27666 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Setting the game folder name
  - Setting which to search first
- See Also
- Other Languages

Sets the write directory for your game. Equivalent to the identity/appendidentity pair for conf.lua. Note that you can only set the name of the folder to store your files in, not the location.

Function

Synopsis

```
love.filesystem.setIdentity( name, appendToPath )
```

Arguments

string name

The new identity that will be used as write directory.

Available since LÖVE 0.9.0

boolean appendToPath (false)

Whether the identity directory will be searched when reading a filepath before or after the game's source directory and any currently.

TRUE: results in searching source before searching save directory; FALSE: results in searching game save directory before searching source directory mounted archives.

Returns

Nothing.

Examples

Setting the game folder name

```
-- Yes:
love.filesystem.setIdentity("monkey_doom_2")
-- No:
love.filesystem.setIdentity("c:/Users/bob/monkey_doom_2")
```

Setting which to search first

```
--Search Source, then the save directory
love.filesystem.setIdentity(love.filesystem.getIdentity(),true)
```

```
--Search Save directory, then the Source Directory
love.filesystem.setIdentity(love.filesystem.getIdentity(),false)
```

See Also

- love.filesystem
- love.filesystem.getIdentity

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.setRequirePath

love.filesystem.setRequirePath · revision 15610 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Sets the filesystem paths that will be searched when require (http://w w w.lua.org/manual/5.1/manual.html#pdf-require) is called.

The paths string given to this function is a sequence of path templates separated by semicolons. The argument passed to *require* will be inserted in place of any question mark ("?") character in each template (after the dot characters in the argument passed to *require* are replaced by directory separators.)

The paths are relative to the game's source and save directories, as well as any paths mounted with love.filesystem.mount.

Function

Synopsis

```
love.filesystem.setRequirePath( paths )
```

Arguments

`string paths`

The paths that the *require* function will check in love's filesystem.

Returns

Nothing.

Notes

The default paths string is `"?.lua;?/init.lua"`, which makes `require("cool")` try to load `cool.lua` and then try `cool/init.lua` if cool.lua doesn't exist.

See Also

- love.filesystem
- love.filesystem.getRequirePath

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.setSource

love.filesystem.setSource · revision 19447 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Sets the source of the game, where the code is present. This function can only be called once, and is normally automatically done by LÖVE.

Function

Synopsis

```
love.filesystem.setSource( path )
```

Arguments

`string path`
: Absolute path to the game's source folder.

Returns

Nothing.

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.setSymlinksEnabled

love.filesystem.setSymlinksEnabled · revision 15748 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Sets whether love.filesystem follows symbolic links. It is enabled by default in version 0.10.0 and newer, and disabled by default in 0.9.2.

Function

Synopsis

```
love.filesystem.setSymlinksEnabled( enable )
```

Arguments

`boolean enable`

Whether love.filesystem should follow symbolic links.

Returns

Nothing.

See Also

- love.filesystem
- love.filesystem.areSymlinksEnabled
- love.filesystem.isSymlink

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.unmount

love.filesystem.unmount · revision 21575 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Mount a zip file and then unmount it.
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Unmounts a zip file or folder previously mounted for reading with love.filesystem.mount.

Function

Synopsis

```
success = love.filesystem.unmount( archive )
```

Arguments

`string archive`

The folder or zip file in the game's save directory which is currently mounted.

Returns

`boolean success`

True if the archive was successfully unmounted, false otherwise.

Examples

Mount a zip file and then unmount it.

```
-- Assuming content.zip exists in the game's save directory and contains a file called 'myimage.png'.
love.filesystem.mount("content.zip", "content")
assert(love.filesystem.getInfo("content/myimage.png"))

love.filesystem.unmount("content.zip")
assert(not love.filesystem.getInfo("content/myimage.png"))
```

See Also

- love.filesystem
- love.filesystem.mount

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

love.filesystem.unmountCommonPath

love.filesystem.unmountCommonPath · revision 30600 · last edited 2026-02-04

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Unmounts any mount that exists in relation to a commonly used directory created by love.filesystem.mountCommonPath.

Function

Synopsis

```
success = love.filesystem.mountCommonPath(commonpath)
-- ..
success = love.filesystem.unmountCommonPath(commonpath)
```

Arguments

`CommonPath commonpath`

The type of common directory to be sought and unmounted from love.filesystem

Returns

`boolean success`

True if the archive was successfully unmounted, false otherwise.

See Also

- love.filesystem
- love.filesystem.unmount
- love.filesystem.mountCommonPath
- love.filesystem.getFullCommonPath
- love.filesystem.mountFullPath

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.unmountFullPath

love.filesystem.unmountFullPath · revision 28109 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Unmounts a zip file or folder previously mounted with love.filesystem.mountFullPath.

Function

Synopsis

```
success = love.filesystem.unmountFullPath( archive )
```

Arguments

`string archive`

The full platform-dependent path to a folder or zip file which is currently mounted via love.filesystem.mountFullPath.

Returns

`boolean success`

True if the archive was successfully unmounted, false otherwise.

Notes

If there are any files open via love.filesystem in the given archive when this API is called, it will fail and return false.

See Also

- love.filesystem
- love.filesystem.mountFullPath

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.filesystem.write

love.filesystem.write · revision 30566 · last edited 2026-01-24

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Example
- Notes
- See Also
- Other Languages

Write data to a file in the save directory. If the file existed already, it will be completely replaced by the new contents.

Function

Synopsis

```
success, message = love.filesystem.write( name, data, size )
```

Arguments

string name
: The name (and path) of the file.

string data
: The string data to write to the file.

number size (all)
: How many bytes to write.

Returns

boolean success
: If the operation was successful.

string message
: Error message if operation was unsuccessful.

Function

Synopsis

```
success, message = love.filesystem.write( name, data, size )
```

Arguments

string name
: The name (and path) of the file.

Data data
: The Data object to write to the file.

number size (all)

How many bytes to write.

Returns

boolean success
If the operation was successful.

string message
Error message if operation was unsuccessful.

Example

```
local success, message =love.filesystem.write( filename, "")
if success then
    print ('file created')
else
    print ('file not created: '..message)
end
```

Notes

If you are getting the error message "Could not set write directory", try setting the save directory. This is done either with love.filesystem.setIdentity or by setting the identity field in love.conf.

Writing to multiple lines: In Windows, some text editors (e.g. Notepad) only treat CRLF ("\r\n") as a new line.

'*The default destination directory* can be found by using love.filesystem.getAppdataDirectory(), the new file will be in the subdirectory "love/application_name/" of this path, where the application_name is the same than the directory containing the main.lua or name of the archive with .love extension.

See Also

- love.filesystem.append
- love.filesystem.read
- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.focus

love.focus · revision 22505 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
- See Also
- Other Languages

Available since LÖVE 0.7.0

This callback is not supported in earlier versions.

Callback function triggered when window receives or loses focus.

Function

Synopsis

```
love.focus( focus )
```

Arguments

`boolean focus`

True if the window gains focus, false if it loses focus.

Returns

Nothing.

Example

```
function love.load()
  text = "FOCUSED"
end

function love.draw()
  love.graphics.print(text,0,0)
end

function love.focus(f)
  if f then
    print("Window is focused.")
    text = "FOCUSED"
  else
    print("Window is not focused.")
    text = "UNFOCUSED"
  end
end
```

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.font

love.font · revision 27326 · last edited 2023-07-05

Available since LÖVE 0.7.0

This module is not supported in earlier versions.

Allows you to work with fonts.

Types

FontData	A FontData represents a font.	Added since 0.7.0	Removed in 0.8.0
GlyphData	A GlyphData represents a drawable symbol of a font.	Added since 0.7.0	
Rasterizer	A Rasterizer represents font data and glyphs.	Added since 0.7.0	

Functions

love.font.newBMFontRasterizer	Creates a new BMFont Rasterizer.	Added since 0.10.0	
love.font.newFontData	Creates a new FontData.	Added since 0.7.0	Removed in 0.8.0
love.font.newGlyphData	Creates a new GlyphData.	Added since 0.7.0	
love.font.newImageRasterizer	Creates a new Image Rasterizer.	Added since 0.10.0	
love.font.newRasterizer	Creates a new Rasterizer.	Added since 0.7.0	
love.font.newTrueTypeRasterizer	Creates a new TrueType Rasterizer.	Added since 0.10.0	

Enums

HintingMode	True Type hinting mode.	Added since 0.10.0
PixelFormat	Pixel formats for Textures, ImageData, and CompressedImageData.	Added since 11.0

See Also

- Font
- love
- love.graphics.newFont

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.font.newBMFontRasterizer

love.font.newBMFontRasterizer · revision 28193 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Creates a new BMFont Rasterizer.

Function

Synopsis

```
rasterizer = love.font.newBMFontRasterizer( imageData, glyphs, dpiscale )
```

Arguments

`ImageData imageData`
: The image data containing the drawable pictures of font glyphs.

`string glyphs`
: The sequence of glyphs in the ImageData.

Available since LÖVE 11.0

`number dpiscale (1)`
: DPI scale.

Returns

`Rasterizer rasterizer`
: The rasterizer.

Function

Synopsis

```
rasterizer = love.font.newBMFontRasterizer( fileName, glyphs, dpiscale )
```

Arguments

`string fileName`
: The path to file containing the drawable pictures of font glyphs.

`string glyphs`
: The sequence of glyphs in the ImageData.

Available since LÖVE 11.0

`number dpiscale (1)`
DPI scale.

Returns

`Rasterizer rasterizer`
The rasterizer.

See Also

- love.font
- love.font.newRasterizer
- love.font.newTrueTypeRasterizer
- love.font.newImageRasterizer
- Rasterizer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.font.newFontData

love.font.newFontData · revision 15947 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.8.0

This function is not supported in earlier or later versions.

Creates a new FontData.

Function

Synopsis

```
fontData = love.font.newFontData( rasterizer )
```

Arguments

`Rasterizer rasterizer`

The Rasterizer containing the font.

Returns

`FontData fontData`

The FontData.

See Also

- love.font
- FontData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.font.newGlyphData

love.font.newGlyphData · revision 5795 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Creates a new GlyphData.

Function

Synopsis

```
glyphData = love.font.newGlyphData( rasterizer, glyph )
```

Arguments

`Rasterizer rasterizer`
: The Rasterizer containing the font.

`number glyph`
: The character code of the glyph.

Returns

`GlyphData glyphData`
: The GlyphData.

See Also

- love.font
- GlyphData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.font.newImageRasterizer

love.font.newImageRasterizer · revision 28195 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Creates a new Image Rasterizer.

Function

Create an ImageRasterizer from the image data.

Synopsis

```
rasterizer = love.font.newImageRasterizer( imageData, glyphs, extraSpacing, dpiscale )
```

Arguments

`ImageData imageData`
: Font image data.

`string glyphs`
: String containing font glyphs.

`number extraSpacing (0)`
: Font extra spacing.

Available since LÖVE 11.0

`number dpiscale (1)`
: Font DPI scale.

Returns

`Rasterizer rasterizer`
: The rasterizer.

See Also

- love.font
- love.font.newRasterizer
- love.font.newBMFontRasterizer
- love.font.newTrueTypeRasterizer

- Rasterizer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.font.newRasterizer

love.font.newRasterizer · revision 25497 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Creates a new Rasterizer.

Function

Synopsis

```
rasterizer = love.font.newRasterizer( filename )
```

Arguments

`string filename`
: The font file.

Returns

`Rasterizer rasterizer`
: The rasterizer.

Function

Synopsis

```
rasterizer = love.font.newRasterizer( data )
```

Arguments

`FileData data`

The FileData of the font file.

Returns

`Rasterizer rasterizer`
The rasterizer.

Function

Create a TrueTypeRasterizer with the default font.

Synopsis

```
rasterizer = love.font.newRasterizer( size, hinting, dpiscale )
```

Arguments

`number size (12)`
The font size.

Available since LÖVE 0.10.0

`HintingMode hinting ("normal")`
True Type hinting mode.

Available since LÖVE 11.0

`number dpiscale (love.window.getDPIScale())`
The font DPI scale.

Returns

`Rasterizer rasterizer`
The rasterizer.

Function

Create a TrueTypeRasterizer with custom font.

Synopsis

```
rasterizer = love.font.newRasterizer( fileName, size, hinting, dpiscale )
```

Arguments

`string fileName`
Path to font file.

`number size (12)`
The font size.

Available since LÖVE 0.10.0

`HintingMode hinting ("normal")`
: True Type hinting mode.

Available since LÖVE 11.0

`number dpiscale (love.window.getDPIScale())`
: The font DPI scale.

Returns

`Rasterizer rasterizer`
: The rasterizer.

Function

Create a TrueTypeRasterizer with custom font.

Synopsis

```
rasterizer = love.font.newRasterizer( fileData, size, hinting, dpiscale )
```

Arguments

`FileData fileData`
: File data containing font.

`number size (12)`
: The font size.

Available since LÖVE 0.10.0

`HintingMode hinting ("normal")`
: True Type hinting mode.

Available since LÖVE 11.0

`number dpiscale (love.window.getDPIScale())`
: The font DPI scale.

Returns

`Rasterizer rasterizer`

The rasterizer.

Function

Creates a new BMFont Rasterizer.

Synopsis

```
rasterizer = love.font.newRasterizer( imageData, glyphs, dpiscale )
```

Arguments

ImageData imageData
The image data containing the drawable pictures of font glyphs.

string glyphs
The sequence of glyphs in the ImageData.

Available since LÖVE 11.0

number dpiscale (1)
DPI scale.

Returns

Rasterizer rasterizer
The rasterizer.

Function

Creates a new BMFont Rasterizer.

Synopsis

```
rasterizer = love.font.newRasterizer( fileName, glyphs, dpiscale )
```

Arguments

string fileName
The path to file containing the drawable pictures of font glyphs.

string glyphs
The sequence of glyphs in the ImageData.

Available since LÖVE 11.0

number dpiscale (1)
DPI scale.

Returns

Rasterizer rasterizer

The rasterizer.

See Also

- love.font
- love.font.newTrueTypeRasterizer
- love.font.newBMFontRasterizer
- love.font.newImageRasterizer
- Rasterizer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.font.newTrueTypeRasterizer

love.font.newTrueTypeRasterizer · revision 28194 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Creates a new TrueType Rasterizer.

Function

Create a TrueTypeRasterizer with the default font.

Synopsis

```
rasterizer = love.font.newTrueTypeRasterizer( size, hinting, dpiscale )
```

Arguments

number size (12)

The font size.

HintingMode hinting ("normal")

True Type hinting mode.

Available since LÖVE 11.0

number dpiscale (love.window.getDPIScale())

The font DPI scale.

Returns

Rasterizer rasterizer

The rasterizer.

Function

Create a TrueTypeRasterizer with custom font.

Synopsis

```
rasterizer = love.font.newTrueTypeRasterizer( fileName, size, hinting, dpiscale )
```

Arguments

string fileName
Path to font file.

number size (12)
The font size.

HintingMode hinting ("normal")
True Type hinting mode.

Available since LÖVE 11.0

number dpiscale (love.window.getDPIScale())
The font DPI scale.

Returns

Rasterizer rasterizer
The rasterizer.

Function

Create a TrueTypeRasterizer with custom font.

Synopsis

```
rasterizer = love.font.newTrueTypeRasterizer( fileData, size, hinting, dpiscale )
```

Arguments

FileData fileData
File data containing font.

number size (12)
The font size.

HintingMode hinting ("normal")
True Type hinting mode.

Available since LÖVE 11.0

number dpiscale (love.window.getDPIScale())

The font DPI scale.

Returns

`Rasterizer rasterizer`

The rasterizer.

See Also

- love.font
- love.font.newRasterizer
- love.font.newBMFontRasterizer
- love.font.newImageRasterizer
- Rasterizer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.gamepadaxis

love.gamepadaxis · revision 27210 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Called when a Joystick's virtual gamepad axis is moved.

Function

Synopsis

```
love.gamepadaxis( joystick, axis, value )
```

Arguments

`Joystick joystick`
: The joystick object.

`GamepadAxis axis`
: The virtual gamepad axis.

`number value`
: The new axis value.

Returns

Nothing.

Example

Update the position of an object when an axis moves.

```
function love.load()
    width, height = love.graphics.getDimensions()
    position      = {x = width/2, y = height/2}
end

function love.gamepadaxis(joystick, axis, value)
    if axis == "leftx" then
        position.x = width/2 + value*width/2
    elseif axis == "lefty" then
        position.y = height/2 + value*height/2
    end
end

function love.draw()
```

```
    love.graphics.circle("fill", position.x, position.y, 50)
end
```

See Also

- love
- Joystick:isGamepad
- Joystick:getGamepadAxis

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.gamepadpressed

love.gamepadpressed · revision 30110 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example: print the MRU gamepad name
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Called when a Joystick's virtual gamepad button is pressed.

Function

Synopsis

```
love.gamepadpressed( joystick, button )
```

Arguments

`Joystick joystick`
: The joystick object.

`GamepadButton button`
: The virtual gamepad button.

Returns

Nothing.

Example: print the MRU gamepad name

Track the most recently used gamepad and print its name when it changes. See also love.joystick.getGamepadMappingString for more detailed names; especially when using love.joystick.loadGamepadMappings.

```
local active_joystick
function love.gamepadpressed(joystick, button)
    if joystick == active_joystick then
        return
    end
    active_joystick = joystick
    local name = joystick:getName()
    local index = joystick:getID()
    print(string.format("Changing active gamepad to #%d '%s'.", index, name))
end
```

See Also

- love
- love.gamepadreleased
- Joystick:isGamepad

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.gamepadreleased

love.gamepadreleased · revision 26455 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Called when a Joystick's virtual gamepad button is released.

Function

Synopsis

```
love.gamepadreleased( joystick, button )
```

Arguments

`Joystick joystick`
: The joystick object.

`GamepadButton button`
: The virtual gamepad button.

Returns

Nothing.

See Also

- love
- love.gamepadpressed
- Joystick:isGamepad

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.getVersion

love.getVersion · revision 31610 · last edited 2026-05-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
  - display the current version
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Gets the current running version of LÖVE.

Function

Synopsis

```
major, minor, revision, codename = love.getVersion( )
```

Arguments

None.

Returns

number major
: The major version of LÖVE, i.e. 0 for version 0.9.1.

number minor
: The minor version of LÖVE, i.e. 9 for version 0.9.1.

number revision
: The revision version of LÖVE, i.e. 1 for version 0.9.1.

string codename
: The codename of the current version, i.e. "Baby Inspector" for version 0.9.1.

Notes

For LÖVE versions below 0.9.1, the following variables can be used instead (and still work in 0.9.2 and newer):

```
love._version_major
love._version_minor
love._version_revision
```

Examples

display the current version

```
function love.draw()
    local major, minor, revision, codename = love.getVersion()
    local str = string.format("Version %d.%d.%d - %s", major, minor, revision, codename)
```

```
    love.graphics.print(str, 20, 20)
end
```

See Also

- love
- love.isVersionCompatible

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics

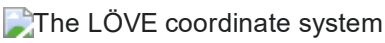
love.graphics · revision 26499 · last edited 2024-06-15

On this page

- Types
- Functions
  - Drawing
  - Object Creation
  - Graphics State
  - Coordinate System
  - Window
  - System Information
- Enums
- See Also
- Other Languages

The primary responsibility for the love.graphics module is the drawing of lines, shapes, text, Images and other Drawable objects onto the screen. Its secondary responsibilities include loading external files (including Images and Fonts) into memory, creating specialized objects (such as ParticleSystems or Canvases) and managing screen geometry.

LÖVE's coordinate system is rooted in the upper-left corner of the screen, which is at location (0, 0). The x axis is horizontal: larger values are further to the right. The y axis is vertical: larger values are further towards the bottom. It is worth noting that the location (0, 0) aligns with the upper-left corner of the pixel as well, meaning that for some functions you may encounter off-by-one problems in the render output when drawing 1 pixel wide lines. You can try aligning the coordinate system with the center of pixels rather than their upper-left corner. Do this by passing x+0.5 and y+0.5 or using love.graphics.translate().


The LÖVE coordinate system

In many cases, you draw images or shapes in terms of their upper-left corner (See the picture above).

A note about angles in LÖVE: Angles are expressed in terms of radians (https://en.wikipedia.org/wiki/Radian), with values in the range of 0 to 2pi (approximately 6.28); you may be more used to working in terms of degrees. Because of how the coordinate system is set up, with an origin in the upper left corner, angles in LÖVE may seem a bit backwards: 0 points right (along the X axis), ¼pi points diagonally down and to the right, ½pi points directly down (along the Y axis), with increasing values continuing **clockwise**.

Many of the functions are used to manipulate the *graphics coordinate system*, which is essentially the way coordinates are mapped to the display. You can change the position, scale, and even change rotation in this way.

Types

Canvas	Off-screen render target.	Added since 0.8.0	
Drawable	Superclass for all things that can be drawn on screen.		
Font	Defines the shape of characters than can be drawn onto the screen.		
Framebuffer	Off-screen render target.	Added since 0.7.0	Removed in 0.8.0
Image	Drawable image type.		
Mesh	A 2D polygon mesh used for drawing arbitrary textured shapes.	Added since 0.9.0	
ParticleSystem	Used to create cool effects, like fire.		
PixelEffect	Pixel shader effect.	Added since 0.8.0	Removed in 0.9.0
Quad	A quadrilateral with texture coordinate information.		
Shader	Shader effect.	Added since 0.9.0	
SpriteBatch	Store image positions in a buffer, and draw it in one call.		
Text	Drawable text.	Added since 0.10.0	
Texture	Superclass for drawable objects which represent a texture.	Added since 0.9.1	

Video	A drawable video.	Added since 0.10.0

Functions

Drawing

love.graphics.arc	Draws an arc.	Added since 0.8.0	
love.graphics.circle	Draws a circle.		
love.graphics.clear	Clears the screen or active Canvas to the specified color.		
love.graphics.discard	Discards the contents of the screen or active Canvas.	Added since 0.10.0	
love.graphics.draw	Draws objects on screen.		
love.graphics.drawInstanced	Draws many instances of a Mesh with a single draw call, using hardware geometry instancing.	Added since 11.0	
love.graphics.drawLayer	Draws a layer of an Array Texture.	Added since 11.0	
love.graphics.drawq	Draw a Quad with the specified Image on screen.		Removed in 0.9.0
love.graphics.ellipse	Draws an ellipse.	Added since 0.10.0	
love.graphics.flushBatch	Immediately renders any pending automatically batched draws.	Added since 11.0	
love.graphics.line	Draws lines between points.		
love.graphics.point	Draws a point.	Added since 0.3.0	Removed in 0.10.0
love.graphics.points	Draws one or more points.	Added since 0.10.0	
love.graphics.polygon	Draw a polygon.	Added since 0.4.0	
love.graphics.present	Displays the results of drawing operations on the screen.		
love.graphics.print	Draws text on screen. If no Font is set, one will be created and set (once) if needed.		
love.graphics.printf	Draws formatted text, with word wrap and alignment.		
love.graphics.quad	Draws a quadrilateral shape.		Removed in 0.9.0
love.graphics.rectangle	Draws a rectangle.	Added since 0.3.2	
love.graphics.stencil	Draws geometry as a stencil.	Added since 0.10.0	
love.graphics.triangle	Draws a triangle.		Removed in 0.9.0

Object Creation

love.graphics.captureScreenshot	Creates a screenshot once the current frame is done.	Added since 11.0	
love.graphics.newArrayImage	Creates a new array Image.	Added since 11.0	
love.graphics.newCanvas	Creates a new Canvas.	Added since 0.8.0	
love.graphics.newCubeImage	Creates a new cubemap Image.	Added since 11.0	
love.graphics.newFont	Creates a new Font from a TrueType Font or BMFont file.		

love.graphics.newFramebuffer	Creates a new Framebuffer.	Added since 0.7.0	Removed in 0.8.0
love.graphics.newImage	Creates a new Image.		
love.graphics.newImageFont	Creates a new Font by loading a specifically formatted image.	Added since 0.2.0	
love.graphics.newMesh	Creates a new Mesh.	Added since 0.9.0	
love.graphics.newParticleSystem	Creates a new ParticleSystem.		
love.graphics.newPixelEffect	Creates a new PixelEffect.	Added since 0.8.0	Removed in 0.9.0
love.graphics.newQuad	Creates a new Quad.		
love.graphics.newScreenshot	Creates a screenshot and returns the ImageData.		Removed in 11.0
love.graphics.newShader	Creates a new Shader.	Added since 0.9.0	
love.graphics.newSpriteBatch	Creates a new SpriteBatch.		
love.graphics.newStencil	Creates a new stencil.	Added since 0.8.0	Removed in 0.9.0
love.graphics.newText	Creates a new drawable Text object.	Added since 0.10.0	
love.graphics.newVideo	Creates a new Video.	Added since 0.10.0	
love.graphics.newVolumeImage	Creates a new volume Image.	Added since 11.0	
love.graphics.setNewFont	Creates and sets a new Font.	Added since 0.8.0	
love.graphics.validateShader	Validates shader code.	Added since 11.0	

Graphics State

love.graphics.getBackgroundColor	Gets the current background color.		
love.graphics.getBlendMode	Gets the blending mode.	Added since 0.2.0	
love.graphics.getCanvas	Returns the current target Canvas.	Added since 0.8.0	
love.graphics.getColor	Gets the current color.		
love.graphics.getColorMask	Gets the active color components used when drawing.	Added since 0.9.0	
love.graphics.getColorMode	Gets the color mode (which controls how images are affected by the current color).	Added since 0.2.0	Removed in 0.9.0
love.graphics.getDefaultFilter	Returns the default scaling filters used with Images, Canvases, and Fonts.	Added since 0.9.0	
love.graphics.getDefaultImageFilter	Returns the default scaling filters.	Added since 0.8.0	Removed in 0.9.0
love.graphics.getDepthMode	Gets the current depth test mode and whether writing to the depth buffer is enabled.	Added since 11.0	

love.graphics.getFont	Gets the current Font object.	Added since 0.9.0	
love.graphics.getFrontFaceWinding	Gets whether triangles with clockwise- or counterclockwise-ordered vertices are considered front-facing.	Added since 11.0	
love.graphics.getLineJoin	Gets the line join style.		
love.graphics.getLineStipple	Gets the current line stipple.		Removed in 0.8.0
love.graphics.getLineStyle	Gets the line style.	Added since 0.3.2	
love.graphics.getLineWidth	Gets the current line width.	Added since 0.3.2	
love.graphics.getMeshCullMode	Gets whether back-facing triangles in a Mesh are culled.	Added since 11.0	
love.graphics.getPixelEffect	Returns the current PixelEffect.	Added since 0.8.0	Removed in 0.9.0
love.graphics.getPointSize	Gets the point size.		
love.graphics.getPointStyle	Gets the current point style.		Removed in 0.10.0
love.graphics.getScissor	Gets the current scissor box.	Added since 0.4.0	
love.graphics.getShader	Gets the current Shader.	Added since 0.9.0	
love.graphics.getStackDepth	Gets the current depth of the transform / state stack (the number of pushes without corresponding pops).	Added since 11.0	
love.graphics.getStencilTest	Gets the current stencil test configuration.	Added since 0.10.0	
love.graphics.intersectScissor	Sets the scissor to the rectangle created by the intersection of the specified rectangle with the existing scissor.	Added since 0.10.0	
love.graphics.isActive	Gets whether the graphics module is able to be used.	Added since 0.10.0	
love.graphics.isGammaCorrect	Gets whether gamma-correct rendering is enabled.	Added since 0.10.0	
love.graphics.isSupported	Checks for the support of graphics related functions.	Added since 0.8.0	Removed in 0.10.0
love.graphics.isWireframe	Gets whether wireframe mode is used when drawing.	Added since 0.9.1	
love.graphics.reset	Resets the current graphics settings.		
love.graphics.setBackgroundColor	Sets the background color.		
love.graphics.setBlendMode	Sets the blending mode.	Added since 0.2.0	

love.graphics.setCanvas	Captures drawing operations to a Canvas	Added since 0.8.0	
love.graphics.setColor	Sets the color used for drawing.		
love.graphics.setColorMask	Sets the color mask. Enables or disables specific color components when rendering.	Added since 0.9.0	
love.graphics.setColorMode	Sets the color mode (which controls how images are affected by the current color).	Added since 0.2.0	Removed in 0.9.0
love.graphics.setDefaultFilter	Sets the default scaling filters used with Images, Canvases, and Fonts.	Added since 0.9.0	
love.graphics.setDefaultImageFilter	Sets the default scaling filters.	Added since 0.8.0	Removed in 0.9.0
love.graphics.setDepthMode	Configures depth testing and writing to the depth buffer.	Added since 11.0	
love.graphics.setFont	Set an already-loaded Font as the current font.		
love.graphics.setFrontFaceWinding	Sets whether triangles with clockwise- or counterclockwise-ordered vertices are considered front-facing.	Added since 11.0	
love.graphics.setInvertedStencil	Defines an inverted stencil.	Added since 0.8.0	Removed in 0.10.0
love.graphics.setLine	Sets the line width and style.		Removed in 0.9.0
love.graphics.setLineJoin	Sets the line join style.		
love.graphics.setLineStipple	Sets the line stipple pattern.		Removed in 0.8.0
love.graphics.setLineStyle	Sets the line style.	Added since 0.3.2	
love.graphics.setLineWidth	Sets the line width.	Added since 0.3.2	
love.graphics.setMeshCullMode	Sets whether back-facing triangles in a Mesh are culled.	Added since 11.0	
love.graphics.setPixelEffect	Routes drawing operations through a pixel shader.	Added since 0.8.0	Removed in 0.9.0
love.graphics.setPoint	Sets the point size and style.		Removed in 0.9.0
love.graphics.setPointSize	Sets the point size.		
love.graphics.setPointStyle	Sets the point style.		Removed in 0.10.0
love.graphics.setRenderTarget	Captures drawing operations to a Framebuffer	Added since 0.7.0	Removed in 0.8.0
love.graphics.setScissor	Sets or disables scissor.	Added since 0.4.0	

love.graphics.setShader	Routes drawing operations through a shader.	Added since 0.9.0	
love.graphics.setStencil	Defines or releases a stencil.	Added since 0.8.0	Removed in 0.10.0
love.graphics.setStencilTest	Configures or disables stencil testing.	Added since 0.10.0	
love.graphics.setWireframe	Sets whether wireframe lines will be used when drawing.	Added since 0.9.1	

Coordinate System

love.graphics.applyTransform	Applies the given Transform object to the current coordinate transformation.	Added since 11.0
love.graphics.inverseTransformPoint	Converts the given 2D position from screen-space into global coordinates.	Added since 11.0
love.graphics.origin	Resets the current coordinate transformation.	Added since 0.9.0
love.graphics.pop	Pops the current coordinate transformation from the transformation stack.	
love.graphics.push	Copies and pushes the current coordinate transformation to the transformation stack.	
love.graphics.replaceTransform	Replaces the current coordinate transformation with the given Transform object.	Added since 11.0
love.graphics.rotate	Rotates the coordinate system in two dimensions.	
love.graphics.scale	Scales the coordinate system in two dimensions.	
love.graphics.shear	Shears the coordinate system.	Added since 0.8.0
love.graphics.transformPoint	Converts the given 2D position from global coordinates into screen-space.	Added since 11.0
love.graphics.translate	Translates the coordinate system in two dimensions.	

Window

love.graphics.checkMode	Checks if a display mode is supported.		Removed in 0.9.0
love.graphics.getCaption	Gets the window caption.		Removed in 0.9.0
love.graphics.getDPIScale	Gets the DPI scale factor of the window.	Added since 11.0	
love.graphics.getDimensions	Gets the width and height of the window.	Added since 0.9.0	
love.graphics.getHeight	Gets the height in pixels of the window.	Added since 0.2.1	
love.graphics.getMode	Returns the current display mode.	Added since 0.8.0	Removed in 0.9.0
love.graphics.getModes	Gets a list of supported fullscreen modes.		Removed in 0.9.0
love.graphics.getPixelDimensions	Gets the width and height in pixels of the window.	Added since 11.0	
love.graphics.getPixelHeight	Gets the height in pixels of the window.	Added since 11.0	
love.graphics.getPixelWidth	Gets the width in pixels of the window.	Added since 11.0	
love.graphics.getWidth	Gets the width in pixels of the window.	Added since 0.2.1	
love.graphics.hasFocus	Checks if the game window has keyboard focus.	Added since 0.8.0	Removed in 0.9.0
love.graphics.isCreated	Checks if the window has been created.		Removed in 0.9.0
love.graphics.setCaption	Sets the window caption.		Removed in 0.9.0
love.graphics.setIcon	Set window icon.	Added since 0.7.0	Removed in 0.9.0

love.graphics.setMode	Changes the display mode.	Removed in 0.9.0
love.graphics.toggleFullscreen	Toggles fullscreen.	Removed in 0.9.0

System Information

love.graphics.getCanvasFormats	Gets the available Canvas formats, and whether each is supported.	Added since 0.9.2	
love.graphics.getCompressedImageFormats	Gets the available compressed image formats, and whether each is supported.	Added since 0.9.2	Removed in 11.0
love.graphics.getImageFormats	Gets the pixel formats usable for Images, and whether each is supported.	Added since 11.0	
love.graphics.getMaxImageSize	Gets the max supported width or height of Images and Canvases.	Added since 0.9.0	Removed in 0.10.0
love.graphics.getMaxPointSize	Gets the max supported point size.		Removed in 0.10.0
love.graphics.getRendererInfo	Gets information about the system's video card and drivers.	Added since 0.9.0	
love.graphics.getStats	Gets performance-related rendering statistics.	Added since 0.9.2	
love.graphics.getSupported	Gets the optional graphics features and whether they're supported.	Added since 0.10.0	
love.graphics.getSystemLimit	Gets the system-dependent maximum value for a love.graphics feature.	Added since 0.9.1	Removed in 0.10.0
love.graphics.getSystemLimits	Gets the system-dependent maximum values for love.graphics features.	Added since 0.10.0	
love.graphics.getTextureTypes	Gets the available texture types, and whether each is supported.	Added since 11.0	

Enums

AlignMode	Text alignment.	
ArcType	Different types of arcs that can be drawn.	Added since 0.10.1
AttributeDataType	Data types used in a Mesh's vertex format.	Added since 0.9.0
BlendAlphaMode	Different ways alpha affects color blending.	Added since 0.10.0
BlendMode	Different ways to do color blending.	Added since 0.2.0
BufferDataUsage	Usage hints for SpriteBatches, Meshes, and GraphicsBuffers to optimize data storage and access.	Added since 0.8.0
CanvasFormat	Canvas texture formats.	Added since 0.9.0

ColorMode	Controls how drawn images are affected by current color.	Added since 0.2.0	Removed in 0.9.0
CompareMode	Different types of stencil test and depth test comparisons.	Added since 0.10.0	
CullMode	How Mesh geometry is culled when rendering.	Added since 11.0	
DrawMode	Controls whether shapes are drawn as an outline, or filled.		
FilterMode	How the image is filtered when scaling.		
GraphicsFeature	Graphics features that can be checked for with love.graphics.getSupported.	Added since 0.8.0	
GraphicsLimit	Types of system-dependent graphics limits.	Added since 0.9.1	
IndexDataType	Vertex map datatype.	Added since 11.0	
LineJoin	Line join style.		
LineStyle	The styles in which lines are drawn.		
MeshDrawMode	How a Mesh's vertices are used when drawing.	Added since 0.9.0	
MipmapMode	Controls whether a Canvas has mipmaps, and its behaviour when it does.	Added since 11.0	
PixelFormat	Pixel formats for Textures, ImageData, and CompressedImageData.	Added since 11.0	
PointStyle	How points should be drawn.		Removed in 0.10.0
StackType	Graphics state stack types used with love.graphics.push.	Added since 0.9.2	
StencilAction	How a stencil function modifies the stencil values of pixels it touches.	Added since 0.10.0	
TextureFormat	Controls the canvas texture format.	Added since 0.9.0	Removed in 0.10.0
TextureType	Types of textures (2D, cubemap, etc.)	Added since 11.0	
VertexAttributeStep	The frequency at which a vertex shader fetches the vertex attribute's data from the Mesh when it's drawn.	Added since 11.0	
VertexWinding	Vertex winding.	Added since 11.0	
WrapMode	How the image wraps inside a large Quad.		

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.applyTransform

love.graphics.applyTransform · revision 21434 · last edited 2026-04-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Applies the given Transform object to the current coordinate transformation.

This effectively multiplies the existing coordinate transformation's matrix with the Transform object's internal matrix to produce the new coordinate transformation.

Function

Synopsis

```
love.graphics.applyTransform( transform )
```

Arguments

`Transform transform`

The Transform object to apply to the current graphics coordinate transform.

Returns

Nothing.

Examples

```
local transform = love.math.newTransform()
transform:translate(200, 0)

function love.draw()
    love.graphics.translate(100, 100)
    love.graphics.rectangle("fill", 0, 0, 50, 50)

    love.graphics.applyTransform(transform)
    love.graphics.rectangle("fill", 0, 0, 50, 50)
end
```

See Also

- love.graphics
- love.math.newTransform
- love.graphics.replaceTransform
- love.graphics.push

- love.graphics.pop

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.arc

love.graphics.arc · revision 29649 · last edited 2025-08-03

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Drawing half a circle
  - Drawing Pacman
  - Drawing a rounded sides rectangle
  - Drawing array of arcs
- Notes
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Draws a filled or unfilled arc at position `(x, y)`. The arc is drawn from `angle1` to `angle2` in radians (https://en.wikipedia.org/wiki/Radian). The `segments` parameter determines how many segments are used to draw the arc. The more segments, the smoother the edge.

Function

Draws an arc using the "pie" ArcType.

Synopsis

```
love.graphics.arc( drawmode, x, y, radius, angle1, angle2, segments )
```

Arguments

`DrawMode drawmode`
: How to draw the arc.

`number x`
: The position of the center along x-axis.

`number y`
: The position of the center along y-axis.

`number radius`
: Radius of the arc.

`number angle1`
: The angle at which the arc begins.

`number angle2`
: The angle at which the arc terminates.

`number segments (10)`
: The number of segments used for drawing the arc.

Returns

Nothing.

Function

Available since LÖVE 0.10.1

This variant is not supported in earlier versions.

Synopsis

```
love.graphics.arc( drawmode, arctype, x, y, radius, angle1, angle2, segments )
```

Arguments

DrawMode drawmode
: How to draw the arc.

ArcType arctype
: The type of arc to draw.

number x
: The position of the center along x-axis.

number y
: The position of the center along y-axis.

number radius
: Radius of the arc.

number angle1
: The angle at which the arc begins.

number angle2
: The angle at which the arc terminates.

number segments (10)
: The number of segments used for drawing the arc.

Returns

Nothing.

Examples

Drawing half a circle

```
function love.draw( )
  love.graphics.arc( "fill", 400, 300, 100, 0, math.pi )
end
```

Drawing Pacman

```
pacwidth = math.pi / 6 -- size of his mouth
function love.draw( )
  love.graphics.setColor( 1, 1, 0 ) -- pacman needs to be yellow
```

```
  love.graphics.arc( "fill", 400, 300, 100, pacwidth, (math.pi * 2) - pacwidth )
end
```

Drawing a rounded sides rectangle

```
quartcircle = math.pi / 2 -- Define 1/4 angle constant
function love.draw( )
  love.graphics.setColor( .7, .7, 1 ) -- Blueish color
  love.graphics.line( 200, 100, 600, 100 )
  love.graphics.line( 200, 300, 600, 300 )
  -- Angles are clockwise, as y value grows when going down
  love.graphics.arc( "line", "open", 200, 200, 100, quartcircle, math.pi+quartcircle )
  love.graphics.arc( "line", "open", 600, 200, 100, -quartcircle, quartcircle)
end
```

Drawing array of arcs

The a is the starting angle (degrees), d is the difference between ending angle and starting angle (degrees), the wite line shows the starting angle:



arcs

```
love.load()
    arcs = {}

    for i = -4, 4 do
        local y = (i+5.5) * 60
        local angle1 = math.rad (i*90)
        for j = -4, 4 do
            local x = (j+8.5) * 120
            local angle2 = math.rad (j*90)
            local arc = {
                i = (i)*90,
                j = (j)*90,
                x=x, y=y,
                angle1=angle1,
                angle2=angle1+angle2,
                radius=25
            }
            table.insert (arcs, arc)
        end
    end
end

function love.draw()
    for i, arc in ipairs (arcs) do
        love.graphics.setColor(1, 1, 0)
        love.graphics.arc ('line', 'open', arc.x, arc.y, arc.radius, arc.angle1, arc.angle2)
        love.graphics.setColor(1, 1, 1)
        love.graphics.line (arc.x, arc.y,
            arc.x+arc.radius*math.cos(arc.angle1), arc.y+arc.radius*math.sin(arc.angle1))
        love.graphics.print ('a: '..arc.i, arc.x-70, arc.y-10)
        love.graphics.print ('d: '..arc.j, arc.x-70, arc.y+4)
```

```
    end
end
```

Notes

The arc is drawn counter clockwise if the starting angle is numerically bigger than the final angle. The arc is drawn clockwise if the final angle is numerically bigger than the starting angle.

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.captureScreenshot

love.graphics.captureScreenshot · revision 22243 · last edited 2026-04-14

On this page

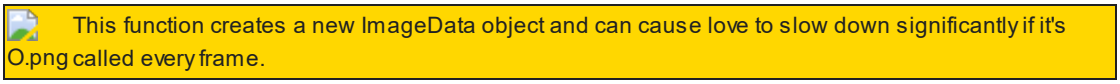
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 11.0

This function replaces love.graphics.newScreenshot.

Creates a screenshot once the current frame is done (after love.draw has finished).

Since this function enqueues a screenshot capture rather than executing it immediately, it can be called from an input callback or love.update and it will still capture all of what's drawn to the screen in that frame.



Function

Capture a screenshot and save it to a file at the end of the current frame.

Synopsis

```
love.graphics.captureScreenshot( filename )
```

Arguments

`string filename`

The filename to save the screenshot to. The encoded image type is determined based on the extension of the filename, and must be one of the ImageFormats.

Returns

Nothing.

Function

Capture a screenshot and call a callback with the generated ImageData at the end of the current frame.

Synopsis

```
love.graphics.captureScreenshot( callback )
```

Arguments

`function callback`
Function which gets called once the screenshot has been captured. An ImageData is passed into the function as its only argument.

Returns

Nothing.

Function

Capture a screenshot and push the generated ImageData to a Channel at the end of the current frame.

Synopsis

```
love.graphics.captureScreenshot( channel )
```

Arguments

`Channel channel`
The Channel to push the generated ImageData to.

Returns

Nothing.

Examples

Create a new screenshot and write it to the save directory.

```
function love.load()
    love.filesystem.setIdentity("screenshot_example")
end

function love.keypressed(key)
    if key == "c" then
        love.graphics.captureScreenshot(os.time() .. ".png")
    end
```

```
end

function love.draw()
    love.graphics.circle("fill", 400, 300, 200)
end
```

See Also

- love.graphics
- ImageData
- ImageData:encode
- Channel

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.checkMode

love.graphics.checkMode · revision 11721 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Removed in LÖVE 0.9.0

This function is not supported in that and later versions.

Checks if a display mode is supported.

Function

Synopsis

```
supported = love.graphics.checkMode( width, height, fullscreen )
```

Arguments

number width
: The display width.

number height
: The display height.

boolean fullscreen (false)
: True to check for fullscreen, false for windowed.

Returns

boolean supported
: True if supported, false if not.

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.circle

love.graphics.circle · revision 28991 · last edited 2023-11-01

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - The effect of the segment argument
- See Also
- Other Languages

Draws a circle.

Function

Synopsis

```
love.graphics.circle( mode, x, y, radius )
```

Arguments

`DrawMode mode`
: How to draw the circle. Has 2 modes: "fill" and "line".

`number x`
: The position of the center along x-axis.

`number y`
: The position of the center along y-axis.

`number radius`
: The radius of the circle.

Returns

Nothing.

Function

Synopsis

```
love.graphics.circle( mode, x, y, radius, segments )
```

Arguments

`DrawMode mode`
: How to draw the circle.

`number x`
: The position of the center along x-axis.

`number y`
: The position of the center along y-axis.

`number radius`
: The radius of the circle.

`number segments`

The number of segments used for drawing the circle. Note: The default variable for the segments parameter varies between different versions of LÖVE.

Returns

Nothing.

Examples

The effect of the segment argument

```
function love.draw()
    love.graphics.setColor(1, 1, 1)
    love.graphics.circle("fill", 300, 300, 50, 100) -- Draw white circle with 100 segments.
    love.graphics.setColor(1, 0, 0)
    love.graphics.circle("fill", 300, 300, 50, 5)   -- Draw red circle with five segments.
end
```

See Also

- love.graphics
- love.graphics.arc
- love.graphics.ellipse

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.clear

love.graphics.clear · revision 21344 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Clear canvas before drawing
- See Also
- Other Languages

Clears the screen or active Canvas to the specified color.

This function is called automatically before love.draw in the default love.run function. See the example in love.run for a typical use of this function.

Note that the scissor area bounds the cleared region.

In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.

In versions prior to 0.10.0, this function clears the screen to the currently set background color instead.

Function

Clears the screen to the background color in 0.9.2 and earlier, or to transparent black (0, 0, 0, 0) in LÖVE 0.10.0 and newer.

Synopsis

```
love.graphics.clear( )
```

Arguments

None.

Returns

Nothing.

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Clears the screen or active Canvas to the specified color.

Synopsis

```
love.graphics.clear( r, g, b, a, clearstencil, cleardepth )
```

Arguments

number r
: The red channel of the color to clear the screen to.

number g
: The green channel of the color to clear the screen to.

number b
: The blue channel of the color to clear the screen to.

number a (1)
: The alpha channel of the color to clear the screen to.

Available since LÖVE 11.0

boolean clearstencil (true)
: Whether to clear the active stencil buffer, if present. It can also be an integer between 0 and 255 to clear the stencil buffer to a specific value.

boolean cleardepth (true)
: Whether to clear the active depth buffer, if present. It can also be a number between 0 and 1 to clear the depth buffer to a specific value.

Returns

Nothing.

Function

Available since LÖVE 0.10.0
This variant is not supported in earlier versions.

Clears multiple active Canvases to different colors, if multiple Canvases are active at once via love.graphics.setCanvas.

Synopsis

```
love.graphics.clear( color, ..., clearstencil, cleardepth )
```

Arguments

table color
: A table in the form of `{r, g, b, a}` containing the color to clear the first active Canvas to.

table ...
: Additional tables for each active Canvas.

Available since LÖVE 11.0

boolean clearstencil (true)
: Whether to clear the active stencil buffer, if present. It can also be an integer between 0 and 255 to clear the stencil buffer to a specific value.

boolean cleardepth (true)
: Whether to clear the active depth buffer, if present. It can also be a number between 0 and 1 to clear the depth buffer to a specific value.

Returns

Nothing.

Notes

A color must be specified for each active Canvas, when this function variant is used.

Function

Available since LÖVE 11.0
This variant is not supported in earlier versions.

Clears the stencil or depth buffers without having to clear the color canvas as well.

Synopsis

```
love.graphics.clear( clearcolor, clearstencil, cleardepth )
```

Arguments

boolean clearcolor
: Whether to clear the active color canvas to transparent black (`0, 0, 0, 0`). Typically this should be set to false with this variant of the function.

boolean clearstencil
: Whether to clear the active stencil buffer, if present. It can also be an integer between 0 and 255 to clear the stencil buffer to a specific value.

boolean cleardepth
: Whether to clear the active depth buffer, if present. It can also be a number between 0 and 1 to clear the depth buffer to a specific value.

Returns

Nothing.

Examples

Clear canvas before drawing

If the c-key is pressed the canvas will be cleared before drawing a new line on the screen.

```
local canvas = love.graphics.newCanvas()
local clear
```

```
function love.update()
    -- Use an anonymous function to draw lines on our canvas.
    canvas:renderTo( function()
        if clear then
            love.graphics.clear() -- Clear the canvas before drawing lines.
        end

        -- Draw lines from the screen's origin to a random x and y coordinate.
        local rx, ry = love.math.random( 0, love.graphics.getWidth() ), love.math.random( 0, love.graphics.getHeight() )
        love.graphics.setColor( love.math.random( ), 0, 0 )
        love.graphics.line( 0, 0, rx, ry )
        love.graphics.setColor( 1, 1, 1 )
    end)
end

function love.draw()
    love.graphics.draw( canvas )
end

function love.keypressed( key )
    if key == "c" then
        clear = not clear
    end
end
```

See Also

- love.graphics.present
- love.graphics.setBackgroundColor
- love.graphics.setScissor
- love.graphics.setCanvas
- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.copyBuffer

love.graphics.copyBuffer · revision 29425 · last edited 2025-01-04

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Copies the contents of one GraphicsBuffer to another.

Function

Synopsis

```
love.graphics.copyBuffer( source, dest, sourceoffset, destoffset, size )
```

Arguments

`GraphicsBuffer source`
: The GraphicsBuffer to copy data from.

`GraphicsBuffer dest`
: The GraphicsBuffer to copy data into.

`number sourceoffset (0)`
: An offset in bytes from the start of the source GraphicsBuffer. Must be a multiple of 4.

`number destoffset (0)`
: An offset in bytes from the start of the destination GraphicsBuffer. Must be a multiple of 4.

`number size (all)`
: An optional size in bytes to copy. All data after the source offset will be copied by default. Must be a multiple of 4.

Returns

Nothing.

Notes

GraphicsBuffers created with the **stream** BufferDataUsage cannot be used as a copy destination.

GraphicsBuffers created with the **readback** BufferDataUsage cannot be used as a copy source.

Copying a GraphicsBuffer to itself is allowed when the source and destination offsets and size do not overlap.

See Also

- love.graphics
- love.graphics.copyBufferToTexture
- love.graphics.copyTextureToBuffer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.discard

love.graphics.discard · revision 16241 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Discards (trashes) the contents of the screen or active Canvas. This is a performance optimization function with niche use cases.

If the active Canvas has just been changed and the "replace" BlendMode is about to be used to draw something which covers the entire screen, calling love.graphics.discard rather than calling love.graphics.clear or doing nothing may improve performance on mobile devices.

On some desktop systems this function may do nothing.



This function effectively replaces the contents of the screen or active Canvas with garbage. Most BlendModes, including the default "alpha" blend mode, blend what's drawn with the contents of the screen - which would cause unexpected glitches if this function is used inappropriately.

Function

Synopsis

```
love.graphics.discard( discardcolor, discardstencil )
```

Arguments

`boolean discardcolor (true)`
: Whether to discard the texture(s) of the active Canvas(es) (the contents of the screen if no Canvas is active.)

`boolean discardstencil (true)`
: Whether to discard the contents of the stencil buffer of the screen / active Canvas.

Returns

Nothing.

Function

Synopsis

```
love.graphics.discard( discardcolors, discardstencil )
```

Arguments

`table discardcolors`

An array containing boolean values indicating whether to discard the texture of each active Canvas, when multiple simultaneous Canvases are active.

`boolean discardstencil (true)`

Whether to discard the contents of the stencil buffer of the screen / active Canvas.

Returns

Nothing.

See Also

- love.graphics
- love.graphics.clear
- Canvas
- love.graphics.setBlendMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.draw

love.graphics.draw · revision 28980 · last edited 2023-10-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Draw an image (the Hamster Ball) at 100 by 100 pixels
  - Draw an image (the Hamster Ball) from the center at 100 by 100 pixels, rotating over time
  - Draw the top half of an image (the Hamster Ball) at 100 by 100 pixels.
  - Draw the flying rocket directed to the mouse pointer
- See Also
- Other Languages

Draws a Drawable object (an Image, Canvas, SpriteBatch, ParticleSystem, Mesh, Text object, or Video) on the screen with optional rotation, scaling and shearing.

Objects are drawn relative to their local coordinate system. The origin is by default located at the top left corner of Image and Canvas objects. All scaling, shearing, and rotation arguments transform the object relative to that point.

It's possible to rotate an object about, for example, its center by offsetting the origin to the center. Angles must be given in radians for rotation. One can also use negative scaling factors to flip the object about its origin.

Note that the offsets are applied before rotation, scaling, or shearing; scaling and shearing are applied before rotation.

The right and bottom edges of the object are shifted at an angle defined by the shearing factors.

When using the default shader anything drawn with this function will be tinted according to the currently selected color. Set it to pure white to preserve the object's original colors.

Function

Synopsis

```
love.graphics.draw( drawable, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

`Drawable drawable`
: A drawable object.

`number x (0)`
: The position to draw the object (x-axis).

`number y (0)`
: The position to draw the object (y-axis).

`number r (0)`
: Orientation (radians).

`number sx (1)`
: Scale factor (x-axis).

`number sy (sx)`
: Scale factor (y-axis).

`number ox (0)`

Origin offset (x-axis).

`number oy (0)`

Origin offset (y-axis).

Available since LÖVE 0.8.0

`number kx (0)`

Shearing factor (x-axis).

`number ky (0)`

Shearing factor (y-axis).

Returns

Nothing.

Function

Available since LÖVE 0.9.0

It has replaced love.graphics.drawq.

Synopsis

```
love.graphics.draw( texture, quad, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

`Texture texture`

A Texture (Image or Canvas) to texture the Quad with.

`Quad quad`

The Quad to draw on screen.

`number x`

The position to draw the object (x-axis).

`number y`

The position to draw the object (y-axis).

`number r (0)`

Orientation (radians).

`number sx (1)`

Scale factor (x-axis).

`number sy (sx)`

Scale factor (y-axis).

`number ox (0)`

Origin offset (x-axis).

`number oy (0)`

Origin offset (y-axis).

`number kx (0)`

Shearing factor (x-axis).

`number ky (0)`

Shearing factor (y-axis).

Returns

Nothing.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
love.graphics.draw( drawable, transform )
```

Arguments

`Drawable drawable`

A drawable object.

`Transform transform`

Transformation object.

Returns

Nothing.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
love.graphics.draw( texture, quad, transform )
```

Arguments

`Texture texture`

A Texture (Image or Canvas) to texture the Quad with.

`Quad quad`

The Quad to draw on screen.

`Transform transform`

Transformation object.

Returns

Nothing.

Examples

Draw an image (the Hamster Ball) at 100 by 100 pixels

```
function love.load()
    hamster = love.graphics.newImage("hamster.png")
end
function love.draw()
    love.graphics.draw(hamster, 100,100)
end
```

Draw an image (the Hamster Ball) from the center at 100 by 100 pixels, rotating over time

```
function love.load()
    hamster = love.graphics.newImage("hamster.png")
end
function love.draw()
    local width  = hamster:getWidth()
    local height = hamster:getHeight()
    local angle  = love.timer.getTime() * 2*math.pi / 2.5 -- Rotate one turn per 2.5 seconds.
    love.graphics.draw(hamster, 100,100, angle, 1,1, width/2, height/2)
end
```

Available since LÖVE 0.9.0

This functionality is not supported in earlier versions.

Draw the top half of an image (the Hamster Ball) at 100 by 100 pixels.

```
function love.load()
    hamster      = love.graphics.newImage("hamster.png")
    local width  = hamster:getWidth()
    local height = hamster:getHeight()
    quad         = love.graphics.newQuad(0,0, width,height/2, width,height)
end
function love.draw()
    love.graphics.draw(hamster,quad, 100,100)
end
```

Draw the flying rocket directed to the mouse pointer

```
function love.load()
    local image = love.graphics.newImage ('rocket.png')
    Rocket ={x=200,y=200,angle=0,image=image,ox=image:getWidth()/2,oy=image:getHeight()/2}
end
function love.update(dt)
    local mx, my = love.mouse.getPosition()
    local dx, dy = mx-Rocket.x, my-Rocket.y
    Rocket.angle = math.atan2 (dy, dx)
```

```
    Rocket.x = Rocket.x + dt*dx
    Rocket.y = Rocket.y + dt*dy
end
function love.draw()
    love.graphics.draw (Rocket.image, Rocket.x, Rocket.y, Rocket.angle, 1, 1, Rocket.ox, Rocket.oy)
end
```

See Also

- love.graphics.drawInstanced
- love.graphics.drawLayer
- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.drawInstanced

love.graphics.drawInstanced · revision 23644 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Use vertex attribute instancing to draw many triangles in a single draw call
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Draws many instances of a Mesh with a single draw call, using hardware geometry instancing.

Each instance can have unique properties (positions, colors, etc.) but will not by default unless a custom Shader along with either per-instance vertex attributes or the `love_InstanceID` GLSL 3 vertex shader variable is used, otherwise they will all render at the same position on top of each other.

Instancing is not supported by some older GPUs that are only capable of using OpenGL ES 2 or OpenGL 2. Use love.graphics.getSupported to check.

Function

Synopsis

```
love.graphics.drawInstanced( mesh, instancecount, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

`Mesh mesh`
: The mesh to render.

`number instancecount`
: The number of instances to render.

`number x (0)`
: The position to draw the instances (x-axis).

`number y (0)`
: The position to draw the instances (y-axis).

`number r (0)`
: Orientation (radians).

`number sx (1)`
: Scale factor (x-axis).

`number sy (sx)`
: Scale factor (y-axis).

`number ox (0)`
: Origin offset (x-axis).

`number oy (0)`
: Origin offset (y-axis).

`number kx (0)`
: Shearing factor (x-axis).

`number ky (0)`

Shearing factor (y-axis).

Returns

Nothing.

Function

Synopsis

```
love.graphics.drawInstanced( mesh, instancecount, transform )
```

Arguments

`Mesh mesh`
: The mesh to render.

`number instancecount`
: The number of instances to render.

`Transform transform`
: A transform object.

Returns

Nothing.

Examples

Use vertex attribute instancing to draw many triangles in a single draw call

```
-- A simple small triangle with the default position, texture coordinate, and color vertex attributes.
local vertices = {
    {0, 0,  0,0, 1.0,0.2,0.2,1.0},
    {20,0,  0,0, 0.2,1.0,0.2,1.0},
    {20,20, 0,0, 0.2,0.2,1.0,1.0},
}

local mesh = love.graphics.newMesh(vertices, "triangles", "static")

-- Unique positions for each instance that will be rendered.
local instancepositions = {}
for y=0, love.graphics.getHeight()-1, 30 do
    for x = 0, love.graphics.getWidth()-1, 30 do
        local pos = {x, y}
        table.insert(instancepositions, pos)
    end
end

-- Create a mesh containing the per-instance position data.
-- It won't be drawn directly, but it will be referenced by the triangle's mesh.
local instancemesh = love.graphics.newMesh({{"InstancePosition", "float", 2}}, instancepositions, nil, "static")

-- When the triangle's mesh is rendered, the vertex shader will pull in a different
-- value of the InstancePosition attribute for each instance, instead of for each vertex.
mesh:attachAttribute("InstancePosition", instancemesh, "perinstance")

-- Vertex shader which uses the InstancePosition vertex attribute.
```

```
local shader = love.graphics.newShader[[
attribute vec2 InstancePosition;

vec4 position(mat4 transform_projection, vec4 vertex_position)
{
    vertex_position.xy += InstancePosition;
    return transform_projection * vertex_position;
}
]]

function love.draw()
    love.graphics.setShader(shader)

    -- Draw the mesh many times in one draw call, using instancing.
    local instancecount = #instancepositions
    love.graphics.drawInstanced(mesh, instancecount, 0, 0)
end
```

See Also

- love.graphics
- Mesh
- Mesh:attachAttribute
- love.graphics.newShader
- GraphicsFeature

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.drawLayer

love.graphics.drawLayer · revision 22245 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Notes
- Examples
  - Draw multiple layers of an Array Image
  - Use a custom shader with love.graphics.drawLayer
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Draws a layer of an Array Texture.

Function

Draws a layer of an Array Texture.

Synopsis

```
love.graphics.drawLayer( texture, layerindex, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

`Texture texture`
: The Array Texture to draw.

`number layerindex`
: The index of the layer to use when drawing.

`number x (0)`
: The position to draw the texture (x-axis).

`number y (0)`
: The position to draw the texture (y-axis).

`number r (0)`
: Orientation (radians).

`number sx (1)`
: Scale factor (x-axis).

`number sy (sx)`
: Scale factor (y-axis).

`number ox (0)`
: Origin offset (x-axis).

`number oy (0)`
: Origin offset (y-axis).

`number kx (0)`
: Shearing factor (x-axis).

`number ky (0)`

Shearing factor (y-axis).

Returns

Nothing.

Function

Draws a layer of an Array Texture using the specified Quad.

Synopsis

```
love.graphics.drawLayer( texture, layerindex, quad, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

`Texture texture`
: The Array Texture to draw.

`number layerindex`
: The index of the layer to use when drawing.

`Quad quad`
: The subsection of the texture's layer to use when drawing.

`number x (0)`
: The position to draw the texture (x-axis).

`number y (0)`
: The position to draw the texture (y-axis).

`number r (0)`
: Orientation (radians).

`number sx (1)`
: Scale factor (x-axis).

`number sy (sx)`
: Scale factor (y-axis).

`number ox (0)`
: Origin offset (x-axis).

`number oy (0)`
: Origin offset (y-axis).

`number kx (0)`
: Shearing factor (x-axis).

`number ky (0)`

Shearing factor (y-axis).

Returns

Nothing.

Notes

The specified layer index overrides any layer index set on the Quad via Quad:setLayer.

Function

Draws a layer of an Array Texture using the specified Transform.

Synopsis

```
love.graphics.drawLayer( texture, layerindex, transform )
```

Arguments

`Texture texture`
: The Array Texture to draw.

`number layerindex`
: The index of the layer to use when drawing.

`Transform transform`
: A transform object.

Returns

Nothing.

Function

Draws a layer of an Array Texture using the specified Quad and Transform.

Synopsis

```
love.graphics.drawLayer( texture, layerindex, quad, transform )
```

Arguments

`Texture texture`
: The Array Texture to draw.

`number layerindex`
: The index of the layer to use when drawing.

`Quad quad`
: The subsection of the texture's layer to use when drawing.

`Transform transform`

A transform object.

Returns

Nothing.

Notes

The specified layer index overrides any layer index set on the Quad via Quad:setLayer.

Notes

In order to use an Array Texture or other non-2D texture types as the main texture in a custom Shader, the void effect() variant must be used in the pixel shader, and MainTex must be declared as an ArrayImage or sampler2DArray like so: `uniform ArrayImage MainTex;`.

Examples

Draw multiple layers of an Array Image

```
function love.load()
    local sprites = {"sprite1.png", "sprite2.png"}
    image = love.graphics.newArrayImage(sprites)
end

function love.draw()
    love.graphics.drawLayer(image, 1, 50, 50)
    love.graphics.drawLayer(image, 2, 250, 50)
end
```

Use a custom shader with love.graphics.drawLayer

```
shader = love.graphics.newShader[[
uniform ArrayImage MainTex;

void effect() {
    // Texel uses a third component of the texture coordinate for the layer index, when an Array Texture is passed in.
    // love sets up the texture coordinates to contain the layer index specified in love.graphics.drawLayer, when
    // rendering the Array Texture.
    love_PixelColor = Texel(MainTex, VaryingTexCoord.xyz) * VaryingColor;
}
]]

function love.load()
    local sprites = {"sprite1.png", "sprite2.png"}
    image = love.graphics.newArrayImage(sprites)
end

function love.draw()
    love.graphics.setShader(shader)
    love.graphics.drawLayer(image, 1, 50, 50)
```

```
    love.graphics.drawLayer(image, 2, 250, 50)
end
```

See Also

- love.graphics
- love.graphics.newArrayImage
- love.graphics.newCanvas
- love.graphics.newShader
- TextureType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.drawq

love.graphics.drawq · revision 23203 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Draw the top half of an image (the Hamster Ball) at 100 by 100 pixels.
- See Also
- Other Languages

Removed in LÖVE 0.9.0

It has been merged into love.graphics.draw.

Draw a Quad with the specified Image on screen.

Function

Synopsis

```
love.graphics.drawq( image, quad, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

`Image image`
: An image to texture the quad with.

`Quad quad`
: The quad to draw on screen.

`number x`
: The position to draw the object (x-axis).

`number y`
: The position to draw the object (y-axis).

`number r (0)`
: Orientation (radians).

`number sx (1)`
: Scale factor (x-axis).

`number sy (sx)`
: Scale factor (y-axis).

`number ox (0)`
: Origin offset (x-axis).

`number oy (0)`
: Origin offset (y-axis).

Available since LÖVE 0.8.0

`number kx (0)`
: Shearing factor (x-axis).

`number ky (0)`

Shearing factor (y-axis).

Returns

Nothing.

Function

Synopsis

```
love.graphics.drawq( canvas, quad, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

Canvas canvas
A canvas to texture the quad with.

Quad quad
The quad to draw on screen.

number x
The position to draw the object (x-axis).

number y
The position to draw the object (y-axis).

number r (0)
Orientation (radians).

number sx (1)
Scale factor (x-axis).

number sy (sx)
Scale factor (y-axis).

number ox (0)
Origin offset (x-axis).

number oy (0)
Origin offset (y-axis).

Available since LÖVE 0.8.0

number kx (0)
Shearing factor (x-axis).

number ky (0)

Shearing factor (y-axis).

Returns

Nothing.

Examples

Draw the top half of an image (the Hamster Ball) at 100 by 100 pixels.

```
image = love.graphics.newImage("hamster.png")
quad = love.graphics.newQuad(0, 0, 128, 64, image:getWidth(), image:getHeight())

function love.draw()
    love.graphics.drawq(image, quad, 100, 100)
end
```

See Also

- love.graphics
- love.graphics.draw

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.ellipse

love.graphics.ellipse · revision 27919 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - The effect of the segment argument
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Draws an ellipse.

Function

Synopsis

```
love.graphics.ellipse( mode, x, y, radiusx, radiusy, segments )
```

Arguments

DrawMode mode
: How to draw the ellipse.

number x
: The position of the center along x-axis.

number y
: The position of the center along y-axis.

number radiusx
: The radius of the ellipse along the x-axis (half the ellipse's width).

number radiusy
: The radius of the ellipse along the y-axis (half the ellipse's height).

number segments (nil)
: The number of segments used for drawing the ellipse (automatically guessed when not set).

Returns

Nothing.

Examples

The effect of the segment argument

```
function love.draw()
    love.graphics.setColor(1, 1, 1)
    love.graphics.ellipse("fill", 300, 300, 75, 50, 100) -- Draw white ellipse with 100 segments.

    love.graphics.setColor(1, 0, 0)
```

```
    love.graphics.ellipse("fill", 300, 300, 75, 50, 5)   -- Draw red ellipse with five segments.
end
```

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.flushBatch

love.graphics.flushBatch · revision 22244 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Immediately renders any pending automatically batched draws.

LÖVE will call this function internally as needed when most state is changed, so it is not necessary to manually call it.

The current batch will be automatically flushed by love.graphics state changes (except for the transform stack and the current color), as well as Shader:send and methods on Textures which change their state. Using a different Image in consecutive love.graphics.draw calls will also flush the current batch.

SpriteBatches, ParticleSystems, Meshes, and Text objects do their own batching and do not affect automatic batching of other draws, aside from flushing the current batch when they're drawn.

Function

Synopsis

```
love.graphics.flushBatch( )
```

Arguments

None.

Returns

Nothing.

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getBackgroundColor

love.graphics.getBackgroundColor · revision 21262 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the current background color.

In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.

Function

Synopsis

```
r, g, b, a = love.graphics.getBackgroundColor( )
```

Arguments

None.

Returns

number r
: The red component (0-1).

number g
: The green component (0-1).

number b
: The blue component (0-1).

number a
: The alpha component (0-1).

See Also

- love.graphics
- love.graphics.setBackgroundColor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getBlendMode

love.graphics.getBlendMode · revision 24729 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.2.0

This function is not supported in earlier versions.

Gets the blending mode.

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Synopsis

```
mode, alphamode = love.graphics.getBlendMode( )
```

Arguments

None.

Returns

`BlendMode mode`

The current blend mode.

`BlendAlphaMode alphamode`

The current blend alpha mode – it determines how the alpha of drawn objects affects blending.

Function

Removed in LÖVE 0.10.0

This variant is not supported in that and later versions.

Synopsis

```
mode = love.graphics.getBlendMode( )
```

Arguments

None.

Returns

`BlendMode mode`

The current blend mode.

See Also

- love.graphics
- love.graphics.setBlendMode
- BlendMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getCanvas

love.graphics.getCanvas · revision 29376 · last edited 2026-04-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Gets the current target Canvas.

Function

Synopsis

```
canvas, ... = love.graphics.getCanvas( )
```

Arguments

None.

Returns

Canvas `canvas`

The Canvas set by setCanvas. Returns `nil` if drawing to the screen.

Canvas `...`

Additional canvases if more than one was set.

See Also

- love.graphics
- Canvas
- love.graphics.setCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getCanvasFormats

love.graphics.getCanvasFormats · revision 25843 · last edited 2026-04-17

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Create a canvas with the format 'rgba16f' if it is supported
  - Display a list of the canvas formats on the screen
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets the available Canvas formats, and whether each is supported.

Function

Synopsis

```
formats = love.graphics.getCanvasFormats( )
```

Arguments

None.

Returns

`table formats`

A table containing PixelFormats as keys, and a boolean indicating whether the format is supported as values. Not all systems support all formats.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
formats = love.graphics.getCanvasFormats( readable )
```

Arguments

`boolean readable`

If true, the returned formats will only be indicated as supported if love.graphics.newCanvas will work with the readable flag set to true for that format, and vice versa if the parameter is false.

Returns

`table formats`

A table containing PixelFormats as keys, and a boolean indicating whether the format is supported as values (taking into account the readable parameter). Not all systems support all formats.

Examples

Create a canvas with the format 'rgba16f' if it is supported

```
local formats = love.graphics.getCanvasFormats()
if formats.rgba16f then
    canvas = love.graphics.newCanvas(800, 600, "rgba16f")
else
    -- The 'rgba16f' format isn't supported. We could have some fallback code, or trigger a message telling the user
their system is unsupported.
end
```

Display a list of the canvas formats on the screen

```
canvasformats = love.graphics.getCanvasFormats()

function love.draw()
    local y = 0
    for formatname, formatsupported in pairs(canvasformats) do
        local str = string.format("Supports format '%s': %s", formatname, tostring(formatsupported))
        love.graphics.print(str, 10, y)
        y = y + 20
    end
end
```

See Also

- love.graphics
- PixelFormat
- love.graphics.newCanvas
- Canvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getCaption

love.graphics.getCaption · revision 17403 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

Moved to the love.window module as love.window.getTitle.

Gets the window caption.

Function

Synopsis

```
caption = love.graphics.getCaption( )
```

Arguments

None.

Returns

`string caption`

The current window caption.

See Also

- love.graphics
- love.graphics.setCaption

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getColor

love.graphics.getColor · revision 21260 · last edited 2026-04-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Gets the current color.

In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.

Function

Synopsis

```
r, g, b, a = love.graphics.getColor( )
```

Arguments

None.

Returns

number r
: The red component (0-1).

number g
: The green component (0-1).

number b
: The blue component (0-1).

number a
: The alpha component (0-1).

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getColorMask

love.graphics.getColorMask · revision 11680 · last edited 2026-04-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the active color components used when drawing. Normally all 4 components are active unless love.graphics.setColorMask has been used.

The color mask determines whether individual components of the colors of drawn objects will affect the color of the screen. They affect love.graphics.clear and Canvas:clear as well.

Function

Synopsis

```
r, g, b, a = love.graphics.getColorMask( )
```

Arguments

None.

Returns

`boolean r`
: Whether the red color component is active when rendering.

`boolean g`
: Whether the green color component is active when rendering.

`boolean b`
: Whether the blue color component is active when rendering.

`boolean a`
: Whether the alpha color component is active when rendering.

See Also

- love.graphics
- love.graphics.setColorMask

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getColorMode

love.graphics.getColorMode · revision 15933 · last edited 2026-04-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.2.0 and removed in LÖVE 0.9.0

This function is not supported in earlier or later versions.

Gets the color mode (which controls how images are affected by the current color).

Function

Synopsis

```
mode = love.graphics.getColorMode( )
```

Arguments

None.

Returns

`ColorMode mode`

The current color mode.

See Also

- love.graphics
- love.graphics.setColorMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getCompressedImageFormats

love.graphics.getCompressedImageFormats · revision 21256 · last edited 2026-04-17

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Display a list of the compressed image formats on the screen
- See Also
- Other Languages

Available since LÖVE 0.9.2 and removed in LÖVE 11.0

It has been replaced by love.graphics.getImageFormats.

Gets the available compressed image formats, and whether each is supported.

Function

Synopsis

```
formats = love.graphics.getCompressedImageFormats( )
```

Arguments

None.

Returns

table formats

A table containing CompressedImageFormats as keys, and a boolean indicating whether the format is supported as values. Not all systems support all formats.

Examples

Display a list of the compressed image formats on the screen

```
formats = love.graphics.getCompressedImageFormats()

function love.draw()
    local y = 0
    for formatname, formatsupported in pairs(formats) do
        local str = string.format("Supports format '%s': %s", formatname, tostring(formatsupported))
        love.graphics.print(str, 10, y)
        y = y + 20
    end
end
```

See Also

- love.graphics
- CompressedImageFormat
- love.graphics.newImage
- love.image.newCompressedData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getDefaultFilter

love.graphics.getDefaultFilter · revision 13535 · last edited 2026-04-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from love.graphics.getDefaultImageFilter.

Returns the default scaling filters used with Images, Canvases, and Fonts.

Function

Synopsis

```
min, mag, anisotropy = love.graphics.getDefaultFilter( )
```

Arguments

None.

Returns

`FilterMode min`
: Filter mode used when scaling the image down.

`FilterMode mag`
: Filter mode used when scaling the image up.

`number anisotropy`
: Maximum amount of Anisotropic Filtering used.

See Also

- love.graphics
- love.graphics.setDefaultFilter
- FilterMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getDefaultImageFilter

love.graphics.getDefaultImageFilter · revision 15934 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.9.0

It has been renamed to love.graphics.getDefaultFilter.

Returns the default scaling filters.

Function

Synopsis

```
min, mag = love.graphics.getDefaultImageFilter( )
```

Arguments

None.

Returns

`FilterMode min`

Filter mode used when scaling the image down.

`FilterMode mag`

Filter mode used when scaling the image up.

See Also

- love.graphics
- love.graphics.setDefaultImageFilter

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getDepthMode

love.graphics.getDepthMode · revision 21282 · last edited 2026-04-14

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the current depth test mode and whether writing to the depth buffer is enabled.

This is low-level functionality designed for use with custom vertex shaders and Meshes with custom vertex attributes. No higher level APIs are provided to set the depth of 2D graphics such as shapes, lines, and Images.


Depth testing and depth writes will have no effect unless the *depth* field is set to *true* in a table passed to love.graphics.setCanvas, or a custom Canvas with a depth PixelFormat is set in the *depthstencil* field in a table passed to setCanvas.


Writing to the depth buffer is generally incompatible with rendering alpha-blended sprites / images. By default depth is determined by geometry (vertices) instead of texture alpha values, the depth buffer only stores a single depth value per pixel, and alpha blending **requires** back-to-front rendering for blending to be correct.

Function

Synopsis

```
comparemode, write = love.graphics.getDepthMode( )
```

Arguments

None.

Returns

`CompareMode comparemode`
: Depth comparison mode used for depth testing.

`boolean write`
: Whether to write update / write values to the depth buffer when rendering.

See Also

- love.graphics
- love.graphics.setDepthMode
- love.graphics.setCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getDimensions

love.graphics.getDimensions · revision 13028 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the width and height in pixels of the window.

Function

Synopsis

```
width, height = love.graphics.getDimensions( )
```

Arguments

None.

Returns

number width
: The width of the window.

number height
: The height of the window.

See Also

- love.graphics
- love.graphics.getWidth
- love.graphics.getHeight

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getDPIScale

love.graphics.getDPIScale · revision 22231 · last edited 2026-04-17

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the DPI scale factor of the window.

The DPI scale factor represents relative pixel density. The pixel density inside the window might be greater (or smaller) than the "size" of the window. For example on a retina screen in Mac OS X with the `highdpi` window flag enabled, the window may take up the same physical size as an 800x600 window, but the area inside the window uses 1600x1200 pixels. `love.graphics.getDPIScale()` would return `2` in that case.

The love.window.fromPixels and love.window.toPixels functions can also be used to convert between units.

The `highdpi` window flag must be enabled to use the full pixel density of a Retina screen on Mac OS X and iOS. The flag currently does nothing on Windows and Linux, and on Android it is effectively always enabled.

Function

Synopsis

```
scale = love.graphics.getDPIScale( )
```

Arguments

None.

Returns

`number scale`

The pixel scale factor associated with the window.

Notes

The units of love.graphics.getWidth, love.graphics.getHeight, love.mouse.getPosition, mouse events, love.touch.getPosition, and touch events are always in DPI-scaled units rather than pixels. In LÖVE 0.10 and older they were in pixels.

See Also

- love.graphics
- love.graphics.getDimensions
- love.graphics.getPixelDimensions
- love.window.toPixels
- love.window.fromPixels
- love.window.setMode

- Config Files

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getFont

love.graphics.getFont · revision 28086 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Gets the current Font object.

Function

Synopsis

```
font = love.graphics.getFont( )
```

Arguments

None.

Returns

`Font font`
: The current Font. Automatically creates and sets the default font, if none is set yet.

Notes

On version before 0.9.0, nil is returned if there are no fonts set beforehand.

See Also

- love.graphics
- love.graphics.setFont
- Font:getHeight
- Font:getWidth
- love.graphics.print

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getFrontFaceWinding

love.graphics.getFrontFaceWinding · revision 22240 · last edited 2026-04-14

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets whether triangles with clockwise- or counterclockwise-ordered vertices are considered front-facing.

This is designed for use in combination with Mesh face culling. Other love.graphics shapes, lines, and sprites are not guaranteed to have a specific winding order to their internal vertices.

Function

Synopsis

```
winding = love.graphics.getFrontFaceWinding( )
```

Arguments

None.

Returns

`VertexWinding winding`

The winding mode being used. The default winding is counterclockwise ("ccw").

See Also

- love.graphics
- love.graphics.setFrontFaceWinding
- love.graphics.setMeshCullMode
- Mesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getHeight

love.graphics.getHeight · revision 14027 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.2.1

This function is not supported in earlier versions.

Gets the height in pixels of the window.

Function

Synopsis

```
height = love.graphics.getHeight( )
```

Arguments

None.

Returns

number height

The height of the window.

See Also

- love.graphics
- love.graphics.getWidth
- love.graphics.getDimensions

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getImageFormats

love.graphics.getImageFormats · revision 22233 · last edited 2026-04-19

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Display a list of the raw and compressed pixel formats usable with Images on the screen
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the raw and compressed pixel formats usable for Images, and whether each is supported.

Function

Synopsis

```
formats = love.graphics.getImageFormats( )
```

Arguments

None.

Returns

table formats

A table containing PixelFormats as keys, and a boolean indicating whether the format is supported as values. Not all systems support all formats.

Examples

Display a list of the raw and compressed pixel formats usable with Images on the screen

```
formats = love.graphics.getImageFormats()

function love.draw()
    local y = 0
    for formatname, formatsupported in pairs(formats) do
        local str = string.format("Supports format '%s': %s", formatname, tostring(formatsupported))
        love.graphics.print(str, 10, y)
        y = y + 20
    end
end
```

See Also

- love.graphics
- PixelFormat
- love.graphics.newImage
- love.image.newImageData

- love.image.newCompressedData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getLineJoin

love.graphics.getLineJoin · revision 12157 · last edited 2026-04-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Gets the line join style.

Function

Synopsis

```
join = love.graphics.getLineJoin( )
```

Arguments

Nothing.

Returns

`LineJoin join`
: The LineJoin style.

See Also

- love.graphics
- love.graphics.setLineJoin

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getLineStipple

love.graphics.getLineStipple · revision 10061 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This function is not supported in that and later versions.

Gets the current line stipple.

Function

Synopsis

```
pattern, repeat = love.graphics.getLineStipple( )
```

Arguments

None.

Returns

number pattern

The 16-bit stipple pattern.

number repeat

The repeat factor.

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getLineStyle

love.graphics.getLineStyle · revision 24722 · last edited 2026-04-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.3.2

This function is not supported in earlier versions.

Gets the line style.

Function

Synopsis

```
style = love.graphics.getLineStyle( )
```

Arguments

None.

Returns

`LineStyle style`

The current line style.

See Also

- love.graphics
- love.graphics.setLineStyle

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getLineWidth

love.graphics.getLineWidth · revision 24724 · last edited 2026-04-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.3.2

This function is not supported in earlier versions.

Gets the current line width.

Function

Synopsis

```
width = love.graphics.getLineWidth( )
```

Arguments

None.

Returns

number width

The current line width.

Notes

This function does not work in 0.8.0, but has been fixed in version 0.9.0. Use the following snippet to circumvent this in 0.8.0;

```
love.graphics._getLineWidth = love.graphics.getLineWidth
love.graphics._setLineWidth = love.graphics.setLineWidth
function love.graphics.getLineWidth() return love.graphics.varlinewidth or 1 end
function love.graphics.setLineWidth(w) love.graphics.varlinewidth = w; return love.graphics._setLineWidth(w) end
```

See Also

- love.graphics
- love.graphics.setLineWidth

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getMaxImageSize

love.graphics.getMaxImageSize · revision 20155 · last edited 2026-04-19

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.0 and removed in LÖVE 0.10.0

Use love.graphics.getSystemLimits instead.

Gets the max supported width or height of Images and Canvases.

Attempting to create an Image with a width *or* height greater than this number will create a checkerboard-patterned image instead. Doing the same for a canvas will result in an error.

The returned number depends on the system running the code. It is safe to assume it will never be less than 1024 and will almost always be 2048 or greater.

Function

Synopsis

```
size = love.graphics.getMaxImageSize( )
```

Arguments

None.

Returns

`number size`

The max supported width or height of Images and Canvases.

Notes

There is an online database (http://feedback.wildfiregames.com/report/opengl/feature/GL_MAX_TEXTURE_SIZE) which has collected info about the max image size for various systems.

See Also

- love.graphics
- Image
- Canvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

love.graphics.getMaxPointSize

love.graphics.getMaxPointSize · revision 20156 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Removed in LÖVE 0.10.0

Use love.graphics.getSystemLimits instead.

Gets the max supported point size.

Function

Synopsis

```
size = love.graphics.getMaxPointSize( )
```

Arguments

None.

Returns

number size

The max supported point size.

Examples

Draws a point at the maximum supported size.

```
function love.draw()
    local max = love.graphics.getMaxPointSize();
    love.graphics.setPointSize(max);
    love.graphics.point(100, 100);
end
```

See Also

- love.graphics
- love.graphics.point
- love.graphics.getPointSize
- love.graphics.setPointSize

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getMeshCullMode

love.graphics.getMeshCullMode · revision 26737 · last edited 2026-04-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets whether back-facing triangles in a Mesh are culled.

Mesh face culling is designed for use with low level custom hardware-accelerated 3D rendering via custom vertex attributes on Meshes, custom vertex shaders, and depth testing with a depth buffer.

Function

Synopsis

```
mode = love.graphics.getMeshCullMode( )
```

Arguments

None.

Returns

`CullMode mode`

The Mesh face culling mode in use (whether to render everything, cull back-facing triangles, or cull front-facing triangles).

See Also

- love.graphics
- love.graphics.setMeshCullMode
- love.graphics.setFrontFaceWinding
- love.graphics.setDepthMode
- Mesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getMode

love.graphics.getMode · revision 15943 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.9.0

Moved to the love.window module as love.window.getMode.

Returns the current display mode.

Function

Synopsis

```
width, height, fullscreen, vsync, fsaa = love.graphics.getMode( )
```

Arguments

None.

Returns

number width
: Display width.

number height
: Display height.

boolean fullscreen
: Fullscreen (true) or windowed (false).

boolean vsync
: True if vertical sync is enabled or false if disabled.

number fsaa
: The number of FSAA samples.

See Also

- love.graphics
- love.graphics.setMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getModes

love.graphics.getModes · revision 11380 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Format of the returned table
  - Sort table to ensure it is in order
- See Also
- Other Languages

Removed in LÖVE 0.9.0

Moved to the love.window module as love.window.getFullscreenModes.

Gets a list of supported fullscreen modes.

Function

Synopsis

```
modes = love.graphics.getModes( )
```

Arguments

None.

Returns

table modes

A table of width/height pairs. (Note that this may not be in order.)

Examples

Format of the returned table

```
modes = love.graphics.getModes()

-- modes = {
--   { width = 320, height = 240 },
--   { width = 640, height = 480 },
--   { width = 800, height = 600 },
--   { width = 1024, height = 768 },
```

```
--  ...
-- }
```

Sort table to ensure it is in order

```
modes = love.graphics.getModes()
table.sort(modes, function(a, b) return a.width*a.height < b.width*b.height end)   -- sort from smallest to largest
```

See Also

- love.graphics
- love.graphics.checkMode
- love.graphics.setMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getPixelDimensions

love.graphics.getPixelDimensions · revision 22232 · last edited 2026-04-17

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the width and height in pixels of the window.

love.graphics.getDimensions gets the dimensions of the window in units scaled by the screen's DPI scale factor, rather than pixels. Use getDimensions for calculations related to drawing to the screen and using the graphics coordinate system (calculating the center of the screen, for example), and getPixelDimensions only when dealing specifically with underlying pixels (pixel-related calculations in a pixel Shader, for example).

Function

Synopsis

```
pixelwidth, pixelheight = love.graphics.getPixelDimensions( )
```

Arguments

None.

Returns

number pixelwidth
: The width of the window in pixels.

number pixelheight
: The width of the window in pixels.

See Also

- love.graphics
- love.graphics.getPixelWidth
- love.graphics.getPixelHeight

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getPixelEffect

love.graphics.getPixelEffect · revision 15936 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.9.0

It has been renamed to love.graphics.getShader.

Returns the current PixelEffect. Returns nil if none is set.

Function

Synopsis

```
pe = love.graphics.getPixelEffect( )
```

Arguments

None.

Returns

`PixelEffect pe`

The current PixelEffect.

See Also

- love.graphics
- love.graphics.setPixelEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getPixelHeight

love.graphics.getPixelHeight · revision 22230 · last edited 2026-04-17

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the height in pixels of the window.

The graphics coordinate system and love.graphics.getHeight use units scaled by the screen's DPI scale factor, rather than raw pixels. Use getHeight for calculations related to drawing to the screen and using the coordinate system (calculating the center of the screen, for example), and getPixelHeight only when dealing specifically with underlying pixels (pixel-related calculations in a pixel Shader, for example).

Function

Synopsis

```
pixelheight = love.graphics.getPixelHeight( )
```

Arguments

None.

Returns

`number pixelheight`

The height of the window in pixels.

See Also

- love.graphics
- love.graphics.getPixelWidth
- love.graphics.getPixelDimensions

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getPixelWidth

love.graphics.getPixelWidth · revision 23471 · last edited 2026-04-17

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the width in pixels of the window.

The graphics coordinate system and love.graphics.getWidth use units scaled by the screen's DPI scale factor, rather than raw pixels. Use getWidth for calculations related to drawing to the screen and using the coordinate system (calculating the center of the screen, for example), and getPixelWidth only when dealing specifically with underlying pixels (pixel-related calculations in a pixel Shader, for example).

Function

Synopsis

```
pixelwidth = love.graphics.getPixelWidth( )
```

Arguments

None.

Returns

`number pixelwidth`

The width of the window in pixels.

See Also

- love.graphics
- love.graphics.getPixelHeight
- love.graphics.getPixelDimensions

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getPointSize

love.graphics.getPointSize · revision 14361 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Gets the point size.

Function

Synopsis

```
size = love.graphics.getPointSize( )
```

Arguments

None.

Returns

number size
The current point size.

Examples

Increase the point size by 1 by using love.graphics.setPointSize.

```
function love.draw()
    local s = love.graphics.getPointSize() + 1;
    love.graphics.setPointSize(s);
    love.graphics.point(100, 100);
end
```

See Also

- love.graphics
- love.graphics.point
- love.graphics.setPointSize
- love.graphics.getMaxPointSize

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getPointStyle

love.graphics.getPointStyle · revision 15733 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Removed in LÖVE 0.10.0

This function is not supported in that and later versions.

Gets the current point style.

Function

Synopsis

```
style = love.graphics.getPointStyle( )
```

Arguments

None.

Returns

`PointStyle style`

The current point style.

Examples

Toggle between point styles with the help of love.graphics.setPointStyle.

```
if love.graphics.getPointStyle() == "rough" then
   love.graphics.setPointStyle("smooth")
else
   love.graphics.setPointStyle("rough")
end
```

See Also

- love.graphics
- love.graphics.point
- love.graphics.setPointStyle

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getRendererInfo

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets information about the system's video card and drivers.



Almost everything returned by this function is highly dependent on the system running the code and should probably not be used to make run-time decisions.

Function

Synopsis

```
name, version, vendor, device = love.graphics.getRendererInfo( )
```

Arguments

None.

Returns

`string name`
: The name of the renderer, e.g. "OpenGL" or "OpenGL ES".

`string version`
: The version of the renderer with some extra driver-dependent version info, e.g. "2.1 INTEL-8.10.44".

`string vendor`
: The name of the graphics card vendor, e.g. "Intel Inc".

`string device`
: The name of the graphics card, e.g. "Intel HD Graphics 3000 OpenGL Engine".

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getScissor

love.graphics.getScissor · revision 29642 · last edited 2026-04-14

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.4.0

This function is not supported in earlier versions.

Gets the current scissor box.

Function

Synopsis

```
x, y, width, height = love.graphics.getScissor( )
```

Arguments

None.

Returns

number x
: The x-component of the top-left point of the box.

number y
: The y-component of the top-left point of the box.

number width
: The width of the box.

number height
: The height of the box.

If there is no box, it returns nil instead.

See Also

- love.graphics
- love.graphics.setScissor
- love.graphics.intersectScissor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getShader

love.graphics.getShader · revision 11079 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from love.graphics.getPixelEffect.

Gets the current Shader. Returns nil if none is set.

Function

Synopsis

```
shader = love.graphics.getShader( )
```

Arguments

None.

Returns

`Shader shader`

The currently active Shader, or nil if none is set.

See Also

- love.graphics
- love.graphics.setShader

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getStackDepth

love.graphics.getStackDepth · revision 27996 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the current depth of the transform / state stack (the number of pushes without corresponding pops).

Function

Synopsis

```
depth = love.graphics.getStackDepth( )
```

Arguments

None.

Returns

number depth

The current depth of the transform and state love.graphics stack.

Examples

```
local function popWholeStack()
    for i = 1, love.graphics.getStackDepth() do
        love.graphics.pop()
    end
end

function love.draw()
    love.graphics.push() ; love.graphics.push() ; love.graphics.push()
    popWholeStack()
end
```

See Also

- love.graphics
- love.graphics.push

- love.graphics.pop

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getStats

love.graphics.getStats · revision 29302 · last edited 2026-04-19

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - display the estimated amount of video memory used for textures
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets performance-related rendering statistics.


The per-frame metrics (drawcalls, canvasswitches, shaderswitches) are reset by love.graphics.present, which for the default implementation of love.run is called right after the execution of love.draw. Therefore this function should probably be called at the end of love.draw.

Function

Synopsis

```
stats = love.graphics.getStats( stats )
```

Arguments

Available since LÖVE 11.0

table stats (nil)
A table which will be filled in with the stat fields below or nil to create a new table.

Returns

table stats
A table with the following fields:

number drawcalls
The number of draw calls made so far during the current frame.

number canvasswitches
The number of times the active Canvas has been switched so far during the current frame.

number texturememory
The estimated total size in bytes of video memory used by all loaded Images, Canvases, and Fonts.

number images
The number of Image objects currently loaded.

number canvases
The number of Canvas objects currently loaded.

number fonts
The number of Font objects currently loaded.

Available since LÖVE 0.10.2

`number shaderswitches`
: The number of times the active Shader has been changed so far during the current frame.

Available since LÖVE 11.0

`number drawcallsbatched`
: The number of draw calls that were saved by LÖVE's automatic batching, since the start of the frame.

Examples

display the estimated amount of video memory used for textures

```
function love.load()
    love.graphics.setNewFont(24)
end

function love.draw()
    -- some drawing code here --

    local stats = love.graphics.getStats()

    local str = string.format("Estimated amount of texture memory used: %.2f MB", stats.texturememory / 1024 / 1024)
    love.graphics.print(str, 10, 10)
end
```

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getStencilMode

love.graphics.getStencilMode · revision 28263 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets the current stencil mode configuration.

Function

Synopsis

```
action, comparemode, value, readmask, writemask = love.graphics.getStencilMode( )
```

Arguments

None.

Returns

`StencilAction action`
: Whether and how to modify any stencil values of pixels that are touched by subsequent draws.

`CompareMode comparemode`
: The type of comparison (if any) being made for each pixel.

`number value`
: The value used when comparing with the stencil value of each pixel, and the new stencil value to use for pixels if the "replace" stencil action is used. Will be an integer between 0 and 255.

`number readmask`
: An 8 bit mask applied to values read from the stencil buffer in subsequent draws.

`number writemask`
: An 8 bit mask applied to values written to the stencil buffer in subsequent draws.

See Also

- love.graphics
- love.graphics.setStencilMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getStencilState

love.graphics.getStencilState · revision 29241 · last edited 2024-06-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets the current low level stencil state.

For simpler APIs see love.graphics.setStencilMode and love.graphics.getStencilMode.

Function

Synopsis

```
action, comparemode, value, readmask, writemask = love.graphics.getStencilState( )
```

Arguments

None.

Returns

`StencilAction action`
: Whether and how to modify any stencil values of pixels that are touched by subsequent draws.

`CompareMode comparemode`
: The type of comparison (if any) being made for each pixel.

`number value`
: The value used when comparing with the stencil value of each pixel, and the new stencil value to use for pixels if the "replace" stencil action is used. Will be an integer between 0 and 255.

`number readmask`
: An 8 bit mask applied to values read from the stencil buffer in subsequent draws.

`number writemask`
: An 8 bit mask applied to values written to the stencil buffer in subsequent draws.

See Also

- love.graphics
- love.graphics.setStencilState

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getStencilTest

love.graphics.getStencilTest · revision 15683 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the current stencil test configuration.

When stencil testing is enabled, the geometry of everything that is drawn afterward will be clipped / stencilled out based on a comparison between the arguments of this function and the stencil value of each pixel that the geometry touches. The stencil values of pixels are affected via love.graphics.stencil.

Each Canvas has its own per-pixel stencil values.

Function

Synopsis

```
comparemode, comparevalue = love.graphics.getStencilTest( )
```

Arguments

None.

Returns

`CompareMode comparemode`
: The type of comparison that is made for each pixel. Will be "always" if stencil testing is disabled.

`number comparevalue`
: The value used when comparing with the stencil value of each pixel.

See Also

- love.graphics
- love.graphics.setStencilTest
- love.graphics.stencil

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getSupported

love.graphics.getSupported · revision 20157 · last edited 2026-04-19

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

It has replaced love.graphics.isSupported.

Gets the optional graphics features and whether they're supported on the system.

Some older or low-end systems don't always support all graphics features.

Function

Synopsis

```
features = love.graphics.getSupported( )
```

Arguments

None.

Returns

`table features`

A table containing GraphicsFeature keys, and boolean values indicating whether each feature is supported.

See Also

- love.graphics
- GraphicsFeature

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getSystemLimit

love.graphics.getSystemLimit · revision 20158 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1 and removed in LÖVE 0.10.0

It has been replaced by love.graphics.getSystemLimits.

Gets the system-dependent maximum value for a love.graphics feature.

Function

Synopsis

```
limit = love.graphics.getSystemLimit( limittype )
```

Arguments

`GraphicsLimit limittype`

The graphics feature to get the maximum value of.

Returns

`number limit`

The system-dependent max value for the feature.

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getSystemLimits

love.graphics.getSystemLimits · revision 20159 · last edited 2026-04-19

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

It has replaced love.graphics.getSystemLimit.

Gets the system-dependent maximum values for love.graphics features.

Function

Synopsis

```
limits = love.graphics.getSystemLimits( )
```

Arguments

None.

Returns

table limits

A table containing GraphicsLimit keys, and number values.

See Also

- love.graphics
- GraphicsLimit

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getTextureFormats

love.graphics.getTextureFormats · revision 29240 · last edited 2026-01-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Create a canvas with the format 'rgba16f' if it is supported
- See Also
- Other Languages

Available since LÖVE 12.0

It has replaced love.graphics.getImageFormats and love.graphics.getCanvasFormats.

Gets the available pixel formats, and whether each is supported for the given Texture usage configuration.

Function

Synopsis

```
formats = love.graphics.getTextureFormats( usage )
```

Arguments

table usage

A table containing fields describing how the pixel formats will be used.

boolean canvas

If true, the returned formats will only be listed as supported if they can be used with a canvas-capable Texture.

boolean readable (nil)

If true, the returned formats will only be listed as supported if they can be used with the readable flag set to true for a Texture using that format, and vice versa if the field is false. When the field is nil, color pixel formats are assumed to be readable and depth/stencil formats are non-readable.

boolean computewrite (false)

If true, the returned formats will only be listed as supported if they can be used with the computewrite flag set to true for a Texture using that format.

Returns

table formats

A table containing PixelFormats as keys, and a boolean indicating whether the format is supported for the given usage configuration as values. Not all systems support all formats.

Examples

Create a canvas with the format 'rgba16f' if it is supported

```
local formats = love.graphics.getTextureFormats({canvas=true})
if formats.rgba16f then
    canvas = love.graphics.newCanvas(800, 600, {format="rgba16f"} )
else
    -- The 'rgba16f' format isn't supported. We could have some fallback code, or trigger a message telling the user
```

```
their system is unsupported.
end
```

See Also

- love.graphics
- PixelFormat
- love.graphics.newCanvas
- love.graphics.newImage
- love.graphics.newTexture
- Texture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getTextureTypes

love.graphics.getTextureTypes · revision 22234 · last edited 2026-04-19

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Display the texture types on screen and whether they're supported
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the available texture types, and whether each is supported.

Function

Synopsis

```
texturetypes = love.graphics.getTextureTypes( )
```

Arguments

None.

Returns

`table texturetypes`

A table containing TextureTypes as keys, and a boolean indicating whether the type is supported as values. Not all systems support all types.

Examples

Display the texture types on screen and whether they're supported

```
textypes = love.graphics.getTextureTypes()

function love.draw()
    local y = 0
    for name, supported in pairs(textypes) do
        local str = string.format("Supports texture type '%s': %s", name, tostring(supported))
        love.graphics.print(str, 10, y)
        y = y + 20
    end
end
```

See Also

- love.graphics
- TextureType
- love.graphics.newCanvas
- love.graphics.newImage
- love.graphics.newArrayImage

- love.graphics.newCubeImage
- love.graphics.newVolumeImage

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.getWidth

love.graphics.getWidth · revision 25460 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.2.1

This function is not supported in earlier versions.

Gets the width in pixels of the window.

Function

Synopsis

```
width = love.graphics.getWidth( )
```

Arguments

None.

Returns

number width

The width of the window.

See Also

- love.graphics
- love.graphics.getHeight
- love.graphics.getDimensions

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.hasFocus

love.graphics.hasFocus · revision 15957 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.9.0

Moved to the love.window module as love.window.hasFocus.

Checks if the game window has keyboard focus.

Function

Synopsis

```
focus = love.graphics.hasFocus( )
```

Arguments

None.

Returns

`boolean focus`

True if the window has the focus or false if not.

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.intersectScissor

love.graphics.intersectScissor · revision 27998 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Limit the drawing area incrementally
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Sets the scissor to the rectangle created by the intersection of the specified rectangle with the existing scissor. If no scissor is active yet, it behaves like love.graphics.setScissor.

The scissor limits the drawing area to a specified rectangle. This affects all graphics calls, including love.graphics.clear.

The dimensions of the scissor is unaffected by graphical transformations (translate, scale, ...).

Function

Synopsis

```
love.graphics.intersectScissor( x, y, width, height )
```

Arguments

number x
: The x-coordinate of the upper left corner of the rectangle to intersect with the existing scissor rectangle.

number y
: The y-coordinate of the upper left corner of the rectangle to intersect with the existing scissor rectangle.

number width
: The width of the rectangle to intersect with the existing scissor rectangle.

number height
: The height of the rectangle to intersect with the existing scissor rectangle.

Returns

Nothing.

Examples

Limit the drawing area incrementally

```
function love.draw()
    local windowWidth, windowHeight = love.graphics.getDimensions()
    local halfWidth  = windowWidth/2
    local halfHeight = windowHeight/2

    love.graphics.setScissor(0,0, halfWidth,windowHeight)
    love.graphics.clear(0, 0, .5) -- Affects the left side.
    love.graphics.intersectScissor(0,0, windowWidth,halfHeight)
```

```
    love.graphics.circle("fill", halfWidth,halfHeight, 100) -- Affects the top left quadrant.
end
```

See Also

- love.graphics
- love.graphics.setScissor
- love.graphics.getScissor
- love.graphics.push
- love.graphics.pop

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.inverseTransformPoint

love.graphics.inverseTransformPoint · revision 22654 · last edited 2026-04-16

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Converts the given 2D position from screen-space into global coordinates.

This effectively applies the reverse of the current graphics transformations to the given position. A similar Transform:inverseTransformPoint method exists for Transform objects.

Function

Synopsis

```
globalX, globalY = love.graphics.inverseTransformPoint( screenX, screenY )
```

Arguments

number screenX
: The x component of the screen-space position.

number screenY
: The y component of the screen-space position.

Returns

number globalX
: The x component of the position in global coordinates.

number globalY
: The y component of the position in global coordinates.

See Also

- love.graphics
- love.graphics.transformPoint
- Transform

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.isActive

love.graphics.isActive · revision 15362 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets whether the graphics module is able to be used. If it is not active, love.graphics function and method calls will not work correctly and may cause the program to crash.

The graphics module is *inactive* if a window is not open, or if the app is in the background on iOS. Typically the app's execution will be automatically paused by the system, in the latter case.

Function

Synopsis

```
active = love.graphics.isActive( )
```

Arguments

None.

Returns

`boolean active`

Whether the graphics module is active and able to be used.

See Also

- love.graphics
- love.window.close
- love.window.setMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.isCreated

love.graphics.isCreated · revision 17410 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

Moved to the love.window module as love.window.isCreated.

Checks if the window has been created.

Function

Synopsis

```
created = love.graphics.isCreated( )
```

Arguments

None.

Returns

`boolean created`

True if the window has been created, false otherwise.

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.isGammaCorrect

love.graphics.isGammaCorrect · revision 27416 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets whether gamma-correct rendering is supported and enabled. It can be enabled by setting `t.gammacorrect = true` in love.conf.

Not all devices support gamma-correct rendering, in which case it will be automatically disabled and this function will return false. It is supported on desktop systems which have graphics cards that are capable of using OpenGL 3 / DirectX 10, and iOS devices that can use OpenGL ES 3.

Function

Synopsis

```
gammacorrect = love.graphics.isGammaCorrect( )
```

Arguments

None.

Returns

`boolean gammacorrect`

True if gamma-correct rendering is supported and was enabled in love.conf, false otherwise.

Notes

When gamma-correct rendering is enabled, many functions and objects perform automatic color conversion between sRGB and linear RGB in order for blending and shader math to be mathematically correct (which they aren't if it's not enabled.)

- The colors passed into love.graphics.setColor, love.graphics.clear, and Shader:sendColor will automatically be converted from sRGB to linear RGB.
- The colors set in SpriteBatches, text with per-character colors, ParticleSystems, points with per-point colors, standard Meshes, and custom Meshes which use the "VertexColor" attribute name will automatically be converted from sRGB to linear RGB when those objects are drawn.
- Images will have their colors automatically converted from sRGB to linear RGB when drawing them (and when getting their pixels in a Shader), unless the flag `linear = true` is set when creating the Image.
- Everything drawn to the screen will be blended in linear RGB and then the result will be converted to sRGB for display.

- Canvases which use the "normal" or "srgb" CanvasFormat will have their content blended in linear RGB and the result will be stored in the canvas in sRGB, when drawing to them. When the Canvas itself is drawn, its pixel colors will be converted from sRGB to linear RGB in the same manner as Images. Keeping the canvas pixel data stored as sRGB allows for better precision (less banding) for darker colors compared to "rgba8".

Because most conversions are automatically handled, your own code doesn't need to worry about sRGB and linear RGB color conversions when gamma-correct rendering is enabled, except in a couple cases:

- If a Mesh with custom vertex attributes is used and one of the attributes is meant to be used as a color in a Shader, and the attribute isn't named "VertexColor".
- If a Shader is used which has uniform / extern variables or other variables that are meant to be used as colors, and Shader:sendColor isn't used.

In both cases, love.math.gammaToLinear can be used to convert color values to linear RGB in Lua code.

In shader code the following functions are available if manual conversions are necessary. They *only* do conversions if gamma-correct rendering is actually enabled. The `LOVE_GAMMA_CORRECT` shader preprocessor define will be set if so:

- `float/vec3/vec4 gammaCorrectColor(float/vec3/vec4 color)` (performant but less precise).
- `float/vec3/vec4 gammaCorrectColorPrecise(float/vec3/vec4 color)` (slower but accurate, equivalent to `love.math.gammaToLinear`).
- `float/vec3/vec4 unGammaCorrectColor(float/vec3/vec4 color)` (performant but less precise).
- `float/vec3/vec4 unGammaCorrectColorPrecise(float/vec3/vec4 color)` (slower but accurate, equivalent to `love.math.linearToGamma`).

Read more about gamma-correct rendering here (https://developer.nvidia.com/gpugems/gpugems3/part-iv-image-effects/chapter-24-importance-being-linear), here (http://filmicworlds.com/blog/linear-space-lighting-i-e-gamma/), and here (http://renderwonk.com/blog/index.php/archive/adventures-with-gamma-correct-rendering/).

See Also

- love.graphics
- Config Files
- love.math.gammaToLinear
- love.math.linearToGamma
- Shader:sendColor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.isSupported

love.graphics.isSupported · revision 15925 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Error smoothly if the graphics card does not support canvases
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.10.0

It has been replaced by love.graphics.getSupported.

Checks if certain graphics functions can be used.

Older and low-end systems do not always support all graphics extensions.

Function

Synopsis

```
isSupported = love.graphics.isSupported( support1, support2, support3, ... )
```

Arguments

`GraphicsFeature supportN`

The graphics feature to check for.

Returns

`boolean isSupported`

True if everything is supported, false otherwise.

Examples

Error smoothly if the graphics card does not support canvases

```
assert(love.graphics.isSupported("canvas"), "Your graphics card does not support canvases, sorry!")
```

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.isWireframe

love.graphics.isWireframe · revision 12932 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Gets whether wireframe mode is used when drawing.

Function

Synopsis

```
wireframe = love.graphics.isWireframe( )
```

Arguments

None.

Returns

`boolean wireframe`

True if wireframe lines are used when drawing, false if it's not.

See Also

- love.graphics
- love.graphics.setWireframe

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.line

love.graphics.line · revision 28705 · last edited 2023-07-21

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Draw the outline of a simple trapezoid
  - Draw a line from the center of the screen to the mouse pointer
  - Draw a zigzag line using a table
  - Generate and draw a grid
  - Draw temp line with mouse
- See Also
- Other Languages

Draws lines between points.

Function

Synopsis

```
love.graphics.line( x1, y1, x2, y2, ... )
```

Arguments

number x1
: The position of first point on the x-axis.

number y1
: The position of first point on the y-axis.

number x2
: The position of second point on the x-axis.

number y2
: The position of second point on the y-axis.

number ...
: You can continue passing point positions to draw a polyline.

Returns

Nothing.

Function

Synopsis

```
love.graphics.line( points )
```

Arguments

table points

A table of point positions, as described above.

Returns

Nothing.

Examples

Draw the outline of a simple trapezoid

```
function love.draw()
    love.graphics.line(200,50, 400,50, 500,300, 100,300, 200,50)   -- last pair is a repeat to complete the trapezoid
end
```

Draw a line from the center of the screen to the mouse pointer

```
function love.draw()
    local mx, my = love.mouse.getPosition()

    local windowWidth, windowHeight = love.graphics.getDimensions()

    love.graphics.line(windowWidth/2, windowHeight/2, mx, my)
end
```

Draw a zigzag line using a table

```
local zigzagLine = {100,100, 200,200, 300,100, 400,200}

function love.draw()
    love.graphics.line(zigzagLine)
end
```

Generate and draw a grid

```
local cellSize  = 30 -- Width and height of cells.
local gridLines = {}

local windowWidth, windowHeight = love.graphics.getDimensions()

-- Vertical lines.
for x = cellSize, windowWidth, cellSize do
    local line = {x, 0, x, windowHeight}
    table.insert(gridLines, line)
end
-- Horizontal lines.
for y = cellSize, windowHeight, cellSize do
    local line = {0, y, windowWidth, y}
    table.insert(gridLines, line)
end

function love.draw()
    love.graphics.setLineWidth(2)

    for i, line in ipairs(gridLines) do
        love.graphics.line(line)
    end
end
```

Draw temp line with mouse

```
tempLine = {}


local function drawLine (line)

end

function love.draw()
--  check if line has 4 coordinates:
    if #tempLine > 3 then
        love.graphics.line (tempLine)
    end
end

function love.mousepressed(x, y)
--  set first point:
    tempLine = {x, y}
end

function love.mousemoved (x, y)
--  update second point:
    tempLine[3] = x
    tempLine[4] = y
end

function love.mousereleased ( x, y)
-- get your temp line coordinates as:
--  local x1, y1 = tempLine[1], tempLine[2]
--  local x2, y2 = tempLine[3], tempLine[4]

-- clean temp line:
    tempLine = {}
end
```

See Also

- love.graphics
- love.graphics.setLineWidth
- love.graphics.setLineStyle

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newArrayImage

love.graphics.newArrayImage · revision 29291 · last edited 2026-04-09

On this page

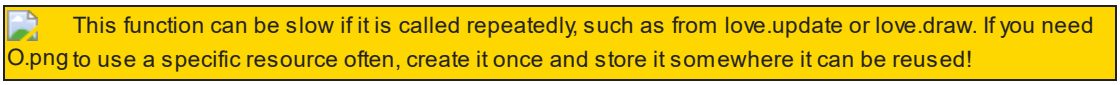
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
  - Draw multiple layers of an Array Image
  - Use a custom shader with love.graphics.drawLayer
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Creates a new array Image.





An array image / array texture is a single object which contains multiple 'layers' or 'slices' of 2D sub-images. It can be thought of similarly to a texture atlas or sprite sheet, but it doesn't suffer from the same tile / quad bleeding artifacts that texture atlases do – although every sub-image must have the same dimensions.

A specific layer of an array image can be drawn with love.graphics.drawLayer / SpriteBatch:addLayer, or with the Quad variant of love.graphics.draw and Quad:setLayer, or via a custom Shader.

To use an array image in a Shader, it must be declared as a `ArrayImage` or `sampler2DArray` type (instead of `Image` or `sampler2D`). The `Texel(ArrayImage image, vec3 texturecoord)` shader function must be used to get pixel colors from a slice of the array image. The vec3 argument contains the texture coordinate in the first two components, and the 0-based slice index in the third component.

Function

Creates an array Image given a different image file for each slice of the resulting array image object.

Synopsis

```
image = love.graphics.newArrayImage( slices, settings )
```

Arguments

`table slices`

A table containing filepaths to images (or File, FileData, ImageData, or CompressedImageData objects), in an array. Each sub-image must have the same dimensions. A table of tables can also be given, where each sub-table contains all mipmap levels for the slice index of that sub-table.

`table settings (nil)`

Optional table of settings to configure the array image, containing the following fields:

`boolean mipmaps (false)`

True to make the image use mipmaps, false to disable them. Mipmaps will be automatically generated if the image isn't a compressed texture format.

boolean linear (false)

True to treat the image's pixels as linear instead of sRGB, when gamma correct rendering is enabled. Most images are authored as sRGB.

number dpiscale (1)

The DPI scale to use when drawing the array image and calling getWidth/getHeight.

Returns

Image image

An Array Image object.

Notes

Illustration of how an array image works: [1] (https://web.archive.org/web/20191118024004/http://codeflow.org/entries/2010/dec/09/minecraft-like-rendering-experiments-in-opengl-4/illustrations/textures.jpg)

A DPI scale of 2 (double the normal pixel density) will result in the image taking up the same space on-screen as an image with half its pixel dimensions that has a DPI scale of 1. This allows for easily swapping out image assets that take the same space on-screen but have different pixel densities, which makes supporting high-dpi / retina resolution require less code logic.

In order to use an Array Texture or other non-2D texture types as the main texture in a custom Shader, the void effect() variant must be used in the pixel shader, and MainTex must be declared as an ArrayImage or sampler2DArray like so: `uniform ArrayImage MainTex;`.

Examples

Draw multiple layers of an Array Image

```
function love.load()
    local sprites = {"sprite1.png", "sprite2.png"}
    image = love.graphics.newArrayImage(sprites)
end

function love.draw()
    love.graphics.drawLayer(image, 1, 50, 50)
    love.graphics.drawLayer(image, 2, 250, 50)
end
```

Use a custom shader with love.graphics.drawLayer

```
shader = love.graphics.newShader[[
uniform ArrayImage MainTex;

void effect() {
    // Texel uses a third component of the texture coordinate for the layer index, when an Array Texture is passed in.
    // love sets up the texture coordinates to contain the layer index specified in love.graphics.drawLayer, when
    // rendering the Array Texture.
    love_PixelColor = Texel(MainTex, VaryingTexCoord.xyz) * VaryingColor;
}
]]

function love.load()
    local sprites = {"sprite1.png", "sprite2.png"}
```

```
    image = love.graphics.newArrayImage(sprites)
end

function love.draw()
    love.graphics.setShader(shader)
    love.graphics.drawLayer(image, 1, 50, 50)
    love.graphics.drawLayer(image, 2, 250, 50)
end
```

See Also

- love.graphics
- Image
- TextureType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newBuffer

love.graphics.newBuffer · revision 31697 · last edited 2026-08-08

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.



This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Creates a new GraphicsBuffer.

Function

Synopsis

```
buffer = love.graphics.newBuffer( format, size, settings )
```

Arguments

`BufferDataFormat format`
: The format of the data stored in the buffer.

`number size`
: The number of elements in the buffer.

`table settings`
: Table of settings to configure the image, containing the following fields:

  `BufferDataUsage usage (nil)`
  : The expected usage of the GraphicsBuffer. The specified usage mode affects the GraphicsBuffer's memory usage and performance.

Returns

`GraphicsBuffer buffer`
: The newly-created graphics buffer.

See Also

- love.graphics
- GraphicsBuffer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newCanvas

love.graphics.newCanvas · revision 26818 · last edited 2025-11-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.8.0

It has been renamed from love.graphics.newFramebuffer.

Creates a new Canvas object for offscreen rendering.


This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Synopsis

```
canvas = love.graphics.newCanvas( )
```

Arguments

None.

Returns

Canvas canvas

A new Canvas with dimensions equal to the window's size in pixels.

Function

Synopsis

```
canvas = love.graphics.newCanvas( width, height )
```

Arguments

number width
: The desired width of the Canvas.

number height
: The desired height of the Canvas.

Returns

Canvas canvas
: A new Canvas with specified width and height.

Function

Available since LÖVE 11.0
This variant is not supported in earlier versions.

Creates a 2D or cubemap Canvas using the given settings.

Synopsis

```
canvas = love.graphics.newCanvas( width, height, settings )
```

Arguments

number width
: The desired width of the Canvas.

number height
: The desired height of the Canvas.

table settings
: A table containing the given fields:

  TextureType type ("2d")
  : The type of Canvas to create.

  PixelFormat format ("normal")
  : The format of the Canvas.

  boolean readable
  : Whether the Canvas is readable (drawable and accessible in a Shader). True by default for regular formats, false by default for depth/stencil formats.

  number msaa (0)
  : The desired number of multisample antialiasing (MSAA) samples used when drawing to the Canvas.

  number dpiscale (love.graphics.getDPIScale())
  : The DPI scale factor of the Canvas, used when drawing to the Canvas as well as when drawing the Canvas to the screen.

`MipmapMode mipmaps ("none")`
: Whether the Canvas has mipmaps, and whether to automatically regenerate them if so.

Returns

`Canvas canvas`
: A new Canvas with specified width and height.

Notes

Some Canvas formats have higher system requirements than the default format. Use love.graphics.getCanvasFormats to check for support.

Function

Available since LÖVE 11.0
This variant is not supported in earlier versions.

Creates a volume or array texture-type Canvas.

Synopsis

```
canvas = love.graphics.newCanvas( width, height, layers, settings )
```

Arguments

`number width`
: The desired width of the Canvas.

`number height`
: The desired height of the Canvas.

`number layers`
: The number of array layers (if the Canvas is an Array Texture), or the volume depth (if the Canvas is a Volume Texture).

`table settings`
: A table containing the given fields:

  `TextureType type ("array")`
  : The type of Canvas to create.

  `PixelFormat format ("normal")`
  : The format of the Canvas.

  `boolean readable`
  : Whether the Canvas is readable (drawable and accessible in a Shader). True by default for regular formats, false by default for depth/stencil formats.

  `number msaa (0)`
  : The desired number of multisample antialiasing (MSAA) samples used when drawing to the Canvas.

  `number dpiscale (love.graphics.getDPIScale())`
  : The DPI scale factor of the Canvas, used when drawing to the Canvas as well as when drawing the Canvas to the screen.

  `MipmapMode mipmaps ("none")`

Whether the Canvas has mipmaps, and whether to automatically regenerate them if so.

Returns

`Canvas canvas`
A new Canvas with specified width and height.

Notes

Not all texture types are supported by all systems. love.graphics.getTextureTypes can check for support.

Function

Available since LÖVE 0.9.0 and removed in LÖVE 11.0
This variant is not supported in earlier or later versions.

Synopsis

```
canvas = love.graphics.newCanvas( width, height, format )
```

Arguments

`number width (window_width)`
The desired width of the Canvas.

`number height (window_height)`
The desired height of the Canvas.

`CanvasFormat format ("normal")`
The desired texture format of the Canvas.

Returns

`Canvas canvas`
A new Canvas with specified width and height.

Notes

Some Canvas formats have higher system requirements than the default format. Use love.graphics.getCanvasFormats to check for support.

Function

Available since LÖVE 0.9.1 and removed in LÖVE 11.0
This variant is not supported in earlier or later versions.

Synopsis

```
canvas = love.graphics.newCanvas( width, height, format, msaa )
```

Arguments

`number width (window_width)`
The desired width of the Canvas.

`number height (window_height)`
: The desired height of the Canvas.

`CanvasFormat format ("normal")`
: The desired texture format of the Canvas.

`number msaa (0)`
: The desired number of multisample antialiasing (MSAA) samples used when drawing to the Canvas.

Returns

`Canvas canvas`
: A new Canvas with specified width and height.

Notes

- Not all texture types are supported by all systems. love.graphics.getTextureTypes can check for support.
- Some Canvas formats have higher system requirements than the default format. Use love.graphics.getCanvasFormats to check for support.
- The supported maximum number of MSAA samples varies depending on the system. Use love.graphics.getSystemLimits to check.
- If the number of MSAA samples specified is greater than the maximum supported by the system, the Canvas will still be created but only using the maximum supported amount (**this includes 0.**)

See Also

- love.graphics
- love.graphics.draw
- Canvas
- love.graphics.setCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newCubeImage

love.graphics.newCubeImage · revision 26879 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Creates a new cubemap Image.


This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Cubemap images have 6 faces (sides) which represent a cube. They can't be rendered directly, they can only be used in Shader code (and sent to the shader via Shader:send).

To use a cubemap image in a Shader, it must be declared as a `CubeImage` or `samplerCube` type (instead of `Image` or `sampler2D`). The `Texel(CubeImage image, vec3 direction)` shader function must be used to get pixel colors from the cubemap. The vec3 argument is a normalized direction from the center of the cube, rather than explicit texture coordinates.

Each face in a cubemap image must have square dimensions.

For variants of this function which accept a single image containing multiple cubemap faces, they must be laid out in one of the following forms in the image:

```
   +y
-x +z +x -z
   -y
```


cubemap-sidecross-earth.png

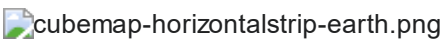
or:

```
+x
-x
+y
-y
+z
-z
```


cubemap-verticalstrip-earth.png

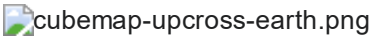
or:

```
+x -x +y -y +z -z
```


cubemap-horizontalstrip-earth.png

or:

```
   +y
+z +x -z
   -y
   -x
```



Note that this form, despite looking like it should fold into a cube, does not do so: the orientation of the individual faces is the same as in the other three layouts.

Function

Creates a cubemap Image given a single image file containing multiple cube faces.

Synopsis

```
image = love.graphics.newCubeImage( filename, settings )
```

Arguments

string filename
: The filepath to a cubemap image file (or a File, FileData, or ImageData).

table settings (nil)
: Optional table of settings to configure the cubemap image, containing the following fields:

  boolean mipmaps (false)
  : True to make the image use mipmaps, false to disable them. Mipmaps will be automatically generated if the image isn't a compressed texture format.

  boolean linear (false)
  : True to treat the image's pixels as linear instead of sRGB, when gamma correct rendering is enabled. Most images are authored as sRGB.

Returns

Image image
: An cubemap Image object.

Function

Creates a cubemap Image given a different image file for each cube face.

Synopsis

```
image = love.graphics.newCubeImage( faces, settings )
```

Arguments

table faces
: A table containing 6 filepaths to images (or File, FileData, ImageData, or CompressedImageData objects), in an array, in the order +x -x +y -y +z -z. Each face image must have the same dimensions. A table of tables can also be given, where each sub-table contains all mipmap levels for the cube face index of that sub-table.

table settings (nil)
: Optional table of settings to configure the cubemap image, containing the following fields:

`boolean mipmaps (false)`

True to make the image use mipmaps, false to disable them. Mipmaps will be automatically generated if the image isn't a compressed texture format.

`boolean linear (false)`

True to treat the image's pixels as linear instead of sRGB, when gamma correct rendering is enabled. Most images are authored as sRGB.

Returns

`Image image`

An cubemap Image object.

See Also

- love.graphics
- Image
- TextureType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newFont

love.graphics.newFont · revision 29716 · last edited 2025-11-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Use new Font to draw a custom styled text
  - Use new Font to draw pixelated text
- See Also
- Other Languages

Creates a new Font from a TrueType font or BMFont (http://www.angelcode.com/products/bmfont/doc/file_format.html) file. OpenType fonts (.otf) work as well but some of its features may not be supported.

Created fonts are not cached, in that calling this function with the same arguments will always create a new Font object.

All variants which accept a filename can also accept a Data object instead.

The default font in LÖVE is 'Bitstream Vera Sans' size 12 (before LÖVE 12.0) and 'Noto Sans' size 13 (after LÖVE 12.0).


This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Create a new BMFont or TrueType font.

Synopsis

```
font = love.graphics.newFont( filename )
```

Arguments

Variant filename

The filepath or filedata to the font file. Types can be below:

string filename

The filepath to the font file. This can be TrueType or, since 0.10.0, BMFont.

Data filename

The filedata of the font file. This can be create by love.filesystem.newFileData.

Returns

Font font

A Font object which can be used to draw text on screen.

Notes

If the file is a TrueType font, it will be size 12. Use the variant below to create a TrueType font with a custom size.

Function

Create a new TrueType font.

Synopsis

```
font = love.graphics.newFont( filename, size, hinting, dpiscale )
```

Arguments

`string filename`
The filepath to the TrueType font file.

`number size`
The size of the font in pixels.

Available since LÖVE 0.10.0

`HintingMode hinting ("normal")`
True Type hinting mode.

Available since LÖVE 11.0

`number dpiscale (love.graphics.getDPIScale())`
The DPI scale factor of the font.

Returns

`Font font`
A Font object which can be used to draw text on screen.

Function

Available since LÖVE 0.10.0
This variant is not supported in earlier versions.

Create a new BMFont.

Synopsis

```
font = love.graphics.newFont( filename, imagefilename )
```

Arguments

string filename
: The filepath to the BMFont file.

string imagefilename
: The filepath to the BMFont's image file. If this argument is omitted, the path specified inside the BMFont file will be used.

Returns

Font font
: A Font object which can be used to draw text on screen.

Function

Create a new instance of the default font (Vera Sans) with a custom size.

Synopsis

```
font = love.graphics.newFont( size, hinting, dpiscale )
```

Arguments

number size (12)
: The size of the font in pixels.

Available since LÖVE 0.10.0

HintingMode hinting ("normal")
: True Type hinting mode.

Available since LÖVE 11.0

number dpiscale (love.graphics.getDPIScale())
: The DPI scale factor of the font.

Returns

Font font

A Font object which can be used to draw text on screen.

Examples

Use newFont to draw a custom styled text

```
-- Create a ttf file font with a custom size of 20 pixels.
mainFont = love.graphics.newFont("anyfont.ttf", 20)

function love.draw()
    -- Setting the font so that it is used when drawning the string.
    love.graphics.setFont(mainFont)

    -- Draws "Hello world!" at position x: 100, y: 200 with the custom font applied.
    love.graphics.print("Hello world!", 100, 200)
end
```

Use newFont to draw pixelated text

```
local font = love.graphics.newFont(7, "mono")
font:setFilter("nearest")
love.graphics.setFont(font)

function love.draw()
    love.graphics.scale(4)
    love.graphics.print ("Hello World 0123", 2, 2)
end
```



Pixelated font size 7

See Also

- Font
- love.graphics
- love.graphics.setFont
- love.graphics.setNewFont
- love.graphics.newImageFont

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newFramebuffer

love.graphics.newFramebuffer · revision 17379 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.8.0

It has been renamed to love.graphics.newCanvas.

Creates a new framebuffer object for offscreen rendering.


Versions prior to 0.8.0 have Framebuffers that are susceptible to power of 2 syndrome.


This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Synopsis

```
framebuffer = love.graphics.newFramebuffer( )
```

Arguments

None.

Returns

`Framebuffer framebuffer`

A new framebuffer with width/height equal to the window width/height.

Function

Synopsis

```
framebuffer = love.graphics.newFramebuffer( width, height )
```

Arguments

`number width`

The desired width of the framebuffer.

`number height`

The desired height of the framebuffer.

Returns

`Framebuffer framebuffer`

A new framebuffer with specified width and height.

See Also

- love.graphics
- Framebuffer
- love.graphics.setRenderTarget

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newImage

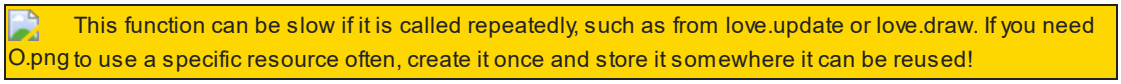
love.graphics.newImage · revision 27840 · last edited 2025-11-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Draw a bunny on the screen
  - Load image if it exists
- See Also
- Other Languages

Creates a new Image from a filepath, FileData, an ImageData, or a CompressedImageData, and optionally generates or specifies mipmaps for the image.

Version 11.0 updated love.graphics.newImage to treat file names ending with "@2x", "@3x", etc. as a pixel density scale factor if none is explicitly supplied.



Function

Synopsis

```
image = love.graphics.newImage( filename, settings )
```

Arguments

string filename

The filepath to the image file (or a FileData or ImageData or CompressedImageData or ByteData object).

Available since LÖVE 0.10.0

table settings (nil)

Optional table of settings to configure the image, containing the following fields:

boolean mipmaps (false)

If true, mipmaps for the image will be automatically generated (or taken from the images's file if possible, if the image originated from a CompressedImageData).

boolean linear (false)

True to treat the image's pixels as linear instead of sRGB, when gamma correct rendering is enabled. Most images are authored as sRGB.

Available since LÖVE 11.0

number dpiscale (1)

The DPI scale to use when drawing the image and calling getWidth/getHeight.

Returns

`Image image`
A new Image object which can be drawn on screen.

Function

Available since LÖVE 0.9.1 and removed in LÖVE 0.10.0
This variant is not supported in earlier or later versions.

Synopsis

```
image = love.graphics.newImage( filename, format )
```

Arguments

`string filename`
The filepath to the image file (or a FileData or ImageData or CompressedImageData object.)
`TextureFormat format`
The format to interpret the image's data as.

Returns

`Image image`
An Image object which can be drawn on screen.

Examples

Draw a bunny on the screen

```
function love.load()
    bunny = love.graphics.newImage("MyBunny.png")
end
function love.draw()
    love.graphics.draw(bunny, 0, 0)
end
```

Load image if it exists

```
function loadImage (path)
    local info = love.filesystem.getInfo( path )
    if info then
        return love.graphics.newImage( path )
    end
end
```

```
image = loadImage ("graphics/image-1.png")
```

See Also

- love.graphics
- love.graphics.draw
- love.graphics.newQuad
- Image
- love.image.newImageData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newImageFont

love.graphics.newImageFont · revision 27946 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
  - Create a simple image font
- See Also
- Other Languages

Creates a new Font by loading a specifically formatted image.

In versions prior to 0.9.0, LÖVE expects ISO 8859-1 encoding for the glyphs string.


This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Synopsis

```
font = love.graphics.newImageFont( filename, glyphs, extraspacing )
```

Arguments

string filename
: The filepath to the image file.

string glyphs
: A string of the characters in the image in order from left to right.

Available since LÖVE 0.10.0

number extraspacing (0)
: Additional spacing (positive or negative) to apply to each glyph in the Font.

Returns

Font font

A Font object which can be used to draw text on screen.

Function

Synopsis

```
font = love.graphics.newImageFont( imageData, glyphs, extraspacing )
```

Arguments

ImageData imageData

The ImageData object to create the font from.

string glyphs

A string of the characters in the image in order from left to right.

Available since LÖVE 0.10.0

number extraspacing (0)

Additional spacing (positive or negative) to apply to each glyph in the Font.

Returns

Font font

A Font object which can be used to draw text on screen.

Notes

Instead of using this limited function, consider using a BMFont generator such as bmfont (http://w w w.angelcode.com/products/bmfont/), littera (http://kvazars.com/littera/), or bmGlyph (https://w w w.bmglyph.com/) with love.graphics.newFont.

Examples

Create a simple image font

Download this image file which will be used by LÖVE to create the font. Of course, when you want to create a font for your game you probably want to make the background transparent instead of black.

```
local font = love.graphics.newImageFont("font_example.png", "
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789")
function love.draw()
    love.graphics.setFont(font)
    love.graphics.print("ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789", 16, 16)
    love.graphics.print("Text is now drawn using the font", 16, 32)
end
```

See Also

- Font
- Image Font Format
- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newMesh

love.graphics.newMesh · revision 28345 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Examples
  - Creates and draws a Mesh identical to a normal drawn image but with different colors at each corner
  - Creates and draws a textured circle with a red tint at the center.
  - Creates a circle and draws it more efficiently than love.graphics.circle.
  - Using custom vertex attributes, create a 3D mesh.
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Creates a new Mesh.

Use Mesh:setTexture if the Mesh should be textured with an Image or Canvas when it's drawn.

In versions prior to 11.0, color and byte component values were within the range of 0 to 255 instead of 0 to 1.



This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Creates a standard Mesh with the specified vertices.

Synopsis

```
mesh = love.graphics.newMesh( vertices, mode, usage )
```

Arguments

table vertices

The table filled with vertex information tables for each vertex as follows:

number [1]

The position of the vertex on the x-axis.

number [2]

The position of the vertex on the y-axis.

number [3] (0)

The u texture coordinate of the vertex. Texture coordinates are normally in the range of [0, 1], but can be greater or less (see WrapMode.)

number [4] (0)

The v texture coordinate of the vertex. Texture coordinates are normally in the range of [0, 1], but can be greater or less (see WrapMode.)

number [5] (1)

The red component of the vertex color.

number [6] (1)

The green component of the vertex color.

number [7] (1)

The blue component of the vertex color.

number [8] (1)

The alpha component of the vertex color.

MeshDrawMode mode ("fan")

How the vertices are used when drawing. The default mode "fan" is sufficient for simple convex polygons.

SpriteBatchUsage usage ("dynamic")

The expected usage of the Mesh. The specified usage mode affects the Mesh's memory usage and performance.

Returns

Mesh mesh

The new mesh.

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Creates a standard Mesh with the specified number of vertices.

Synopsis

```
mesh = love.graphics.newMesh( vertexcount, mode, usage )
```

Arguments

number vertexcount

The total number of vertices the Mesh will use. Each vertex is initialized to `{0,0, 0,0, 1,1,1,1}`.

MeshDrawMode mode ("fan")

How the vertices are used when drawing. The default mode "fan" is sufficient for simple convex polygons.

SpriteBatchUsage usage ("dynamic")

The expected usage of the Mesh. The specified usage mode affects the Mesh's memory usage and performance.

Returns

Mesh mesh

The new mesh.

Notes

Mesh:setVertices or Mesh:setVertex and Mesh:setDrawRange can be used to specify vertex information once the Mesh is created.

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Creates a Mesh with custom vertex attributes and the specified vertex data.

Synopsis

```
mesh = love.graphics.newMesh( vertexformat, vertices, mode, usage )
```

Arguments

table vertexformat

A table in the form of `{attribute, ...}`. Each attribute is a table which specifies a custom vertex attribute used for each vertex.

table attribute

A table containing the attribute's name, its data type, and the number of components in the attribute, in the form of `{name, datatype, components}`.

table ...

Additional vertex attribute format tables.

table vertices

The table filled with vertex information tables for each vertex, in the form of `{vertex, ...}` where each vertex is a table in the form of `{attributecomponent, ...}`.

number attributecomponent

The first component of the first vertex attribute in the vertex.

number ...

Additional components of all vertex attributes in the vertex.

MeshDrawMode mode ("fan")

How the vertices are used when drawing. The default mode "fan" is sufficient for simple convex polygons.

SpriteBatchUsage usage ("dynamic")

The expected usage of the Mesh. The specified usage mode affects the Mesh's memory usage and performance.

Returns

Mesh mesh

The new mesh.

Notes

The values in each vertex table are in the same order as the vertex attributes in the specified vertex format. If no value is supplied for a specific vertex attribute component, it will be set to a default value of 0 if its data type is "float", or 1 if its data type is "byte".

If the data type of an attribute is "float", components can be in the range 1 to 4, if the data type is "byte" it must be 4.

If a custom vertex attribute uses the name "VertexPosition", "VertexTexCoord", or "VertexColor", then the vertex data for that vertex attribute will be used for the standard vertex positions, texture coordinates, or vertex colors respectively, when drawing the Mesh. Otherwise a Vertex Shader is required in order to make use of the vertex attribute when the Mesh is drawn.

A Mesh **must** have a "VertexPosition" attribute in order to be drawn, but it can be attached from a different Mesh via Mesh:attachAttribute.

To use a custom named vertex attribute in a Vertex Shader, it must be declared as an `attribute` variable of the same name. Variables can be sent from Vertex Shader code to Pixel Shader code by making a `varying` variable. For example:

Vertex Shader code

```
attribute vec2 CoolVertexAttribute;

varying vec2 CoolVariable;

vec4 position(mat4 transform_projection, vec4 vertex_position)
{
    CoolVariable = CoolVertexAttribute;
    return transform_projection * vertex_position;
}
```

Pixel Shader code

```
varying vec2 CoolVariable;

vec4 effect(vec4 color, Image tex, vec2 texcoord, vec2 pixcoord)
{
    vec4 texcolor = Texel(tex, texcoord + CoolVariable);
    return texcolor * color;
}
```

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Creates a Mesh with custom vertex attributes and the specified number of vertices.

Synopsis

```
mesh = love.graphics.newMesh( vertexformat, vertexcount, mode, usage )
```

Arguments

table vertexformat

A table in the form of `{attribute, ...}`. Each attribute is a table which specifies a custom vertex attribute used for each vertex.

`table attribute`

A table containing the attribute's name, it's data type, and the number of components in the attribute, in the form of `{name, datatype, components}`.

`table ...`

Additional vertex attribute format tables.

`number vertexcount`

The total number of vertices the Mesh will use.

`MeshDrawMode mode ("fan")`

How the vertices are used when drawing. The default mode "fan" is sufficient for simple convex polygons.

`SpriteBatchUsage usage ("dynamic")`

The expected usage of the Mesh. The specified usage mode affects the Mesh's memory usage and performance.

Returns

`Mesh mesh`

The new mesh.

Notes

Each vertex attribute component is initialized to 0 if its data type is "float", or 1 if its data type is "byte". Mesh:setVertices or Mesh:setVertex and Mesh:setDrawRange can be used to specify vertex information once the Mesh is created.

If the data type of an attribute is "float", components can be in the range 1 to 4, if the data type is "byte" it must be 4.

If a custom vertex attribute uses the name "VertexPosition", "VertexTexCoord", or "VertexColor", then the vertex data for that vertex attribute will be used for the standard vertex positions, texture coordinates, or vertex colors respectively, when drawing the Mesh. Otherwise a Vertex Shader is required in order to make use of the vertex attribute when the Mesh is drawn.

A Mesh **must** have a "VertexPosition" attribute in order to be drawn, but it can be attached from a different Mesh via Mesh:attachAttribute.

Function

Removed in LÖVE 0.10.0

This variant is not supported in that and later versions.

Synopsis

```
mesh = love.graphics.newMesh( vertices, texture, mode )
```

Arguments

`table vertices`

The table filled with vertex information tables for each vertex as follows:

`number [1]`

The position of the vertex on the x-axis.

`number [2]`

The position of the vertex on the y-axis.

`number [3]`
: The u texture coordinate. Texture coordinates are normally in the range of [0, 1], but can be greater or less (see WrapMode.)

`number [4]`
: The v texture coordinate. Texture coordinates are normally in the range of [0, 1], but can be greater or less (see WrapMode.)

`number [5] (255)`
: The red color component.

`number [6] (255)`
: The green color component.

`number [7] (255)`
: The blue color component.

`number [8] (255)`
: The alpha color component.

`Texture texture (nil)`
: The Image or Canvas to use when drawing the Mesh. May be nil to use no texture.

`MeshDrawMode mode ("fan")`
: How the vertices are used when drawing. The default mode "fan" is sufficient for simple convex polygons.

Returns

`Mesh mesh`
: The new mesh.

Function

Available since LÖVE 0.9.1 and removed in LÖVE 0.10.0
This variant is not supported in earlier or later versions.

Synopsis

```
mesh = love.graphics.newMesh( vertexcount, texture, mode )
```

Arguments

`number vertexcount`
: The total number of vertices the Mesh will use. Each vertex is initialized to `{0,0, 0,0, 255,255,255,255}`.

`Texture texture (nil)`
: The Image or Canvas to use when drawing the Mesh. May be nil to use no texture.

`MeshDrawMode mode ("fan")`
: How the vertices are used when drawing. The default mode "fan" is sufficient for simple convex polygons.

Returns

`Mesh mesh`

The new mesh.

Notes

Mesh:setVertices or Mesh:setVertex and Mesh:setDrawRange can be used to specify vertex information once the Mesh is created.

Examples

Creates and draws a Mesh identical to a normal drawn image but with different colors at each corner

```
function love.load()
    image = love.graphics.newImage("pig.png")

    local vertices = {
        {
            -- top-left corner (red-tinted)
            0, 0, -- position of the vertex
            0, 0, -- texture coordinate at the vertex position
            1, 0, 0, -- color of the vertex
        },
        {
            -- top-right corner (green-tinted)
            image:getWidth(), 0,
            1, 0, -- texture coordinates are in the range of [0, 1]
            0, 1, 0
        },
        {
            -- bottom-right corner (blue-tinted)
            image:getWidth(), image:getHeight(),
            1, 1,
            0, 0, 1
        },
        {
            -- bottom-left corner (yellow-tinted)
            0, image:getHeight(),
            0, 1,
            1, 1, 0
        },
    }

    -- the Mesh DrawMode "fan" works well for 4-vertex Meshes.
    mesh = love.graphics.newMesh(vertices, "fan")
        mesh:setTexture(image)
end

function love.draw()
    love.graphics.draw(mesh, 0, 0)
end
```


pig.png

Creates and draws a textured circle with a red tint at the center.

```
function CreateTexturedCircle(image, segments)
    segments = segments or 40
    local vertices = {}
```

```
    -- The first vertex is at the center, and has a red tint. We're centering the circle around the origin (0, 0).
    table.insert(vertices, {0, 0, 0.5, 0.5, 255, 0, 0})

    -- Create the vertices at the edge of the circle.
    for i=0, segments do
        local angle = (i / segments) * math.pi * 2

        -- Unit-circle.
        local x = math.cos(angle)
        local y = math.sin(angle)

        -- Our position is in the range of [-1, 1] but we want the texture coordinate to be in the range of [0, 1].
        local u = (x + 1) * 0.5
        local v = (y + 1) * 0.5

        -- The per-vertex color defaults to white.
        table.insert(vertices, {x, y, u, v})
    end

    -- The "fan" draw mode is perfect for our circle.
    local mesh = love.graphics.newMesh(vertices, "fan")
        mesh:setTexture(image)

        return mesh
end

function love.load()
    image = love.graphics.newImage("pig.png")
    mesh = CreateTexturedCircle(image)
end

function love.draw()
    local radius = 100
    local mx, my = love.mouse.getPosition()

    -- We created a unit-circle, so we can use the scale parameter for the radius directly.
    love.graphics.draw(mesh, mx, my, 0, radius, radius)
end
```

Creates a circle and draws it more efficiently than love.graphics.circle.

```
function CreateCircle(segments)
    segments = segments or 40
    local vertices = {}

    -- The first vertex is at the origin (0, 0) and will be the center of the circle.
    table.insert(vertices, {0, 0})

    -- Create the vertices at the edge of the circle.
    for i=0, segments do
        local angle = (i / segments) * math.pi * 2

        -- Unit-circle.
        local x = math.cos(angle)
        local y = math.sin(angle)

        table.insert(vertices, {x, y})
    end

    -- The "fan" draw mode is perfect for our circle.
    return love.graphics.newMesh(vertices, "fan")
```

```
end

function love.load()
    mesh = CreateCircle()
end

function love.draw()
    local radius = 100
    local mx, my = love.mouse.getPosition()

    -- We created a unit-circle, so we can use the scale parameter for the radius directly.
    love.graphics.draw(mesh, mx, my, 0, radius, radius)
end
```

Using custom vertex attributes, create a 3D mesh.

```
local vertexFormat = {
    {"VertexPosition", "float", 3},
    {"VertexTexCoord", "float", 2},
}

-- Vertices are defined in the order that the vertexFormat table is in
-- So, the first 3 numbers in each vertex are "VertexPosition" and the last 2 are "VertexTexCoord"

local vertices = {
    {-0.5, -0.5, -0.5, 0, 0},
    { 0.5, -0.5, -0.5, 1, 1},
    { 0.5,  0.5, -0.5, 0, 1},
    --and so on.
}

-- The "triangles" mode creates a separate triangle for each set of 3 vertices.
mesh = love.graphics.newMesh(vertexFormat, vertices, "triangles")
```

See Also

- love.graphics
- Mesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

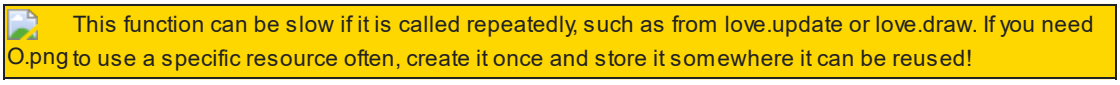
love.graphics.newParticleSystem

love.graphics.newParticleSystem · revision 22510 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Creation and usage of a particle system
- See Also
- Other Languages

Creates a new ParticleSystem.



Function

Synopsis

```
system = love.graphics.newParticleSystem( image, buffer )
```

Arguments

`Image image`
: The image to use.

`number buffer (1000)`
: The max number of particles at the same time.

Returns

`ParticleSystem system`
: A new ParticleSystem.

Function

Available since LÖVE 0.9.1

This variant is not supported in earlier versions.

Synopsis

```
system = love.graphics.newParticleSystem( texture, buffer )
```

Arguments

`Texture texture`
: The texture (Image or Canvas) to use.

`number buffer (1000)`

The max number of particles at the same time.

Returns

`ParticleSystem system`
A new ParticleSystem.

Examples

Creation and usage of a particle system

You can use the LÖVE logo as an example image.

```
function love.load()
    local img = love.graphics.newImage('logo.png')

    psystem = love.graphics.newParticleSystem(img, 32)
    psystem:setParticleLifetime(2, 5) -- Particles live at least 2s and at most 5s.
    psystem:setEmissionRate(5)
    psystem:setSizeVariation(1)
    psystem:setLinearAcceleration(-20, -20, 20, 20) -- Random movement in all directions.
    psystem:setColors(1, 1, 1, 1, 1, 1, 1, 0) -- Fade to transparency.
end

function love.draw()
    -- Draw the particle system at the center of the game window.
    love.graphics.draw(psystem, love.graphics.getWidth() * 0.5, love.graphics.getHeight() * 0.5)
end

function love.update(dt)
    psystem:update(dt)
end
```

See Also

- love.graphics
- ParticleSystem:clone
- ParticleSystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newPixelEffect

love.graphics.newPixelEffect · revision 18026 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Effect Language
- Effect Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.9.0

It has been renamed to love.graphics.newShader.

Creates a new PixelEffect object for hardware-accelerated pixel level effects.

A PixelEffect contains at least one function, named `effect`, which is the effect itself, but it can contain additional functions.

Function

Synopsis

```
pixeleffect = love.graphics.newPixelEffect( code )
```

Arguments

`string code`

The pixel effect code.

Returns

`PixelEffect pixeleffect`

A PixelEffect object for use in drawing operations.

Effect Language

Pixel effects are not programmed in Lua, but by using a special effect language instead. The effect language is basically GLSL 1.20 (http://www.opengl.org/sdk/docs/manglsl/) (specs (http://www.opengl.org/registry/doc/GLSLangSpec.Full.1.20.8.pdf)) with a few aliases added for existing types:

GLSL	Effect language
float	number
sampler2D	Image
uniform	extern

texture2D(tex, uv) Texel(tex, uv)

Effect Function

Synopsis

```
vec4 effect( vec4 color, Image texture, vec2 texture_coords, vec2 screen_coords )
```

Arguments

vec4 color
: The drawing color set with love.graphics.setColor.

Image texture
: The texture of the image or canvas being drawn.

vec2 texture_coords
: Coordinates of the pixel relative to the texture. The y-axis of canvases are inverted. Coordinates (1,1) would be the top right corner of the canvas.

vec2 screen_coords
: Coordinates of the pixel on the screen. Pixel coordinates are not normalized (unlike texture coordinates). (0.5, 0.5) is the bottom left of the screen.

Returns

vec4 output_color
: The color of the pixel.

See Also

- love.graphics
- PixelEffect
- love.graphics.setPixelEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newQuad

love.graphics.newQuad · revision 28245 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Use a Quad to display part of an Image:
- See Also
- Other Languages

Creates a new Quad.

The purpose of a Quad is to use a fraction of a texture to draw objects, as opposed to drawing the entire texture. It is most useful for sprite sheets and atlases: in a sprite atlas, multiple sprites reside in the same texture, quad is used to draw a specific sprite from that texture; in animated sprites with all frames residing in the same texture, quad is used to draw specific frame from the animation.


This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Synopsis

```
quad = love.graphics.newQuad( x, y, width, height, sw, sh )
```

Arguments

number x
: The top-left position in the Texture along the x-axis.

number y
: The top-left position in the Texture along the y-axis.

number width
: The width of the Quad in the Texture. (Must be greater than 0.)

number height
: The height of the Quad in the Texture. (Must be greater than 0.)

number sw
: The reference width, the width of the Texture. (Must be greater than 0.)

number sh
: The reference height, the height of the Texture. (Must be greater than 0.)

Returns

Quad quad

The new Quad.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
quad = love.graphics.newQuad( x, y, width, height, texture )
```

Arguments

number x

The top-left position in the Texture along the x-axis.

number y

The top-left position in the Texture along the y-axis.

number width

The width of the Quad in the Texture. (Must be greater than 0.)

number height

The height of the Quad in the Texture. (Must be greater than 0.)

Texture texture

The texture whose width and height will be used as the reference width and height.

Returns

Quad quad

The new Quad.

Examples

Use a Quad to display part of an Image:

```
img = love.graphics.newImage("mushroom-64x64.png")

-- Let's say we want to display only the top-left
-- 32x32 quadrant of the Image:
top_left = love.graphics.newQuad(0, 0, 32, 32, img)

-- And here is bottom left:
bottom_left = love.graphics.newQuad(0, 32, 32, 32, img)

function love.draw()
    love.graphics.draw(img, top_left, 50, 50)
    love.graphics.draw(img, bottom_left, 50, 200)
end
```

See Also

- love.graphics
- Quad

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newScreenshot

love.graphics.newScreenshot · revision 21258 · last edited 2023-07-05

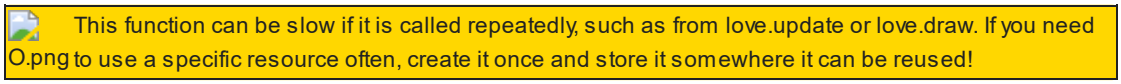
On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Removed in LÖVE 11.0

It has been replaced by love.graphics.captureScreenshot.

Creates a screenshot and returns the image data.



Function

Synopsis

```
screenshot = love.graphics.newScreenshot( )
```

Arguments

None.

Returns

`ImageData screenshot`

The image data of the screenshot.

Function

Available since LÖVE 0.9.0

This variant is not supported in earlier versions.

Synopsis

```
screenshot = love.graphics.newScreenshot( copyAlpha )
```

Arguments

`boolean copyAlpha (false)`

Whether to include the screen's alpha channel in the ImageData. If false, the screenshot will be fully opaque.

Returns

`ImageData screenshot`
The image data of the screenshot.

Examples

Create a new screenshot and write it to the save directory.

```
function love.load()
    love.filesystem.setIdentity('screenshot_example');
end

function love.keypressed()
    local screenshot = love.graphics.newScreenshot();
    screenshot:encode('png', os.time() .. '.png');
end
```

See Also

- love.graphics
- ImageData
- ImageData:encode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newShader

love.graphics.newShader · revision 25026 · last edited 2026-04-11

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Shader Language
- Pixel Shader Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Vertex Shader Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Notes
- Built-in variables
- Examples
  - Create a shader using vertex and pixel shader code which behaves as if no shader is set.
  - Access the pre-transformed vertex position in the pixel shader with the varying keyword.
  - Combine the above example into one string or file with the help of #ifdef.
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from love.graphics.newPixelEffect.



This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Creates a new Shader object for hardware-accelerated vertex and pixel effects. A Shader contains either vertex shader code, pixel shader code, or both.

Shaders are small programs which are run on the graphics card when drawing. Vertex shaders are run once for each vertex (for example, an image has 4 vertices - one at each corner. A Mesh might have many more.) Pixel shaders are run once for each pixel on the screen which the drawn object touches. Pixel shader code is executed after all the object's vertices have been processed by the vertex shader.

Function

Synopsis

```
shader = love.graphics.newShader( code )
```

Arguments

`string code`

The pixel shader or vertex shader code, or a filename pointing to a file with the code.

Returns

`Shader shader`

A Shader object for use in drawing operations.

Function

Synopsis

```
shader = love.graphics.newShader( pixelcode, vertexcode )
```

Arguments

`string pixelcode`
: The pixel shader code, or a filename pointing to a file with the code.

`string vertexcode`
: The vertex shader code, or a filename pointing to a file with the code.

Returns

`Shader shader`
: A Shader object for use in drawing operations.

Notes

The pixelcode and vertexcode arguments can be in any order.

Shader Language

Shaders are not programmed in Lua, but by using a special shader language – GLSL, with a few aliases and a different entry point for convenience – instead. GLSL has very similar syntax to C. None of the aliases LÖVE provides are mandatory, but using `Texel` instead of `texture2D` is recommended since `Texel` works in all glsl versions, whereas `texture2D` does not work in GLSL 3.

GLSL	LÖVE shader language
sampler2D	Image
sampler2DArray	ArrayImage
samplerCube	CubeImage
sampler3D	VolumeImage
texture2D(tex, uv) (in GLSL 1)	Texel(tex, uv)
texture(tex, uv) (in GLSL 3)	Texel(tex, uv)
float	number (deprecated)
uniform	extern (deprecated)

The version of GLSL used depends on whether the `#pragma language glsl3` line is added to the top of a shader file, as well as whether LÖVE is running on a desktop or mobile device:

LÖVE shader language	desktop GLSL version	mobile GLSL version
glsl1 (default)	GLSL 1.20 (http://www.opengl.org/registry/doc/GLSLangSpec.Full.1.20.8.pdf)	GLSL ES 1.00 (https://www.khronos.org/files/opengles_shading_language.pdf)

glsl3	GLSL 3.30 (https://www.khronos.org/registry/OpenGL/specs/gl/GLSLangSpec.3.30.pdf)	GLSL ES 3.00 (https://www.khronos.org/registry/OpenGL/specs/es/3.0/GLSL_ES_Specification_3.00.pdf)

GLSL 3 is not supported on some older systems. Use love.graphics.getSupported to check at run-time.

Vertex shader code must contain at least one function, named `position`, which is the function that will produce transformed vertex positions of drawn objects in screen-space.

Pixel shader code must contain at least one function, named `effect`, which is the function that will produce the color which is blended onto the screen for each pixel a drawn object touches.

LÖVE provides several useful Shader Variables by default. Additionally, LÖVE exposes a function `VideoTexel(uv)` which yields the color value of the currently drawn video at that position. Since Videos are drawn as YUV data in multiple textures, and then converted in the shader, the Texel function cannot be used.

Pixel Shader Function

When an object is drawn, the pixel shader `effect` function is called hundreds or thousands of times: once for each pixel on the screen that the object touches. The pixel shader is run after the vertex shader, if there is one.

Synopsis

```
vec4 effect( vec4 color, Image tex, vec2 texture_coords, vec2 screen_coords )
```

Arguments

`vec4 color`
: The drawing color set with love.graphics.setColor or the per-vertex Mesh color.

`Image tex`
: The texture of the image or canvas being drawn.

`vec2 texture_coords`
: The location inside the texture to get pixel data from. Texture coordinates are usually normalized to the range of (0, 0) to (1, 1), with the top-left corner being (0, 0).

`vec2 screen_coords`
: Coordinates of the pixel on the screen. Pixel coordinates are not normalized (unlike texture coordinates). (0.5, 0.5) represents the top left of the screen (bottom left in LÖVE versions prior to 0.10.0).

Returns

`vec4 output_color`
: The color of the pixel.

Notes


On mobile devices, variables in pixel shaders use `mediump` (16 bit float) precision by default instead of 32 bit float, for performance reasons. This may cause numeric instability or visual artifacts for larger numbers. Use the `highp` qualifier when declaring a variable (for example `highp float pos;`) to make it use always 32 bit float precision. Furthermore, `highp` precision is not supported on all devices, particularly GLES2 devices. Use love.graphics.getSupported to check!

If no pixel shader is used, LÖVE internally uses a default. This is its code:

```
vec4 effect(vec4 color, Image tex, vec2 texture_coords, vec2 screen_coords)
{
```

```
    vec4 texturecolor = Texel(tex, texture_coords);
    return texturecolor * color;
}
```

Or for Video

```
vec4 effect(vec4 color, Image tex, vec2 texture_coords, vec2 screen_coords)
{
    vec4 texturecolor = VideoTexel(texture_coords);
    return texturecolor * color;
}
```

If multiple canvases are being rendered to simultaneously (by giving multiple Canvas parameters to love.graphics.setCanvas), you can use the **void effect** (no arguments!) function instead of **vec4 effect** in order to output a separate color to each Canvas. It has the following prototype:

```
void effect()
{
    // love_Canvases is a writable array of vec4 colors. Each index corresponds to a Canvas.
    // IMPORTANT: If you don't assign a value to all active canvases, bad things will happen.
    love_Canvases[0] = color;
    love_Canvases[1] = color + vec4(0.5);
    // etc.
}
```

If you wish to get the arguments that are passed to the single-canvas version (`vec4 effect`), see the built-in Shader Variables. `color` will be in `VaryingColor`, `texture_coords` will be in `VaryingTexCoord` and `screen_coords` is in `love_PixelCoord`. And if you wish to access the texture used in the drawing operation, you can do that by defining a uniform texture (of the appropriate type) named `MainTex` or by sending it yourself via Shader:send.

Vertex Shader Function

Synopsis

```
vec4 position( mat4 transform_projection, vec4 vertex_position )
```

Arguments

`mat4 transform_projection`
: The transformation matrix affected by love.graphics.translate and friends combined with the orthographic projection matrix.

`vec4 vertex_position`
: The raw un-transformed position of the current vertex.

Returns

`vec4 output_pos`
: The final transformed position of the current vertex.

Notes

If no vertex shader code is used, LÖVE uses a default. This is its code:

```
vec4 position(mat4 transform_projection, vec4 vertex_position)
{
    // The order of operations matters when doing matrix multiplication.
```

```
    return transform_projection * vertex_position;
}
```

Notes

Vertex shader code is run for every vertex drawn to the screen (for example, love.graphics.rectangle will produce 4 vertices) and is used to transform the vertex positions from their original coordinates into screen-space, as well as to send information such as per-vertex color and texture coordinate values to the pixel shader.

Pixel shader code is run for every pixel on the screen that a drawn object touches, and is used to produce the color that will be blended onto the screen at that pixel, often by reading from an image. Pixel shaders are sometimes called fragment shaders.

The `varying` keyword can be used to set a value in the vertex shader and have it interpolated in between vertices and passed down to the pixel shader.

Vertex and Pixel shader code can be combined into one file or string if you wrap the vertex-specific code in `#ifdef VERTEX .. #endif` and the pixel-specific code in `#ifdef PIXEL .. #endif`.

Built-in variables

LÖVE provides several built-in variables for both pixel and vertex shaders. The full list can be found here: Shader Variables.

Examples

Create a shader using vertex and pixel shader code which behaves as if no shader is set.

```
local pixelcode = [[
    vec4 effect( vec4 color, Image tex, vec2 texture_coords, vec2 screen_coords )
    {
        vec4 texcolor = Texel(tex, texture_coords);
        return texcolor * color;
    }
]]

local vertexcode = [[
    vec4 position( mat4 transform_projection, vec4 vertex_position )
    {
        return transform_projection * vertex_position;
    }
]]

shader = love.graphics.newShader(pixelcode, vertexcode)

function love.draw()
    love.graphics.setShader(shader)
    -- draw things
    love.graphics.setShader()
```

```
    -- draw more things
end
```

Access the pre-transformed vertex position in the pixel shader with the **varying** keyword.

vertex shader code

```
varying vec4 vpos;

vec4 position( mat4 transform_projection, vec4 vertex_position )
{
    vpos = vertex_position;
    return transform_projection * vertex_position;
}
```

pixel shader code

```
varying vec4 vpos;

vec4 effect( vec4 color, Image tex, vec2 texture_coords, vec2 screen_coords )
{
    texture_coords += vec2(cos(vpos.x), sin(vpos.y));
    vec4 texcolor = Texel(tex, texture_coords);
    return texcolor * color;
}
```

Combine the above example into one string or file with the help of **#ifdef**.

```
varying vec4 vpos;

#ifdef VERTEX
vec4 position( mat4 transform_projection, vec4 vertex_position )
{
    vpos = vertex_position;
    return transform_projection * vertex_position;
}
#endif

#ifdef PIXEL
vec4 effect( vec4 color, Image tex, vec2 texture_coords, vec2 screen_coords )
{
    texture_coords += vec2(cos(vpos.x), sin(vpos.y));
    vec4 texcolor = Texel(tex, texture_coords);
    return texcolor * color;
}
#endif
```

See Also

- love.graphics
- Shader
- love.graphics.setShader
- Shader Variables

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newSpriteBatch

love.graphics.newSpriteBatch · revision 27764 · last edited 2024-06-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Creates a new SpriteBatch object.


This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Synopsis

```
spriteBatch = love.graphics.newSpriteBatch( image, maxsprites )
```

Arguments

`Image image`
: The Image to use for the sprites.

`number maxsprites (1000)`
: The maximum number of sprites that the SpriteBatch can contain at any given time. Since version 11.0, additional sprites added past this number will automatically grow the spritebatch.

Returns

`SpriteBatch spriteBatch`
: The new SpriteBatch.

Function

Available since LÖVE 0.8.0
This variant is not supported in earlier versions.

Synopsis

```
spriteBatch = love.graphics.newSpriteBatch( image, maxsprites, usage )
```

Arguments

`Image image`
: The Image to use for the sprites.

number maxsprites (1000)
: The maximum number of sprites that the SpriteBatch can contain at any given time. Since version 11.0, additional sprites added past this number will automatically grow the spritebatch.

SpriteBatchUsage usage ("dynamic")
: The expected usage of the SpriteBatch. The specified usage mode affects the SpriteBatch's memory usage and performance.

Returns

SpriteBatch spriteBatch
: The new SpriteBatch.

Function

Available since LÖVE 0.9.1
This variant is not supported in earlier versions.

Synopsis

```
spriteBatch = love.graphics.newSpriteBatch( texture, maxsprites, usage )
```

Arguments

Texture texture
: The Image or Canvas to use for the sprites.

number maxsprites (1000)
: The maximum number of sprites that the SpriteBatch can contain at any given time. Since version 11.0, additional sprites added past this number will automatically grow the spritebatch.

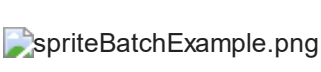
SpriteBatchUsage usage ("dynamic")
: The expected usage of the SpriteBatch. The specified usage mode affects the SpriteBatch's memory usage and performance.

Returns

SpriteBatch spriteBatch
: The new SpriteBatch.

Examples

This example uses this image for all the graphics:




```
local entities = {}
local spriteBatch, protagonist

function love.load()
    local image = love.graphics.newImage("spriteBatchExample.png")
    spriteBatch = love.graphics.newSpriteBatch(image)

    local quadPlayer   = love.graphics.newQuad(0,  0,  32, 32, image:getDimensions())
    local quadTree     = love.graphics.newQuad(32, 0,  32, 32, image:getDimensions())
    local quadSign     = love.graphics.newQuad(0,  32, 32, 32, image:getDimensions())
    local quadDogHouse = love.graphics.newQuad(32, 32, 32, 32, image:getDimensions())
```

```
    local windowWidth, windowHeight = love.graphics.getDimensions()

    love.graphics.setBackgroundColor(.05, .15, .05)

    -- Spawn lots of random things on an uneven grid.
    for baseY = 0, windowHeight, 30 do
        for baseX = 0, windowWidth, 40 do
            local quad = (
                love.math.random() < .10 and quadSign     or
                love.math.random() < .03 and quadDogHouse or
                quadTree
            )
            table.insert(entities, {
                quad   = quad,
                x      = baseX + math.random(-12, 12),
                y      = baseY + math.random(-8,  8 ),
                serial = #entities,
            })
        end
    end

    -- Spawn protagonist.
    protagonist = {quad=quadPlayer, x=0, y=0, serial=#entities} -- We set the position in love.update().
    table.insert(entities, protagonist)
end

function love.update(dt)
    local windowWidth, windowHeight = love.graphics.getDimensions()

    -- Make the protagonist move around in a circle.
    local angle    = .4 * love.timer.getTime()
    protagonist.x = windowWidth/2  + .3*windowWidth  * math.cos(angle)
    protagonist.y = windowHeight/2 + .3*windowHeight * math.sin(angle)
end

function love.draw()
    -- Sort entities by their Y position.
    table.sort(entities, function(entity1, entity2)
        if entity1.y ~= entity2.y then
            return entity1.y < entity2.y
        end
        return entity1.serial < entity2.serial -- If Y is the same, use the serial number as fallback.
    end)

    -- Add entities to sprite batch for drawing.
    spriteBatch:clear()
    for _, entity in ipairs(entities) do
        spriteBatch:add(entity.quad, math.floor(entity.x), math.floor(entity.y))
    end

    -- Finally, draw the sprite batch to the screen.
    love.graphics.draw(spriteBatch)
end
```

See Also

- love.graphics
- SpriteBatch

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newStencil

love.graphics.newStencil · revision 15932 · last edited 2026-04-14

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.9.0

Lua functions can be used directly with love.graphics.stencil (or love.graphics.setStencil in 0.9).

Creates a new stencil.

Function

Synopsis

```
myStencil = love.graphics.newStencil( stencilFunction )
```

Arguments

`function stencilFunction`

Function that draws the stencil.

Returns

`function myStencil`

Function that defines the new stencil.

See Also

- love.graphics
- love.graphics.setStencil
- love.graphics.setInvertedStencil

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newText

love.graphics.newText · revision 29032 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
- Returns
- Example
- Example
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Creates a new drawable Text object.


This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Synopsis

```
text = love.graphics.newText( font, textstring )
```

Arguments

`Font font`

The font to use for the text.

`string textstring (nil)`

The initial string of text that the new Text object will contain. May be nil.

Returns

`Text text`

The new drawable Text object.

Function

Synopsis

```
text = love.graphics.newText( font, coloredtext )
```

Arguments

`Font font`

The font to use for the text.

`table coloredtext`

A table containing colors and strings to add to the object, in the form of `{color1, string1, color2, string2, ...}`.

`table color1`
: A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

`string string1`
: A string of text which has a color specified by the previous color.

`table color2`
: A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

`string string2`
: A string of text which has a color specified by the previous color.

`tables and strings ...`
: Additional colors and strings.

Returns

`Text text`
: The new drawable Text object.

Example

```lua
local font = love.graphics.getFont()
--regular text
local plainText = love.graphics.newText(font, "Hello world")
--colored text
local coloredText = love.graphics.newText(font, {{1, 0, 0}, "Hello ", {0, 0, 1}, " world"})
```

Example

Create text and reflection:

```lua
love.graphics.setFont (love.graphics.newFont (50))

font = love.graphics.getFont ()
text = love.graphics.newText(font)
height = font:getHeight( )
text:add( {{1,1,1}, "Hello World!"}, 0, 0)
text:add( {{0.8,0.8,0}, "Hello World!"}, 0, 0, 0, 1, -1, 0.32*height, 1.6*height,
-0.4,0)

function love.draw()
    love.graphics.draw (text, 10, 10)
end
```



Text object

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text, when drawing the Text object.

See Also

- love.graphics
- Text
- Font

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newTextBatch

love.graphics.newTextBatch · revision 28969 · last edited 2025-11-25

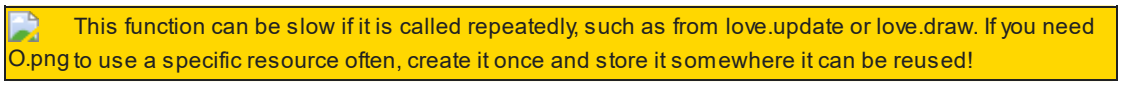
On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Example
  - Notes
- See Also
- Other Languages

Available since LÖVE 12.0

It has been renamed from Text.

Creates a new drawable TextBatch object.



Function

Synopsis

```
text = love.graphics.newTextBatch( font, textstring )
```

Arguments

`Font font`
: The font to use for the text.

`string textstring (nil)`
: The initial string of text that the new TextBatch object will contain. May be nil.

Returns

`TextBatch text`
: The new drawable TextBatch object.

Function

Synopsis

```
text = love.graphics.newTextBatch( font, coloredtext )
```

Arguments

`Font font`
: The font to use for the text.

`table coloredtext`
: A table containing colors and strings to add to the object, in the form of `{color1, string1, color2, string2, ...}`.

table color1
: A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string1
: A string of text which has a color specified by the previous color.

table color2
: A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string2
: A string of text which has a color specified by the previous color.

tables and strings ...
: Additional colors and strings.

Returns

TextBatch text
: The new drawable TextBatch object.

Example

```
local font = love.graphics.getFont()
--regular text
local plainText = love.graphics.newTextBatch(font, "Hello world")
--colored text
local coloredText = love.graphics.newTextBatch(font, {{1, 0, 0}, "Hello ", {0, 0, 1}, " world"})
```

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text, when drawing the TextBatch object.

See Also

- love.graphics
- TextBatch
- Font

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newTexture

love.graphics.newTexture · revision 29379 · last edited 2024-12-02

love.graphics.newTexture is an unreleased feature in LÖVE 12.0.

love.graphics.newVideo

love.graphics.newVideo · revision 29128 · last edited 2026-04-06

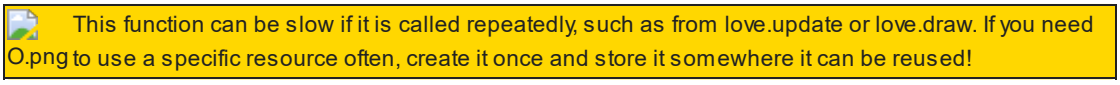
On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Creates a new drawable Video. Currently only Ogg Theora video files are supported.



Function

Synopsis

```
video = love.graphics.newVideo( filename )
```

Arguments

`string filename`

The file path to the Ogg Theora video file.

Returns

`Video video`

A new Video.

Function

Synopsis

```
video = love.graphics.newVideo( videostream )
```

Arguments

`VideoStream videostream`

A video stream object.

Returns

`Video video`
A new Video.

Function

Available since LÖVE 11.0
This variant is not supported in earlier versions.

Synopsis

```
video = love.graphics.newVideo( filename, settings )
```

Arguments

`string filename`
The file path to the Ogg Theora video file (or VideoStream).

`table settings`
A table containing the following fields:

`boolean audio (false)`
Whether to try to load the video's audio into an audio Source. If not explicitly set to true or false, it will try without causing an error if the video has no audio.

`number dpiscale (love.graphics.getDPIScale())`
The DPI scale factor of the video.

Returns

`Video video`
A new Video.

Function

Available since LÖVE 0.10.0 and removed in LÖVE 11.0
This variant is not supported in earlier or later versions.

Synopsis

```
video = love.graphics.newVideo( filename, loadaudio )
```

Arguments

`string filename`
The file path to the Ogg Theora video file.

`boolean loadaudio (nil)`

Whether to try to load the video's audio into an audio Source. If not explicitly set to true or false, it will try without causing an error if the video has no audio.

Returns

`Video video`
A new Video.

Function

Available since LÖVE 0.10.0 and removed in LÖVE 11.0
This variant is not supported in earlier or later versions.

Synopsis

```
video = love.graphics.newVideo( videostream, loadaudio )
```

Arguments

`VideoStream videostream`
A video stream object.

`boolean loadaudio (nil)`
Whether to try to load the video's audio into an audio Source. If not explicitly set to true or false, it will try without causing an error if the video has no audio.

Returns

`Video video`
A new Video.

Examples

Load and play a video.

```
function love.load()
    video = love.graphics.newVideo("myvideo.ogv")
    video:play()
end

function love.draw()
    love.graphics.draw(video, 0, 0)
end
```

Notes

Löve only provides the basic functionality provided by libtheora. If your video plays fine in a video player but not in Löve, try different encoding options.

See Also

- love.graphics
- Video

- love.graphics.draw

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.newVolumeImage

love.graphics.newVolumeImage · revision 25502 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Creates a new volume (3D) Image.



This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Volume images are 3D textures with width, height, and depth. They can't be rendered directly, they can only be used in Shader code (and sent to the shader via Shader:send). Or, more accurately, the default shaders can't render volume images, and you need to make one yourself to do it.

To use a volume image in a Shader, it must be declared as a `VolumeImage` or `sampler3D` type (instead of `Image` or `sampler2D`). The `Texel(VolumeImage image, vec3 texcoords)` shader function must be used to get pixel colors from the volume image. The vec3 argument is a normalized texture coordinate with the z component representing the depth to sample at (ranging from [0, 1]).

Volume images are typically used as lookup tables in shaders for color grading, for example, because sampling using a texture coordinate that is partway in between two pixels can interpolate across all 3 dimensions in the volume image, resulting in a smooth gradient even when a small-sized volume image is used as the lookup table.

Array images are a much better choice than volume images for storing multiple different sprites in a single array image for directly drawing them.

Function

Creates a volume Image given multiple image files with matching dimensions.

Synopsis

```
image = love.graphics.newVolumeImage( layers, settings )
```

Arguments

`table layers`

A table containing filepaths to images (or File, FileData, ImageData, or CompressedImageData objects), in an array. A table of tables can also be given, where each sub-table represents a single mipmap level and contains all layers for that mipmap.

`table settings (nil)`

Optional table of settings to configure the volume image, containing the following fields:

`boolean mipmaps (false)`

True to make the image use mipmaps, false to disable them. Mipmaps will be automatically generated if the image isn't a compressed texture format.

`boolean linear (false)`

True to treat the image's pixels as linear instead of sRGB, when gamma correct rendering is enabled. Most images are authored as sRGB.

Returns

`Image image`
A volume Image object.

Notes

Volume images are not supported on some older mobile devices. Use love.graphics.getTextureTypes to check at runtime.

See Also

- love.graphics
- Image
- TextureType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.origin

love.graphics.origin · revision 30159 · last edited 2026-04-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Resets the current coordinate transformation to its default state.

This function reverses any previous calls to love.graphics.translate, love.graphics.rotate, love.graphics.scale or love.graphics.shear.

It is called automatically right before love.draw.

Function

Synopsis

```
love.graphics.origin()
```

Arguments

None

Returns

Nothing.

Example

```
local image = love.graphics.newImage("path_to_your_image")
function love.draw()
    love.graphics.push()   -- stores the coordinate system
        love.graphics.scale(0.5, 0.5)   -- reduce everything by 50% in both X and Y coordinates
        love.graphics.draw(image, 0, 0)  -- you can see a scaled image that you loaded on the left top of the screen.
        love.graphics.print("Scaled text", 50, 50)   -- print half-sized text at 25x25
        love.graphics.draw(image, 0, 0)
            love.graphics.push()
            love.graphics.origin()  -- Reset the state to the defaults.
            love.graphics.draw(image, 0, 0) -- Draw the image on screen as if nothing was scaled.
        love.graphics.pop()   -- return to our scaled coordinate state.
        love.graphics.print("Scaled text", 100, 100)   -- print half-sized text at 50x50
    love.graphics.pop()   -- return to the previous stored coordinated
    love.graphics.print("Normal text", 50, 50)
end
```

See Also

- love.graphics

- love.graphics.pop
- love.graphics.push
- love.graphics.rotate
- love.graphics.scale
- love.graphics.shear
- love.graphics.translate

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.point

love.graphics.point · revision 28084 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- See Also
- Other Languages

Available since LÖVE 0.3.0 and removed in LÖVE 0.10.0

It has been replaced by love.graphics.points.

Draws a point.

Function

Synopsis

```
love.graphics.point( x, y )
```

Arguments

number x
: The position on the x-axis.

number y
: The position on the y-axis.

Returns

Nothing.

Notes

The pixel grid is actually offset to the center of each pixel. So to get clean pixels drawn use 0.5 + integer increments.

Points are not affected by love.graphics.scale - their size is always in pixels.

Examples

Render a starfield

```
function love.load()
   local screen_width, screen_height = love.graphics.getDimensions()
   local max_stars = 100   -- how many stars we want

   stars = {}   -- table which will hold our stars

   for i=1, max_stars do   -- generate the coords of our stars
      local x = love.math.random(5, screen_width-5)   -- generate a "random" number for the x coord of this star
      local y = love.math.random(5, screen_height-5)   -- both coords are limited to the screen size, minus 5 pixels of
padding
      stars[i] = {x, y}   -- stick the values into the table
   end
```

```
end

function love.draw()
   for i, star in ipairs(stars) do   -- loop through all of our stars
      love.graphics.point(star[1], star[2])   -- draw each point
   end
end
```

See Also

- love.graphics
- love.graphics.setPointSize
- love.graphics.setPointStyle

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.points

love.graphics.points · revision 26777 · last edited 2026-04-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Notes
- Examples
- See Also
- Other Languages

Available since LÖVE 0.10.0

It has replaced love.graphics.point.

Draws one or more points.

Function

Synopsis

```
love.graphics.points( x, y, ... )
```

Arguments

number x
: The position of the first point on the x-axis.

number y
: The position of the first point on the y-axis.

number ...
: The x and y coordinates of additional points.

Returns

Nothing.

Function

Synopsis

```
love.graphics.points( points )
```

Arguments

table points
: A table containing multiple point positions, in the form of `{x, y, ...}`.

  number x
  : The position of the first point on the x-axis.

  number y
  : The position of the first point on the y-axis.

number ...
The x and y coordinates of additional points.

Returns

Nothing.

Function

Draws one or more individually colored points.

In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.

Synopsis

```
love.graphics.points( points )
```

Arguments

table points
A table containing multiple individually colored points, in the form of `{point, ...}`.

table point
A table containing the position and color of the first point, in the form of `{x, y, r, g, b, a}`. The color components are optional.

table ...
Additional tables containing the position and color of more points, in the form of `{x, y, r, g, b, a}`. The color components are optional.

Returns

Nothing.

Notes

The global color set by love.graphics.setColor is modulated (multiplied) with the per-point colors.

Notes

The pixel grid is actually offset to the center of each pixel. So to get clean pixels drawn use 0.5 + integer increments.

Points are not affected by love.graphics.scale - their size is always in pixels.

Examples

Render a starfield

```
function love.load()
   local screen_width, screen_height = love.graphics.getDimensions()
   local max_stars = 100   -- how many stars we want

   stars = {}   -- table which will hold our stars

   for i=1, max_stars do   -- generate the coords of our stars
      local x = love.math.random(5, screen_width-5)   -- generate a "random" number for the x coord of this star
```

```
      local y = love.math.random(5, screen_height-5)   -- both coords are limited to the screen size, minus 5 pixels of
padding
      stars[i] = {x, y}   -- stick the values into the table
   end
end

function love.draw()
   love.graphics.points(stars)  -- draw all stars as points
end
```

See Also

- love.graphics
- love.graphics.setPointSize

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.polygon

love.graphics.polygon · revision 29933 · last edited 2025-09-30

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Two ways of drawing the same triangle
  - Draw concave polygon
  - Draw rotated rectangle
  - Draw rotated boid (directed triangle)
  - Pixel-perfect Hexadecagon
- See Also
- Other Languages

Available since LÖVE 0.4.0

This function is not supported in earlier versions.

Draws a polygon.

Following the mode argument, this function can accept multiple numeric arguments or a single table of numeric arguments. In either case the arguments are interpreted as alternating x and y coordinates of the polygon's vertices.



When in **fill** mode, the polygon must be convex (http://en.wikipedia.org/wiki/Convex_polygon) and simple (http://en.wikipedia.org/wiki/Simple_polygon) or rendering artifacts may occur. love.math.triangulate and love.math.isConvex can be used in 0.9.0+.

Function

Synopsis

```
love.graphics.polygon( mode, ... )
```

Arguments

`DrawMode mode`
: How to draw the polygon.

`number ...`
: The vertices of the polygon.

Returns

Nothing.

Function

Synopsis

```
love.graphics.polygon( mode, vertices )
```

Arguments

`DrawMode mode`

How to draw the polygon.

table vertices

The vertices of the polygon as a table.

Returns

Nothing.

Examples

Two ways of drawing the same triangle

This example shows how to give the coordinates explicitly and how to pass a table argument.



Triangle drawn using love.graphics.polygon

```
-- Giving the coordinates directly.
love.graphics.polygon("fill", 100,100, 200,100, 150,200)

-- Defining a table with the coordinates.
-- This table could be built incrementally too.
local vertices = {100,100, 200,100, 150,200}

-- Passing the table to the function as a second argument.
love.graphics.polygon("fill", vertices)
```

Draw concave polygon

```
local vertices  = {100,100, 200,100, 200,200, 300,200, 300,300, 100,300} -- Concave "L" shape.
local triangles = love.math.triangulate(vertices)

for i, triangle in ipairs(triangles) do
    love.graphics.polygon("fill", triangle)
end
```

Draw rotated rectangle

This is one way to draw a rotated rectangle.

```
function drawRotatedRectangle(mode, x, y, width, height, angle)
    local cosa, sina = math.cos(angle), math.sin(angle)

    local dx1, dy1 = width*cosa,   width*sina
    local dx2, dy2 = -height*sina, height*cosa

    local px1, py1 = x,          y
    local px2, py2 = x+dx1,      y+dy1
    local px3, py3 = x+dx1+dx2, y+dy1+dy2
    local px4, py4 = x+dx2,      y+dy2

    love.graphics.polygon(mode, px1,py1, px2,py2, px3,py3, px4,py4)
end
```

Draw rotated boid (directed triangle)

```
function drawBoid (mode, x, y, length, width , angle) -- position, length, width and angle
    love.graphics.push()
    love.graphics.translate(x, y)
```

```
    love.graphics.rotate( angle )
    love.graphics.polygon(mode, -length/2, -width /2, -length/2, width /2, length/2, 0)
    love.graphics.pop()
end
x, y, angle = 200, 100, math.pi/4
drawBoid ("fill", x, y, 20, 10 , angle)
```



Triangle drawn using love.graphics.poly

Pixel-perfect Hexadecagon (https://en.wikipedia.org/wiki/Hexadecagon)

For better result it can be used with "rough" line style and "nearest" canvas filter.



Hexadecagon using love.graphics.polygon

```
function hexadecagon (mode, x, y, radius) -- same as love.graphics.circle
    local w1 = math.atan(2/5) -- ratio of two integers, 21.8 degrees
    local w2 = math.atan(5/5) -- ratio of two integers, 45 degrees
    local k1 = (w1+w2)/2
    local k2 = math.cos((w2-w1)/2)
    local a = radius
    local b = radius*math.tan (w1 / 2)
    local c = radius*math.cos (k1) / k2
    local d = radius*math.sin (k1) / k2
    local vertices = {
         a, b,  c, d,  d, c,  b, a,
        -b, a, -d, c, -c, d, -a, b,
        -a,-b, -c,-d, -d,-c, -b,-a,
         b,-a,  d,-c,  c,-d,  a,-b}
    love.graphics.translate (x+0.5, y+0.5)
    love.graphics.polygon (mode, vertices)
    love.graphics.translate (-x-0.5, -y-0.5)
end

canvas = love.graphics.newCanvas(width, height)
canvas:setFilter("nearest", "nearest")
love.graphics.setLineStyle("rough")
love.graphics.setCanvas (canvas)
    love.graphics.setColor (0,1,0)
    hexadecagon ("line", 200, 100, 60)
love.graphics.setCanvas ()

function love.draw()
    love.graphics.setColor (1,1,1)
    love.graphics.draw (canvas)
end
```

See Also

- love.graphics
- love.math.triangulate
- love.math.isConvex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.pop

love.graphics.pop · revision 29123 · last edited 2026-04-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Pops the current coordinate transformation from the transformation stack.

This function is always used to reverse a previous push operation. It returns the current transformation state to what it was before the last preceding push.

Function

Synopsis

```
love.graphics.pop()
```

Arguments

None

Returns

Nothing.

Examples

Draw two lines of text, one scaled and one normal, using love.graphics.scale. We use love.graphics.pop to return to normal render scale, after having used love.graphics.push.

```
function love.draw()
   love.graphics.push()   -- stores the coordinate system
-- love.graphics.push("all") -- stores all: transform, colors etc.
   love.graphics.scale(0.5, 0.5)   -- reduce everything by 50% in both X and Y coordinates
   love.graphics.print("Scaled text", 50, 50)   -- print half-sized text at 25x25
   love.graphics.pop()   -- return to stored coordinated
   love.graphics.print("Normal text", 50, 50)
end
```

See Also

- love.graphics
- love.graphics.push
- love.graphics.translate
- love.graphics.rotate
- love.graphics.scale
- love.graphics.shear
- love.graphics.origin

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.present

love.graphics.present · revision 26784 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
  - Show a progress bar while the game loads
- See Also
- Other Languages

Displays the results of drawing operations on the screen.

This function is used when writing your own love.run function. It presents all the results of your drawing operations on the screen. See the example in love.run for a typical use of this function.

Function

Synopsis

```
love.graphics.present( )
```

Arguments

None.

Returns

Nothing.

Notes

- If love.window.setMode has `vsync` equal to `true`, this function can't run more frequently than the refresh rate (e.g. 60 Hz), and will halt the program until ready if necessary.

Examples

Show a progress bar while the game loads

```
-- Dummy functions for the example.
local function loadSounds()  love.timer.sleep(1) end
local function loadSprites() love.timer.sleep(1) end
local function loadLevels()  love.timer.sleep(1) end

local font = nil

local function drawLoadingScreen(progress, text)
    local windowWidth, windowHeight = love.graphics.getDimensions()

    -- Draw text.
    font = font or love.graphics.newFont(26)

    local textX = math.floor((windowWidth - font:getWidth(text)) / 2)
    local textY = math.floor(windowHeight/2) - font:getHeight()
```

```
    love.graphics.setColor(1, 1, 1)
    love.graphics.setFont(font)
    love.graphics.print(text, textX, textY)

    -- Draw progress bar.
    local progressWidthFull    = 400
    local progressWidthCurrent = progress * progressWidthFull
    local progressHeight       = 20
    local progressX            = math.floor((windowWidth - progressWidthFull) / 2)
    local progressY            = math.floor(windowHeight/2)

    love.graphics.setColor(.2, .2, .2)
    love.graphics.rectangle("fill", progressX, progressY, progressWidthFull, progressHeight)
    love.graphics.setColor(.1, .3, 1)
    love.graphics.rectangle("fill", progressX, progressY, progressWidthCurrent, progressHeight)
end

local function presentLoadingScreen(progress, text)
    love.graphics.clear()
    drawLoadingScreen(progress, text)
    love.graphics.present()
end

function love.load()
    presentLoadingScreen(0/3, "Loading sprites...") ; loadSprites()
    presentLoadingScreen(1/3, "Loading sounds...")  ; loadSounds()
    presentLoadingScreen(2/3, "Loading levels...")  ; loadLevels()
end
function love.draw()
    drawLoadingScreen(3/3, "Done loading!")
end
```

See Also

- love.graphics.clear
- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.print

love.graphics.print · revision 26881 · last edited 2025-11-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Examples
  - A lame example
  - Draw centered text
- Notes
- See Also
- Other Languages

Draws text on screen. If no Font is set, one will be created and set (once) if needed.

As of LOVE 0.7.1, when using translation and scaling functions while drawing text, this function assumes the scale occurs first. If you don't script with this in mind, the text won't be in the right position, or possibly even on screen.

love.graphics.print and love.graphics.printf both support UTF-8 encoding. You'll also need a proper Font for special characters.

In versions prior to 11.0, color and byte component values were within the range of 0 to 255 instead of 0 to 1.


Text may appear blurry if it's rendered at non-integer pixel locations.

Function

Synopsis

```
love.graphics.print( text, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

`string text`
: The text to draw.

`number x (0)`
: The position to draw the object (x-axis).

`number y (0)`
: The position to draw the object (y-axis).

`number r (0)`
: Orientation (radians).

number sx (1)
Scale factor (x-axis).

number sy (sx)
Scale factor (y-axis).

number ox (0)
Origin offset (x-axis).

number oy (0)
Origin offset (y-axis).

Available since LÖVE 0.8.0

number kx (0)
Shearing factor (x-axis).

number ky (0)
Shearing factor (y-axis).

Returns

Nothing.

Function

Available since LÖVE 11.0
This variant is not supported in earlier versions.

Synopsis

```
love.graphics.print( text, font, x, y, r, sx, sy, ox, oy, kx, ky )
```

Arguments

string text
The text to draw.

Font font
The Font object to use.

number x (0)
The position of the text on the x-axis.

number y (0)
The position of the text on the y-axis.

number angle (0)
The orientation of the text in radians.

number sx (1)
Scale factor on the x-axis.

number sy (sx)
Scale factor on the y-axis.

number ox (0)
Origin offset on the x-axis.

number oy (0)
Origin offset on the y-axis.

number kx (0)

Shearing / skew factor on the x-axis.

number ky (0)

Shearing / skew factor on the y-axis.

Returns

Nothing.

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Synopsis

```
love.graphics.print( coloredtext, x, y, angle, sx, sy, ox, oy, kx, ky )
```

Arguments

table coloredtext

A table containing colors and strings to add to the object, in the form of `{color1, string1, color2, string2, ...}`.

table color1

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string1

A string of text which has a color specified by the previous color.

table color2

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string2

A string of text which has a color specified by the previous color.

tables and strings ...

Additional colors and strings.

number x (0)

The position of the text on the x-axis.

number y (0)

The position of the text on the y-axis.

number angle (0)

The orientation of the text in radians.

number sx (1)

Scale factor on the x-axis.

number sy (sx)

Scale factor on the y-axis.

number ox (0)

Origin offset on the x-axis.

number oy (0)

Origin offset on the y-axis.

number kx (0)

Shearing / skew factor on the x-axis.

number ky (0)

Shearing / skew factor on the y-axis.

Returns

Nothing.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
love.graphics.print( coloredtext, font, x, y, angle, sx, sy, ox, oy, kx, ky )
```

Arguments

table coloredtext

A table containing colors and strings to add to the object, in the form of `{color1, string1, color2, string2, ...}`.

table color1

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string1

A string of text which has a color specified by the previous color.

table color2

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string2

A string of text which has a color specified by the previous color.

tables and strings ...

Additional colors and strings.

Font font

The Font object to use.

number x (0)

The position of the text on the x-axis.

number y (0)

The position of the text on the y-axis.

number angle (0)

The orientation of the text in radians.

number sx (1)

Scale factor on the x-axis.

number sy (sx)

Scale factor on the y-axis.

number ox (0)

Origin offset on the x-axis.

number oy (0)

Origin offset on the y-axis.

number kx (0)
: Shearing / skew factor on the x-axis.

number ky (0)
: Shearing / skew factor on the y-axis.

Returns

Nothing.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text.

Function

Available since LÖVE 11.0
This variant is not supported in earlier versions.

Synopsis

```
love.graphics.print( text, transform )
```

Arguments

string text
: The text to draw.

Transform transform
: Transformation object.

Returns

Nothing.

Function

Available since LÖVE 11.0
This variant is not supported in earlier versions.

Synopsis

```
love.graphics.print( text, font, transform )
```

Arguments

string text
: The text to draw.

Font font
: The Font object to use.

Transform transform

Transformation object.

Returns

Nothing.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
love.graphics.print( coloredtext, transform )
```

Arguments

`table coloredtext`

A table containing colors and strings to add to the object, in the form of `{color1, string1, color2, string2, ...}`.

`table color1`

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

`string string1`

A string of text which has a color specified by the previous color.

`table color2`

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

`string string2`

A string of text which has a color specified by the previous color.

`tables and strings ...`

Additional colors and strings.

`Transform transform`

Transformation object.

Returns

Nothing.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
love.graphics.print( coloredtext, font, transform )
```

Arguments

table coloredtext

A table containing colors and strings to add to the object, in the form of `{color1, string1, color2, string2, ...}`.

table color1

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string1

A string of text which has a color specified by the previous color.

table color2

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string2

A string of text which has a color specified by the previous color.

tables and strings ...

Additional colors and strings.

Font font

The Font object to use.

Transform transform

Transformation object.

Returns

Nothing.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text.

Examples

A lame example

```
function love.draw()
    love.graphics.setColor(0, 1, 0, 1)
    love.graphics.print("This is a pretty lame example.", 10, 200)
    love.graphics.setColor(1, 0, 0, 1)
    love.graphics.print("This lame example is twice as big.", 10, 250, 0, 2, 2)
    love.graphics.setColor(0, 0, 1, 1)
    love.graphics.print("This example is lamely vertical.", 300, 30, math.pi/2)
end
```

Draw centered text

```
function drawCenteredText(rectX, rectY, rectWidth, rectHeight, text)
    local font       = love.graphics.getFont()
```

```
    local textWidth  = font:getWidth(text)
    local textHeight = font:getHeight()
    love.graphics.print(text, rectX+rectWidth/2, rectY+rectHeight/2, 0, 1, 1, textWidth/2, textHeight/2)
end

function love.draw()
    local x, y = 100, 60
    local w, h = 400, 40
    love.graphics.rectangle("line", x, y, w, h)
    drawCenteredText(x, y, w, h, "I print my LÖVE")
end
```

Notes

In version 0.8.0 and older, love.graphics.print stops at the first '\0' (null) character. This can bite you if you are appending keystrokes to form your string, as some of those are multi-byte unicode characters which will likely contain null bytes.

See Also

- love.graphics
- love.graphics.printf
- Font
- love.graphics.newFont
- love.graphics.setFont
- love.graphics.getFont

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.printf

love.graphics.printf · revision 31637 · last edited 2026-05-31

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Examples
- Notes
- See Also
- Other Languages

Draws formatted text, with word wrap and alignment.

See additional notes in love.graphics.print.

The word wrap limit is applied before any scaling, rotation, and other coordinate transformations. Therefore the amount of text per line stays constant given the same wrap limit, even if the scale arguments change.

In version 0.9.2 and earlier, wrapping was implemented by breaking up words by spaces and putting them back together to make sure things fit nicely within the limit provided. However, due to the way this is done, extra spaces between words would end up missing when printed on the screen, and some lines could overflow past the provided wrap limit. In version 0.10.0 and newer this is no longer the case.

In versions prior to 11.0, color and byte component values were within the range of 0 to 255 instead of 0 to 1.


Aligning does not work as one might expect! It doesn't align to the x/y coordinates given, but in a rectangle, where the limit is the width.


Text may appear blurry if it's rendered at non-integer pixel locations.

Function

Synopsis

```
love.graphics.printf( text, x, y, limit, align, r, sx, sy, ox, oy, kx, ky )
```

Arguments

string text

A text string.

`number x`
: The position on the x-axis.

`number y`
: The position on the y-axis.

`number limit`
: Maximum width in pixels of each line before it gets automatically wrapped to a new line.

`AlignMode align ("left")`
: The alignment.

Available since LÖVE 0.9.0

`number r (0)`
: Orientation (radians).

`number sx (1)`
: Scale factor (x-axis).

`number sy (sx)`
: Scale factor (y-axis).

`number ox (0)`
: Origin offset (x-axis).

`number oy (0)`
: Origin offset (y-axis).

`number kx (0)`
: Shearing factor (x-axis).

`number ky (0)`
: Shearing factor (y-axis).

Returns

Nothing.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
love.graphics.printf( text, font, x, y, limit, align, r, sx, sy, ox, oy, kx, ky )
```

Arguments

`string text`
: A text string.

`Font font`
: The Font object to use.

`number x`
: The position on the x-axis.

`number y`
: The position on the y-axis.

number limit
: Maximum width in pixels of each line before it gets automatically wrapped to a new line.

AlignMode align ("left")
: The alignment.

number r (0)
: Orientation (radians).

number sx (1)
: Scale factor (x-axis).

number sy (sx)
: Scale factor (y-axis).

number ox (0)
: Origin offset (x-axis).

number oy (0)
: Origin offset (y-axis).

number kx (0)
: Shearing factor (x-axis).

number ky (0)
: Shearing factor (y-axis).

Returns

Nothing.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
love.graphics.printf( text, transform, limit, align )
```

Arguments

string text
: A text string.

Transform transform
: Transformation object.

number limit
: Maximum width in pixels of each line before it gets automatically wrapped to a new line.

AlignMode align ("left")
: The alignment.

Returns

Nothing.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
love.graphics.printf( text, font, transform, limit, align )
```

Arguments

string text
: A text string.

Font font
: The Font object to use.

Transform transform
: Transformation object.

number limit
: Maximum width in pixels of each line before it gets automatically wrapped to a new line.

AlignMode align ("left")
: The alignment.

Returns

Nothing.

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Synopsis

```
love.graphics.printf( coloredtext, x, y, limit, align, angle, sx, sy, ox, oy, kx, ky )
```

Arguments

table coloredtext
: A table containing colors and strings to add to the object, in the form of `{color1, string1, color2, string2, ...}`.

  table color1
  : A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

  string string1
  : A string of text which has a color specified by the previous color.

  table color2
  : A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

  string string2
  : A string of text which has a color specified by the previous color.

  tables and strings ...
  : Additional colors and strings.

number x
: The position of the text (x-axis).

number y

The position of the text (y-axis).

number limit

Maximum width in pixels of each line before it gets automatically wrapped to a new line.

AlignMode align

The alignment of the text.

number angle (0)

Orientation (radians).

number sx (1)

Scale factor (x-axis).

number sy (sx)

Scale factor (y-axis).

number ox (0)

Origin offset (x-axis).

number oy (0)

Origin offset (y-axis).

number kx (0)

Shearing / skew factor (x-axis).

number ky (0)

Shearing / skew factor (y-axis).

Returns

Nothing.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
love.graphics.printf( coloredtext, font, x, y, limit, align, angle, sx, sy, ox, oy, kx, ky )
```

Arguments

table coloredtext

A table containing colors and strings to add to the object, in the form of `{color1, string1, color2, string2, ...}`.

table color1

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string1

A string of text which has a color specified by the previous color.

table color2

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string2

A string of text which has a color specified by the previous color.

tables and strings ...

Additional colors and strings.

Font font
The Font object to use.

number x
The position on the x-axis.

number y
The position on the y-axis.

number limit
Maximum width in pixels of each line before it gets automatically wrapped to a new line.

AlignMode align ("left")
The alignment.

number angle (0)
Orientation (radians).

number sx (1)
Scale factor (x-axis).

number sy (sx)
Scale factor (y-axis).

number ox (0)
Origin offset (x-axis).

number oy (0)
Origin offset (y-axis).

number kx (0)
Shearing factor (x-axis).

number ky (0)
Shearing factor (y-axis).

Returns

Nothing.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text.

Function

Available since LÖVE 11.0
This variant is not supported in earlier versions.

Synopsis

```
love.graphics.printf( coloredtext, transform, limit, align )
```

Arguments

table coloredtext
A table containing colors and strings to add to the object, in the form of `{color1, string1, color2, string2, ...}`.

table color1
A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string1

A string of text which has a color specified by the previous color.

table color2

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string2

A string of text which has a color specified by the previous color.

tables and strings ...

Additional colors and strings.

Transform transform

Transformation object.

number limit

Maximum width in pixels of each line before it gets automatically wrapped to a new line.

AlignMode align ("left")

The alignment.

Returns

Nothing.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
love.graphics.printf( coloredtext, font, transform, limit, align )
```

Arguments

table coloredtext

A table containing colors and strings to add to the object, in the form of `{color1, string1, color2, string2, ...}`.

table color1

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string1

A string of text which has a color specified by the previous color.

table color2

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

string string2

A string of text which has a color specified by the previous color.

tables and strings ...

Additional colors and strings.

Font font

The Font object to use.

`Transform transform`
: Transformation object.

`number limit`
: Maximum width in pixels of each line before it gets automatically wrapped to a new line.

`AlignMode align ("left")`
: The alignment.

Returns

Nothing.

Notes

The color set by love.graphics.setColor will be combined (multiplied) with the colors of the text.

Examples

Draw text to the screen with right alignment and a horizontal limit of 125.

```
love.graphics.printf("This text is aligned right, and wraps when it gets too big.", 25, 25, 125, "right")
```

Notes

Note that the limit argument affects the position of your text for 'center' and 'right' alignment.

```
love.graphics.printf("This text is aligned center",100, 100, 200,"center") -- center your text around x = 200/2 + 100 = 200
love.graphics.printf("This text is aligned right",100, 100, 200,"right") -- align right to x = 100 + 200 = 300
```

See Also

- love.graphics.print
- love.graphics
- Font:getWrap

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.push

love.graphics.push · revision 29644 · last edited 2026-04-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Copies and pushes the current coordinate transformation to the transformation stack.

This function is always used to prepare for a corresponding pop operation later. It stores the current coordinate transformation state into the transformation stack and keeps it active. Later changes to the transformation can be undone by using the pop operation, which returns the coordinate transform to the state it was in before calling push.

Function

Pushes the graphics state to the stack.

Synopsis

```
love.graphics.push( stack )
```

Arguments

Available since LÖVE 0.9.2

`StackType stack ("transform")`
: The type of stack to push (e.g. just transformation state, or all love.graphics state).

Returns

Nothing.

Examples

Modify and restore the coordinate system.

```
function love.draw()
    love.graphics.push() -- stores the default coordinate system
    love.graphics.translate(...) -- move the camera position
    love.graphics.scale(...) -- zoom the camera
    -- use the new coordinate system to draw the viewed scene
    love.graphics.pop() -- return to the default coordinates
    -- draw the status display using the screen coordinates
end
```

Available since LÖVE 0.9.2

This example is not supported in earlier versions.

Modify love.graphics state in a function, and restore it easily so other code isn't disturbed.

```
function DrawCoolThing()
  -- save all love.graphics state so any changes can be restored with pop
```

```
  love.graphics.push("all")

  love.graphics.setColor(0, 0, 1)
  love.graphics.setBlendMode("subtract")

  -- push only stores state, it doesn't change it; the translation is still in effect so this will draw at y = 250
  love.graphics.circle("fill", 400, 300, 80)

  -- reset is the counterpart to push("all"), it not only resets the transformation but also all other state
  love.graphics.reset()

  -- color is also a state, it has been reset to white
  love.graphics.setBlendMode("subtract")

  love.graphics.circle("fill", 400, 350, 80)

  -- restore the saved love.graphics state
  love.graphics.pop()
end

function love.draw()
  love.graphics.setColor(1, 0.5, 0.5)
  love.graphics.translate(0, -50)
  love.graphics.circle("fill", 400, 350, 100)

  DrawCoolThing()

  -- still uses the color set at the top of love.draw and the translation of y by -50
  love.graphics.rectangle("fill", 600, 250, 200, 200)
end
```

See Also

- love.graphics
- love.graphics.pop
- love.graphics.translate
- love.graphics.rotate
- love.graphics.scale
- love.graphics.shear
- love.graphics.origin
- love.graphics.reset
- StackType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.quad

love.graphics.quad · revision 14353 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- A quadrilateral that has its top left corner located at 0,0 and w hose w idth and height are 100 pixels.
- See Also
- Other Languages

Removed in LÖVE 0.9.0

This function is not supported in that and later versions. Use love.graphics.polygon instead.

Draws a quadrilateral shape.

Function

Synopsis

```
love.graphics.quad( mode, x1, y1, x2, y2, x3, y3, x4, y4 )
```

Arguments

DrawMode mode
: How to draw the quad.

number x1
: The position of the top left corner along x-axis.

number y1
: The position of the top left corner along y-axis.

number x2
: The position of the top right corner along x-axis.

number y2
: The position of the top right corner along y-axis.

number x3
: The position of the bottom right corner along x-axis.

number y3
: The position of the bottom right corner along y-axis.

number x4
: The position of the bottom left corner along x-axis.

number y4

The position of the bottom left corner along y-axis.

Returns

Nothing.

Examples

A quadrilateral that has its top left corner located at 0,0 and whose width and height are 100 pixels.

```
function love.draw()
    love.graphics.quad("fill", 0, 0, 100, 0, 100, 100, 0, 100)
end
```

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.readbackBuffer

love.graphics.readbackBuffer · revision 29586 · last edited 2025-04-18

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Generates or updates ByteData from the contents of the given GraphicsBuffer.

Unlike love.graphics.readbackBufferAsync, this will not finish and return until the GPU completes its own asynchronous work which may take a frame's worth of time or more.



This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use this while the game is interactive instead of loading, consider love.graphics.readbackBufferAsync instead.

Function

Synopsis

```
data = love.graphics.readbackBuffer( buffer, offset, size, dest, destoffset )
```

Arguments

`GraphicsBuffer buffer`
: The Buffer to read from.

`number offset (0)`
: Optional offset in bytes from the start of the Buffer to read from.

`number size (all)`
: Optional number of bytes to read. The entire contents until the end of the Buffer will be read if no size is given.

`ByteData dest (nil)`
: Optional existing ByteData to fill instead of generating a new one.

`number destoffset (0)`
: Optional offset in bytes from the start of the *dest* ByteData to copy into.

Returns

`ByteData data`
: The new ByteData made from the Buffer's contents, or the updated existing ByteData if one was given.

See Also

- love.graphics
- love.graphics.readbackBufferAsync

- ByteData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.readbackTexture

love.graphics.readbackTexture · revision 30717 · last edited 2026-04-04

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

It has replaced Canvas:newImageData.

Generates or updates ImageData from the contents of the given Texture.

Unlike love.graphics.readbackTextureAsync, this will not finish and return until the GPU completes its own asynchronous work which may take a frame's worth of time or more.



This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use this while the game is interactive instead of loading, consider love.graphics.readbackTextureAsync instead.

Function

Synopsis

```
data = love.graphics.readbackTexture( texture )
```

Arguments

`Texture texture`

The Texture to capture.

Returns

`ImageData data`

The new ImageData made from the Texture's contents.

Function

Synopsis

```
data = love.graphics.readbackTexture( texture, slice, mipmap, x, y, width, height, dest, destx, desty )
```

Arguments

`Texture texture`

The Texture to capture.

number slice

The cubemap face index, array index, or depth layer for cubemap, array, or volume type Textures, respectively. This argument is ignored for regular 2D textures.

number mipmap (1)

The mipmap index to use, for Textures with mipmaps.

number x

The x-axis of the top-left corner (in pixels) of the area within the Texture to capture.

number y

The y-axis of the top-left corner (in pixels) of the area within the Texture to capture.

number width

The width in pixels of the area within the Texture to capture.

number height

The height in pixels of the area within the Texture to capture.

ImageData dest (nil)

Optional existing destination ImageData that will be used instead of creating a new ImageData object.

number destx (0)

The x-axis of the top-left corner (in pixels) of the area within the destination ImageData to use.

number desty (0)

The y-axis of the top-left corner (in pixels) of the area within the destination ImageData to use.

Returns

ImageData data

The new ImageData made from the Texture's contents, or the updated existing ImageData if one was given.

See Also

- love.graphics
- love.graphics.readbackTextureAsync
- love.graphics.captureScreenshot
- ImageData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.rectangle

love.graphics.rectangle · revision 27428 · last edited 2025-01-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
  - Draw a rectangle at 20,50 with a size of 60x120
  - Draw a rotated rectangle
- See Also
- Other Languages

Available since LÖVE 0.3.2

This function is not supported in earlier versions.

Draws a rectangle.

Function

Synopsis

```
love.graphics.rectangle( mode, x, y, width, height, rx, ry, segments )
```

Arguments

DrawMode mode
: How to draw the rectangle.

number x
: The position of top-left corner along the x-axis.

number y
: The position of top-left corner along the y-axis.

number width
: Width of the rectangle.

number height
: Height of the rectangle.

Available since LÖVE 0.10.0

number rx (nil)
: The x-axis radius of each round corner. Cannot be greater than half the rectangle's width.

number ry (rx)
: The y-axis radius of each round corner. Cannot be greater than half the rectangle's height.

number segments (nil)

The number of segments used for drawing the round corners. A default amount will be chosen if no number is given.

Returns

Nothing.

Notes

Custom shaders which use texture coordinates will not work correctly with love.graphics.rectangle (or other shape-drawing functions), since primitive shapes don't have UV texture coordinates associated with the shape's vertex positions.

Examples

Draw a rectangle at 20,50 with a size of 60x120

```
function love.draw()
    love.graphics.rectangle("fill", 20,50, 60,120)
end
```

Draw a rotated rectangle

```
function drawRotatedRectangle(mode, x, y, width, height, angle)
    -- We cannot rotate the rectangle directly, but we
    -- can move and rotate the coordinate system.
    love.graphics.push()
    love.graphics.translate(x, y)
    love.graphics.rotate(angle)
    love.graphics.rectangle(mode, 0, 0, width, height) -- origin in the top left corner
--  love.graphics.rectangle(mode, -width/2, -height/2, width, height) -- origin in the middle
    love.graphics.pop()
end

local angle = 0
function love.update(dt)
    angle = angle + dt * 1 -- Rotate at one radian per second.
end

function love.draw()
    drawRotatedRectangle("fill", 150,150, 100,60, angle)
end
```

See Also

- love.graphics
- love.graphics.polygon
- love.graphics.draw

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.replaceTransform

love.graphics.replaceTransform · revision 22652 · last edited 2026-04-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Replaces the current coordinate transformation with the given Transform object.

Function

Synopsis

```
love.graphics.replaceTransform( transform )
```

Arguments

`Transform transform`

The Transform object to replace the current graphics coordinate transform with.

Returns

Nothing.

Examples

```
local transform = love.math.newTransform()
transform:translate(200, 0)

function love.draw()
    love.graphics.translate(100, 100)
    love.graphics.rectangle("fill", 0, 0, 50, 50)

    love.graphics.replaceTransform(transform)
    love.graphics.rectangle("fill", 0, 0, 50, 50)
end
```

See Also

- love.graphics
- love.math.newTransform
- love.graphics.applyTransform
- love.graphics.push
- love.graphics.pop

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.reset

love.graphics.reset · revision 25397 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Resets the current graphics settings.

Calling reset makes the current drawing color white, the current background color black, disables any active Canvas or Shader, and removes any scissor settings. It sets the BlendMode to `alpha`, enables all color component masks, disables wireframe mode and resets the current graphics transformation to the origin. It also sets both the point and line drawing modes to `smooth` and their sizes to `1.0`.

Function

Synopsis

```
love.graphics.reset( )
```

Arguments

None

Returns

Nothing

Notes

This function, like any other function that change graphics state, only affects the current stack. Calling love.graphics.pop will restore the previous transformation stack or the whole graphics settings, depending on how love.graphics.push was called.

See Also

- love.graphics
- love.graphics.setBackgroundColor
- love.graphics.setColor
- love.graphics.setLineStyle
- love.graphics.setPointStyle
- love.graphics.origin

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.rotate

love.graphics.rotate · revision 27872 · last edited 2026-04-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Rotating a static scene
  - Rotating the entire screen
- See Also
- Other Languages

Rotates the coordinate system in two dimensions.

Calling this function affects all future drawing operations by rotating the coordinate system around the origin by the given amount of radians.

This change lasts until the next love.draw call, or a love.graphics.pop reverts to a previous love.graphics.push, or love.graphics.origin is called - whichever comes first.

Function

Synopsis

```
love.graphics.rotate( angle )
```

Arguments

number angle

The amount to rotate the coordinate system in radians.

Returns

Nothing.

Examples

Rotating a static scene

This example shows how to rotate a static scene around a given point. Since the rotation is always around the origin, translating the center of the screen to the origin and back around the rotate operation makes the effective rotation point be the center of the screen. This is demonstrated by drawing a rectangle that shows the rotation and a point that is right in the center and thus does not move when the scene rotates. Note that the drawing commands have coordinates that depend solely on the screen size.

```
local angle = 0

function love.update(dt)
    -- Rotate a quarter turn per second.
    angle = angle + .5*math.pi * dt
end

function love.draw()
    local width, height = love.graphics.getDimensions()

    local centerX = width/2
    local centerY = height/2
```

```
    -- Rotate around the center of the screen by angle radians.
    love.graphics.translate(centerX, centerY)
    love.graphics.rotate(angle)
    love.graphics.translate(-centerX, -centerY)

    -- Draw a white rectangle slightly off center.
    love.graphics.setColor(1, 1, 1)
    love.graphics.rectangle("fill", centerX-100, centerY-100, 300, 400)

    -- Draw some black text with the top left corner at the center.
    love.graphics.setColor(0, 0, 0)
    love.graphics.print("Lorem ipsum.", centerX, centerY)

    -- Draw a five-pixel-wide blue point at the center.
    love.graphics.setPointSize(5)
    love.graphics.setColor(0, 0, 1)
    love.graphics.points(centerX, centerY)
end
```

Rotating the entire screen

This example shows how you can rotate the screen 90 degrees, counter clockwise. Especially useful if you just distribute .love files for android, where the game always starts in landscape mode. After this, the width and height of the canvas will be "swapped". Don't forget to always translate the input and do all the screen bounds specific checks accordingly.

```
-- Simple function for translating on-screen coordinates to in-game coordinates.
local function screenToGame(xOnScreen, yOnScreen)
    local xInGame = love.graphics.getHeight() - yOnScreen
    local yInGame = xOnScreen

    return xInGame, yInGame
end

function love.draw()
    local screenWidth, screenHeight = love.graphics.getDimensions()

    -- Transform the coordinate system so the top left in-game corner is in
    -- the bottom left corner of the screen.
    love.graphics.translate(0, screenHeight)
    love.graphics.rotate(-math.pi/2)

    -- Draw some text in the (translated) top left corner.
    love.graphics.setColor(1, 1, 1)
    love.graphics.print("Lorem ipsum.", 5, 5)

    -- Draw a rectangle that lights up when hovered.
    local rectX      = 150
    local rectY      = 250
    local rectWidth  = 180
    local rectHeight = 110

    local mouseX, mouseY = love.mouse.getPosition()     -- On-screen.
    mouseX, mouseY       = screenToGame(mouseX, mouseY) -- In-game.

    local isHovered = (mouseX >= rectX and mouseY >= rectY and mouseX < rectX+rectWidth and mouseY < rectY+rectHeight)

    if isHovered then
        love.graphics.setColor(1, 1, 1)
    else
```

```
        love.graphics.setColor(1, 0, 0)
    end
    love.graphics.rectangle("fill", rectX, rectY, rectWidth, rectHeight)
end
```

See Also

- love.graphics
- love.graphics.push
- love.graphics.pop
- love.graphics.scale
- love.graphics.shear
- love.graphics.translate
- love.graphics.origin

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.scale

love.graphics.scale · revision 28110 · last edited 2026-04-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Example w ith translate and scale
- See Also
- Other Languages

Scales the coordinate system in two dimensions.

By default the coordinate system in LÖVE corresponds to the display pixels in horizontal and vertical directions one-to-one, and the x-axis increases towards the right while the y-axis increases downwards. Scaling the coordinate system changes this relation.

After scaling by sx and sy, all coordinates are treated as if they were multiplied by sx and sy. Every result of a drawing operation is also correspondingly scaled, so scaling by (2, 2) for example would mean making everything twice as large in both x- and y-directions. Scaling by a negative value flips the coordinate system in the corresponding direction, which also means everything will be drawn flipped or upside down, or both. Scaling by zero is not a useful operation.

Scale and translate are not commutative operations, therefore, calling them in different orders will change the outcome.

This change lasts until the next love.draw call, or a love.graphics.pop reverts to a previous love.graphics.push, or love.graphics.origin is called - whichever comes first.

Function

Synopsis

```
love.graphics.scale( sx, sy )
```

Arguments

number sx
: The scaling in the direction of the x-axis.

number sy (sx)
: The scaling in the direction of the y-axis. If omitted, it defaults to same as parameter sx.

Returns

Nothing.

Examples

Draw two lines of text, one scaled and one normal. Uses love.graphics.push and love.graphics.pop to return to normal render scale.

```
function love.draw()
   love.graphics.push()
   love.graphics.scale(0.5, 0.5)   -- reduce everything by 50% in both X and Y coordinates
   love.graphics.print("Scaled text", 50, 50)
   love.graphics.pop()
```

```
    love.graphics.print("Normal text", 50, 50)
end
```

Example with translate and scale

```
function love.load ()
    -- if your code was optimized for fullHD:
    window = {translateX = 40, translateY = 40, scale = 2, width = 1920, height = 1080}
    width, height = love.graphics.getDimensions ()
    love.window.setMode (width, height, {resizable=true, borderless=false})
    resize (width, height) -- update new translation and scale
end

function love.update (dt)
    -- mouse position with applied translate and scale:
    local mx = math.floor ((love.mouse.getX()-window.translateX)/window.scale+0.5)
    local my = math.floor ((love.mouse.getY()-window.translateY)/window.scale+0.5)
    -- your code here, use mx and my as mouse X and Y positions
end

function love.draw ()
    -- first translate, then scale
    love.graphics.translate (window.translateX, window.translateY)
    love.graphics.scale (window.scale)
    -- your graphics code here, optimized for fullHD
    love.graphics.rectangle('line', 0, 0, 1920, 1080)
end

function resize (w, h) -- update new translation and scale:
    local w1, h1 = window.width, window.height -- target rendering resolution
    local scale = math.min (w/w1, h/h1)
    window.translateX, window.translateY, window.scale = (w-w1*scale)/2, (h-h1*scale)/2, scale
end

function love.resize (w, h)
    resize (w, h) -- update new translation and scale
end
```

See Also

- love.graphics
- love.graphics.pop
- love.graphics.push
- love.graphics.rotate
- love.graphics.shear
- love.graphics.translate
- love.graphics.origin

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setBackgroundColor

love.graphics.setBackgroundColor · revision 29118 · last edited 2024-02-18

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Sets the background color.


In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.

Function

Synopsis

```
love.graphics.setBackgroundColor( red, green, blue, alpha )
```

Arguments

number red
The red component (0-1).

number green
The green component (0-1).

number blue
The blue component (0-1).

Available since LÖVE 0.8.0

number alpha (1)
The alpha component (0-1).

Returns

Nothing.

Function

Available since LÖVE 0.7.0

This variant is not supported in earlier versions.

Synopsis

```
love.graphics.setBackgroundColor( rgb )
```

Arguments

table rgb
: A numerical indexed table with the red, green and blue values as numbers.

Returns

Nothing.

Function

Available since LÖVE 0.8.0
This variant is not supported in earlier versions.

Synopsis

```
love.graphics.setBackgroundColor( rgba )
```

Arguments

table rgba
: A numerical indexed table with the red, green, blue and alpha values as numbers.

Returns

Nothing.

Examples

```
-- set background color to #84C1EE (132, 193, 238)
local r, g, b = love.math.colorFromBytes(132, 193, 238)
love.graphics.setBackgroundColor(r, g, b)


function love.draw()
    -- draw any geometry:
    love.graphics.setColor(1,1,1)
    love.graphics.circle('fill', 400, 300, 250)
    love.graphics.setColor(0,0,0)
    love.graphics.circle('line', 400, 300, 250)
end
```

See Also

- love.graphics
- love.graphics.clear
- love.graphics.getBackgroundColor

- love.run

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setBlendMode

love.graphics.setBlendMode · revision 28995 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- See Also
- Other Languages

Available since LÖVE 0.2.0

This function is not supported in earlier versions.

Sets the blending mode.

Function

Synopsis

```
love.graphics.setBlendMode( mode, alphamode )
```

Arguments

`BlendMode mode`

The blend mode to use.

Available since LÖVE 0.10.0

`BlendAlphaMode alphamode ("alphamultiply")`

What to do with the alpha of drawn objects when blending.

Returns

Nothing.

Notes

The default "alphamultiply" alpha mode should normally be preferred except when drawing content with pre-multiplied alpha. If content is drawn to a Canvas using the "alphamultiply" mode, the Canvas texture will have pre-multiplied alpha afterwards, so the "premultiplied" alpha mode should generally be used when drawing a Canvas to the screen.

Examples

```
function love.load()
    love.graphics.setBackgroundColor(.21, .67, .97)
end

function love.draw()
    love.graphics.setBlendMode("alpha") -- Default blend mode.
    love.graphics.setColor(.90, .17, .48)
    love.graphics.rectangle("fill", 50,50, 100,100)

    love.graphics.setColor(.04, .39, .90)
    love.graphics.setBlendMode("multiply", "premultiplied")
```

```
    love.graphics.rectangle("fill", 75,75, 125,125)
end
```

See Also

- love.graphics
- love.graphics.getBlendMode
- BlendMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setBlendState

love.graphics.setBlendState · revision 29161 · last edited 2024-04-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Sets the low-level blending state. love.graphics.setBlendMode is a simpler function for setting a higher level blending mode.

Function

Synopsis

```
love.graphics.setBlendState( operation, srcFactor, dstFactor )
```

Arguments

`BlendOperation operation`
: The blend operation to use for RGB and alpha.

`BlendFactor srcFactor`
: The blend factor to use for source RGB and source alpha.

`BlendFactor dstFactor`
: The blend factor to use for destination RGB and destination alpha.

Returns

Nothing.

Function

Synopsis

```
love.graphics.setBlendState( operationRGB, operationA, srcFactorRGB, srcFactorA, dstFactorRGB, dstFactorA )
```

Arguments

`BlendOperation operationRGB`
: The blend operation to use for RGB.

`BlendOperation operationA`
: The blend operation to use for alpha.

`BlendFactor srcFactorRGB`
: The blend factor to use for source RGB.

`BlendFactor srcFactorA`
: The blend factor to use for source alpha.

`BlendFactor dstFactorRGB`
: The blend factor to use for destination RGB.

`BlendFactor dstFactorA`
: The blend factor to use for destination alpha.

Returns

Nothing.

Function

Disables all blending.

Synopsis

```
love.graphics.setBlendState( )
```

Arguments

None.

Returns

Nothing.

Notes

You can use Visual glBlendFunc + glBlendEquation Tool (https://w w w.andersriggelsen.dk/glblendfunc.php) to (approximately) visualize the blending state.

See Also

- love.graphics
- love.graphics.getBlendState
- love.graphics.setBlendMode
- BlendMode Formulas
- BlendOperation
- BlendFactor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setCanvas

love.graphics.setCanvas · revision 22740 · last edited 2025-11-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Drawing to a canvas
  - Advanced setup with the table variant of love.graphics.setCanvas
- See Also
- Other Languages

Available since LÖVE 0.8.0

It has been renamed from love.graphics.setRenderTarget.

Captures drawing operations to a Canvas.

Function

Sets the render target to a specified Canvas. All drawing operations until the next *love.graphics.setCanvas* call will be redirected to the Canvas and not shown on the screen.

When using a stencil or depth testing with an active Canvas, the stencil buffer or depth buffer must be explicitly enabled in setCanvas via the variants below.

Note that no canvas should be active when *love.graphics.present* is called. *love.graphics.present* is called at the end of love.draw in the default love.run, hence if you activate a canvas using this function, you normally need to deactivate it at some point before *love.draw* finishes.

Synopsis

```
love.graphics.setCanvas( canvas, mipmap )
```

Arguments

`Canvas canvas`

The new target.

Available since LÖVE 11.0

`number mipmap (1)`

The mipmap level to render to, for Canvases with mipmaps.

Returns

Nothing.

Function

Resets the render target to the screen, i.e. re-enables drawing to the screen.

Synopsis

```
love.graphics.setCanvas( )
```

Arguments

None.

Returns

Nothing.

Function

Available since LÖVE 0.9.0
This variant is not supported in earlier versions.

Sets the render target to multiple simultaneous 2D Canvases. All drawing operations until the next *love.graphics.setCanvas* call will be redirected to the specified canvases and not shown on the screen.

Synopsis

```
love.graphics.setCanvas( canvas1, canvas2, ... )
```

Arguments

Canvas canvas1
The first render target.

Canvas canvas2
The second render target.

Canvas ...
More canvases.

Returns

Nothing.

Notes

Normally all drawing operations will draw only to the first canvas passed to the function, but that can be changed if a pixel shader is used with the `void effect` function instead of the regular `vec4 effect`.

All canvas arguments must have the same widths and heights and the same texture type. Not all computers which support Canvases will support multiple render targets. If love.graphics.isSupported("multicanvas") returns true, at least 4 simultaneously active canvases are supported.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Sets the render target to the specified layer/slice and mipmap level of the given non-2D Canvas. All drawing operations until the next *love.graphics.setCanvas* call will be redirected to the Canvas and not shown on the screen.

Synopsis

```
love.graphics.setCanvas( canvas, slice, mipmap )
```

Arguments

`Canvas canvas`

The new render target.

`number slice`

For cubemaps this is the cube face index to render to (between 1 and 6). For Array textures this is the array layer. For volume textures this is the depth slice. 2D canvases should use a value of 1.

`number mipmap (1)`

The mipmap level to render to, for Canvases with mipmaps.

Returns

Nothing.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Sets the active render target(s) and active stencil and depth buffers based on the specified setup information. All drawing operations until the next *love.graphics.setCanvas* call will be redirected to the specified Canvases and not shown on the screen.

Synopsis

```
love.graphics.setCanvas( setup )
```

Arguments

`table setup`

A table specifying the active Canvas(es), their mipmap levels and active layers if applicable, and whether to use a stencil and/or depth buffer.

`RenderTargetSetup [1]`

The Canvas to render to.

`RenderTargetSetup [2] (nil)`

An additional Canvas to render to, if multiple simultaneous render targets are wanted.

`RenderTargetSetup ...`

Additional Canvases to render to, if multiple simultaneous render targets are wanted.

boolean stencil (false)

Whether an internally managed stencil buffer should be used, if the `depthstencil` field isn't set.

boolean depth (false)

Whether an internally managed depth buffer should be used, if the `depthstencil` field isn't set.

RenderTargetSetup depthstencil (nil)

An optional custom depth/stencil formatted Canvas to use for the depth and/or stencil buffer.

Returns

Nothing.

Notes

The `RenderTargetSetup` parameters can either be a Canvas object, or a table in the following format: `{canvas, mipmap=#, layer=#, face=#}`

Canvas [1]

The Canvas to use for this active render target.

number mipmap (1)

The mipmap level to render to, for Canvases with mipmaps.

number layer (1)

Only used for Volume and Array-type Canvases. For Array textures this is the array layer to render to. For volume textures this is the depth slice.

number face (1)

Only used for Cubemap-type Canvases. The cube face index to render to (between 1 and 6)

Examples

Drawing to a canvas

```
function love.load()
    -- create canvas
    canvas = love.graphics.newCanvas()

    -- direct drawing operations to the canvas
    love.graphics.setCanvas(canvas)

    -- draw colored square to canvas
    love.graphics.setColor(0.8, 0.9, 0.4)
    love.graphics.rectangle("fill", 0, 0, 100, 100)

    -- re-enable drawing to the main screen
    love.graphics.setCanvas()
end

function love.draw()
    -- draw scaled canvas to screen
    love.graphics.setColor(1, 1, 1)
    love.graphics.draw(canvas, 200, 100, 0, 0.5, 0.5)
end
```

Advanced setup with the table variant of love.graphics.setCanvas

```
-- Allow love.graphics.stencil calls when drawing to the given Canvas.
love.graphics.setCanvas({canvas, stencil=true})

-- Use multiple simultaneous render targets when drawing to several canvases of the Array Texture type,
-- and use a custom depth buffer as well.
canvas1 = love.graphics.newCanvas(128, 128, 4, {type="array"})
canvas2 = love.graphics.newCanvas(128, 128, 8, {type="array"})
depthcanvas = love.graphics.newCanvas(128, 128, {format="depth24", readable=true})

love.graphics.setCanvas({
    {canvas1, layer = 3},
    {canvas2, layer = 1},
    depthstencil = depthcanvas,
})
```

See Also

- love.graphics
- love.graphics.getCanvas
- Canvas
- Canvas:renderTo

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setCaption

love.graphics.setCaption · revision 11697 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

Moved to the love.window module as love.window.setTitle.

Sets the window caption.

Function

Synopsis

```
love.graphics.setCaption( caption )
```

Arguments

`string caption`

The new window caption.

Returns

Nothing.

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setColor

love.graphics.setColor · revision 29165 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Set the color from r, g, b bytes
  - Draw a red, blue and green circle
  - Display a Venn diagram
- See Also
- Other Languages

Sets the color used for drawing. The color is used for all subsequent draw operations.

In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.

Function

Synopsis

```
love.graphics.setColor( red, green, blue, alpha )
```

Arguments

number red
: The amount of red.

number green
: The amount of green.

number blue
: The amount of blue.

number alpha (1)
: The amount of alpha.

Returns

Nothing.

Function

Available since LÖVE 0.7.0

This variant is not supported in earlier versions.

Synopsis

```
love.graphics.setColor( rgba )
```

Arguments

table rgba

A numerical indexed table with the red, green, blue and alpha values as numbers. The alpha is optional and defaults to 1 if it is left out.

Returns

Nothing.

Examples

Set the color from r, g, b bytes

Using love.math.colorFromBytes we can specify color components in the range 0..255.

```
love.graphics.setColor(love.math.colorFromBytes(128, 234, 255))
```

Draw a red, blue and green circle

```
function love.draw()
    love.graphics.setColor(1, 0, 0)
    love.graphics.circle("fill", 50,50, 20)

    love.graphics.setColor(0, 0, 1)
    love.graphics.circle("fill", 50,100, 20)

    local myColor = {0, 1, 0, 1}
    love.graphics.setColor(myColor)
    love.graphics.circle("fill", 50,150, 20)
end
```

Display a Venn diagram

```
function love.load()
    love.graphics.setBackgroundColor(1, 1, 1)
end

function love.draw()
    local baseX   = love.graphics.getWidth()/2
    local baseY   = 400
    local radius  = 100
    local offsetY = radius/2 * math.sqrt(3)

    love.graphics.setBlendMode("multiply", "premultiplied")
    love.graphics.setColor(1, .6, .6)
    love.graphics.circle("fill", baseX-radius/2, baseY, radius)
    love.graphics.setColor(.6, 1, .6)
    love.graphics.circle("fill", baseX, baseY-offsetY, radius)
    love.graphics.setColor(.6, .6, 1)
    love.graphics.circle("fill", baseX+radius/2, baseY, radius)
end
```

See Also

- love.graphics
- love.math.colorFromBytes
- HSL color (an alternate color space, based on human perception)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setColorMask

love.graphics.setColorMask · revision 16591 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the color mask. Enables or disables specific color components when rendering and clearing the screen. For example, if **red** is set to **false**, no further changes will be made to the red component of any pixels.

Function

Enables color masking for the specified color components.

Synopsis

```
love.graphics.setColorMask( red, green, blue, alpha )
```

Arguments

`boolean red`
: Render red component.

`boolean green`
: Render green component.

`boolean blue`
: Render blue component.

`boolean alpha`

Render alpha component.

Returns

Nothing.

Function

Disables color masking.

Synopsis

```
love.graphics.setColorMask( )
```

Arguments

None.

Returns

Nothing.

See Also

- love.graphics
- love.graphics.getColorMask

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setColorMode

love.graphics.setColorMode · revision 27192 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.2.0 and removed in LÖVE 0.9.0

This function is not supported in earlier or later versions.

Sets the color mode (which controls how images are affected by the current color).

Function

Synopsis

```
love.graphics.setColorMode( mode )
```

Arguments

`ColorMode mode`

The color mode to use.

Returns

Nothing.

See Also

- love.graphics
- love.graphics.getColorMode
- love.graphics.setBlendMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setDefaultFilter

love.graphics.setDefaultFilter · revision 29306 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from love.graphics.setDefaultImageFilter.

Sets the default scaling filters used with Images, Canvases, and Fonts.



This function does not apply retroactively to loaded images. Existing objects retain their current scaling filters.

Function

Synopsis

```
love.graphics.setDefaultFilter( min, mag, anisotropy )
```

Arguments

`FilterMode min`
: Filter mode used when scaling the image down.

`FilterMode mag (min)`
: Filter mode used when scaling the image up.

`number anisotropy (1)`
: Maximum amount of Anisotropic Filtering used.

Returns

Nothing.

See Also

- love.graphics
- love.graphics.getDefaultFilter
- FilterMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setDefaultImageFilter

love.graphics.setDefaultImageFilter · revision 17406 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.9.0

It has been renamed to love.graphics.setDefaultFilter.

Sets the default scaling filters.

Function

Synopsis

```
love.graphics.setDefaultImageFilter( min, mag )
```

Arguments

`FilterMode min`
: Filter mode used when scaling the image down.

`FilterMode mag`
: Filter mode used when scaling the image up.

Returns

Nothing.

Notes

This function does not apply retroactively to loaded images.

See Also

- love.graphics
- love.graphics.getDefaultImageFilter

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setDepthMode

love.graphics.setDepthMode · revision 29159 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Configures depth testing and writing to the depth buffer.

This is low-level functionality designed for use with custom vertex shaders and Meshes with custom vertex attributes. No higher level APIs are provided to set the depth of 2D graphics such as shapes, lines, and Images.


O.png

Depth writes will not work without an active depth buffer.

- When drawing to the main screen, the window's depth buffer must be enabled in love.conf or love.window.setMode.
- When drawing to a Canvas, the *depth* field must be set to true in a table passed to love.graphics.setCanvas, or a custom Canvas with a depth PixelFormat must be set in the *depthstencil* field in a table passed to setCanvas.


O.png

Writing to the depth buffer is generally incompatible with rendering alpha-blended sprites / images. By default depth is determined by geometry (vertices) instead of texture alpha values, the depth buffer only stores a single depth value per pixel, and alpha blending **requires** back-to-front rendering for blending to be correct.

Function

Synopsis

```
love.graphics.setDepthMode( comparemode, write  )
```

Arguments

`CompareMode comparemode`

Depth comparison mode used for depth testing.

`boolean write`

Whether to write update / write values to the depth buffer when rendering.

Returns

Nothing.

Function

Disables depth testing and depth writes.

Synopsis

```
love.graphics.setDepthMode( )
```

Arguments

None.

Returns

Nothing.

See Also

- love.graphics
- love.graphics.getDepthMode
- love.graphics.setCanvas
- love.graphics.clear
- PixelFormat
- Mesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setFont

love.graphics.setFont · revision 28001 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Draw some text with default font, 18px
  - Draw some text with custom font, 18px
- See Also
- Other Languages

Set an already-loaded Font as the current font or create and load a new one from the file and size.

It's recommended that Font objects are created with love.graphics.newFont in the loading stage and then passed to this function in the drawing stage.

Function

Synopsis

```
love.graphics.setFont( font )
```

Arguments

Font font
: The Font object to use.

Returns

Nothing.

Function

Removed in LÖVE 0.8.0

This variant is not supported in that and later versions.

Synopsis

```
love.graphics.setFont( filename, size )
```

Arguments

string filename
: The filepath to the font.

number size (12)

The size of the font.

Returns

Nothing.

Function

Removed in LÖVE 0.8.0
This variant is not supported in that and later versions.

This variant creates a new font using the default font and the size specified, and sets it as the current font. **Do not use this function in love.update or love.draw.** That would create a new font every frame, eating up memory very quickly.

Synopsis

```
love.graphics.setFont( size )
```

Arguments

number size (12)
The size of the font.

Returns

Nothing.

Examples

Draw some text with default font, 18px

```
function love.load()
    font = love.graphics.newFont(18)
end

function love.draw()
    love.graphics.setFont(font)
    love.graphics.print("Hello, World!", 200, 100)
end
```

Draw some text with custom font, 18px

```
function love.load()
    font = love.graphics.newFont("MyFont.ttf", 18)
end

function love.draw()
    love.graphics.setFont(font)
```

```
    love.graphics.print("Hello, World!", 200, 100)
end
```

See Also

- love.graphics
- love.graphics.newFont
- love.graphics.getFont
- love.graphics.setNewFont

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setFrontFaceWinding

love.graphics.setFrontFaceWinding · revision 22237 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Sets whether triangles with clockwise- or counterclockwise-ordered vertices are considered front-facing.

This is designed for use in combination with Mesh face culling. Other love.graphics shapes, lines, and sprites are not guaranteed to have a specific winding order to their internal vertices.

Function

Synopsis

```
love.graphics.setFrontFaceWinding( winding )
```

Arguments

`VertexWinding winding`

The winding mode to use. The default winding is counterclockwise ("ccw").

Returns

Nothing.

See Also

- love.graphics
- love.graphics.getFrontFaceWinding
- love.graphics.setMeshCullMode
- Mesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setIcon

love.graphics.setIcon · revision 28506 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.9.0

Moved to the love.window module as love.window.setIcon.

Set window icon. This feature is not completely supported on Windows (apparently an SDL bug (https://github.com/love2d/love/issues/148), not a LÖVE bug).

Function

Synopsis

```
love.graphics.setIcon( image )
```

Arguments

`Image image`
: The window icon.

Returns

Nothing.

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setInvertedStencil

love.graphics.setInvertedStencil · revision 17694 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.10.0

It has been replaced by love.graphics.stencil and love.graphics.setStencilTest.

Defines an inverted stencil for the drawing operations or releases the active one.

It's the same as love.graphics.setStencil with the mask inverted.

Function

Synopsis

```
love.graphics.setInvertedStencil( stencilFunction )
```

Arguments

`function stencilFunction`
: Function that draws the stencil.

Returns

Nothing.

Function

Releases the active stencil.

Synopsis

```
love.graphics.setInvertedStencil( )
```

Arguments

None.

Returns

Nothing.

See Also

- love.graphics
- love.graphics.newStencil

- love.graphics.setStencil

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setLine

love.graphics.setLine · revision 10771 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
- See Also
- Other Languages

Removed in LÖVE 0.9.0

Use love.graphics.setLineWidth and love.graphics.setLineStyle.

Sets the line width and style.

Function

Synopsis

```
love.graphics.setLine( width, style )
```

Arguments

number width

The width of the line.

LineStyle style ("smooth")

The LineStyle to use.

Returns

Nothing.

Example

```
love.graphics.setLine(2, "smooth")
love.graphics.line(15, 25, 69, 89)
```

See Also

- love.graphics
- love.graphics.setLineWidth
- love.graphics.setLineStyle

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setLineJoin

love.graphics.setLineJoin · revision 28002 · last edited 2026-04-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Showcase the styles
- See Also
- Other Languages

Sets the line join style. See LineJoin for the possible options.

Function

Synopsis

```
love.graphics.setLineJoin( join )
```

Arguments

`LineJoin join`
: The LineJoin to use.

Returns

Nothing.

Examples

Showcase the styles

```
function love.draw()
    local line = {0,0, 100,20, 40,40, 50,80}
    love.graphics.setLineWidth(15)
    love.graphics.setColor(1, 1, 1, .7)

    love.graphics.translate(300, 100)
    love.graphics.setLineJoin("miter")
    love.graphics.line(line)

    love.graphics.translate(0, 150)
    love.graphics.setLineJoin("none")
    love.graphics.line(line)

    love.graphics.translate(0, 150)
    love.graphics.setLineJoin("bevel")
    love.graphics.line(line)
end
```

See Also

- love.graphics

- love.graphics.getLineJoin

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setLineStipple

love.graphics.setLineStipple · revision 10081 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Additional Information
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This function is not supported in that and later versions.

Sets the line stipple pattern.

Function

Synopsis

```
love.graphics.setLineStipple( pattern, repeat )
```

Arguments

number pattern
: A 16-bit pattern.

number repeat (1)
: Repeat factor.

Returns

Nothing.

Additional Information

A stipple pattern is made up of a 16bit sequence of 0s and 1s, Just like binary.

The pattern is repeated to complete the line. Using the repeat will stretch the pattern and multiplies each 1 and 0. For example if three 1s are consecutively placed they are stretched to six if the repeat is 2. The maximum repeat is 255, and the minimum repeat is 1.

A 1 is a a cue to draw a pixel whereas 0 is the opposite and does not draw a pixel, This is done per pixel for the given line.

A pattern is read from back to front.

0x3F07 would equal to 0011111100000111 in binary.

You can visualise the pattern by reading the binary sequence backwards.

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setLineStyle

love.graphics.setLineStyle · revision 29307 · last edited 2025-10-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
- See Also
- Other Languages

Available since LÖVE 0.3.2

This function is not supported in earlier versions.

Sets the line style.

Function

Synopsis

```
love.graphics.setLineStyle( style )
```

Arguments

`LineStyle style`

The LineStyle to use. Line styles include `smooth` and `rough`.

Returns

Nothing.

Example

Draw lines

```
function love.load()
    love.graphics.setLineStyle( "rough" )
end

function love.draw()
    love.graphics.line (0,0, 200, 50)
    love.graphics.line (0,0, 200, 100)
    love.graphics.line (0,0, 200, 150)
    love.graphics.line (0,0, 200, 200)
    love.graphics.line (0,0, 200, 250)
end
```

See Also

- love.graphics
- love.graphics.getLineStyle
- love.graphics.setLineWidth

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setLineWidth

love.graphics.setLineWidth · revision 24723 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.3.2

This function is not supported in earlier versions.

Sets the line width.

Function

Synopsis

```
love.graphics.setLineWidth( width )
```

Arguments

`number width`
: The width of the line.

Returns

Nothing.

See Also

- love.graphics
- love.graphics.getLineWidth
- love.graphics.setLineStyle

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setMeshCullMode

love.graphics.setMeshCullMode · revision 23945 · last edited 2026-04-16

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Sets whether back-facing triangles in a Mesh are culled.

This is designed for use with low level custom hardware-accelerated 3D rendering via custom vertex attributes on Meshes, custom vertex shaders, and depth testing with a depth buffer.

By default, both front- and back-facing triangles in Meshes are rendered.

Function

Synopsis

```
love.graphics.setMeshCullMode( mode )
```

Arguments

`CullMode mode`

The Mesh face culling mode to use (whether to render everything, cull back-facing triangles, or cull front-facing triangles).

Returns

Nothing.

See Also

- love.graphics
- love.graphics.getMeshCullMode
- love.graphics.setFrontFaceWinding
- love.graphics.setDepthMode
- Mesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setMode

love.graphics.setMode · revision 17411 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Removed in LÖVE 0.9.0

Moved to the love.window module as love.window.setMode.

Changes the window size, or the display mode if fullscreen.

If width or height is 0, setMode will use the width or height of the desktop.

Changing the display mode may have side effects: for example, Canvases will be cleared; make sure to save their contents beforehand.

Function

Synopsis

```
success = love.graphics.setMode( width, height, fullscreen, vsync, fsaa )
```

Arguments

`number width`
: Display width.

`number height`
: Display height.

`boolean fullscreen (false)`
: Fullscreen (true), or windowed (false).

`boolean vsync (true)`
: True if LÖVE should wait for vsync, false otherwise.

`number fsaa (0)`
: The number of MSAA samples.

Returns

`boolean success`
: True if successful, false otherwise.

Notes

If you have disabled the screen in conf.lua and use this function to manually create the window, then you must not call any other love.graphics.* function before this one. Doing so will result in undefined behavior and/or crashes.

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setNewFont

love.graphics.setNewFont · revision 14691 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Creates and sets a new Font.


This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Synopsis

```
font = love.graphics.setNewFont( size )
```

Arguments

number size (12)

The size of the font.

Returns

Font font

The new font.

Function

Synopsis

```
font = love.graphics.setNewFont( filename, size )
```

Arguments

string filename

The path and name of the file with the font.

number size (12)
The size of the font.

Returns

Font font
The new font.

Function

Synopsis

```
font = love.graphics.setNewFont( file, size )
```

Arguments

File file
A File with the font.

number size (12)
The size of the font.

Returns

Font font
The new font.

Function

Synopsis

```
font = love.graphics.setNewFont( data, size )
```

Arguments

Data data
A Data with the font.

number size (12)
The size of the font.

Returns

Font font

The new font.

Function

Synopsis

```
font = love.graphics.setNewFont( rasterizer )
```

Arguments

`Rasterizer rasterizer`
A rasterizer.

Returns

`Font font`
The new font.

See Also

- love.graphics
- love.graphics.newFont
- love.graphics.setFont

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setPixelEffect

love.graphics.setPixelEffect · revision 15939 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Examples
  - Drawing a rectangle using a pixel effect
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.9.0

It has been renamed to love.graphics.setShader.

Sets or resets a PixelEffect as the current pixel effect. All drawing operations until the next *love.graphics.setPixelEffect* will be drawn using the PixelEffect object specified.

Function

Synopsis

```
love.graphics.setPixelEffect( pixeleffect )
```

Arguments

`PixelEffect pixeleffect`

The new pixel effect.

Returns

Nothing.

Notes

Sets the current pixel shader to a specified PixelEffect. All drawing operations until the next *love.graphics.setPixelEffect* will be drawn using the PixelEffect object specified.

Function

Synopsis

```
love.graphics.setPixelEffect( )
```

Arguments

None.

Returns

Nothing.

Notes

Disables pixel effects, allowing unfiltered drawing operations.

Examples

Drawing a rectangle using a pixel effect

```
function love.load()
    effect = love.graphics.newPixelEffect [[
        extern number time;
        vec4 effect(vec4 color, Image texture, vec2 texture_coords, vec2 pixel_coords)
        {
            return vec4((1.0+sin(time))/2.0, abs(cos(time)), abs(sin(time)), 1.0);
        }
    ]]
end

function love.draw()
    -- boring white
    love.graphics.setPixelEffect()
    love.graphics.rectangle('fill', 10,10,790,285)

    -- LOOK AT THE PRETTY COLORS!
    love.graphics.setPixelEffect(effect)
    love.graphics.rectangle('fill', 10,305,790,285)
end

local t = 0
function love.update(dt)
    t = t + dt
```

```
    effect:send("time", t)
end
```

See Also

- love.graphics
- PixelEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setPoint

love.graphics.setPoint · revision 10770 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Removed in LÖVE 0.9.0

Use love.graphics.setPointSize and love.graphics.setPointStyle.

Sets the point size and style.

Function

Synopsis

```
love.graphics.setPoint( size, style )
```

Arguments

`number size`
: The new point size.

`PointStyle style`
: The new point style.

Returns

Nothing.

Examples

Increase the point size by 1 and set it to smooth.

```
local s = love.graphics.getPointSize() + 1
love.graphics.setPoint(s, "smooth")
```

See Also

- love.graphics
- love.graphics.setPointSize
- love.graphics.setPointStyle

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setPointSize

love.graphics.setPointSize · revision 31705 · last edited 2026-09-24

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- See Also
- Other Languages

Sets the point size.


The sizes of points are not affected by love.graphics.scale – they're always in pixels, or since 11.0, in DPI-scaled units.

Function

Synopsis

```
love.graphics.setPointSize( size )
```

Arguments

number size
: The new point size.

Returns

Nothing.

Notes

The provided size must be strictly greater than zero, otherwise the function has no effect.

Examples

Increase the point size by 1, by using love.graphics.getPointSize.

```
function love.draw()
    local s = love.graphics.getPointSize() + 1
    love.graphics.setPointSize(s)
    love.graphics.points(100, 100)
end
```

See Also

- love.graphics
- love.graphics.points
- love.graphics.setPointStyle
- love.graphics.getPointSize

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setPointStyle

love.graphics.setPointStyle · revision 15732 · last edited 2025-11-30

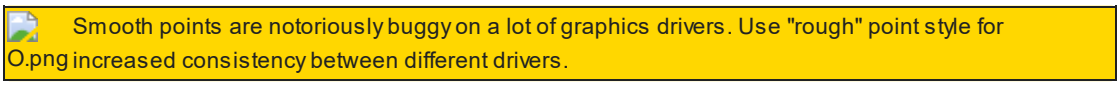
On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Removed in LÖVE 0.10.0

This function is not supported in that and later versions.

Sets the point style.



Function

Synopsis

```
love.graphics.setPointStyle( style )
```

Arguments

`PointStyle style`

The new point style.

Returns

Nothing.

Examples

Toggle between point styles with the help of love.graphics.getPointStyle.

```
if love.graphics.getPointStyle() == "rough" then
   love.graphics.setPointStyle("smooth")
else
   love.graphics.setPointStyle("rough")
end
```

See Also

- love.graphics
- love.graphics.point
- love.graphics.setPointSize
- love.graphics.getPointStyle

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setProjection

love.graphics.setProjection · revision 31698 · last edited 2026-08-15

On this page

- Function
  - Synopsis
  - Arguments
- Function
  - Synopsis
  - Arguments
- Function
  - Synopsis
  - Arguments
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Sets the projection matrix used for rendering.

Function

Synopsis

```
transform = love.graphics.setProjection( e1_1, e1_2, ..., e4_4 )
```

Arguments

number e1_1
: The first column of the first row of the matrix.

number e1_2
: The second column of the first row of the matrix.

number ...
: Additional matrix elements.

number e4_4
: The fourth column of the fourth row of the matrix.

Function

Synopsis

```
transform = love.graphics.setProjection( transform )
```

Arguments

Transform transform

The transform containing the matrix.

Function

Synopsis

```
transform = love.graphics.setProjection( layout, matrix )
```

Arguments

`MatrixLayout layout`
: How to interpret the matrix element arguments (row-major or column-major).

`table matrix`
: A flat table containing the 16 matrix elements.

See Also

- love.graphics
- love.graphics.resetProjection
- Transform:setMatrix

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setRenderTarget

love.graphics.setRenderTarget · revision 15956 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Examples
  - Drawing to a framebuffer
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.8.0

It has been renamed to love.graphics.setCanvas.

Sets or resets a Framebuffer as render target. All drawing operations until the next *love.graphics.setRenderTarget* will be directed to the Framebuffer object specified.

Function

Synopsis

```
love.graphics.setRenderTarget( framebuffer )
```

Arguments

`Framebuffer framebuffer`

The new render target.

Returns

Nothing.

Notes

Sets the render target to a specified Framebuffer. The specified Framebuffer will be cleared. All drawing operations until the next *love.graphics.setRenderTarget* will be redirected to the Framebuffer and not shown on the screen.

Function

Synopsis

```
love.graphics.setRenderTarget( )
```

Arguments

None.

Returns

Nothing.

Notes

Resets the render target to the screen, i.e. re-enables drawing to the screen.

Examples

Drawing to a framebuffer

```
-- draw colored square to framebuffer
love.graphics.setRenderTarget(framebuffer)
love.graphics.setColor(230,240,120)
love.graphics.rectangle('fill',0,0,100,100)
love.graphics.setRenderTarget()

-- draw scaled framebuffer to screen
love.graphics.setColor(255,255,255)
love.graphics.draw(framebuffer, 200,100, 0, .5,.5)
```

See Also

- love.graphics
- Framebuffer
- Framebuffer:renderTo

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setScissor

love.graphics.setScissor · revision 28334 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Only draw on the left side of the screen
- See Also
- Other Languages

Available since LÖVE 0.4.0

This function is not supported in earlier versions.

Sets or disables scissor.

The scissor limits the drawing area to a specified rectangle. This affects all graphics calls, including love.graphics.clear.

The dimensions of the scissor are unaffected by graphical transformations (translate, scale, ...), and it operates on whole pixels rather than having sub-pixel precision.

Function

Limits the drawing area to a specified rectangle.

Synopsis

```
love.graphics.setScissor( x, y, width, height )
```

Arguments

number x

The x-coordinate of the upper left corner of the clipping rectangle.

number y

The y-coordinate of the upper left corner of the clipping rectangle.

number width

The width of the clipping rectangle.

number height

The height of the clipping rectangle.

Returns

Nothing.

Function

Disables scissor.

Synopsis

```
love.graphics.setScissor( )
```

Arguments

None.

Returns

Nothing.

Examples

Only draw on the left side of the screen

```
function love.draw()
    local windowWidth, windowHeight = love.graphics.getDimensions()

    love.graphics.setScissor(0,0, windowWidth/2,windowHeight)
    love.graphics.clear(0, 0, .5)
    love.graphics.circle("fill", windowWidth/2,windowHeight/2, 100)
end
```

See Also

- love.graphics
- love.graphics.getScissor
- love.graphics.intersectScissor
- love.graphics.stencil

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setShader

love.graphics.setShader · revision 12644 · last edited 2026-04-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Examples
  - Drawing a rectangle using a pixel effect shader
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from love.graphics.setPixelEffect.

Sets or resets a Shader as the current pixel effect or vertex shaders. All drawing operations until the next *love.graphics.setShader* will be drawn using the Shader object specified.

Function

Synopsis

```
love.graphics.setShader( shader )
```

Arguments

`Shader shader`

The new shader.

Returns

Nothing.

Notes

Sets the current shader to a specified Shader. All drawing operations until the next *love.graphics.setShader* will be drawn using the Shader object specified.

Function

Synopsis

```
love.graphics.setShader( )
```

Arguments

None.

Returns

Nothing.

Notes

Disables shaders, allowing unfiltered drawing operations.

Examples

Drawing a rectangle using a pixel effect shader

```
function love.load()
    effect = love.graphics.newShader [[
        extern number time;
        vec4 effect(vec4 color, Image texture, vec2 texture_coords, vec2 pixel_coords)
        {
            return vec4((1.0+sin(time))/2.0, abs(cos(time)), abs(sin(time)), 1.0);
        }
    ]]
end

function love.draw()
    -- boring white
    love.graphics.setShader()
    love.graphics.rectangle('fill', 10,10,780,285)

    -- LOOK AT THE PRETTY COLORS!
    love.graphics.setShader(effect)
    love.graphics.rectangle('fill', 10,305,780,285)
end

local t = 0
function love.update(dt)
    t = t + dt
```

```
    effect:send("time", t)
end
```

See Also

- love.graphics
- Shader

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setStencil

love.graphics.setStencil · revision 17693 · last edited 2026-04-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Drawing circles masked by a rectangle
  - Drawing a circle with a hole
  - Drawing two masked triangles with different colors
  - Using an Image as a stencil mask
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.10.0

It has been replaced by love.graphics.stencil and love.graphics.setStencilTest.

Defines or releases a stencil for the drawing operations.

The passed function draws to the stencil instead of the screen, creating an image with transparent and opaque pixels. While active, it is used to test where pixels will be drawn or discarded. Image contents do not directly affect the stencil, but see below for a workaround.

Calling the function without arguments releases the active stencil.

Function

Synopsis

```
love.graphics.setStencil( stencilFunction )
```

Arguments

function stencilFunction

Function that draws the stencil.

Returns

Nothing.

Function

Releases the active stencil.

Synopsis

```
love.graphics.setStencil( )
```

Arguments

None.

Returns

Nothing.

Examples

Drawing circles masked by a rectangle

```
myStencilFunction = function()
   love.graphics.rectangle("fill", 225, 200, 350, 300)
end

love.graphics.setStencil(myStencilFunction)

love.graphics.setColor(255, 0, 0, 120)
love.graphics.circle("fill", 300, 300, 150, 50)
love.graphics.setColor(0, 255, 0, 120)
love.graphics.circle("fill", 500, 300, 150, 50)
love.graphics.setColor(0, 0, 255, 120)
love.graphics.circle("fill", 400, 400, 150, 50)
```

Drawing a circle with a hole

```
myStencilFunction = function()
   love.graphics.circle("fill", 400, 300, 50)
end

love.graphics.setInvertedStencil(myStencilFunction)
love.graphics.circle("fill", 400, 300, 150)
```

Drawing two masked triangles with different colors

```
myStencilFunction = function()
   love.graphics.circle("fill", 400, 300, 60, 25)
end

love.graphics.setStencil(myStencilFunction)
```

```
love.graphics.setColor(155, 0, 128)
love.graphics.polygon("fill", 400, 200, 486, 350, 314, 350)


love.graphics.setInvertedStencil(myStencilFunction)
love.graphics.setColor(144, 214, 128)
love.graphics.polygon("fill", 400, 200, 486, 350, 314, 350)
```

Using an Image as a stencil mask

```
-- a black/white mask image: black pixels will mask, white pixels will pass.
local mask = love.graphics.newImage("mymask.png")

local mask_effect = love.graphics.newShader[[
   vec4 effect (vec4 color, Image texture, vec2 texture_coords, vec2 screen_coords) {
      if (Texel(texture, texture_coords).rgb == vec3(0.0)) {
         // a discarded pixel wont be applied as the stencil.
         discard;
      }
      return vec4(1.0);
   }
]]

function myStencilFunction()
   love.graphics.setShader(mask_effect)
   love.graphics.draw(mask, 0, 0)
   love.graphics.setShader()
end

love.graphics.setStencil(myStencilFunction)
love.graphics.rectangle("fill", 0, 0, 256, 256)
love.graphics.setStencil()
```

See Also

- love.graphics
- love.graphics.setInvertedStencil

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setStencilState

love.graphics.setStencilState · revision 29056 · last edited 2024-06-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function replaces love.graphics.stencil and love.graphics.setStencilTest.

Low-level function to configure the screen's per-pixel stencil buffer. For a simpler API see love.graphics.setStencilMode.

The geometry drawn after configuring the stencil state can set invisible stencil values of pixels, in addition to affecting pixel colors.

The stencil buffer (which contains those stencil values) can act like a mask / stencil - this function can be used to determine how further rendering is affected by the stencil values in each pixel, via the **comparemode** parameter.

Note that unlike love.graphics.setStencilMode this function does **not** prevent subsequent draws from affecting pixel colors, unless love.graphics.setColorMask is used.

Function

Synopsis

```
love.graphics.setStencilState( action, comparemode, value, readmask, writemask )
```

Arguments

`StencilAction action`

Whether and how to modify any stencil values of pixels that are touched by subsequent draws.

`CompareMode comparemode`

The type of comparison (if any) to make for each pixel.

`number value (0)`

The value to use when comparing with the stencil value of each pixel, and the new stencil value to use for pixels if the "replace" stencil action is used. Must be an integer between 0 and 255.

`number readmask (255)`

An 8 bit mask applied to values read from the stencil buffer in subsequent draws.

`number writemask (255)`

An 8 bit mask applied to values written to the stencil buffer in subsequent draws.

Returns

Nothing.

Function

Disables writing to the stencil buffer and using its values, for subsequent draws.

Synopsis

```
love.graphics.setStencilState( )
```

Arguments

None.

Returns

Nothing.

See Also

- love.graphics
- love.graphics.getStencilState
- love.graphics.setStencilMode
- love.graphics.setCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setStencilTest

love.graphics.setStencilTest · revision 27455 · last edited 2026-04-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Drawing circles masked by rectangle, polygon and circle
  - Drawing a circle with a hole
  - Drawing two masked triangles with different colors
- See Also
- Other Languages

Available since LÖVE 0.10.0

Together with love.graphics.stencil, it has replaced love.graphics.setStencil.

Configures or disables stencil testing.

When stencil testing is enabled, the geometry of everything that is drawn afterward will be clipped / stencilled out based on a comparison between the arguments of this function and the stencil value of each pixel that the geometry touches. The stencil values of pixels are affected via love.graphics.stencil.



Starting with version 11.0, a stencil buffer must be set or requested in love.graphics.setCanvas when using stencils with a Canvas. `love.graphics.setCanvas{canvas, stencil=true}` is an easy way to use an automatically provided temporary stencil buffer in that case.

Function

Synopsis

```
love.graphics.setStencilTest( comparemode, comparevalue )
```

Arguments

`CompareMode comparemode`

The type of comparison to make for each pixel.

`number comparevalue`

The value to use when comparing with the stencil value of each pixel. Must be between 0 and 255.

Returns

Nothing.

Function

Disables stencil testing.

Synopsis

```
love.graphics.setStencilTest( )
```

Arguments

None.

Returns

Nothing.

Examples

Drawing circles masked by rectangle, polygon and circle

```
local function myStencilFunction1()
    love.graphics.rectangle("fill", 225, 200, 350, 300)
end

local function myStencilFunction2()
    love.graphics.polygon ("fill", 225, 200, 400, 100, 575, 200)
end

local function myStencilFunction3()
    love.graphics.circle ("fill", 400, 200, 60)
end

function love.draw()

    -- draw a rectangle as a stencil. Each pixel touched by the rectangle will have its stencil value set to 1. The rest will be 0.
    love.graphics.stencil (myStencilFunction1, "replace", 1)

    -- draw polygon as a stencil. Each pixel touched by the polygon will have its stencil value set to 1. The rest will be not changed.
    love.graphics.stencil (myStencilFunction2, "replace", 2, true)

    -- remove circle from stencil. Each pixel touched by the circle will have its stencil value decrement to 1. The rest will be not changed.
    love.graphics.stencil (myStencilFunction3, "decrement", 1, true)

    -- Only allow rendering on pixels whose stencil value is greater than 0.
    love.graphics.setStencilTest ("greater", 0)

    love.graphics.setColor(1, 0, 0, 0.8)
    love.graphics.circle("fill", 300, 300, 150)
```

```
    love.graphics.setColor(0, 1, 0, 0.6)
    love.graphics.circle("fill", 500, 300, 150)

    love.graphics.setColor(0, 0, 1, 0.6)
    love.graphics.circle("fill", 400, 400, 150)

    love.graphics.setColor(1, 1, 1, 1)
    love.graphics.circle("fill", 400, 100, 117)

    love.graphics.setStencilTest()
end
```

Drawing a circle with a hole

```
local function myStencilFunction()
   -- Draw a small circle as a stencil. This will be the hole.
   love.graphics.circle("fill", 400, 300, 50)
end

function love.draw()
   -- Each pixel touched by the circle will have its stencil value set to 1. The rest will be 0.
   love.graphics.stencil(myStencilFunction, "replace", 1)

   -- Configure the stencil test to only allow rendering on pixels whose stencil value is equal to 0.
   -- This will end up being every pixel *except* ones that were touched by the circle drawn as a stencil.
   love.graphics.setStencilTest("equal", 0)
   love.graphics.circle("fill", 400, 300, 150)
   love.graphics.setStencilTest()
end
```

Drawing two masked triangles with different colors

```
local function myStencilFunction()
   love.graphics.circle("fill", 400, 300, 60, 25)
end

function love.draw()
   -- Each pixel touched by the circle will have its stencil value set to 1. The rest will be 0.
   love.graphics.stencil(myStencilFunction, "replace", 1)

   -- Only allow rendering on pixels whose stencil value is greater than 0.
   love.graphics.setStencilTest("greater", 0)
   love.graphics.setColor(0.6, 0, 0.5)
   love.graphics.polygon("fill", 400, 200, 486, 350, 314, 350)

   -- Now only allow rendering on pixels whose stencil value is equal to 0.
   love.graphics.setStencilTest("equal", 0)
   love.graphics.setColor(0.55, 0.85, 0.5)
   love.graphics.polygon("fill", 400, 200, 486, 350, 314, 350)

   love.graphics.setStencilTest()
end
```

The love.graphics.stencil wiki page includes more examples.

See Also

- love.graphics

- love.graphics.getStencilTest
- love.graphics.stencil

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.setWireframe

love.graphics.setWireframe · revision 15941 · last edited 2026-04-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Sets whether wireframe lines will be used when drawing.


This function does nothing on mobile devices and other systems which use OpenGL ES 2.


Wireframe mode should only be used for debugging. The lines drawn with it enabled do not behave like regular love.graphics lines: their widths don't scale with the coordinate transformations or with love.graphics.setLineWidth, and they don't use the smooth LineStyle.

Function

Synopsis

```
love.graphics.setWireframe( enable )
```

Arguments

`boolean enable`

True to enable wireframe mode when drawing, false to disable it.

Returns

Nothing.

See Also

- love.graphics
- love.graphics.isWireframe

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.shear

love.graphics.shear · revision 27870 · last edited 2026-04-16

On this page

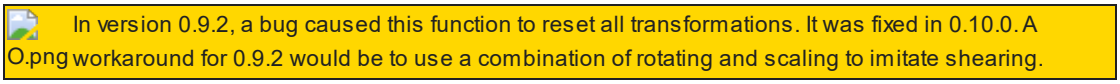
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Squish a rectangle
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Shears the coordinate system.

This change lasts until the next love.draw call, or a love.graphics.pop reverts to a previous love.graphics.push, or love.graphics.origin is called - whichever comes first.



Function

Synopsis

```
love.graphics.shear( kx, ky )
```

Arguments

number kx
: The shear factor on the x-axis.

number ky
: The shear factor on the y-axis.

Returns

Nothing.

Examples

Squish a rectangle

```
function love.draw()
    love.graphics.translate(100, 100)
    local t = love.timer.getTime()
    love.graphics.shear(math.cos(t), math.cos(t * 1.3))
    love.graphics.rectangle('fill', 0, 0, 100, 50)
end
```

See Also

- love.graphics

- love.graphics.pop
- love.graphics.push
- love.graphics.rotate
- love.graphics.scale
- love.graphics.translate
- love.graphics.origin

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.stencil

love.graphics.stencil · revision 21345 · last edited 2026-04-13

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- Drawing circles masked by a rectangle
- Using an Image as a stencil mask
- Allow drawing everywhere except where multiple circles intersect
- See Also
- Other Languages

Available since LÖVE 0.10.0

Together with love.graphics.setStencilTest, it has replaced love.graphics.setStencil.

Draws geometry as a stencil.

The geometry drawn by the supplied function sets invisible stencil values of pixels, instead of setting pixel colors. The stencil buffer (which contains those stencil values) can act like a mask / stencil - love.graphics.setStencilTest can be used afterward to determine how further rendering is affected by the stencil values in each pixel.

Stencil values are integers within the range of [0, 255].



Starting with version 11.0, a stencil buffer must be set or requested in love.graphics.setCanvas when using stencils with a Canvas. `love.graphics.setCanvas{canvas, stencil=true}` is an easy way to use an automatically provided temporary stencil buffer in that case.

Function

Synopsis

```
love.graphics.stencil( stencilfunction, action, value, keepvalues )
```

Arguments

`function stencilfunction`

Function which draws geometry. The stencil values of pixels, rather than the color of each pixel, will be affected by the geometry.

`StencilAction action ("replace")`

How to modify any stencil values of pixels that are touched by what's drawn in the stencil function.

`number value (1)`

The new stencil value to use for pixels if the "replace" stencil action is used. Has no effect with other stencil actions. Must be between 0 and 255.

`boolean keepvalues (false)`

True to preserve old stencil values of pixels, false to re-set every pixel's stencil value to 0 before executing the stencil function. love.graphics.clear will also re-set all stencil values.

Returns

Nothing.

Notes

It is possible to draw to the screen and to the stencil values of pixels at the same time, by using love.graphics.setColorMask inside the stencil function to enable drawing to all color components.

Examples

Drawing circles masked by a rectangle

```
local function myStencilFunction()
   love.graphics.rectangle("fill", 225, 200, 350, 300)
end

function love.draw()
    -- draw a rectangle as a stencil. Each pixel touched by the rectangle will have its stencil value set to 1. The rest will be 0.
    love.graphics.stencil(myStencilFunction, "replace", 1)

   -- Only allow rendering on pixels which have a stencil value greater than 0.
    love.graphics.setStencilTest("greater", 0)

    love.graphics.setColor(1, 0, 0, 0.45)
    love.graphics.circle("fill", 300, 300, 150, 50)

    love.graphics.setColor(0, 1, 0, 0.45)
    love.graphics.circle("fill", 500, 300, 150, 50)

    love.graphics.setColor(0, 0, 1, 0.45)
    love.graphics.circle("fill", 400, 400, 150, 50)

    love.graphics.setStencilTest()
end
```

Using an Image as a stencil mask

```
-- a black/white mask image: black pixels will mask, white pixels will pass.
local mask = love.graphics.newImage("mymask.png")

local mask_shader = love.graphics.newShader[[
   vec4 effect(vec4 color, Image texture, vec2 texture_coords, vec2 screen_coords) {
      if (Texel(texture, texture_coords).rgb == vec3(0.0)) {
         // a discarded pixel wont be applied as the stencil.
         discard;
      }
      return vec4(1.0);
   }
]]

local function myStencilFunction()
   love.graphics.setShader(mask_shader)
   love.graphics.draw(mask, 0, 0)
```

```
   love.graphics.setShader()
end

function love.draw()
    love.graphics.stencil(myStencilFunction, "replace", 1)
    love.graphics.setStencilTest("greater", 0)
    love.graphics.rectangle("fill", 0, 0, 256, 256)
    love.graphics.setStencilTest()
end
```

Allow drawing everywhere except where multiple circles intersect

```
local function myStencilFunction()
   -- Draw four overlapping circles as a stencil.
   love.graphics.circle("fill", 200, 250, 100)
   love.graphics.circle("fill", 300, 250, 100)
   love.graphics.circle("fill", 250, 200, 100)
   love.graphics.circle("fill", 250, 300, 100)
end

function love.draw()
   -- Each pixel touched by each circle will have its stencil value incremented by 1.
   -- The stencil values for pixels where the circles overlap will be at least 2.
   love.graphics.stencil(myStencilFunction, "increment")

   -- Only allow drawing in areas which have stencil values that are less than 2.
   love.graphics.setStencilTest("less", 2)

   -- Draw a big rectangle.
   love.graphics.rectangle("fill", 0, 0, 500, 500)

   love.graphics.setStencilTest()
end
```

The love.graphics.setStencilTest wiki page includes more examples.

See Also

- love.graphics
- love.graphics.setStencilTest
- love.graphics.clear

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.toggleFullscreen

love.graphics.toggleFullscreen · revision 29009 · last edited 2023-11-28

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

Moved to the love.window module as love.window.setFullscreen.

Toggles fullscreen.

Function

Synopsis

```
success = love.graphics.toggleFullscreen( )
```

Arguments

None.

Returns

`boolean success`

True if successful, false otherwise.

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.transformPoint

love.graphics.transformPoint · revision 22653 · last edited 2026-04-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Converts the given 2D position from global coordinates into screen-space.

This effectively applies the current graphics transformations to the given position. A similar Transform:transformPoint method exists for Transform objects.

Function

Synopsis

```
screenX, screenY = love.graphics.transformPoint( globalX, globalY )
```

Arguments

number globalX
: The x component of the position in global coordinates.

number globalY
: The y component of the position in global coordinates.

Returns

number screenX
: The x component of the position with graphics transformations applied.

number screenY
: The y component of the position with graphics transformations applied.

See Also

- love.graphics
- love.graphics.inverseTransformPoint
- Transform

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.translate

love.graphics.translate · revision 27869 · last edited 2026-04-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Translates the coordinate system in two dimensions.

When this function is called with two numbers, dx, and dy, all the following drawing operations take effect as if their x and y coordinates were x+dx and y+dy.

Scale and translate are not commutative operations, therefore, calling them in different orders will change the outcome.

This change lasts until the next love.draw call, or a love.graphics.pop reverts to a previous love.graphics.push, or love.graphics.origin is called - whichever comes first.

Translating using whole numbers will prevent tearing/blurring of images and fonts draw after translating.

Function

Synopsis

```
love.graphics.translate( dx, dy )
```

Arguments

number dx
: The translation relative to the x-axis.

number dy
: The translation relative to the y-axis.

Returns

Nothing.

Examples

Translate down and to the right by 10 pixels. Remember, the translation is reset at the end of each love.draw.

```
function love.draw()
   love.graphics.translate(10, 10)
   love.graphics.print("Text", 5, 5)   -- will effectively render at 15x15
end
```

Move the coordinate system with the mouse:

```
tx=0
ty=0
function love.draw()
    mx = love.mouse.getX()
```

```
    my = love.mouse.getY()
    if love.mouse.isDown(1) then
        if not mouse_pressed then
            mouse_pressed = true
            dx = tx-mx
            dy = ty-my
        else
            tx = mx+dx
            ty = my+dy
        end
    elseif mouse_pressed then
        mouse_pressed = false
    end
    love.graphics.translate(tx, ty)

    -- example graphics:
    love.graphics.circle( "line", 0, 0, 400 )
    love.graphics.line(-440, 0, 440, 0)
    love.graphics.line(0, -440, 0, 440)
end

-- restore position with the right mouse button:
function love.mousepressed(x, y, button, istouch)
   if button == 2 then
      tx = 0
      ty = 0
   end
end
```

See Also

- love.graphics
- love.graphics.pop
- love.graphics.push
- love.graphics.rotate
- love.graphics.scale
- love.graphics.shear
- love.graphics.origin

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.triangle

love.graphics.triangle · revision 27204 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

This function is not supported in that and later versions. Use love.graphics.polygon instead.

Draws a triangle.

Function

Synopsis

```
love.graphics.triangle( mode, x1, y1, x2, y2, x3, y3 )
```

Arguments

`DrawMode mode`
: How to draw the triangle.

`number x1`
: The position of first point on the x-axis.

`number y1`
: The position of first point on the y-axis.

`number x2`
: The position of second point on the x-axis.

`number y2`
: The position of second point on the y-axis.

`number x3`
: The position of third point on the x-axis.

`number y3`
: The position of third point on the y-axis.

Returns

Nothing.

See Also

- love.graphics
- love.graphics.polygon

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.graphics.validateShader

love.graphics.validateShader · revision 22711 · last edited 2026-04-14

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Validates shader code. Check if specified shader code does not contain any errors.

Function

Synopsis

```
status, message = love.graphics.validateShader( gles, code )
```

Arguments

boolean gles
: Validate code as GLSL ES shader.

string code
: The pixel shader or vertex shader code, or a filename pointing to a file with the code.

Returns

boolean status
: `true` if specified shader code doesn't contain any errors. `false` otherwise.

string message
: Reason why shader code validation failed (or `nil` if validation succeded).

Function

Synopsis

```
status, message = love.graphics.validateShader( gles, pixelcode, vertexcode )
```

Arguments

boolean gles
: Validate code as GLSL ES shader.

string pixelcode
: The pixel shader code, or a filename pointing to a file with the code.

string vertexcode

The vertex shader code, or a filename pointing to a file with the code.

Returns

`boolean status`

`true` if specified shader code doesn't contain any errors. `false` otherwise.

`string message`

Reason why shader code validation failed (or `nil` if validation succeded).

See Also

- love.graphics
- love.graphics.newShader

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.hasDeprecationOutput

love.hasDeprecationOutput · revision 26313 · last edited 2026-05-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets whether LÖVE displays warnings when using deprecated functionality. It is disabled by default in fused mode, and enabled by default otherwise.

When deprecation output is enabled, the first use of a formally deprecated LÖVE API will show a message at the bottom of the screen for a short time, and print the message to the console.

Function

Synopsis

```
enabled = love.hasDeprecationOutput( )
```

Arguments

None.

Returns

`boolean enabled`

Whether deprecation output is enabled.

See Also

- love
- love.setDeprecationOutput

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.image

love.image · revision 29169 · last edited 2024-05-01

Provides an interface to decode encoded image data.

Types

CompressedImageData	Compressed image data designed to stay compressed in RAM and on the GPU.	Added since 0.9.0
ImageData	Raw (decoded) image data.	

Functions

love.image.isCompressed	Determines whether a file can be loaded as CompressedImageData.	Added since 0.9.0
love.image.newCompressedData	Create a new CompressedImageData object from a compressed image file.	Added since 0.9.0
love.image.newEncodedImageData	Encodes ImageData.	Removed in 0.8.0
love.image.newImageData	Creates a new ImageData object.	

Enums

CompressedImageFormat	Compressed image data formats.	Added since 0.9.0
ImageEncodeFormat	Image file formats supported by ImageData:encode.	
PixelFormat	Pixel formats for Textures, ImageData, and CompressedImageData.	Added since 11.0

See Also

- love
- Image - the love.graphics data type
- Image Formats
- love.graphics.newImage

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.image.isCompressed

love.image.isCompressed · revision 16008 · last edited 2026-04-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Determines whether a file can be loaded as CompressedImageData.

Function

Synopsis

```
compressed = love.image.isCompressed( filename )
```

Arguments

`string filename`

The filename of the potentially compressed image file.

Returns

`boolean compressed`

Whether the file can be loaded as CompressedImageData or not.

Function

Synopsis

```
compressed = love.image.isCompressed( fileData )
```

Arguments

`FileData fileData`

A FileData potentially containing a compressed image.

Returns

`boolean compressed`

Whether the FileData can be loaded as CompressedImageData or not.

See Also

- love.image

- love.image.newCompressedData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.image.newCompressedData

love.image.newCompressedData · revision 15377 · last edited 2026-04-22

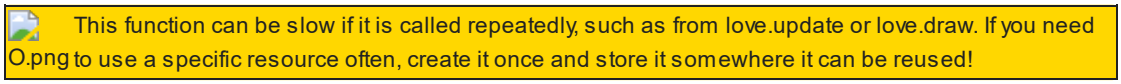
On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Create a new CompressedImageData object from a compressed image file. LÖVE supports several compressed texture formats, enumerated in the CompressedImageFormat page.



Function

Synopsis

```
compressedImageData = love.image.newCompressedData( filename )
```

Arguments

`string filename`

The filename of the compressed image file.

Returns

`CompressedImageData compressedImageData`

The new CompressedImageData object.

Function

Synopsis

```
compressedImageData = love.image.newCompressedData( fileData )
```

Arguments

`FileData fileData`

A FileData containing a compressed image.

Returns

`CompressedImageData compressedImageData`

The new CompressedImageData object.

See Also

- love.image
- love.image.isCompressed
- love.graphics.newImage
- CompressedImageData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.image.newEncodedImageData

love.image.newEncodedImageData · revision 25492 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

It has been replaced by ImageData:encode.

Encodes ImageData.

Function

Synopsis

```
data = love.image.newEncodedImageData( imageData, format )
```

Arguments

`ImageData imageData`

The raw ImageData to encode.

`ImageFormat format`

The format to encode the image in.

Returns

`Data data`

The encoded image data.

See Also

- love.image

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.image.newImageData

love.image.newImageData · revision 27762 · last edited 2026-04-24

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Creates a new ImageData object.


This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Synopsis

```
imageData = love.image.newImageData( width, height, format, rawdata )
```

Arguments

number width
The width of the ImageData.

number height
The height of the ImageData.

Available since LÖVE 11.0

PixelFormat format ("rgba8")
The pixel format of the ImageData.

string or Data rawdata (nil)
Optional raw byte data to load into the ImageData, in the format specified by *format*.

Returns

ImageData imageData

The new ImageData object. If data isn't supplied, each pixel's color values (including the alpha values!) will be set to zero.

Function

Synopsis

```
imageData = love.image.newImageData( filename )
```

Arguments

`string filename`
The filename of the image file.

Returns

`ImageData imageData`
The new ImageData object.

Function

Synopsis

```
imageData = love.image.newImageData( filedata )
```

Arguments

`FileData filedata`
The encoded file data to decode into image data.

Returns

`ImageData imageData`
The new ImageData object.

Function

Available since LÖVE 11.0
This variant is not supported in earlier versions.

Synopsis

```
imageData = love.image.newImageData( encodeddata )
```

Arguments

`Data encodeddata`

The encoded data to load into the ImageData.

Returns

ImageData imageData
The new ImageData object.

Function

Available since LÖVE 0.10.0 and removed in LÖVE 11.0
This variant is not supported in earlier or later versions.

Synopsis

```
imageData = love.image.newImageData( width, height, rawdata )
```

Arguments

number width
The width of the ImageData.

number height
The height of the ImageData.

string rawdata
The data to load into the ImageData (RGBA bytes, left to right and top to bottom).

Returns

ImageData imageData
The new ImageData object.

See Also

- love.image
- ImageData
- love.graphics.newImage

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.isVersionCompatible

love.isVersionCompatible · revision 26822 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets whether the given version is compatible with the current running version of LÖVE.

Function

Synopsis

```
compatible = love.isVersionCompatible( version )
```

Arguments

string version

The version to check (for example `'11.3'` or `'0.10.2'`).

Returns

boolean compatible

Whether the given version is compatible with the current running version of LÖVE.

Function

Synopsis

```
compatible = love.isVersionCompatible( major, minor, revision )
```

Arguments

number major

The major version to check (for example `11` for 11.3 or `0` for 0.10.2).

number minor

The minor version to check (for example `3` for 11.3 or `10` for 0.10.2).

number revision

The revision of version to check (for example `0` for 11.3 or `2` for 0.10.2).

Returns

boolean compatible

Whether the given version is compatible with the current running version of LÖVE.

See Also

- love
- love.getVersion

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick

love.joystick · revision 22002 · last edited 2025-09-28

Available since LÖVE 0.5.0

This module is not supported in earlier versions.

Provides an interface to connected joysticks.

Types

Joystick	Represents a physical joystick.	Added since 0.9.0

Functions

love.joystick.close	Closes a joystick.	Added since 0.5.0	Removed in 0.9.0
love.joystick.getAxes	Returns the position of each axis.	Added since 0.5.0	Removed in 0.9.0
love.joystick.getAxis	Returns the direction of the axis.	Added since 0.5.0	Removed in 0.9.0
love.joystick.getBall	Returns the change in ball position.	Added since 0.5.0	Removed in 0.9.0
love.joystick.getGamepadMappingString	Gets the full gamepad mapping string of the Joysticks which have the given GUID, or nil if the GUID isn't recognized as a gamepad.	Added since 11.3	
love.joystick.getHat	Returns the direction of a hat.	Added since 0.5.0	Removed in 0.9.0
love.joystick.getJoystickCount	Gets the number of connected joysticks.	Added since 0.9.0	
love.joystick.getJoysticks	Gets a list of connected Joysticks.	Added since 0.9.0	
love.joystick.getName	Returns the name of a joystick.	Added since 0.5.0	Removed in 0.9.0
love.joystick.getNumAxes	Returns the number of axes on the joystick.	Added since 0.5.0	Removed in 0.9.0
love.joystick.getNumBalls	Returns the number of balls on the joystick.	Added since 0.5.0	Removed in 0.9.0
love.joystick.getNumButtons	Returns the number of buttons on the joystick.	Added since 0.5.0	Removed in 0.9.0
love.joystick.getNumHats	Returns the number of hats on the joystick.	Added since 0.5.0	Removed in 0.9.0

love.joystick.getNumJoysticks	Returns how many joysticks are available.	Added since 0.5.0	Removed in 0.9.0
love.joystick.isDown	Checks if a button on a joystick is pressed.	Added since 0.5.0	Removed in 0.9.0
love.joystick.isOpen	Checks if the joystick is open.	Added since 0.5.0	Removed in 0.9.0
love.joystick.loadGamepadMappings	Loads a gamepad mappings string or file created with love.joystick.saveGamepadMappings.	Added since 0.9.2	
love.joystick.open	Opens up a joystick to be used.	Added since 0.5.0	Removed in 0.9.0
love.joystick.saveGamepadMappings	Saves the virtual gamepad mappings of all recently-used Joysticks that are recognized as gamepads.	Added since 0.9.2	
love.joystick.setGamepadMapping	Binds a virtual gamepad input to a button, axis or hat.	Added since 0.9.2	

Enums

GamepadAxis	Virtual gamepad axes.	Added since 0.9.0
GamepadButton	Virtual gamepad buttons.	Added since 0.9.0
JoystickHat	Joystick hat positions.	Added since 0.5.0
JoystickInputType	Types of Joystick inputs.	Added since 0.9.0

See Also

- love
- love.gamepadpressed
- love.gamepadreleased
- love.joystickpressed
- love.joystickreleased
- love.joystickadded
- love.joystickremoved

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.close

love.joystick.close · revision 24699 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.5.0 and removed in LÖVE 0.9.0

This function is not supported in earlier or later versions.

Closes a joystick, i.e. stop using it for generating events and in query functions.

Function

Synopsis

```
love.joystick.close( joystick )
```

Arguments

`number joystick`
: The joystick to be closed

Returns

Nothing.

See Also

- love.joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.getAxes

love.joystick.getAxes · revision 24700 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.5.0 and removed in LÖVE 0.9.0

It has been moved to Joystick:getAxes.

Returns the position of each axis.

Function

Synopsis

```
axisDir1, axisDir2, axisDirN = love.joystick.getAxes( joystick )
```

Arguments

number joystick
: The joystick to be checked

Returns

number axisDir1
: Direction of axis1

number axisDir2
: Direction of axis2

number axisDirN
: Direction of axisN

See Also

- love.joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.getAxis

love.joystick.getAxis · revision 24701 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.5.0 and removed in LÖVE 0.9.0

It has been moved to Joystick:getAxis.

Returns the direction of the axis.

Function

Synopsis

```
direction = love.joystick.getAxis( joystick, axis )
```

Arguments

number joystick
: The joystick to be checked

number axis
: The axis to be checked

Returns

number direction
: Current value of the axis

See Also

- love.joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.getBall

love.joystick.getBall · revision 24702 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.5.0 and removed in LÖVE 0.9.0

This function is not supported in earlier or later versions.

Returns the change in ball position.

Function

Synopsis

```
dx, dy = love.joystick.getBall( joystick, ball )
```

Arguments

number joystick
: The joystick to be checked

number ball
: The ball to be checked

Returns

number dx
: Change in x of the ball position.

number dy
: Change in y of the ball position.

See Also

- love.joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.getGamepadMappingString

love.joystick.getGamepadMappingString · revision 30111 · last edited 2025-10-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example: print the MRU gamepad name
- See Also
- Other Languages

Available since LÖVE 11.3

This function is not supported in earlier versions.

Gets the full gamepad mapping string of the Joysticks which have the given GUID, or nil if the GUID isn't recognized as a gamepad.

The mapping string contains binding information used to map the Joystick's buttons an axes to the standard gamepad layout, and can be used later with love.joystick.loadGamepadMappings.

Function

Synopsis

```
mappingstring = love.joystick.getGamepadMappingString( guid )
```

Arguments

string guid

The GUID value to get the mapping string for.

Returns

string mappingstring (nil)

A string containing the Joystick's gamepad mappings, or nil if the GUID is not recognized as a gamepad.

Example: print the MRU gamepad name

Track the most recently used gamepad and print its name when it changes.

```
local active_joystick
function love.gamepadpressed(joystick, button)
    if joystick == active_joystick then
        return
    end
    active_joystick = joystick
    -- Pull out the second column from the csv string.
    local name = joystick:getGamepadMappingString():match("^%x*,(.-),")
    if not name or name:len() < 3 then
        -- Names like "*" are not useful, so fall back to device name.
        name = joystick:getName()
    end
    local index = joystick:getID()
```

```
    print(string.format("Changing primary gamepad to #%d '%s'.", index, name))
end
```

See Also

- love.joystick
- Joystick:getGamepadMappingString
- Joystick:isGamepad
- love.joystick.loadGamepadMappings
- love.joystick.saveGamepadMappings
- love.joystick.setGamepadMapping
- Joystick:getGamepadMapping

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.getHat

love.joystick.getHat · revision 24703 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.5.0 and removed in LÖVE 0.9.0

It has been moved to Joystick:getHat.

Returns the direction of a hat.

Function

Synopsis

```
direction = love.joystick.getHat( joystick, hat )
```

Arguments

`number joystick`
: The joystick to be checked

`number hat`
: The hat to be checked

Returns

`JoystickConstant direction`
: The direction the hat is pushed

See Also

- love.joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.getJoystickCount

love.joystick.getJoystickCount · revision 11320 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

It has been renamed from love.joystick.getNumJoysticks.

Gets the number of connected joysticks.

Function

Synopsis

```
joystickcount = love.joystick.getJoystickCount( )
```

Arguments

None.

Returns

number joystickcount

The number of connected joysticks.

See Also

- love.joystick
- love.joystick.getJoysticks

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.getJoysticks

love.joystick.getJoysticks · revision 27898 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets a list of connected Joysticks.

Function

Synopsis

```
joysticks = love.joystick.getJoysticks( )
```

Arguments

None.

Returns

table joysticks

The list of currently connected Joysticks.

Examples

```
function love.draw()
    local joysticks = love.joystick.getJoysticks()
    for i, joystick in ipairs(joysticks) do
        love.graphics.print(joystick:getName(), 10, i * 20)
    end
end
```

See Also

- Joystick
- love.joystick
- love.joystick.getJoystickCount
- love.joystickadded
- love.joystickremoved

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

love.joystick.getName

love.joystick.getName · revision 24704 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.5.0 and removed in LÖVE 0.9.0

It has been moved to Joystick:getName.

Returns the name of a joystick.

Function

Synopsis

```
name = love.joystick.getName( joystick )
```

Arguments

number joystick
: The joystick to be checked

Returns

string name
: The name

See Also

- love.joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.getNumAxes

love.joystick.getNumAxes · revision 24705 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.5.0 and removed in LÖVE 0.9.0

It has been moved to Joystick:getAxisCount.

Returns the number of axes on the joystick.

Function

Synopsis

```
axes = love.joystick.getNumAxes( joystick )
```

Arguments

number joystick
: The joystick to be checked

Returns

number axes
: The number of axes available

See Also

- love.joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.getNumBalls

love.joystick.getNumBalls · revision 24706 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.5.0 and removed in LÖVE 0.9.0

This function is not supported in earlier or later versions.

Returns the number of balls on the joystick.

Function

Synopsis

```
balls = love.joystick.getNumBalls( joystick )
```

Arguments

number joystick
: The joystick to be checked

Returns

number balls
: The number of balls available

See Also

- love.joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.getNumButtons

love.joystick.getNumButtons · revision 24707 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.5.0 and removed in LÖVE 0.9.0

It has been renamed to Joystick:getButtonCount.

Returns the number of buttons on the joystick.

Function

Synopsis

```
buttons = love.joystick.getNumButtons( joystick )
```

Arguments

number joystick
: The joystick to be checked

Returns

number buttons
: The number of buttons available

See Also

- love.joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.getNumHats

love.joystick.getNumHats · revision 24708 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.5.0 and removed in LÖVE 0.9.0

It has been moved to Joystick:getHatCount.

Returns the number of hats on the joystick.

Function

Synopsis

```
hats = love.joystick.getNumHats( joystick )
```

Arguments

number joystick
: The joystick to be checked

Returns

number hats
: How many hats the joystick has

See Also

- love.joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.getNumJoysticks

love.joystick.getNumJoysticks · revision 24709 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.5.0 and removed in LÖVE 0.9.0

It has been renamed to love.joystick.getJoystickCount.

Returns how many joysticks are available.

Function

Synopsis

```
joysticks = love.joystick.getNumJoysticks( )
```

Arguments

None.

Returns

number joysticks

The number of joysticks available

See Also

- love.joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.isDown

love.joystick.isDown · revision 24710 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.5.0 and removed in LÖVE 0.9.0

It has been moved to Joystick:isDown.

Checks if a button on a joystick is pressed.

Function

Synopsis

```
down = love.joystick.isDown( joystick, button )
```

Arguments

number joystick
: The joystick to be checked

number button
: The button to be checked

Returns

boolean down
: True if the button is down, false if it is not

Function

Available since LÖVE 0.7.2

This variant is not supported in earlier versions.

Synopsis

```
anyDown = love.joystick.isDown( joystick, button1, button2, button3, ... )
```

Arguments

number joystick
: The joystick to be checked

number buttonN
: A button to check

Returns

boolean anyDown

True if any supplied button is down, false if not.

See Also

- love.joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.isOpen

love.joystick.isOpen · revision 24711 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.5.0 and removed in LÖVE 0.9.0

This function is not supported in earlier or later versions.

Checks if the joystick is open.

Function

Synopsis

```
open = love.joystick.isOpen( joystick )
```

Arguments

`number joystick`

The joystick to be checked

Returns

`boolean open`

True if the joystick is open, false if it is closed.

See Also

- love.joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.loadGamepadMappings

love.joystick.loadGamepadMappings · revision 21586 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Loads a gamepad mappings string or file created with love.joystick.saveGamepadMappings.

It also recognizes any SDL gamecontroller mapping string, such as those created with Steam's Big Picture controller configure interface, or this nice database (https://raw.githubusercontent.com/gabomdq/SDL_GameControllerDB/master/gamecontrollerdb.txt). If a new mapping is loaded for an already known controller GUID, the later version will overwrite the one currently loaded.

Function

Loads a gamepad mappings string from a file.

Synopsis

```
love.joystick.loadGamepadMappings( filename )
```

Arguments

`string filename`
: The filename to load the mappings string from.

Returns

Nothing.

Function

Loads a gamepad mappings string directly.

Synopsis

```
love.joystick.loadGamepadMappings( mappings )
```

Arguments

`string mappings`

The mappings string to load.

Returns

Nothing.

See Also

- love.joystick
- love.joystick.saveGamepadMappings
- love.joystick.setGamepadMapping
- Joystick:getGamepadMapping
- Joystick:isGamepad

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.open

love.joystick.open · revision 24712 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.5.0 and removed in LÖVE 0.9.0

This function is not supported in earlier or later versions.

Opens up a joystick to be used, i.e. makes it ready to use. By default joysticks that are available at the start of your game will be opened.


Unlike conventional Lua indexes, joysticks begin counting from 0 in LÖVE 0.7.2 and below. To to open the first joystick, you would use `love.joystick.open(0)`. This is not the case in LÖVE 0.8.0 and later.

Function

Synopsis

```
open = love.joystick.open( joystick )
```

Arguments

`number joystick`

The joystick to be opened

Returns

`boolean open`

True if the joystick has been successfully opened or false on failure.

See Also

- love.joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.saveGamepadMappings

love.joystick.saveGamepadMappings · revision 13777 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Saves the virtual gamepad mappings of all Joysticks that are recognized as gamepads and have either been recently used or their gamepad bindings have been modified.

The mappings are stored as a string for use with love.joystick.loadGamepadMappings.

Function

Saves the gamepad mappings of all relevant joysticks to a file.

Synopsis

```
mappings = love.joystick.saveGamepadMappings( filename )
```

Arguments

`string filename`
: The filename to save the mappings string to.

Returns

`string mappings`
: The mappings string that was written to the file.

Function

Returns the mappings string without writing to a file.

Synopsis

```
mappings = love.joystick.saveGamepadMappings( )
```

Arguments

None.

Returns

`string mappings`

The mappings string.

See Also

- love.joystick
- love.joystick.loadGamepadMappings
- love.joystick.setGamepadMapping
- Joystick:getGamepadMapping
- Joystick:isGamepad

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystick.setGamepadMapping

love.joystick.setGamepadMapping · revision 28414 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Binds a virtual gamepad input to a button, axis or hat for all Joysticks of a certain type. For example, if this function is used with a GUID returned by a Dualshock 3 controller in OS X, the binding will affect Joystick:getGamepadAxis and Joystick:isGamepadDown for *all* Dualshock 3 controllers used with the game when run in OS X.

LÖVE includes built-in gamepad bindings for many common controllers. This function lets you change the bindings or add new ones for types of Joysticks which aren't recognized as gamepads by default.

The virtual gamepad buttons and axes are designed around the Xbox 360 controller layout.

Function

Synopsis

```
success = love.joystick.setGamepadMapping( guid, button, inputtype, inputindex, hatdir )
```

Arguments

`string guid`
: The OS-dependent GUID for the type of Joystick the binding will affect.

`GamepadButton button`
: The virtual gamepad button to bind.

`JoystickInputType inputtype`
: The type of input to bind the virtual gamepad button to.

`number inputindex`
: The 1-based index of the axis, button, or hat to bind the virtual gamepad button to.

`JoystickHat hatdir (nil)`
: The direction of the hat, if the virtual gamepad button will be bound to a hat. nil otherwise.

Returns

`boolean success`

Whether the virtual gamepad button was successfully bound.

Function

Synopsis

```
success = love.joystick.setGamepadMapping( guid, axis, inputtype, inputindex, hatdir )
```

Arguments

string guid
The OS-dependent GUID for the type of Joystick the binding will affect.

GamepadAxis axis
The virtual gamepad axis to bind.

JoystickInputType inputtype
The type of input to bind the virtual gamepad axis to.

number inputindex
The 1-based index of the axis, button, or hat to bind the virtual gamepad axis to.

JoystickHat hatdir (nil)
The direction of the hat, if the virtual gamepad axis will be bound to a hat. nil otherwise.

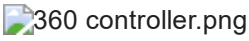
Returns

boolean success
Whether the virtual gamepad axis was successfully bound.

Notes

LÖVE versions older than 0.10.0 have a bug which requires any mapping that is a suffix of another (for example "x" is a suffix to "leftx") to be mapped first.

The physical locations for the bound gamepad axes and buttons should correspond as closely as possible to the layout of a standard Xbox 360 controller.


360 controller.png

See Also

- love.joystick
- Joystick:getGUID
- Joystick:isGamepad
- Joystick:getGamepadMapping

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystickadded

love.joystickadded · revision 11897 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Called when a Joystick is connected.

Function

Synopsis

```
love.joystickadded( joystick )
```

Arguments

`Joystick joystick`
: The newly connected Joystick object.

Returns

Nothing.

Notes

This callback is also triggered after love.load for every Joystick which was already connected when the game started up.

See Also

- love
- love.joystick
- love.joystickremoved
- Joystick:isConnected

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystickaxis

love.joystickaxis · revision 27229 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Show the value of the first two joystick axes
- See Also
- Other Languages

Available since LÖVE 0.9.0

This callback is not supported in earlier versions.

Called when a joystick axis moves.

Function

Synopsis

```
love.joystickaxis( joystick, axis, value )
```

Arguments

`Joystick joystick`
: The joystick object.

`number axis`
: The axis number.

`number value`
: The new axis value.

Returns

Nothing.

Examples

Show the value of the first two joystick axes

```
local lastAxis1Value = 0
local lastAxis2Value = 0

function love.joystickaxis(joystick, axis, value)
    if axis == 1 then
        lastAxis1Value = value
    elseif axis == 2 then
        lastAxis2Value = value
    end
end

function love.draw()
    love.graphics.print(string.format("Axis 1: %.2f", lastAxis1Value), 100, 100)
```

```
    love.graphics.print(string.format("Axis 2: %.2f", lastAxis2Value), 100, 120)
end
```

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystickhat

love.joystickhat · revision 11899 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This callback is not supported in earlier versions.

Called when a joystick hat direction changes.

Function

Synopsis

```
love.joystickhat( joystick, hat, direction )
```

Arguments

`Joystick joystick`
: The joystick object.

`number hat`
: The hat number.

`JoystickHat direction`
: The new hat direction.

Returns

Nothing.

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystickpressed

love.joystickpressed · revision 21579 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Use joystickpressed to map stimulate the jumping of a player
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Called when a joystick button is pressed.

Function

Available since LÖVE 0.9.0

This variant is not supported in earlier versions.

Synopsis

```
love.joystickpressed( joystick, button )
```

Arguments

`Joystick joystick`
: The joystick object.

`number button`
: The button number.

Returns

Nothing.

Examples

Use joystickpressed to map stimulate the jumping of a player

```
function love.joystickpressed(joystick,button)
   player:jumping()
end

--inside playerClass
function player:jumping()
   if joyStick:isGamepadDown('a') then
        if self.jump then
           self.speedY = self.jumpSpeed
            self.jump = false
        end
```

```
    end
end
```

Function

Removed in LÖVE 0.9.0

This variant is not supported in that and later versions.

Synopsis

```
love.joystickpressed( joystick, button )
```

Arguments

number joystick

The joystick number.

number button

The button number.

Returns

Nothing.

See Also

- love
- love.joystickreleased

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystickreleased

love.joystickreleased · revision 11901 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Called when a joystick button is released.

Function

Available since LÖVE 0.9.0

This variant is not supported in earlier versions.

Synopsis

```
love.joystickreleased( joystick, button )
```

Arguments

`Joystick joystick`
: The joystick object.

`number button`
: The button number.

Returns

Nothing.

Function

Removed in LÖVE 0.9.0

This variant is not supported in that and later versions.

Synopsis

```
love.joystickreleased( joystick, button )
```

Arguments

`number joystick`
: The joystick number.

`number button`

The button number.

Returns

Nothing.

See Also

- love
- love.joystickpressed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joystickremoved

love.joystickremoved · revision 11902 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Called when a Joystick is disconnected.

Function

Synopsis

```
love.joystickremoved( joystick )
```

Arguments

`Joystick joystick`
: The now-disconnected Joystick object.

Returns

Nothing.

See Also

- love
- love.joystickadded
- Joystick:isConnected

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.joysticksensorupdated

love.joysticksensorupdated · revision 28972 · last edited 2024-06-15

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This callback is not supported in earlier versions.

Called when a Joystick's sensor is updated with new values.

Only Joystick sensors enabled with Joystick:setSensorEnabled will trigger this event.

Function

Synopsis

```
love.joysticksensorupdated( joystick, sensorType, x, y, z )
```

Arguments

`Joystick joystick`
: The Joystick object.

`SensorType sensorType`
: The type of sensor.

`number x`
: The new sensor 1st value.

`number y`
: The new sensor 2nd value.

`number z`
: The new sensor 3rd value.

Returns

Nothing

See Also

- love
- Joystick:setSensorEnabled
- Joystick:getSensorData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.keyboard

love.keyboard · revision 31646 · last edited 2026-06-16

Provides an interface to the user's keyboard.

Functions

love.keyboard.getKeyFromScancode	Gets the key corresponding to the given hardware scancode.	Added since 0.9.2	
love.keyboard.getKeyRepeat	Returns the delay and interval of key repeating.		Removed in 0.9.0
love.keyboard.getScancodeFromKey	Gets the hardware scancode corresponding to the given key.	Added since 0.9.2	
love.keyboard.hasKeyRepeat	Gets whether key repeat is enabled.	Added since 0.9.0	
love.keyboard.hasScreenKeyboard	Gets whether screen keyboard is supported.	Added since 0.10.0	
love.keyboard.hasTextInput	Gets whether text input events are enabled.	Added since 0.9.0	
love.keyboard.isDown	Checks whether a certain key is down.		
love.keyboard.isScancodeDown	Checks whether the specified Scancodes are pressed.	Added since 0.10.0	
love.keyboard.setKeyRepeat	Enables or disables key repeat for love.keypressed.		
love.keyboard.setTextInput	Enables or disables text input events.	Added since 0.9.0	

Enums

KeyConstant	All the keys you can press.	
Scancode	Keyboard scancodes.	Added since 0.9.2

See Also

- love
- love.keypressed
- love.keyreleased
- love.textinput
- love.textedited

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.keyboard.getKeyFromScancode

love.keyboard.getKeyFromScancode · revision 15788 · last edited 2026-04-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets the key corresponding to the given hardware scancode.

Unlike key constants, Scancodes are keyboard layout-independent. For example the scancode "w" will be generated if the key in the same place as the "w" key on an American keyboard is pressed, no matter what the key is labelled or what the user's operating system settings are.

Scancodes are useful for creating default controls that have the same physical locations on on all systems.

Function

Synopsis

```
key = love.keyboard.getKeyFromScancode( scancode )
```

Arguments

`Scancode scancode`
: The scancode to get the key from.

Returns

`KeyConstant key`
: The key corresponding to the given scancode, or "unknown" if the scancode doesn't map to a KeyConstant on the current system.

See Also

- love.keyboard
- love.keyboard.getScancodeFromKey
- love.keyboard.isScancodeDown
- love.keypressed
- love.keyreleased

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.keyboard.getKeyRepeat

love.keyboard.getKeyRepeat · revision 11210 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

This function is not supported in that and later versions.

Returns the delay and interval of key repeating.

Function

Synopsis

```
delay, interval = love.keyboard.getKeyRepeat( )
```

Arguments

None.

Returns

number delay
: The amount of time before repeating the key (in seconds)

number interval
: The amount of time between repeats (in seconds)

See Also

- love.keyboard

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.keyboard.getScancodeFromKey

love.keyboard.getScancodeFromKey · revision 15789 · last edited 2026-04-07

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets the hardware scancode corresponding to the given key.

Unlike key constants, Scancodes are keyboard layout-independent. For example the scancode "w" will be generated if the key in the same place as the "w" key on an American keyboard is pressed, no matter what the key is labelled or what the user's operating system settings are.

Scancodes are useful for creating default controls that have the same physical locations on on all systems.

Function

Synopsis

```
scancode = love.keyboard.getScancodeFromKey( key )
```

Arguments

`KeyConstant key`

The key to get the scancode from.

Returns

`Scancode scancode`

The scancode corresponding to the given key, or "unknown" if the given key has no known physical representation on the current system.

See Also

- love.keyboard
- love.keyboard.getKeyFromScancode
- love.keyboard.isScancodeDown
- love.keypressed
- love.keyreleased

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.keyboard.hasKeyRepeat

love.keyboard.hasKeyRepeat · revision 11412 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets whether key repeat is enabled.

Function

Synopsis

```
enabled = love.keyboard.hasKeyRepeat( )
```

Arguments

None.

Returns

`boolean enabled`

Whether key repeat is enabled.

See Also

- love.keyboard
- love.keyboard.setKeyRepeat
- love.keypressed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.keyboard.hasScreenKeyboard

love.keyboard.hasScreenKeyboard · revision 25516 · last edited 2026-04-07

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets whether screen keyboard is supported.

Function

Synopsis

```
supported = love.keyboard.hasScreenKeyboard( )
```

Arguments

None.

Returns

`boolean supported`

Whether screen keyboard is supported.

See Also

- love.keyboard

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.keyboard.hasTextInput

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets whether text input events are enabled.

Function

Synopsis

```
enabled = love.keyboard.hasTextInput( )
```

Arguments

None.

Returns

`boolean enabled`

Whether text input events are enabled.

See Also

- love.keyboard
- love.keyboard.setTextInput
- love.textinput

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.keyboard.isDown

love.keyboard.isDown · revision 28145 · last edited 2025-11-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Increase a value while a key is held down
- See Also
- Other Languages

Checks whether a certain key is down. Not to be confused with love.keypressed or love.keyreleased.

Function

Synopsis

```
down = love.keyboard.isDown( key )
```

Arguments

KeyConstant key
: The key to check.

Returns

boolean down
: True if the key is down, false if not.

Function

Available since LÖVE 0.7.2

This variant is not supported in earlier versions.

Synopsis

```
anyDown = love.keyboard.isDown( key, ... )
```

Arguments

KeyConstant key
: A key to check.

KeyConstant ...
: Additional keys to check.

Returns

boolean anyDown

True if any supplied key is down, false if not.

Function

Available since LÖVE 0.10.2

This variant is not supported in earlier versions.

Synopsis

```
anyDown = love.keyboard.isDown({ key, ... })
```

Arguments

table keys

Table of keys to check.

KeyConstant key

A key to check.

KeyConstant ...

Additional keys to check.

Returns

boolean anyDown

True if any supplied key is down, false if not.

Examples

Increase a value while a key is held down

```
local val = 0;
function love.update(dt)
    -- We will increase the variable by 1 for every second the key is held down.
    if love.keyboard.isDown("up") then
        val = val + dt
        print(val)
    end

    -- We will decrease the variable by 1/s if any of the wasd keys is pressed.
    if love.keyboard.isDown('w', 'a', 's', 'd') then
        val = val - dt
        print(val)
    end
end
```

See Also

- love.keyboard
- love.keypressed
- love.keyreleased
- love.keyboard.isScancodeDown

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.keyboard.isModifierActive

love.keyboard.isModifierActive · revision 28065 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Checks whether a modifier key is active.

Example of modifier keys are caps lock, num lock, and scroll lock while also have press/release state, additionally also have active state.

Function

Synopsis

```
active = love.keyboard.isModifierActive( modifier )
```

Arguments

`ModifierKey modifier`

Modifier key to check.

Returns

`boolean active`

Wheter the specified modifier key is active or not.

See Also

- love.keyboard
- ModifierKey

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.keyboard.isScancodeDown

love.keyboard.isScancodeDown · revision 29060 · last edited 2026-04-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Standard WASD or arrow keys for movement
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Checks whether the specified Scancodes are pressed. Not to be confused with love.keypressed or love.keyreleased.

Unlike regular KeyConstants, Scancodes are keyboard layout-independent. The scancode "w" is used if the key in the same place as the "w" key on an American keyboard (https://en.wikipedia.org/wiki/British_and_American_keyboards#/media/File:KB_United_States-NoAltGr.svg) is pressed, no matter what the key is labelled or what the user's operating system settings are.

Function

Synopsis

```
down = love.keyboard.isScancodeDown( scancode, ... )
```

Arguments

`Scancode scancode`
: A Scancode to check.

`Scancode ...`
: Additional Scancodes to check.

Returns

`boolean down`
: True if any supplied Scancode is down, false if not.

Function

Available since LÖVE 0.10.2

This variant is not supported in earlier versions.

Synopsis

```
down = love.keyboard.isScancodeDown({ scancode, ... })
```

Arguments

`table scancodes`
: Table of Scancodes to check.

  `Scancode scancode`

A Scancode to check.

`Scancode ...`
Additional Scancodes to check.

Returns

`boolean down`
True if any supplied Scancode is down, false if not.

Examples

Standard WASD or arrow keys for movement

```
function love.update(dt)
    local up    = love.keyboard.isScancodeDown('w', 'up')
    local down  = love.keyboard.isScancodeDown('s', 'down')
    local left  = love.keyboard.isScancodeDown('a', 'left')
    local right = love.keyboard.isScancodeDown('d', 'right')
end
```

Or

```
function love.update(dt)
    local speed = 100
    local dx, dy = 0,0
    local isScancodeDown = love.keyboard.isScancodeDown
    if isScancodeDown ('a') then dx = dx - dt*speed end
    if isScancodeDown ('d') then dx = dx + dt*speed end
    if isScancodeDown ('w') then dy = dy - dt*speed end
    if isScancodeDown ('s') then dy = dy + dt*speed end
end
```

See Also

- love.keyboard
- love.keyboard.isDown
- love.keyboard.getKeyFromScancode
- love.keyboard.getScancodeFromKey
- love.keypressed
- love.keyreleased

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.keyboard.setKeyRepeat

love.keyboard.setKeyRepeat · revision 25244 · last edited 2026-04-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
- Synopsis
- Arguments
- Returns
- Examples
- See Also
- Other Languages

Enables or disables key repeat for love.keypressed. It is disabled by default.

Function

Available since LÖVE 0.9.0

This variant is not supported in earlier versions.

Synopsis

```
love.keyboard.setKeyRepeat( enable )
```

Arguments

`boolean enable`

Whether repeat keypress events should be enabled when a key is held down.

Returns

Nothing.

Notes

The interval between repeats depends on the user's system settings. This function doesn't affect whether love.textinput is called multiple times while a key is held down.

Function

Removed in LÖVE 0.9.0

This variant is not supported in that and later versions.

Enables key repeating and sets the delay and interval.

Synopsis

```
love.keyboard.setKeyRepeat( delay, interval )
```

Arguments

`number delay`

The amount of time before repeating the key (in seconds). 0 disables key repeat.

`number interval`

The amount of time between repeats (in seconds)

Returns

Nothing.

Examples

Hold left or right to change the position.

```
function love.load()
    love.keyboard.setKeyRepeat(true)
    x = 50
end

function love.keypressed(key, scancode, isrepeat)
    if key == "right" then
        x = (x + 80) % love.graphics.getWidth()
    elseif key == "left" then
        x = (x - 80) % love.graphics.getWidth()
    end
end

function love.draw()
    love.graphics.circle("fill", x, 100, 50, 50)
end
```

See Also

- love.keyboard
- love.keyboard.hasKeyRepeat
- love.keypressed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.keyboard.setTextInput

love.keyboard.setTextInput · revision 20212 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Enables or disables text input events. It is enabled by default on Windows, Mac, and Linux, and disabled by default on iOS and Android.

On touch devices, this shows the system's native on-screen keyboard when it's enabled.

Function

Synopsis

```
love.keyboard.setTextInput( enable )
```

Arguments

`boolean enable`
: Whether text input events should be enabled.

Returns

Nothing.

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

On iOS and Android this variant tells the OS that the specified rectangle is where text will show up in the game, which prevents the system on-screen keyboard from covering the text.

Synopsis

```
love.keyboard.setTextInput( enable, x, y, w, h )
```

Arguments

`boolean enable`
: Whether text input events should be enabled.

`number x`
: Text rectangle x position.

`number y`
: Text rectangle y position.

`number w`
: Text rectangle width.

`number h`
: Text rectangle height.

Returns

Nothing.

See Also

- love.keyboard
- love.keyboard.hasTextInput
- love.textinput

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.keypressed

love.keypressed · revision 29309 · last edited 2025-10-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Examples
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Callback function triggered when a key is pressed.

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Synopsis

```
love.keypressed( key, scancode, isrepeat )
```

Arguments

`KeyConstant key`
: Character of the pressed key.

`Scancode scancode`
: The scancode representing the pressed key.

`boolean isrepeat`
: Whether this keypress event is a repeat. The delay between key repeats depends on the user's system settings.

Returns

Nothing.

Notes

Scancodes are keyboard layout-independent, so the scancode "w" will be generated if the key in the same place as the "w" key on an American keyboard (https://en.w ikipedia.org/w iki/British_and_American_keyboards#/media/File:KB_United_States-NoAltGr.s vg) is pressed, no matter what the key is labelled or what the user's operating system settings are.

Key repeat needs to be enabled with love.keyboard.setKeyRepeat for repeat keypress events to be received. This does not affect love.textinput.

Examples

Exit the game when the player presses the Escape key, using love.event.quit.

```
function love.keypressed(key, scancode, isrepeat)
   if key == "escape" then
```

```
        love.event.quit()
    end
end
```

Function

Available since LÖVE 0.9.0 and removed in LÖVE 0.10.0

This variant is not supported in earlier or later versions.

Synopsis

```
love.keypressed( key, isrepeat )
```

Arguments

`KeyConstant key`

Character of the key pressed.

`boolean isrepeat`

Whether this keypress event is a repeat. The delay between key repeats depends on the user's system settings.

Returns

Nothing.

Notes

Key repeat needs to be enabled with love.keyboard.setKeyRepeat for repeat keypress events to be received.

Function

Removed in LÖVE 0.9.0

Unicode text input is now handled separately via love.textinput.

Synopsis

```
love.keypressed( key, unicode )
```

Arguments

`KeyConstant key`

Character of the key pressed.

`number unicode`

The unicode number of the key pressed.

Returns

Nothing.

Examples

A movement key input system, where for one step need to press the key once.

```
function love.keypressed( key, scancode, isrepeat )
   local dx, dy = 0, 0
   if scancode == "d" then -- move right
      dx = 1
   elseif scancode == "a" then -- move left
      dx = -1
   elseif scancode == "s" then -- move down
      dy = 1
   elseif scancode == "w" then -- move up
      dy = -1
   end
   move (dx, dy) -- need to be some function to move the active object
end
```

Removed in LÖVE 0.9.0

Text input is now handled separately via love.textinput.

Record and print text the user writes (0.8.0 and below.)

```
function love.load()
    text = "Type away! -- "
end

function love.keypressed(key, unicode)
    -- ignore non-printable characters (see http://www.ascii-code.com/)
    if unicode > 31 and unicode < 127 then
        text = text .. string.char(unicode)
    end
end

function love.draw()
    love.graphics.printf(text, 0, 0, 800)
end
```

See Also

- love
- love.keyreleased
- love.keyboard.isDown
- love.keyboard.isScancodeDown
- love.textinput

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.keyreleased

love.keyreleased · revision 24819 · last edited 2025-10-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- See Also
- Other Languages

Callback function triggered when a keyboard key is released.

Function

Synopsis

```
love.keyreleased( key, scancode )
```

Arguments

KeyConstant key

Character of the released key.

Available since LÖVE 0.10.0

Scancode scancode

The scancode representing the released key.

Returns

Nothing.

Notes

Scancodes are keyboard layout-independent, so the scancode "w" will be used if the key in the same place as the "w" key on an American keyboard (https://en.w ikipedia.org/w iki/British_and_American_keyboards#/media/File:KB_United_States-NoAltGr.svg) is released, no matter what the key is labelled or what the user's operating system settings are.

Examples

Exit the game when the player releases the Escape key, using love.event.quit.

```
function love.keyreleased(key)
   if key == "escape" then
      love.event.quit()
   end
end
```

See Also

- love
- love.keypressed

- love.keyboard.isDown

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.load

love.load · revision 29420 · last edited 2024-12-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- See Also
- Other Languages

This function is called by the default love.run exactly once at the beginning of the game.

Function

Synopsis

```
love.load( arg, unfilteredArg )
```

Arguments

table arg
: Command-line arguments given to the game.

Available since LÖVE 11.0

table unfilteredArg
: Unfiltered command-line arguments given to the executable (see Notes).

Returns

Nothing.

Notes

In LÖVE 11.0, the passed arguments excludes the game name and the fused command-line flag (if exist) when runs from non-fused LÖVE executable. Previous version pass the argument as-is without any filtering.

Examples

Establish some variables/resources on the game load, so that they can be used repeatedly in other functions (such as love.draw).

Run love with arguments to see them displayed: *love . --hello*.

```
local text, pos

function love.load(args)
   local msg = args[1] or 'no arguments'
   text = love.graphics.newText(love.graphics.getFont(), msg)
   pos = {
       x = 50,
       y = 50,
```

```
   }
end

function love.update(dt)
   if love.keyboard.isDown('right') then
      pos.x = pos.x + 1
   end
end

function love.draw()
   love.graphics.draw(text, pos.x, pos.y)
end
```

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.localechanged

love.localechanged · revision 29149 · last edited 2025-12-01

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This callback is not supported in earlier versions.

Callback function triggered when the user's system locale preferences have changed.

Function

Synopsis

```
love.localechanged( )
```

Arguments

None.

Returns

Nothing.

See Also

- love
- love.system.getPreferredLocales

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.lowmemory

love.lowmemory · revision 24982 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 0.10.0

This callback is not supported in earlier versions.

Callback function triggered when the system is running out of memory on mobile devices.

Mobile operating systems may forcefully kill the game if it uses too much memory, so any non-critical resource should be removed if possible (by setting all variables referencing the resources to **nil**), when this event is triggered. Sounds and images in particular tend to use the most memory.

Function

Synopsis

```
love.lowmemory( )
```

Arguments

None.

Returns

Nothing.

Examples

Clear unused data and collect the garbage when memory is low.

```
local cachetable = {}
for i = 0, math.pi * 2, math.pi / 1000 do
    cachetable[i] = math.sin(i)
end

function love.lowmemory()
    cachetable = {}
    collectgarbage()
end
```

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.maker

love.maker · revision 29360 · last edited 2024-10-09

love.maker

This library was written in Lua specifically for the LÖVE framework. love.maker is a library that can minify, compile and compress your project folder. If you choose to compile your Lua script files, the resulting .love file will only run on the exact same version of LÖVE or LuaJIT that was used when building. This library allows you to output .love files anywhere on your system using absolute paths.

Example

```lua
love.maker = require("maker")
love.maker.setExtensions('lua', 'png', 'txt') -- include only the specified extensions

local build = love.maker.newBuild("C://path/to/project/folder/") -- create from source folder
build:ignore('/readme.txt') -- exclude a specific file
build:ignoreMatch('^/%.git') -- exclude based on pattern matching
build:allow('/images/exception.jpg') -- whitelist a specific file

build:save('C://path/to/output/game.love', 'DEMO') -- build the .love project file
local comment = love.maker.getComment(dest)
print(comment)
```

Documentation and source code

https://github.com/2dengine/love.maker ↗

https://2dengine.com/doc/maker.html ↗

love.math

love.math · revision 22611 · last edited 2023-07-05

Available since LÖVE 0.9.0

This module is not supported in earlier versions.

Provides system-independent mathematical functions.

Types

BezierCurve	ABézier curve object that can evaluate and render Bézier curves of arbitrary degree.	Added since 0.9.0
RandomGenerator	Arandom number generation object which has its own random state.	Added since 0.9.0
Transform	Object containing a coordinate system transformation.	Added since 11.0

Functions

love.math.colorFromBytes	Converts a color from 0..255 to 0..1 range.	Added since 11.3	
love.math.colorToBytes	Converts a color from 0..1 to 0..255 range.	Added since 11.3	
love.math.compress	Compresses a string or data using a specific compression algorithm.	Added since 0.10.0	Deprecated in 11.0
love.math.decompress	Decompresses a CompressedData or previously compressed string or Data object.	Added since 0.10.0	Deprecated in 11.0
love.math.gammaToLinear	Converts a color from gamma-space (sRGB) to linear-space (RGB).	Added since 0.9.1	
love.math.getRandomSeed	Gets the seed of the random number generator.	Added since 0.9.0	
love.math.getRandomState	Gets the current state of the random number generator.	Added since 0.9.1	
love.math.isConvex	Checks whether a polygon is convex.	Added since 0.9.0	
love.math.linearToGamma	Converts a color from linear-space (RGB) to gamma-space (sRGB).	Added since 0.9.1	
love.math.newBezierCurve	Creates a new BezierCurve object.	Added since 0.9.0	
love.math.newRandomGenerator	Creates a new RandomGenerator object.	Added since 0.9.0	
love.math.newTransform	Creates a new Transform object.	Added since 11.0	
love.math.noise	Generates a Simplex noise value in 1-4 dimensions.	Added since 0.9.0	
love.math.random	Get uniformly distributed pseudo-random number	Added since 0.9.0	
love.math.randomNormal	Get a normally distributed pseudo random number.	Added since 0.9.0	
love.math.setRandomSeed	Sets the seed of the random number generator.	Added since 0.9.0	
love.math.setRandomState	Sets the current state of the random number generator.	Added since 0.9.1	

love.math.triangulate	Decomposes a simple polygon into triangles.	Added since 0.9.0

Enums

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.colorFromBytes

love.math.colorFromBytes · revision 31636 · last edited 2026-05-27

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.3

This function is not supported in earlier versions.

Converts a color from 0..255 to 0..1 range.

Function

Synopsis

```
r, g, b, a = love.math.colorFromBytes( rb, gb, bb, ab )
```

Arguments

number rb
: Red color component in 0..255 range.

number gb
: Green color component in 0..255 range.

number bb
: Blue color component in 0..255 range.

number ab (nil)
: Alpha color component in 0..255 range.

Returns

number r
: Red color component in 0..1 range.

number g
: Green color component in 0..1 range.

number b
: Blue color component in 0..1 range.

number a (nil)
: Alpha color component in 0..1 range or nil if alpha is not specified.

Notes

Here's implementation for 11.2 and earlier.

```
function love.math.colorFromBytes(r, g, b, a)
    if type(r) == "table" then
        r, g, b, a = r[1], r[2], r[3], r[4]
    end
    r = clamp01(floor(r + 0.5) / 255)
    g = clamp01(floor(g + 0.5) / 255)
```

```
    b = clamp01(floor(b + 0.5) / 255)
    a = a ~= nil and clamp01(floor(a + 0.5) / 255) or nil
    return r, g, b, a
end
```

Where `clamp01` is defined as follows

```
local function clamp01(x)
    return math.min(math.max(x, 0), 1)
end
```

See Also

- love.math
- love.graphics.setColor
- love.math.colorToBytes

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.colorToBytes

love.math.colorToBytes · revision 22058 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.3

This function is not supported in earlier versions.

Converts a color from 0..1 to 0..255 range.

Function

Synopsis

```
rb, gb, bb, ab = love.math.colorToBytes( r, g, b, a )
```

Arguments

number r
: Red color component.

number g
: Green color component.

number b
: Blue color component.

number a (nil)
: Alpha color component.

Returns

number rb
: Red color component in 0..255 range.

number gb
: Green color component in 0..255 range.

number bb
: Blue color component in 0..255 range.

number ab (nil)
: Alpha color component in 0..255 range or nil if alpha is not specified.

Notes

Here's implementation for 11.2 and earlier.

```
function love.math.colorToBytes(r, g, b, a)
    if type(r) == "table" then
        r, g, b, a = r[1], r[2], r[3], r[4]
    end
    r = floor(clamp01(r) * 255 + 0.5)
    g = floor(clamp01(g) * 255 + 0.5)
```

```
    b = floor(clamp01(b) * 255 + 0.5)
    a = a ~= nil and floor(clamp01(a) * 255 + 0.5) or nil
    return r, g, b, a
end
```

Where `clamp01` is defined as follows

```
local function clamp01(x)
    return math.min(math.max(x, 0), 1)
end
```

See Also

- love.math
- love.math.colorFromBytes

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.compress

love.math.compress · revision 25102 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Deprecated in LÖVE 11.0

This function is deprecated and is replaced by love.data.compress.

Compresses a string or data using a specific compression algorithm.



This function, depending on the compression format and level, can be slow if called repeatedly, such as from love.update or love.draw. Some benchmarks are available here (https://github.com/Cyan4973/lz4#user-content-benchmarks).

Function

Synopsis

```
compressedData = love.math.compress( rawstring, format, level )
```

Arguments

`string rawstring`
: The raw (un-compressed) string to compress.

`CompressedDataFormat format ("lz4")`
: The format to use when compressing the string.

`number level (-1)`
: The level of compression to use, between 0 and 9. -1 indicates the default level. The meaning of this argument depends on the compression format being used.

Returns

`CompressedData compressedData`

A new Data object containing the compressed version of the string.

Function

Synopsis

```
compressedData = love.math.compress( data, format, level )
```

Arguments

`Data data`
A Data object containing the raw (un-compressed) data to compress.

`CompressedDataFormat format ("lz4")`
The format to use when compressing the data.

`number level (-1)`
The level of compression to use, between 0 and 9. -1 indicates the default level. The meaning of this argument depends on the compression format being used.

Returns

`CompressedData compressedData`
A new Data object containing the compressed version of the raw data.

See Also

- love.math
- love.math.decompress
- CompressedData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.decompress

love.math.decompress · revision 22627 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Deprecated in LÖVE 11.0

This function is deprecated and is replaced by love.data.decompress.

Decompresses a CompressedData or previously compressed string or Data object.

Function

Synopsis

```
rawstring = love.math.decompress( compressedData )
```

Arguments

`CompressedData compressedData`

The compressed data to decompress.

Returns

`string rawstring`

A string containing the raw decompressed data.

Function

Synopsis

```
rawstring = love.math.decompress( compressedString, format )
```

Arguments

`string compressedstring`

A string containing data previously compressed with love.math.compress.

`CompressedDataFormat format`

The format that was used to compress the given string.

Returns

`string rawstring`

A string containing the raw decompressed data.

Function

Synopsis

```
rawstring = love.math.decompress( data, format )
```

Arguments

`Data data`
A Data object containing data previously compressed with love.math.compress.

`CompressedDataFormat format`
The format that was used to compress the given data.

Returns

`string rawstring`
A string containing the raw decompressed data.

See Also

- love.math
- love.math.compress

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.gammaToLinear

love.math.gammaToLinear · revision 21277 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Pre-multiply an image's alpha with its RGB values in linear RGB space
- See Also
- Other Languages

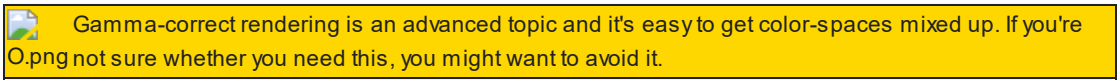
Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Converts a color from gamma-space (sRGB) to linear-space (RGB). This is useful when doing gamma-correct rendering and you need to do math in linear RGB in the few cases where LÖVE doesn't handle conversions automatically.

Read more about gamma-correct rendering here (https://developer.nvidia.com/gpugems/GPUGems3/gpugems3_ch24.html), here (http://filmicgames.com/archives/299), and here (http://renderwonk.com/blog/index.php/archive/adventures-with-gamma-correct-rendering/).

In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.



Function

Synopsis

```
lr, lg, lb = love.math.gammaToLinear( r, g, b )
```

Arguments

number r
: The red channel of the sRGB color to convert.

number g
: The green channel of the sRGB color to convert.

number b
: The blue channel of the sRGB color to convert.

Returns

number lr
: The red channel of the converted color in linear RGB space.

number lg
: The green channel of the converted color in linear RGB space.

number lb

The blue channel of the converted color in linear RGB space.

Notes

An alpha value can be passed into the function as a fourth argument, but it will be returned unchanged because alpha is always linear.

Function

Synopsis

```
lr, lg, lb = love.math.gammaToLinear( color )
```

Arguments

table color
An array with the red, green, and blue channels of the sRGB color to convert.

Returns

number lr
The red channel of the converted color in linear RGB space.

number lg
The green channel of the converted color in linear RGB space.

number lb
The blue channel of the converted color in linear RGB space.

Function

Synopsis

```
lc = love.math.gammaToLinear( c )
```

Arguments

number c
The value of a color channel in sRGB space to convert.

Returns

number lc
The value of the color channel in linear RGB space.

Examples

Pre-multiply an image's alpha with its RGB values in linear RGB space

```
local function PremultiplyLinearPixel(x, y, r, g, b, a)
   r = r * a
   g = g * a
   b = b * a
```

```
   return r, g, b, a
end

local function PremultiplyGammaPixel(x, y, r, g, b, a)
   r, g, b = love.math.gammaToLinear(r, g, b)
   r = r * a
   g = g * a
   b = b * a
   r, g, b = love.math.linearToGamma(r, g, b)
   return r, g, b, a
end

-- Loads an image and pre-multiplies its RGB values with its alpha, for use with the ('alpha', 'premultiplied') blend mode.
-- The multiplication correctly accounts for the color-space of the image.
function NewPremultipliedImage(filepath, flags)
    local imagedata = love.image.newImageData(filepath)

    local mapfunction = (flags and flags.linear) and PremultiplyLinearPixel or PremultiplyGammaPixel
    imagedata:mapPixel(mapfunction)

    return love.graphics.newImage(imagedata, flags)
end

image = NewPremultipliedImage("pig.png")
```

See Also

- love.math
- love.math.linearToGamma

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.getRandomSeed

love.math.getRandomSeed · revision 11980 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the seed of the random number generator.

The seed is split into two numbers due to Lua's use of doubles (http://en.w ikipedia.org/w iki/Double-precision_floating-point_format) for all number values - doubles can't accurately represent integer values above 2^53, but the seed can be an integer value up to 2^64.

Function

Synopsis

```
low, high = love.math.getRandomSeed( )
```

Arguments

None.

Returns

`number low`
: Integer number representing the lower 32 bits of the random number generator's 64 bit seed value.

`number high`
: Integer number representing the higher 32 bits of the random number generator's 64 bit seed value.

See Also

- love.math
- love.math.setRandomSeed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.getRandomState

love.math.getRandomState · revision 22610 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Gets the current state of the random number generator. This returns an opaque implementation-dependent string which is only useful for later use with love.math.setRandomState or RandomGenerator:setState.

This is different from love.math.getRandomSeed in that getRandomState gets the random number generator's current state, whereas getRandomSeed gets the previously set seed number.

Function

Synopsis

```
state = love.math.getRandomState( )
```

Arguments

None.

Returns

`string state`
: The current state of the random number generator, represented as a string.

Notes

The value of the state string does not depend on the current operating system.

See Also

- love.math
- love.math.setRandomState

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.isConvex

love.math.isConvex · revision 17909 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Checks whether a polygon is convex.

PolygonShapes in love.physics, some forms of Meshes, and polygons drawn with love.graphics.polygon must be simple convex polygons.

Function

Synopsis

```
convex = love.math.isConvex( vertices )
```

Arguments

table vertices
: The vertices of the polygon as a table in the form of `{x1, y1, x2, y2, x3, y3, ...}`.

Returns

boolean convex
: Whether the given polygon is convex.

Function

Synopsis

```
convex = love.math.isConvex( x1, y1, x2, y2, x3, y3, ... )
```

Arguments

number x1
: The position of the first vertex of the polygon on the x-axis.

number y1
: The position of the first vertex of the polygon on the y-axis.

number x2
: The position of the second vertex of the polygon on the x-axis.

number y2
: The position of the second vertex of the polygon on the y-axis.

number x3
: The position of the third vertex of the polygon on the x-axis.

number y3

The position of the third vertex of the polygon on the y-axis.

Returns

boolean convex

Whether the given polygon is convex.

See Also

- love.math.triangulate
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.linearToGamma

love.math.linearToGamma · revision 21278 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Pre-multiply an image's alpha with its RGB values in linear RGB space
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Converts a color from linear-space (RGB) to gamma-space (sRGB). This is useful when storing linear RGB color values in an image, because the linear RGB color space has less precision than sRGB for dark colors, which can result in noticeable color banding when drawing.

In general, colors chosen based on what they look like on-screen are already in gamma-space and should not be double-converted. Colors calculated using math are often in the linear RGB space.

Read more about gamma-correct rendering here (http://http.developer.nvidia.com/GPUGems3/gpugems3_ch24.html), here (http://filmicgames.com/archives/299), and here (http://renderwonk.com/blog/index.php/archive/adventures-with-gamma-correct-rendering/).

In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.


Gamma-correct rendering is an advanced topic and it's easy to get color-spaces mixed up. If you're not sure whether you need this, you might want to avoid it.

Function

Synopsis

```
cr, cg, cb = love.math.linearToGamma( lr, lg, lb )
```

Arguments

number lr
: The red channel of the linear RGB color to convert.

number lg
: The green channel of the linear RGB color to convert.

number lb
: The blue channel of the linear RGB color to convert.

Returns

number cr
: The red channel of the converted color in gamma sRGB space.

number cg
: The green channel of the converted color in gamma sRGB space.

number cb
: The blue channel of the converted color in gamma sRGB space.

Notes

An alpha value can be passed into the function as a fourth argument, but it will be returned unchanged because alpha is always linear.

Function

Synopsis

```
cr, cg, cb = love.math.linearToGamma( color )
```

Arguments

table color
: An array with the red, green, and blue channels of the linear RGB color to convert.

Returns

number cr
: The red channel of the converted color in gamma sRGB space.

number cg
: The green channel of the converted color in gamma sRGB space.

number cb
: The blue channel of the converted color in gamma sRGB space.

Function

Synopsis

```
c = love.math.linearToGamma( lc )
```

Arguments

number lc
: The value of a color channel in linear RGB space to convert.

Returns

number c
: The value of the color channel in gamma sRGB space.

Examples

Pre-multiply an image's alpha with its RGB values in linear RGB space

```
local function PremultiplyLinearPixel(x, y, r, g, b, a)
   r = r * a
   g = g * a
```

```
    b = b * a
    return r, g, b, a
end

local function PremultiplyGammaPixel(x, y, r, g, b, a)
    r, g, b = love.math.gammaToLinear(r, g, b)
    r = r * a
    g = g * a
    b = b * a
    r, g, b = love.math.linearToGamma(r, g, b)
    return r, g, b, a
end

-- Loads an image and pre-multiplies its RGB values with its alpha, for use with the ('alpha', 'premultiplied') blend mode.
-- The multiplication correctly accounts for the color-space of the image.
function NewPremultipliedImage(filepath, flags)
    local imagedata = love.image.newImageData(filepath)

    local mapfunction = (flags and flags.linear) and PremultiplyLinearPixel or PremultiplyGammaPixel
    imagedata:mapPixel(mapfunction)

    return love.graphics.newImage(imagedata, flags)
end

image = NewPremultipliedImage("pig.png")
```

See Also

- love.math
- love.math.gammaToLinear

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.newBezierCurve

love.math.newBezierCurve · revision 27216 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Creates a new BezierCurve object.

The number of vertices in the control polygon determines the degree of the curve, e.g. three vertices define a quadratic (degree 2) Bézier curve, four vertices define a cubic (degree 3) Bézier curve, etc.

Function

Synopsis

```
curve = love.math.newBezierCurve( vertices )
```

Arguments

`table vertices`
: The vertices of the control polygon as a table in the form of `{x1, y1, x2, y2, x3, y3, ...}`.

Returns

`BezierCurve curve`
: A Bézier curve object.

Function

Synopsis

```
curve = love.math.newBezierCurve( x1, y1, x2, y2, x3, y3, ... )
```

Arguments

`number x1`
: The position of the first vertex of the control polygon on the x-axis.

`number y1`
: The position of the first vertex of the control polygon on the y-axis.

`number x2`
: The position of the second vertex of the control polygon on the x-axis.

`number y2`
: The position of the second vertex of the control polygon on the y-axis.

`number x3`
: The position of the third vertex of the control polygon on the x-axis.

number y3

The position of the third vertex of the control polygon on the y-axis.

Returns

BezierCurve curve

A Bézier curve object.

See Also

- love.math
- BezierCurve
- BezierCurve:render

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.newRandomGenerator

love.math.newRandomGenerator · revision 25557 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Creates a new RandomGenerator object which is completely independent of other RandomGenerator objects and random functions.

Function

Synopsis

```
rng = love.math.newRandomGenerator( )
```

Arguments

None

Returns

`RandomGenerator rng`

The new Random Number Generator object.

Notes

The default seed used in version 11.x is the following low/high pair: 0xCBBF7A44, 0x0139408D.

Function

Synopsis

```
rng = love.math.newRandomGenerator( seed )
```

Arguments

`number seed`

The initial seed number to use for this object.

Returns

`RandomGenerator rng`

The new Random Number Generator object.

Notes

See RandomGenerator:setSeed.

Function

Synopsis

```
rng = love.math.newRandomGenerator( low, high )
```

Arguments

number low
The lower 32 bits of the seed number to use for this object.

number high
The higher 32 bits of the seed number to use for this object.

Returns

RandomGenerator rng
The new Random Number Generator object.

Notes

See RandomGenerator:setSeed.

Examples

Creates a new RandomGenerator object, then generates a number between 1 and 100 inclusive.

```
function love.load()
    rng = love.math.newRandomGenerator()
    randomNumber = rng:random(1,100)
end
```

See Also

- love.math
- RandomGenerator

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.newTransform

love.math.newTransform · revision 29487 · last edited 2026-04-20

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Creates a new Transform object.

Function

Creates a Transform with no transformations applied. Call methods on the returned object to apply transformations.

Synopsis

```
transform = love.math.newTransform( )
```

Arguments

None.

Returns

`Transform transform`
: The new Transform object.

Function

Creates a Transform with the specified transformation applied on creation.

Synopsis

```
transform = love.math.newTransform( x, y, angle, sx, sy, ox, oy, kx, ky )
```

Arguments

`number x`
: The position of the new Transform on the x-axis.

`number y`
: The position of the new Transform on the y-axis.

`number angle (0)`
: The orientation of the new Transform in radians.

`number sx (1)`
: Scale factor on the x-axis.

`number sy (sx)`

Scale factor on the y-axis.

`number ox (0)`

Origin offset on the x-axis.

`number oy (0)`

Origin offset on the y-axis.

`number kx (0)`

Shearing / skew factor on the x-axis.

`number ky (0)`

Shearing / skew factor on the y-axis.

Returns

`Transform transform`

The new Transform object.

Examples

Creates a new Transform object and uses it to position and rotate a rectangle around its center:

```
function love.load()
    rectwidth = 100
    rectheight = 100

    -- arguments are: x, y, angle, scalex, scaley, offsetx, offsety
    transform = love.math.newTransform(100, 100, math.pi/4, 1, 1, rectwidth / 2, rectheight / 2)
end

function love.draw()
    love.graphics.applyTransform(transform)
    love.graphics.rectangle("fill", 0, 0, rectwidth, rectheight)
end
```

See Also

- love.math
- Transform

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.noise

love.math.noise · revision 28024 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Generates a Simplex or Perlin noise value in 1-4 dimensions. The return value will always be the same, given the same arguments.

Simplex noise (http://en.w ikipedia.org/w iki/Simplex_noise) is closely related to Perlin noise (http://en.w ikipedia.org/w iki/Perlin_noise). It is widely used for procedural content generation.

There are many webpages (http://libnoise.sourceforge.net/noisegen/) which discuss Perlin and Simplex noise in detail.



The return value might be constant if only integer arguments are used. Avoid solely passing in integers, to get varying return values.

Function

Generates Simplex noise from 1 dimension.

Synopsis

```
value = love.math.noise( x )
```

Arguments

number x

The number used to generate the noise value.

Returns

number value

The noise value in the range of [0, 1].

Function

Generates Simplex noise from 2 dimensions.

Synopsis

```
value = love.math.noise( x, y )
```

Arguments

number x
The first value of the 2-dimensional vector used to generate the noise value.

number y
The second value of the 2-dimensional vector used to generate the noise value.

Returns

number value
The noise value in the range of [0, 1].

Function

Generates Perlin noise (Simplex noise in version 0.9.2 and older) from 3 dimensions.

Synopsis

```
value = love.math.noise( x, y, z )
```

Arguments

number x
The first value of the 3-dimensional vector used to generate the noise value.

number y
The second value of the 3-dimensional vector used to generate the noise value.

number z
The third value of the 3-dimensional vector used to generate the noise value.

Returns

number value

The noise value in the range of [0, 1].

Function

Generates Perlin noise (Simplex noise in version 0.9.2 and older) from 4 dimensions.

Synopsis

```
value = love.math.noise( x, y, z, w )
```

Arguments

number x
: The first value of the 4-dimensional vector used to generate the noise value.

number y
: The second value of the 4-dimensional vector used to generate the noise value.

number z
: The third value of the 4-dimensional vector used to generate the noise value.

number w
: The fourth value of the 4-dimensional vector used to generate the noise value.

Returns

number value
: The noise value in the range of [0, 1].

Examples

Fills a two-dimensional grid with simplex noise each time a key is pressed.

```
local grid = {}

local function generateNoiseGrid()
    -- Fill each tile in our grid with noise.
    local baseX = 10000 * love.math.random()
    local baseY = 10000 * love.math.random()
    for y = 1, 60 do
        grid[y] = {}
        for x = 1, 60 do
            grid[y][x] = love.math.noise(baseX+.1*x, baseY+.2*y)
        end
    end
end

function love.load()
    generateNoiseGrid()
end
function love.keypressed()
    generateNoiseGrid()
end

function love.draw()
    local tileSize = 8
    for y = 1, #grid do
        for x = 1, #grid[y] do
            love.graphics.setColor(1, 1, 1, grid[y][x])
```

```
            love.graphics.rectangle("fill", x*tileSize, y*tileSize, tileSize-1, tileSize-1)
        end
    end
end
```

See Also

- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.perlinNoise

love.math.perlinNoise · revision 28063 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function replaces love.math.noise.

Generates a Perlin noise (http://en.w ikipedia.org/w iki/Perlin_noise) value in 1-4 dimensions. The return value will always be the same, given the same arguments.

It is widely used for procedural content generation.


The return value might be constant if only integer arguments are used. Avoid solely passing in integers, to get varying return values.

Function

Generates Perlin noise from 1 dimension.

Synopsis

```
value = love.math.perlinNoise( x )
```

Arguments

number x

The number used to generate the noise value.

Returns

number value

The noise value in the range of [0, 1].

Function

Generates Perlin noise from 2 dimensions.

Synopsis

```
value = love.math.perlinNoise( x, y )
```

Arguments

number x
The first value of the 2-dimensional vector used to generate the noise value.

number y
The second value of the 2-dimensional vector used to generate the noise value.

Returns

number value
The noise value in the range of [0, 1].

Function

Generates Perlin noise from 3 dimensions.

Synopsis

```
value = love.math.perlinNoise( x, y, z )
```

Arguments

number x
The first value of the 3-dimensional vector used to generate the noise value.

number y
The second value of the 3-dimensional vector used to generate the noise value.

number z
The third value of the 3-dimensional vector used to generate the noise value.

Returns

number value

The noise value in the range of [0, 1].

Function

Generates Perlin noise from 4 dimensions.

Synopsis

```
value = love.math.perlinNoise( x, y, z, w )
```

Arguments

number x
The first value of the 4-dimensional vector used to generate the noise value.

number y
The second value of the 4-dimensional vector used to generate the noise value.

number z
The third value of the 4-dimensional vector used to generate the noise value.

number w
The fourth value of the 4-dimensional vector used to generate the noise value.

Returns

number value
The noise value in the range of [0, 1].

Notes

There are many webpages (http://libnoise.sourceforge.net/noisegen/) which discuss Perlin and Simplex noise in detail.

See Also

- love.math
- love.math.simplexNoise

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.random

love.math.random · revision 29675 · last edited 2025-09-11

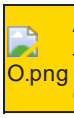
On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Generates a pseudo-random number in a platform independent manner. This function is seeded at startup, so you generally don't need to seed it yourself.



As with RandomGenerator and math.random (https://www.lua.org/manual/5.1/manual.html#pdf-math.random), this function is not a cryptographically secure (https://en.wikipedia.org/wiki/Cryptographically_secure_pseudorandom_number_generator) random number generator, and thus should not be used for cryptography!

Function

Get uniformly distributed pseudo-random **real** number between 0 inclusive to 1 exclusive.

Synopsis

```
number = love.math.random( )
```

Arguments

None.

Returns

number `number`

The pseudo-random number.

Function

Get a uniformly distributed pseudo-random **integer** between 1 inclusive to `max` inclusive.

Synopsis

```
number = love.math.random( max )
```

Arguments

number `max`

The maximum possible value it should return.

Returns

number number
The pseudo-random integer number.

Function

Get uniformly distributed pseudo-random **integer** between `min` inclusive to `max` inclusive.

Synopsis

```
number = love.math.random( min, max )
```

Arguments

number min
The minimum possible value it should return.

number max
The maximum possible value it should return.

Returns

number number
The pseudo-random integer number.

Examples

Generates a number between 1 and 100 (both inclusive).

```
function love.load()
    randomNumber = love.math.random(1, 100)
end
```

Generate a random permutation of list:

```
function shuffle (list)
    -- backward iteration from last to second element:
    for i = #list, 2, -1 do
        -- choose one of elements:
        local j = love.math.random(i) -- between 1 to i (both inclusive)
        -- replace both elements each other:
        list[i], list[j] = list[j], list[i]
```

```
    end
end
```

Notes

When using the 2nd and 3rd variant, numbers passed will be rounded, thus, `love.math.random(0, 76.767)` may return 77

See Also

- love.math
- love.math.setRandomSeed
- love.math.randomNormal
- love.math.newRandomGenerator

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.randomNormal

love.math.randomNormal · revision 27914 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

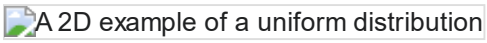
Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Generates a normally-distributed (https://en.w ikipedia.org/w iki/Normal_distribution) pseudo-random number. This function is seeded at startup, so you generally don't need to seed it yourself.

While a typical uniform distribution looks like this:





A typical normal distribution looks like this (note the values aggregating at the center, and the shape of a bell curve):





Function

Synopsis

```
number = love.math.randomNormal( stddev, mean )
```

Arguments

`number stddev (1)`
: Standard deviation of the distribution.

`number mean (0)`
: The mean of the distribution.

Returns

`number number`
: Normally distributed random number with variance (stddev)² and the specified mean.

See Also

- love.math
- love.math.random

- RandomGenerator

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.setRandomSeed

love.math.setRandomSeed · revision 22607 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Set a random seed
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the seed of the random number generator using the specified integer number. This is called internally at startup, so you generally don't need to call it yourself.

Function

Synopsis

```
love.math.setRandomSeed( seed )
```

Arguments

number seed
: The integer number with which you want to seed the randomization. Must be within the range of [0, 2^53 - 1].

Returns

Nothing.

Notes

Due to Lua's use of double-precision floating point numbers, integer values above 2^53 cannot be accurately represented. Use the other variant of the function if you want to use a larger number.

Function

Combines two 32-bit integer numbers into a 64-bit integer value and sets the seed of the random number generator using the value.

Synopsis

```
love.math.setRandomSeed( low, high )
```

Arguments

number low
: The lower 32 bits of the seed value. Must be within the range of [0, 2^32 - 1].

number high

The higher 32 bits of the seed value. Must be within the range of [0, 2^32 - 1].

Returns

Nothing.

Examples

Set a random seed

```
love.math.setRandomSeed(love.timer.getTime())
```

See Also

- love.math
- love.math.getRandomSeed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.setRandomState

love.math.setRandomState · revision 22609 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Sets the current state of the random number generator. The value used as an argument for this function is an opaque implementation-dependent string and should only originate from a previous call to love.math.getRandomState.

This is different from love.math.setRandomSeed in that setRandomState directly sets the random number generator's current implementation-dependent state, whereas setRandomSeed gives it a new seed value.

Function

Synopsis

```
love.math.setRandomState( state )
```

Arguments

`string state`
: The new state of the random number generator, represented as a string. This should originate from a previous call to love.math.getRandomState.

Returns

Nothing.

Notes

The effect of the state string does not depend on the current operating system.

See Also

- love.math
- love.math.getRandomState

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.simplexNoise

love.math.simplexNoise · revision 28064 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function replaces love.math.noise.

Generates a Simplex noise (http://en.w ikipedia.org/w iki/Simplex_noise) value in 1-4 dimensions. The return value will always be the same, given the same arguments.

It is widely used for procedural content generation.


The return value might be constant if only integer arguments are used. Avoid solely passing in integers, to get varying return values.

Function

Generates Simplex noise from 1 dimension.

Synopsis

```
value = love.math.simplexNoise( x )
```

Arguments

number x

The number used to generate the noise value.

Returns

number value

The noise value in the range of [0, 1].

Function

Generates Simplex noise from 2 dimensions.

Synopsis

```
value = love.math.simplexNoise( x, y )
```

Arguments

number x
The first value of the 2-dimensional vector used to generate the noise value.

number y
The second value of the 2-dimensional vector used to generate the noise value.

Returns

number value
The noise value in the range of [0, 1].

Function

Generates Simplex noise from 3 dimensions.

Synopsis

```
value = love.math.simplexNoise( x, y, z )
```

Arguments

number x
The first value of the 3-dimensional vector used to generate the noise value.

number y
The second value of the 3-dimensional vector used to generate the noise value.

number z
The third value of the 3-dimensional vector used to generate the noise value.

Returns

number value

The noise value in the range of [0, 1].

Function

Generates Simplex noise from 4 dimensions.

Synopsis

```
value = love.math.simplexNoise( x, y, z, w )
```

Arguments

number x
The first value of the 4-dimensional vector used to generate the noise value.

number y
The second value of the 4-dimensional vector used to generate the noise value.

number z
The third value of the 4-dimensional vector used to generate the noise value.

number w
The fourth value of the 4-dimensional vector used to generate the noise value.

Returns

number value
The noise value in the range of [0, 1].

Notes

There are many webpages (http://libnoise.sourceforge.net/noisegen/) which discuss Perlin and Simplex noise in detail.

See Also

- love.math
- love.math.perlinNoise

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.math.triangulate

love.math.triangulate · revision 15259 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Decomposes a simple convex or concave polygon into triangles.

Function

Synopsis

```
triangles = love.math.triangulate( polygon )
```

Arguments

table polygon

Polygon to triangulate. Must not intersect itself.

Returns

table triangles

List of triangles the polygon is composed of, in the form of `{{x1, y1, x2, y2, x3, y3}, {x1, y1, x2, y2, x3, y3}, ...}`.

Function

Synopsis

```
triangles = love.math.triangulate( x1, y1, x2, y2, x3, y3, ... )
```

Arguments

number x1

The position of the first vertex of the polygon on the x-axis.

number y1

The position of the first vertex of the polygon on the y-axis.

number x2

The position of the second vertex of the polygon on the x-axis.

number y2

The position of the second vertex of the polygon on the y-axis.

number x3

The position of the third vertex of the polygon on the x-axis.

number y3

The position of the third vertex of the polygon on the y-axis.

Returns

table triangles

List of triangles the polygon is composed of, in the form of `{{x1, y1, x2, y2, x3, y3}, {x1, y1, x2, y2, x3, y3}, ...}`.

See Also

- love.math
- love.graphics.polygon
- love.physics.newPolygonShape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse

love.mouse · revision 16211 · last edited 2025-03-24

Provides an interface to the user's mouse.

Types

Cursor	Represents a hardware cursor.	Added since 0.9.0

Functions

love.mouse.getCursor	Gets the current Cursor.	Added since 0.9.0	
love.mouse.getPosition	Returns the current position of the mouse.	Added since 0.3.2	
love.mouse.getRelativeMode	Gets whether relative mode is enabled for the mouse.	Added since 0.9.2	
love.mouse.getSystemCursor	Gets a Cursor object representing a system-native hardware cursor.	Added since 0.9.0	
love.mouse.getX	Returns the current x-position of the mouse.		
love.mouse.getY	Returns the current y-position of the mouse.		
love.mouse.hasCursor	Gets whether cursor functionality is supported.	Added since 0.10.0	Removed in 11.0
love.mouse.isCursorSupported	Gets whether cursor functionality is supported.	Added since 11.0	
love.mouse.isDown	Checks whether a certain button is down.		
love.mouse.isGrabbed	Checks if the mouse is grabbed.		
love.mouse.isVisible	Checks if the cursor is visible.		
love.mouse.newCursor	Creates a new hardware Cursor object from an image.	Added since 0.9.0	
love.mouse.setCursor	Sets the current mouse cursor.	Added since 0.9.0	
love.mouse.setGrab	Grabs the mouse and confines it to the window.		Removed in 0.9.0
love.mouse.setGrabbed	Grabs the mouse and confines it to the window.	Added since 0.9.0	
love.mouse.setPosition	Sets the current position of the mouse.		
love.mouse.setRelativeMode	Sets whether relative mode is enabled for the mouse.	Added since 0.9.2	
love.mouse.setVisible	Sets the current visibility of the cursor.		
love.mouse.setX	Sets the current X position of the mouse.	Added since 0.9.0	
love.mouse.setY	Sets the current Y position of the mouse.	Added since 0.9.0	

Enums

CursorType	Types of hardware cursors.	Added since 0.9.0

MouseConstant	Mouse buttons.	Removed in 0.10.0

See Also

- love
- love.mousepressed
- love.mousereleased
- love.mousemoved
- love.wheelmoved

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.getCursor

love.mouse.getCursor · revision 27187 · last edited 2025-03-24

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the current Cursor.

Function

Synopsis

```
cursor = love.mouse.getCursor( )
```

Arguments

None.

Returns

`Cursor cursor (nil)`

The current cursor, or nil if no cursor is set.

See Also

- love.mouse
- love.mouse.setCursor
- love.mouse.getSystemCursor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.getGlobalPosition

love.mouse.getGlobalPosition · revision 29247 · last edited 2024-06-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets the global position of the mouse on the screen. This can be used when implementing custom drag areas in the window.

The global mouse position is in the same coordinate space as love.window.getPosition and love.window.setPosition.

Function

Synopsis

```
x, y, displayindex = love.mouse.getGlobalPosition( )
```

Arguments

None.

Returns

number x
: The x-coordinate of the mouse's global position, within its current display.

number y
: The y-coordinate of the mouse's global position, within its current display.

number displayindex
: The index of the display that the mouse is in.

See Also

- love.mouse
- love.mouse.getPosition
- love.window.getPosition
- love.window.setPosition

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.getPosition

love.mouse.getPosition · revision 14850 · last edited 2025-03-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Use getPosition to help draw a custom mouse image
- See Also
- Other Languages

Returns the current position of the mouse.

Function

Synopsis

```
x, y = love.mouse.getPosition( )
```

Arguments

None.

Returns

number x
: The position of the mouse along the x-axis.

number y
: The position of the mouse along the y-axis.

Examples

Use getPosition to help draw a custom mouse image

```
function love.load()
   love.mouse.setVisible(false) -- make default mouse invisible
   img = love.graphics.newImage("mouse.png") -- load in a custom mouse image
end
function love.draw()
   local x, y = love.mouse.getPosition() -- get the position of the mouse
   love.graphics.draw(img, x, y) -- draw the custom mouse image
end
```

See Also

- love.mouse
- love.mouse.getX
- love.mouse.getY

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.getRelativeMode

love.mouse.getRelativeMode · revision 14669 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets whether relative mode is enabled for the mouse.

If relative mode is enabled, the cursor is hidden and doesn't move when the mouse does, but relative mouse motion events are still generated via love.mousemoved. This lets the mouse move in any direction indefinitely without the cursor getting stuck at the edges of the screen.

The reported position of the mouse is not updated while relative mode is enabled, even when relative mouse motion events are generated.

Function

Synopsis

```
enabled = love.mouse.getRelativeMode( )
```

Arguments

None.

Returns

`boolean enabled`

True if relative mode is enabled, false if it's disabled.

See Also

- love.mouse
- love.mouse.setRelativeMode
- love.mousemoved

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.getSystemCursor

love.mouse.getSystemCursor · revision 11862 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets a Cursor object representing a system-native hardware cursor.

Hardware cursors are framerate-independent and work the same way as normal operating system cursors. Unlike drawing an image at the mouse's current coordinates, hardware cursors never have visible lag between when the mouse is moved and when the cursor position updates, even at low framerates.

Function

Synopsis

```
cursor = love.mouse.getSystemCursor( ctype )
```

Arguments

`CursorType ctype`

The type of system cursor to get.

Returns

`Cursor cursor`

The Cursor object representing the system cursor type.

Notes

The "image" CursorType is not a valid argument. Use love.mouse.newCursor to create a hardware cursor using a custom image.

Examples

```
function love.load()
    i_beam_cursor = love.mouse.getSystemCursor("ibeam")
    love.mouse.setCursor(i_beam_cursor)
end
```

See Also

- love.mouse
- love.mouse.setCursor

- Cursor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.getX

love.mouse.getX · revision 7227 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Returns the current x-position of the mouse.

Function

Synopsis

```
x = love.mouse.getX( )
```

Arguments

None.

Returns

number x
: The position of the mouse along the x-axis.

Examples

Draw a vertical line at the mouse's x-position.

```
function love.draw()
   local x = love.mouse.getX()
   love.graphics.line(x,0, x,love.graphics.getHeight())
end
```

See Also

- love.mouse
- love.mouse.getY
- love.mouse.getPosition

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.getY

love.mouse.getY · revision 7228 · last edited 2025-03-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Returns the current y-position of the mouse.

Function

Synopsis

```
y = love.mouse.getY( )
```

Arguments

None.

Returns

number y
: The position of the mouse along the y-axis.

Examples

Draw a horizontal line at the mouse's y-position.

```
function love.draw()
   local y = love.mouse.getY()
   love.graphics.line(0,y, love.graphics.getWidth(),y)
end
```

See Also

- love.mouse
- love.mouse.getX
- love.mouse.getPosition

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.hasCursor

love.mouse.hasCursor · revision 22770 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Deprecated in LÖVE 11.0

This function has been renamed to love.mouse.isCursorSupported.

Gets whether cursor functionality is supported.

If it isn't supported, calling love.mouse.newCursor and love.mouse.getSystemCursor will cause an error. Mobile devices do not support cursors.

Function

Synopsis

```
hascursor = love.mouse.hasCursor( )
```

Arguments

None.

Returns

`boolean hascursor`

Whether the system has cursor functionality.

See Also

- love.mouse

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.isCursorSupported

love.mouse.isCursorSupported · revision 28079 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 11.0

It has been renamed from love.mouse.hasCursor.

Gets whether cursor functionality is supported.

If it isn't supported, calling love.mouse.newCursor and love.mouse.getSystemCursor will cause an error. Some mobile devices do not support cursors.

Function

Synopsis

```
supported = love.mouse.isCursorSupported( )
```

Arguments

None.

Returns

`boolean supported`

Whether the system has cursor functionality.

See Also

- love.mouse

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.isDown

love.mouse.isDown · revision 29459 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Example
  - Increase a value w hile the right mouse button is held
- See Also
- Other Languages

Checks whether a certain mouse button is down.

This function does not detect mouse wheel scrolling; you must use the love.wheelmoved (or love.mousepressed in version 0.9.2 and older) callback for that.

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Synopsis

```
down = love.mouse.isDown( button, ... )
```

Arguments

`number button`

The index of a button to check. 1 is the primary mouse button, 2 is the secondary mouse button and 3 is the middle button. Further buttons are mouse dependant.

`number ...`

Additional button indices to check.

Returns

`boolean down`

True if any specified button is down.

Function

Available since LÖVE 0.10.2

This variant is not supported in earlier versions.

Synopsis

```
down = love.mouse.isDown({ button, ... })
```

Arguments

table buttons

Table containing indices of mouse buttons to check.

table button

The index of a button to check. 1 is the primary mouse button, 2 is the secondary mouse button and 3 is the middle button. Further buttons are mouse dependant.

number ...

Additional button indices to check.

Returns

boolean down

True if any specified button is down.

Function

Removed in LÖVE 0.10.0

This variant is not supported in that and later versions.

Synopsis

```
down = love.mouse.isDown( button )
```

Arguments

MouseConstant button

The button to check.

Returns

boolean down

True if the specified button is down.

Function

Available since LÖVE 0.7.2 and removed in LÖVE 0.10.0

This variant is not supported in earlier or later versions.

Synopsis

```
anyDown = love.mouse.isDown( button1, button2, button3, ... )
```

Arguments

MouseConstant buttonN

A button to check.

Returns

boolean anyDown

True if any specified button is down, false otherwise.

Example

Increase a value while the right mouse button is held

```
val = 0   -- establish a variable for later use
function love.update(dt)
    if love.mouse.isDown(2) then
        val = val + dt   -- we will increase the variable by 1 for every second the button is held down
    end
end
```

See Also

- Mouse Constant / buttons
- love.mouse
- love.mousepressed
- love.mousereleased
- love.keyboard.isDown

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.isGrabbed

love.mouse.isGrabbed · revision 14418 · last edited 2025-03-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Checks if the mouse is grabbed.

Function

Synopsis

```
grabbed = love.mouse.isGrabbed( )
```

Arguments

None.

Returns

boolean grabbed

True if the cursor is grabbed, false if it is not.

Examples

Toggles whether the mouse is grabbed by pressing tab, using love.mouse.setGrabbed.

```
function love.keypressed(key)
   if key == "tab" then
      local state = not love.mouse.isGrabbed()   -- the opposite of whatever it currently is
      love.mouse.setGrabbed(state) --Use love.mouse.setGrab(state) for 0.8.0 or lower
   end
end
```

See Also

- love.mouse
- love.mouse.setGrabbed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.isVisible

love.mouse.isVisible · revision 7230 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Checks if the cursor is visible.

Function

Synopsis

```
visible = love.mouse.isVisible( )
```

Arguments

None.

Returns

boolean visible

True if the cursor to visible, false if the cursor is hidden.

Examples

Toggle mouse visibility by pressing tab (using setVisible).

```
function love.keypressed(key)
   if key == "tab" then
      local state = not love.mouse.isVisible()   -- the opposite of whatever it currently is
      love.mouse.setVisible(state)
   end
end
```

See Also

- love.mouse
- love.mouse.setVisible

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.newCursor

love.mouse.newCursor · revision 27188 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

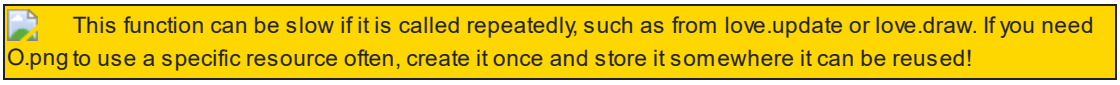
Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Creates a new hardware Cursor object from an image file or ImageData.

Hardware cursors are framerate-independent and work the same way as normal operating system cursors. Unlike drawing an image at the mouse's current coordinates, hardware cursors never have visible lag between when the mouse is moved and when the cursor position updates, even at low framerates.

The hot spot is the point the operating system uses to determine what was clicked and at what position the mouse cursor is. For example, the normal arrow pointer normally has its hot spot at the top left of the image, but a crosshair cursor might have it in the middle.



Function

Synopsis

```
cursor = love.mouse.newCursor( imageData, hotx, hoty )
```

Arguments

ImageData imageData

The ImageData to use for the new Cursor.

number hotx (0)

The x-coordinate in the ImageData of the cursor's hot spot.

number hoty (0)

The y-coordinate in the ImageData of the cursor's hot spot.

Returns

Cursor cursor

The new Cursor object.

Function

Synopsis

```
cursor = love.mouse.newCursor( filename, hotx, hoty )
```

Arguments

string filename
Path to the image to use for the new Cursor.

number hotx (0)
The x-coordinate in the image of the cursor's hot spot.

number hoty (0)
The y-coordinate in the image of the cursor's hot spot.

Returns

Cursor cursor
The new Cursor object.

Function

Synopsis

```
cursor = love.mouse.newCursor( fileData, hotx, hoty )
```

Arguments

FileData fileData
Data representing the image to use for the new Cursor.

number hotx (0)
The x-coordinate in the image of the cursor's hot spot.

number hoty (0)
The y-coordinate in the image of the cursor's hot spot.

Returns

Cursor cursor
The new Cursor object.

See Also

- love.mouse
- love.mouse.setCursor
- love.mouse.getCursor
- love.mouse.getSystemCursor
- Cursor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.setCursor

love.mouse.setCursor · revision 23835 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the current mouse cursor.

Function

Synopsis

```
love.mouse.setCursor( cursor )
```

Arguments

`Cursor cursor`

The Cursor object to use as the current mouse cursor.

Returns

Nothing.

Function

Synopsis

```
love.mouse.setCursor( )
```

Arguments

None.

Returns

Nothing.

Notes

Resets the current mouse cursor to the default.

Examples

```
function love.load()
    cursor = love.mouse.getSystemCursor("hand")
    -- Alternative: cursor = love.mouse.newCursor("pig.png", 0, 0)
end

function love.mousepressed(x, y, button)
    -- Use a custom cursor when the left mouse button is pressed.
    if button == 1 then
        love.mouse.setCursor(cursor)
    end
end

function love.mousereleased(x, y, button)
    -- Go back to the default cursor when the left mouse button is released.
    if button == 1 then
        love.mouse.setCursor()
    end
end
```

See Also

- love.mouse
- love.mouse.newCursor

- love.mouse.getCursor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.setGrab

love.mouse.setGrab · revision 11713 · last edited 2025-03-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Removed in LÖVE 0.9.0

It has been renamed to love.mouse.setGrabbed.

Grabs the mouse and confines it to the window.

Function

Synopsis

```
love.mouse.setGrab( grab )
```

Arguments

boolean grab

True to confine the mouse, false to let it leave the window.

Returns

Nothing.

Examples

Toggles whether the mouse is grabbed by pressing tab, with the help of love.mouse.isGrabbed.

```
function love.keypressed(key)
   if key == "tab" then
      local state = not love.mouse.isGrabbed()   -- the opposite of whatever it currently is
      love.mouse.setGrab(state)
   end
end
```

See Also

- love.mouse
- love.mouse.isGrabbed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

love.mouse.setGrabbed

love.mouse.setGrabbed · revision 11714 · last edited 2025-03-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from love.mouse.setGrab.

Grabs the mouse and confines it to the window.

Function

Synopsis

```
love.mouse.setGrabbed( grab )
```

Arguments

boolean grab

True to confine the mouse, false to let it leave the window.

Returns

Nothing.

Examples

Toggles whether the mouse is grabbed by pressing tab, with the help of love.mouse.isGrabbed.

```
function love.keypressed(key)
   if key == "tab" then
      local state = not love.mouse.isGrabbed()   -- the opposite of whatever it currently is
      love.mouse.setGrabbed(state)
   end
end
```

See Also

- love.mouse
- love.mouse.isGrabbed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.setPosition

love.mouse.setPosition · revision 19471 · last edited 2025-03-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Sets the current position of the mouse. Non-integer values are floored.

Function

Synopsis

```
love.mouse.setPosition( x, y )
```

Arguments

number x
: The new position of the mouse along the x-axis.

number y
: The new position of the mouse along the y-axis.

Returns

Nothing.

Examples

Snap the mouse to the horizontal center of the screen while maintaining the coordinate along the y-axis by using love.mouse.getY.

```
function love.keypressed( )
  love.mouse.setPosition( love.graphics.getWidth() * 0.5, love.mouse.getY() )
end
```

See Also

- love.mouse
- love.mouse.getPosition
- love.mouse.getX
- love.mouse.getY

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.setRelativeMode

love.mouse.setRelativeMode · revision 14902 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Sets whether relative mode is enabled for the mouse.

When relative mode is enabled, the cursor is hidden and doesn't move when the mouse does, but relative mouse motion events are still generated via love.mousemoved. This lets the mouse move in any direction indefinitely without the cursor getting stuck at the edges of the screen.

The reported position of the mouse may not be updated while relative mode is enabled, even when relative mouse motion events are generated.

Function

Synopsis

```
love.mouse.setRelativeMode( enable )
```

Arguments

`boolean enable`
: True to enable relative mode, false to disable it.

Returns

Nothing.

See Also

- love.mouse
- love.mouse.getRelativeMode
- love.mousemoved

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.setVisible

love.mouse.setVisible · revision 7229 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Sets the current visibility of the cursor.

Function

Synopsis

```
love.mouse.setVisible( visible )
```

Arguments

boolean visible
: True to set the cursor to visible, false to hide the cursor.

Returns

Nothing.

Examples

Toggle mouse visibility by pressing tab.

```
function love.keypressed(key)
   if key == "tab" then
      local state = not love.mouse.isVisible()   -- the opposite of whatever it currently is
      love.mouse.setVisible(state)
   end
end
```

See Also

- love.mouse
- love.mouse.isVisible

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.setX

love.mouse.setX · revision 19473 · last edited 2025-03-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the current X position of the mouse. Non-integer values are floored.

Function

Synopsis

```
love.mouse.setX( x )
```

Arguments

`number x`
: The new position of the mouse along the x-axis.

Returns

Nothing.

See Also

- love.mouse
- love.mouse.setPosition
- love.mouse.setY
- love.mouse.getX

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mouse.setY

love.mouse.setY · revision 19472 · last edited 2025-03-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the current Y position of the mouse. Non-integer values are floored.

Function

Synopsis

```
love.mouse.setY( y )
```

Arguments

`number y`
: The new position of the mouse along the y-axis.

Returns

Nothing.

See Also

- love.mouse
- love.mouse.setPosition
- love.mouse.setX
- love.mouse.getY

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mousefocus

love.mousefocus · revision 22496 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
- See Also
- Other Languages

Available since LÖVE 0.9.0

This callback is not supported in earlier versions.

Callback function triggered when window receives or loses mouse focus.

Function

Synopsis

```
love.mousefocus( focus )
```

Arguments

`boolean focus`

Whether the window has mouse focus or not.

Returns

Nothing.

Example

```
function love.load()
  text = "Mouse is in the window!"
end

function love.draw()
  love.graphics.print(text,0,0)
end

function love.mousefocus(f)
  if not f then
    text = "Mouse is not in the window!"
    print("LOST MOUSE FOCUS")
  else
    text = "Mouse is in the window!"
    print("GAINED MOUSE FOCUS")
  end
end
```

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mousemoved

love.mousemoved · revision 28200 · last edited 2025-11-08

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Callback function triggered when the mouse is moved.

Function

Synopsis

```
love.mousemoved( x, y, dx, dy, istouch )
```

Arguments

`number x`
: The mouse position on the x-axis.

`number y`
: The mouse position on the y-axis.

`number dx`
: The amount moved along the x-axis since the last time love.mousemoved was called.

`number dy`
: The amount moved along the y-axis since the last time love.mousemoved was called.

Available since LÖVE 0.10.0

`boolean istouch`
: True if the mouse button press originated from a touchscreen touch-press.

Returns

Nothing.

Notes

If Relative Mode is enabled for the mouse, the **dx** and **dy** arguments of this callback will update but **x** and **y** are not guaranteed to.

Examples

Position a string ("Text") wherever the mouse, also the delta x and y.

```
function love.load()
    printx = 0
    printy = 0
    printdx = 0
    printdy = 0
end

function love.draw()
    love.graphics.print("Text", printx, printy)
    love.graphics.print('dx: '..printdx .. ' dy: '.. printdy, 0, 0)
end

function love.mousemoved( x, y, dx, dy, istouch )
    printx = x
    printy = y
    printdx = dx
    printdy = dy
end
```

See Also

- love
- love.mouse
- love.mousepressed
- love.mousereleased
- love.mouse.setPosition
- love.mouse.getPosition
- love.mouse.setRelativeMode
- love.mouse.getRelativeMode
- love.mouse.isDown

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mousepressed

love.mousepressed · revision 30107 · last edited 2026-03-27

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
- Synopsis
- Arguments
- Returns
- Examples
- See Also
- Other Languages

Callback function triggered when a mouse button is pressed.

Function

Synopsis

```
love.mousepressed( x, y, button, istouch, presses )
```

Arguments

number x
: Mouse x position, in pixels.

number y
: Mouse y position, in pixels.

number button
: The button index that was pressed. 1 is the primary mouse button, 2 is the secondary mouse button, and 3 is the middle button. Further buttons are mouse dependent.

boolean istouch
: True if the mouse button press originated from a touchscreen touch-press.

Available since LÖVE 11.0

number presses
: The number of presses in a short time frame and small area, used to simulate double, triple clicks.

Returns

Nothing.

Notes

Use love.wheelmoved to detect mouse wheel motion. It will not register as a button press in version 0.10.0 and newer.

Function

Removed in LÖVE 0.10.0

This variant is not supported in that and later versions.

Synopsis

```
love.mousepressed( x, y, button )
```

Arguments

number x
: Mouse x position.

number y
: Mouse y position.

MouseConstant button
: Mouse button pressed.

Returns

Nothing.

Examples

Position a string ("Text") wherever the user left-clicks.

```
function love.load()
   printx = 0
   printy = 0
end

function love.draw()
   love.graphics.print("Text", printx, printy)
end

function love.mousepressed(x, y, button, istouch)
   if button == 1 then -- Versions prior to 0.10.0 use the MouseConstant 'l'
      printx = x
      printy = y
   end
end
```

See Also

- love
- love.mouse
- love.mousemoved
- love.mousereleased
- love.mouse.isDown

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.mousereleased

love.mousereleased · revision 30108 · last edited 2025-10-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Callback function triggered when a mouse button is released.

Function

Synopsis

```
love.mousereleased( x, y, button, istouch, presses )
```

Arguments

number x
: Mouse x position, in pixels.

number y
: Mouse y position, in pixels.

number button
: The button index that was released. 1 is the primary mouse button, 2 is the secondary mouse button, and 3 is the middle button. Further buttons are mouse dependent.

boolean istouch
: True if the mouse button release originated from a touchscreen touch-release.

Available since LÖVE 11.0

number presses
: The number of presses in a short time frame and small area, used to simulate double, triple clicks.

Returns

Nothing.

Function

Removed in LÖVE 0.10.0

This variant is not supported in that and later versions.

Synopsis

```
love.mousereleased( x, y, button )
```

Arguments

number x
: Mouse x position.

number y
: Mouse y position.

MouseConstant button
: Mouse button released, except Mouse Wheel.

Returns

Nothing.

Examples

Position a string ("Text") wherever the user releases the primary mouse button.

```
function love.load()
   printx = 0
   printy = 0
end

function love.draw()
   love.graphics.print("Text", printx, printy)
end

function love.mousereleased(x, y, button)
   if button == 1 then
      printx = x
      printy = y
   end
end
```

See Also

- love
- love.mouse
- love.mousepressed
- love.mousemoved

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

love.occluded

love.occluded · revision 29597 · last edited 2026-02-13

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 12.0

This callback is not supported in earlier versions.

Callback function triggered when the window is completely covered by other windows.

An occluded window is distinct from a window which is not visible.

If the window is uncovered after being covered, the love.exposed callback will be called.

Function

Synopsis

```
love.occluded( )
```

Arguments

None.

Returns

Nothing.

See Also

- love
- love.window.isOccluded
- love.exposed
- love.visible

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics

love.physics · revision 30451 · last edited 2025-11-30

On this page

- Types
- Functions
- Enums
- Notes
- Box2D's architecture, concepts, and terminologies.
- See Also
- Other Languages

Available since LÖVE 0.4.0

This module is not supported in earlier versions.

Can simulate 2D rigid bodies in a realistic manner. This module is essentially just a binding to Box2D (http://box2d.org/about/) (version 2.3.0 - manual (https://box2d.org/documentation/)).

For simpler (and more common) use cases, a small number of libraries exist, which are usually more popularly used than love.physics and can be found here: https://github.com/love2d-community/awesome-love2d#physics ↗

Types

Body	Bodies are objects with velocity and position.	
Contact	Contacts are objects created to manage collisions in worlds.	
Fixture	Fixtures attach shapes to bodies.	Added since 0.8.0
Joint	Attach multiple bodies together to interact in unique ways.	
Shape	Shapes are objects used to control mass and collisions.	
World	Aworld is an object that contains all bodies and joints.	

Functions

love.physics.getDistance	Returns the two closest points between two fixtures and their distance.	Added since 0.8.0
love.physics.getMeter	Returns the meter scale factor.	Added since 0.8.0
love.physics.newBody	Creates a new body.	
love.physics.newChainShape	Creates a new ChainShape.	Added since 0.8.0
love.physics.newCircleShape	Creates a new CircleShape.	
love.physics.newDistanceJoint	Creates a DistanceJoint between two bodies.	
love.physics.newEdgeShape	Creates a new EdgeShape.	Added since 0.8.0
love.physics.newFixture	Creates and attaches a fixture.	Added since 0.8.0
love.physics.newFrictionJoint	AFrictionJoint applies friction to a body.	Added since 0.8.0
love.physics.newGearJoint	Create a GearJoint connecting two Joints.	
love.physics.newMotorJoint	Creates a joint between two bodies which controls the relative motion between them.	Added since 0.9.0
love.physics.newMouseJoint	Create a joint between a body and the mouse.	
love.physics.newPolygonShape	Creates a new PolygonShape.	
love.physics.newPrismaticJoint	Creates a PrismaticJoint between two bodies.	
love.physics.newPulleyJoint	Creates a PulleyJoint to join two bodies to each other and the ground.	
love.physics.newRectangleShape	Shorthand for creating rectangular PolygonShapes.	
love.physics.newRevoluteJoint	Creates a pivot joint between two bodies.	

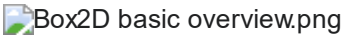
love.physics.newRopeJoint	Creates a joint between two bodies that enforces a max distance between them.	Added since 0.8.0
love.physics.newWeldJoint	A WeldJoint essentially glues two bodies together.	Added since 0.8.0
love.physics.newWheelJoint	Creates a wheel joint.	Added since 0.8.0
love.physics.newWorld	Creates a new World.	
love.physics.setMeter	Sets the meter scale factor.	Added since 0.8.0

Enums

BodyType	The types of a Body.
JointType	Different types of joints.
ShapeType	The different types of Shapes, as returned by Shape:getType.

Notes

Box2D's architecture, concepts, and terminologies.



See Also

- love
- Tutorial:Physics
- Tutorial:PhysicsCollisionCallbacks
- Box2D Gotchas (http://w w w.iforce2d.net/b2dtut/gotchas) (recommended reading)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.getDistance

love.physics.getDistance · revision 9778 · last edited 2025-11-30

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Returns the two closest points between two fixtures and their distance.



This function does not work correctly in 0.8.0 and may terminate LÖVE without errors.

Function

Synopsis

```
distance, x1, y1, x2, y2 = love.physics.getDistance( fixture1, fixture2 )
```

Arguments

`Fixture fixture1`
: The first fixture.

`Fixture fixture2`
: The second fixture.

Returns

`number distance`
: The distance of the two points.

`number x1`
: The x-coordinate of the first point.

`number y1`
: The y-coordinate of the first point.

`number x2`
: The x-coordinate of the second point.

`number y2`
: The y-coordinate of the second point.

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.getMeter

love.physics.getMeter · revision 29415 · last edited 2024-12-17

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Returns the meter scale factor.

All coordinates in the physics module are divided by this number, creating a convenient way to draw the objects directly to the screen without the need for graphics transformations.

It is recommended to create shapes no larger than 10 times the scale. This is important because Box2D is tuned to work well with shape sizes from 0.1 to 10 meters.

Function

Synopsis

```
scale = love.physics.getMeter( )
```

Arguments

None.

Returns

`number scale`

The scale factor as an integer.

See Also

- love.physics
- love.physics.setMeter

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newBody

love.physics.newBody · revision 27755 · last edited 2025-09-29

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Creates a new body.

There are three types of bodies.

- Static bodies do not move, have an infinite mass, and can be used for level boundaries.
- Dynamic bodies are the main actors in the simulation, they collide with everything.
- Kinematic bodies do not react to forces and only collide with dynamic bodies.

The mass of the body gets calculated when a Fixture is attached or removed, but can be changed at any time with Body:setMass or Body:resetMassData.


Making changes to a World is not allowed inside of the beginContact, endContact, preSolve, and postSolve callback functions, as BOX2D locks the world during these callbacks.

Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
body = love.physics.newBody( world, x, y, type )
```

Arguments

`World world`
: The world to create the body in.

`number x (0)`
: The x position of the body.

`number y (0)`
: The y position of the body.

`BodyType type ("static")`
: The type of the body.

Returns

`Body body`

A new body.

Function

Removed in LÖVE 0.8.0
This variant is not supported in that and later versions.

Synopsis

```
body = love.physics.newBody( world, x, y, m, i )
```

Arguments

`World world`
: The world to create the body in.

`number x (0)`
: The x position of the body.

`number y (0)`
: The y position of the body.

`number m (0)`
: The mass of the body.

`number i (0)`
: The rotational inertia of the body.

Returns

`Body body`
: A new body.

Examples

```
love.physics.setMeter(40) -- meter is 40 px/units
world = love.physics.newWorld(0, 9.81*40, true) -- new world with gravity
body1 = love.physics.newBody(world, 250, 300, "static") -- new static body at x=250, y=300
body2 = love.physics.newBody(world, 251, 100, "dynamic") -- new dynamic body at x=251, y=100
```

See Also

- love.physics
- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newChainShape

love.physics.newChainShape · revision 28975 · last edited 2023-10-03

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Creates a new ChainShape.



Due to a limitation of the current collision algorithm, collision with chain shapes can react in a odd manner. see [here (https://code.google.com/p/box2d/issues/detail?id=286)]

Function

Synopsis

```
shape = love.physics.newChainShape( loop, x1, y1, x2, y2, ... )
```

Arguments

`boolean loop`
: If the chain should loop back to the first point.

`number x1`
: The x position of the first point.

`number y1`
: The y position of the first point.

`number x2`
: The x position of the second point.

`number y2`
: The y position of the second point.

`number ...`
: Additional point positions.

Returns

`ChainShape shape`
: The new shape.

Function

Available since LÖVE 0.9.2

This variant is not supported in earlier versions.

Synopsis

```
shape = love.physics.newChainShape( loop, points )
```

Arguments

`boolean loop`
: If the chain should loop back to the first point.

`table points`
: A list of points to construct the ChainShape, in the form of `{x1, y1, x2, y2, ...}`.

Returns

`ChainShape shape`
: The new shape.

See Also

- love.physics
- ChainShape
- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newCircleBody

love.physics.newCircleBody · revision 30712 · last edited 2026-04-04

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Creates a new Body with an attached CircleShape.

This is a convenience constructor which effectively calls love.physics.newBody and love.physics.newCircleShape.

Function

Synopsis

```
body, shape = love.physics.newCircleBody( world, type, x, y, radius )
```

Arguments

`World world`
: The world to create the body in.

`BodyType type`
: The type of the body.

`number x`
: The x position of the body in the world.

`number y`
: The y position of the body in the world.

`number radius`
: The radius of the circle.

Returns

`Body body`
: The new Body.

`CircleShape shape`
: The CircleShape attached to the new Body. The shape's local origin is at the center of the Body.

See Also

- love.physics
- Body
- Shape
- CircleShape
- love.physics.newBody
- love.physics.newCircleShape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newCircleShape

love.physics.newCircleShape · revision 29277 · last edited 2024-06-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Creates a new CircleShape.

Function

Available since LÖVE 0.8.0

These variants are not supported in earlier versions.

Synopsis

```
shape = love.physics.newCircleShape( radius )
```

Arguments

number radius
: The radius of the circle.

Returns

CircleShape shape
: The new shape.

Function

Synopsis

```
shape = love.physics.newCircleShape( x, y, radius )
```

Arguments

number x
: The x position of the circle.

number y
: The y position of the circle.

number radius
: The radius of the circle.

Returns

CircleShape shape

The new shape.

Function

Removed in LÖVE 0.8.0
This variant is not supported in that and later versions.

Synopsis

```
shape = love.physics.newCircleShape( body, x, y, radius )
```

Arguments

`Body body`
The body to attach the shape to.

`number x`
The x offset of the circle.

`number y`
The y offset of the circle.

`number radius`
The radius of the circle.

Returns

`CircleShape shape`
A new CircleShape.

See Also

- love.physics
- CircleShape
- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newDistanceJoint

love.physics.newDistanceJoint · revision 21859 · last edited 2025-11-28

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Creates a DistanceJoint between two bodies.

This joint constrains the distance between two points on two bodies to be constant. These two points are specified in world coordinates and the two bodies are assumed to be in place when this joint is created. The first anchor point is connected to the first body and the second to the second body, and the points define the length of the distance joint.





Making changes to a World is not allowed inside of the beginContact, endContact, preSolve, and postSolve callback functions, as BOX2D locks the world during these callbacks.

Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
joint = love.physics.newDistanceJoint( body1, body2, x1, y1, x2, y2, collideConnected )
```

Arguments

`Body body1`
: The first body to attach to the joint.

`Body body2`
: The second body to attach to the joint.

`number x1`
: The x position of the first anchor point (world space).

`number y1`
: The y position of the first anchor point (world space).

`number x2`
: The x position of the second anchor point (world space).

`number y2`
: The y position of the second anchor point (world space).

`boolean collideConnected (false)`
: Specifies whether the two bodies should collide with each other.

Returns

`DistanceJoint joint`

The new distance joint.

Function

Removed in LÖVE 0.8.0
This variant is not supported in that and later versions.

Synopsis

```
joint = love.physics.newDistanceJoint( body1, body2, x1, y1, x2, y2 )
```

Arguments

Body body1
The first body to attach to the joint.

Body body2
The second body to attach to the joint.

number x1
The x position of the first anchor point (world space).

number y1
The y position of the first anchor point (world space).

number x2
The x position of the second anchor point (world space).

number y2
The y position of the second anchor point (world space).

Returns

DistanceJoint joint
The new distance joint.

See Also

- love.physics
- DistanceJoint
- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newEdgeBody

love.physics.newEdgeBody · revision 30715 · last edited 2026-04-04

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Creates a new Body with an attached EdgeShape.

This is a convenience constructor which effectively calls love.physics.newBody and love.physics.newEdgeShape.

Function

Synopsis

```
body, shape = love.physics.newEdgeBody( world, type, x1, y1, x2, y2 )
```

Arguments

World world
: The world to create the body in.

BodyType type
: The type of the body.

number x1
: The x position of the first point.

number y1
: The y position of the first point.

number x2
: The x position of the second point.

number y2
: The y position of the second point.

Returns

Body body
: The new Body with an attached EdgeShape. The body's world position is computed based on the geometric center of the given vertices, and the created shape's local origin is at that center point.

EdgeShape shape

The EdgeShape attached to the new Body.

Function

Creates a new Body with an attached EdgeShape which has ghost vertices on each side, to help prevent unwanted collisions when a flat shape slides along the edge and moves over to an adjacent shape.

Synopsis

```
body, shape = love.physics.newEdgeBody( world, type, x1, y1, x2, y2, prevx, prevy, nextx, nexty )
```

Arguments

World world
: The world to create the body in.

BodyType type
: The type of the body.

number x1
: The x position of the first point.

number y1
: The y position of the first point.

number x2
: The x position of the second point.

number y2
: The y position of the second point.

number prevx
: The x position of the previous point.

number prevy
: The y position of the previous point.

number nextx
: The x position of the next point.

number nexty
: The y position of the next point.

Returns

Body body
: The new Body with an attached EdgeShape. The body's world position is computed based on the geometric center of the given vertices, and the created shape's local origin is at that center point.

EdgeShape shape
: The EdgeShape attached to the new Body.

See Also

- love.physics
- Body
- EdgeShape
- Shape
- love.physics.newBody
- love.physics.newEdgeShape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newEdgeShape

love.physics.newEdgeShape · revision 8761 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Creates a new EdgeShape.

Function

Synopsis

```
shape = love.physics.newEdgeShape( x1, y1, x2, y2 )
```

Arguments

number x1
: The x position of the first point.

number y1
: The y position of the first point.

number x2
: The x position of the second point.

number y2
: The y position of the second point.

Returns

EdgeShape shape
: The new shape.

See Also

- love.physics
- EdgeShape
- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newFixture

love.physics.newFixture · revision 27952 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Creates and attaches a Fixture to a body.

Note that the Shape object is copied rather than kept as a reference when the Fixture is created. To get the Shape object that the Fixture owns, use Fixture:getShape.

Function

Synopsis

```
fixture = love.physics.newFixture( body, shape, density )
```

Arguments

`Body body`
: The body which gets the fixture attached.

`Shape shape`
: The shape to be copied to the fixture.

`number density (1)`
: The density of the fixture.

Returns

`Fixture fixture`
: The new fixture.

Notes

Creating a Fixture may change the mass of the Body it's attached to.

See Also

- love.physics
- Fixture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newFrictionJoint

love.physics.newFrictionJoint · revision 10592 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Create a friction joint between two bodies. A FrictionJoint applies friction to a body.

Function

Synopsis

```
joint = love.physics.newFrictionJoint( body1, body2, x, y, collideConnected )
```

Arguments

`Body body1`
: The first body to attach to the joint.

`Body body2`
: The second body to attach to the joint.

`number x`
: The x position of the anchor point.

`number y`
: The y position of the anchor point.

`boolean collideConnected (false)`
: Specifies whether the two bodies should collide with each other.

Returns

`FrictionJoint joint`
: The new FrictionJoint.

Function

Synopsis

```
joint = love.physics.newFrictionJoint( body1, body2, x1, y1, x2, y2, collideConnected )
```

Arguments

`Body body1`
: The first body to attach to the joint.

`Body body2`
: The second body to attach to the joint.

number x1
: The x position of the first anchor point.

number y1
: The y position of the first anchor point.

number x2
: The x position of the second anchor point.

number y2
: The y position of the second anchor point.

boolean collideConnected (false)
: Specifies whether the two bodies should collide with each other.

Returns

FrictionJoint joint
: The new FrictionJoint.

See Also

- love.physics
- FrictionJoint
- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newGearJoint

love.physics.newGearJoint · revision 19233 · last edited 2025-11-30

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Create a GearJoint connecting two Joints.

The gear joint connects two joints that must be either prismatic or revolute joints. Using this joint requires that the joints it uses connect their respective bodies to the ground and have the ground as the first body. When destroying the bodies and joints you must make sure you destroy the gear joint before the other joints.

The gear joint has a ratio the determines how the angular or distance values of the connected joints relate to each other. The formula `coordinate1 + ratio * coordinate2` always has a constant value that is set when the gear joint is created.



Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
joint = love.physics.newGearJoint( joint1, joint2, ratio, collideConnected )
```

Arguments

`Joint joint1`
: The first joint to connect with a gear joint.

`Joint joint2`
: The second joint to connect with a gear joint.

`number ratio (1)`
: The gear ratio.

`boolean collideConnected (false)`
: Specifies whether the two bodies should collide with each other.

Returns

`GearJoint joint`
: The new gear joint.

Function

Removed in LÖVE 0.8.0

This variant is not supported in that and later versions.

Synopsis

```
joint = love.physics.newGearJoint( joint1, joint2, ratio )
```

Arguments

`Joint joint1`
: The first joint to connect with a gear joint.

`Joint joint2`
: The second joint to connect with a gear joint.

`number ratio (1)`
: The gear ratio.

Returns

`GearJoint joint`
: The new gear joint.

See Also

- love.physics
- GearJoint
- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newMotorJoint

love.physics.newMotorJoint · revision 16190 · last edited 2025-11-30

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Creates a joint between two bodies which controls the relative motion between them.

Position and rotation offsets can be specified once the MotorJoint has been created, as well as the maximum motor force and torque that will be be applied to reach the target offsets.

Function

Synopsis

```
joint = love.physics.newMotorJoint( body1, body2, correctionFactor )
```

Arguments

`Body body1`
: The first body to attach to the joint.

`Body body2`
: The second body to attach to the joint.

`number correctionFactor (0.3)`
: The joint's initial position correction factor, in the range of [0, 1].

Returns

`MotorJoint joint`
: The new MotorJoint.

Function

Available since LÖVE 0.10.1

This variant is not supported in earlier versions.

Synopsis

```
joint = love.physics.newMotorJoint( body1, body2, correctionFactor, collideConnected )
```

Arguments

`Body body1`
: The first body to attach to the joint.

`Body body2`

The second body to attach to the joint.

`number correctionFactor (0.3)`

The joint's initial position correction factor, in the range of [0, 1].

`boolean collideConnected (false)`

Specifies whether the two bodies should collide with each other.

Returns

`MotorJoint joint`

The new MotorJoint.

See Also

- love.physics
- MotorJoint
- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newMouseJoint

love.physics.newMouseJoint · revision 19234 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Create a joint between a body and the mouse.

This joint actually connects the body to a fixed point in the world. To make it follow the mouse, the fixed point must be updated every timestep (example below).

The advantage of using a MouseJoint instead of just changing a body position directly is that collisions and reactions to other joints are handled by the physics engine.

Function

Synopsis

```
joint = love.physics.newMouseJoint( body, x, y )
```

Arguments

Body body
: The body to attach to the mouse.

number x
: The x position of the connecting point.

number y
: The y position of the connecting point.

Returns

MouseJoint joint
: The new mouse joint.

Examples

```
function love.load()
    world = love.physics.newWorld(0, 0)
    body = love.physics.newBody(world, love.mouse.getX(), love.mouse.getY(), "dynamic")
    shape = love.physics.newCircleShape(20)
    fixture = love.physics.newFixture(body, shape)
    joint = love.physics.newMouseJoint(body, love.mouse.getPosition())
end

function love.draw()
    love.graphics.circle("fill", body:getX(), body:getY(), shape:getRadius())
end

function love.update(dt)
    joint:setTarget(love.mouse.getPosition())
```

```
    world:update(dt)
end
```

See Also

- love.physics
- MouseJoint
- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newPolygonBody

love.physics.newPolygonBody · revision 30714 · last edited 2026-04-04

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Creates a new Body with an attached PolygonShape.

This is a convenience constructor which effectively calls love.physics.newBody and love.physics.newPolygonShape.

The shape can have 8 vertices at most, and must be convex.

Function

Synopsis

```
body, shape = love.physics.newPolygonBody( world, type, x1, y1, x2, y2, x3, y3, ... )
```

Arguments

`World world`
: The world to create the body in.

`BodyType type`
: The type of the body.

`number x1`
: The x position of the first point, in world space.

`number y1`
: The y position of the first point, in world space.

`number x2`
: The x position of the second point, in world space.

`number y2`
: The y position of the second point, in world space.

`number x3`
: The x position of the third point, in world space.

`number y3`
: The y position of the third point, in world space.

`number ...`
: You can continue passing more point positions to create the object.

Returns

`Body body`
: The new Body with an attached PolygonShape. The body's world position is computed based on the geometric center of the given vertices, and the created shape's local origin is at that center point.

`PolygonShape shape`
: The PolygonShape attached to the new Body.

Function

Synopsis

```
body = love.physics.newPolygonBody( world, type, vertices )
```

Arguments

`World world`
: The world to create the body in.

`BodyType type`
: The type of the body.

`table vertices`
: A list of vertices to construct the polygon, in the form of `{x1, y1, x2, y2, x3, y3, ...}`.

Returns

`Body body`
: The new Body with an attached PolygonShape. The body's world position is computed based on the geometric center of the given vertices, and the created shape's local origin is at that center point.

See Also

- love.physics
- Body
- Shape
- PolygonShape
- love.physics.newBody
- love.physics.newPolygonShape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newPolygonShape

love.physics.newPolygonShape · revision 27921 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Creates a new PolygonShape.

This shape can have 8 vertices at most, and must form a convex shape.

Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
shape = love.physics.newPolygonShape( x1, y1, x2, y2, x3, y3, ... )
```

Arguments

`number x1`
: The x position of the first point, relative to the body it is attached to.

`number y1`
: The y position of the first point, relative to the body it is attached to.

`number x2`
: The x position of the second point, relative to the body it is attached to.

`number y2`
: The y position of the second point, relative to the body it is attached to.

`number x3`
: The x position of the third point, relative to the body it is attached to.

`number y3`
: The y position of the third point, relative to the body it is attached to.

`number ...`
: You can continue passing more point positions to create the PolygonShape.

Returns

`PolygonShape shape`
: A new PolygonShape.

Function

Available since LÖVE 0.9.2

This variant is not supported in earlier versions.

Synopsis

```
shape = love.physics.newPolygonShape( vertices )
```

Arguments

`table vertices`
A list of vertices to construct the polygon, in the form of `{x1, y1, x2, y2, x3, y3, ...}`.

Returns

`PolygonShape shape`
A new PolygonShape.

Function

Removed in LÖVE 0.8.0
This variant is not supported in that and later versions.

Synopsis

```
shape = love.physics.newPolygonShape( body, ... )
```

Arguments

`Body body`
The body to attach the shape to.

`numbers ...`
Vertices of the polygon.

Returns

`PolygonShape shape`
A new PolygonShape.

See Also

- love.physics
- PolygonShape
- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newPrismaticJoint

love.physics.newPrismaticJoint · revision 18115 · last edited 2025-11-30

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Creates a PrismaticJoint between two bodies.

A prismatic joint constrains two bodies to move relatively to each other on a specified axis. It does not allow for relative rotation. Its definition and operation are similar to a revolute joint, but with translation and force substituted for angle and torque.



Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
joint = love.physics.newPrismaticJoint( body1, body2, x, y, ax, ay, collideConnected )
```

Arguments

`Body body1`
: The first body to connect with a prismatic joint.

`Body body2`
: The second body to connect with a prismatic joint.

`number x`
: The x coordinate of the anchor point.

`number y`
: The y coordinate of the anchor point.

`number ax`
: The x coordinate of the axis vector.

`number ay`
: The y coordinate of the axis vector.

`boolean collideConnected (false)`
: Specifies whether the two bodies should collide with each other.

Returns

`PrismaticJoint joint`

The new prismatic joint.

Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
joint = love.physics.newPrismaticJoint( body1, body2, x1, y1, x2, y2, ax, ay, collideConnected )
```

Arguments

`Body body1`
The first body to connect with a prismatic joint.

`Body body2`
The second body to connect with a prismatic joint.

`number x1`
The x coordinate of the first anchor point.

`number y1`
The y coordinate of the first anchor point.

`number x2`
The x coordinate of the second anchor point.

`number y2`
The y coordinate of the second anchor point.

`number ax`
The x coordinate of the axis unit vector.

`number ay`
The y coordinate of the axis unit vector.

`boolean collideConnected (false)`
Specifies whether the two bodies should collide with each other.

Returns

`PrismaticJoint joint`
The new prismatic joint.

Function

Available since LÖVE 0.10.2

This variant is not supported in earlier versions.

Synopsis

```
joint = love.physics.newPrismaticJoint( body1, body2, x1, y1, x2, y2, ax, ay, collideConnected, referenceAngle )
```

Arguments

`Body body1`
The first body to connect with a prismatic joint.

`Body body2`

The second body to connect with a prismatic joint.

number x1
The x coordinate of the first anchor point.

number y1
The y coordinate of the first anchor point.

number x2
The x coordinate of the second anchor point.

number y2
The y coordinate of the second anchor point.

number ax
The x coordinate of the axis unit vector.

number ay
The y coordinate of the axis unit vector.

boolean collideConnected (false)
Specifies whether the two bodies should collide with each other.

number referenceAngle (0)
The reference angle between body1 and body2, in radians.

Returns

PrismaticJoint joint
The new prismatic joint.

Function

Removed in LÖVE 0.8.0
This variant is not supported in that and later versions.

Synopsis

```
joint = love.physics.newPrismaticJoint( body1, body2, x, y, ax, ay )
```

Arguments

Body body1
The first body to connect with a prismatic joint.

Body body2
The second body to connect with a prismatic joint.

number x
The x coordinate of the anchor point.

number y
The y coordinate of the anchor point.

number ax
The x coordinate of the axis unit vector.

number ay
The y coordinate of the axis unit vector.

Returns

PrismaticJoint joint
The new prismatic joint.

See Also

- love.physics
- PrismaticJoint
- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newPulleyJoint

love.physics.newPulleyJoint · revision 19232 · last edited 2025-12-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Creates a PulleyJoint to join two bodies to each other and the ground.

The pulley joint simulates a pulley with an optional block and tackle. If the ratio parameter has a value different from one, then the simulated rope extends faster on one side than the other. In a pulley joint the total length of the simulated rope is the constant `length1 + ratio * length2`, which is set when the pulley joint is created.

Pulley joints can behave unpredictably if one side is fully extended. It is recommended that the method setMaxLengths be used to constrain the maximum lengths each side can attain.



Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
joint = love.physics.newPulleyJoint( body1, body2, gx1, gy1, gx2, gy2, x1, y1, x2, y2, ratio, collideConnected )
```

Arguments

Body body1
: The first body to connect with a pulley joint.

Body body2
: The second body to connect with a pulley joint.

number gx1
: The x coordinate of the first body's ground anchor.

number gy1
: The y coordinate of the first body's ground anchor.

number gx2
: The x coordinate of the second body's ground anchor.

number gy2
: The y coordinate of the second body's ground anchor.

number x1
: The x coordinate of the pulley joint anchor in the first body.

number y1
: The y coordinate of the pulley joint anchor in the first body.

number x2
: The x coordinate of the pulley joint anchor in the second body.

number y2
: The y coordinate of the pulley joint anchor in the second body.

number ratio (1)

The joint ratio.

boolean collideConnected (true)

Specifies whether the two bodies should collide with each other.

Returns

PulleyJoint joint

The new pulley joint.

Function

Removed in LÖVE 0.8.0

This variant is not supported in that and later versions.

Synopsis

```
joint = love.physics.newPulleyJoint( body1, body2, gx1, gy1, gx2, gy2, x1, y1, x2, y2, ratio )
```

Arguments

Body body1

The first body to connect with a pulley joint.

Body body2

The second body to connect with a pulley joint.

number gx1

The x coordinate of the first body's ground anchor.

number gy1

The y coordinate of the first body's ground anchor.

number gx2

The x coordinate of the second body's ground anchor.

number gy2

The y coordinate of the second body's ground anchor.

number x1

The x coordinate of the pulley joint anchor in the first body.

number y1

The y coordinate of the pulley joint anchor in the first body.

number x2

The x coordinate of the pulley joint anchor in the second body.

number y2

The y coordinate of the pulley joint anchor in the second body.

number ratio (1)

The joint ratio.

Returns

PulleyJoint joint

The new pulley joint.

See Also

- love.physics

- PulleyJoint
- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newRectangleBody

love.physics.newRectangleBody · revision 30713 · last edited 2026-04-04

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Creates a new Body with an attached rectangle PolygonShape.

This is a convenience constructor which effectively calls love.physics.newBody and love.physics.newRectangleShape.

Function

Synopsis

```
body, shape = love.physics.newRectangleBody( world, type, x, y, width, height, angle )
```

Arguments

`World world`
: The world to create the body in.

`BodyType type`
: The type of the body.

`number x`
: The x position of the body in the world.

`number y`
: The y position of the body in the world.

`number width`
: The width of the rectangle.

`number height`
: The height of the rectangle.

`number angle (0)`
: The initial local angle of the rectangle relative to the body.

Returns

`Body body`
: The new Body.

`PolygonShape shape`
: The PolygonShape attached to the new Body. The shape's local origin is at the center of the Body.

See Also

- love.physics
- Body
- Shape
- PolygonShape

- love.physics.newBody
- love.physics.newRectangleShape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newRectangleShape

love.physics.newRectangleShape · revision 29290 · last edited 2024-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Shorthand for creating rectangular PolygonShapes.

By default, the local origin is located at the **center** of the rectangle as opposed to the top left for graphics.

Function

Available since LÖVE 0.8.0

These variants are not supported in earlier versions.

Synopsis

```
shape = love.physics.newRectangleShape( width, height )
```

Arguments

number width
: The width of the rectangle.

number height
: The height of the rectangle.

Returns

PolygonShape shape
: A new PolygonShape.

Function

Synopsis

```
shape = love.physics.newRectangleShape( x, y, width, height, angle )
```

Arguments

number x
: The offset along the x-axis.

number y
: The offset along the y-axis.

number width
: The width of the rectangle.

number height

The height of the rectangle.

number angle (0)

The initial angle of the rectangle.

Returns

PolygonShape shape

A new PolygonShape.

Function

Removed in LÖVE 0.8.0

This variant is not supported in that and later versions.

Synopsis

```
s = love.physics.newRectangleShape( body, x, y, width, height, angle )
```

Arguments

Body body

The Body to attach the Shape to.

number x

The offset along the x-axis.

number y

The offset along the y-axis.

number width

The width of the rectangle.

number height

The height of the rectangle.

number angle (0)

The initial angle of the rectangle.

Returns

PolygonShape s

A new PolygonShape.

See Also

- love.physics
- PolygonShape
- Shape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newRevoluteJoint

love.physics.newRevoluteJoint · revision 19235 · last edited 2025-12-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Creates a pivot joint between two bodies.

This joint connects two bodies to a point around which they can pivot.



Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
joint = love.physics.newRevoluteJoint( body1, body2, x, y, collideConnected )
```

Arguments

`Body body1`
: The first body.

`Body body2`
: The second body.

`number x`
: The x position of the connecting point.

`number y`
: The y position of the connecting point.

`boolean collideConnected (false)`
: Specifies whether the two bodies should collide with each other.

Returns

`RevoluteJoint joint`
: The new revolute joint.

Function

Available since LÖVE 0.10.2

This variant is not supported in earlier versions.

Synopsis

```
joint = love.physics.newRevoluteJoint( body1, body2, x1, y1, x2, y2, collideConnected, referenceAngle )
```

Arguments

Body body1
: The first body.

Body body2
: The second body.

number x1
: The x position of the first connecting point.

number y1
: The y position of the first connecting point.

number x2
: The x position of the second connecting point.

number y2
: The y position of the second connecting point.

boolean collideConnected (false)
: Specifies whether the two bodies should collide with each other.

number referenceAngle (0)
: The reference angle between body1 and body2, in radians.

Returns

RevoluteJoint joint
: The new revolute joint.

Function

Removed in LÖVE 0.8.0
This variant is not supported in that and later versions.

Synopsis

```
joint = love.physics.newRevoluteJoint( body1, body2, x, y )
```

Arguments

Body body1
: The first body to connect with a Revolute Joint.

Body body2
: The second body to connect with a Revolute Joint.

number x
: The x position of the connecting point.

number y

The y position of the connecting point.

Returns

`RevoluteJoint joint`
The new revolute joint.

See Also

- love.physics
- RevoluteJoint
- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newRopeJoint

love.physics.newRopeJoint · revision 21010 · last edited 2025-12-12

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Creates a joint between two bodies. Its only function is enforcing a max distance between these bodies.



There is a bug in version 0.8.0 where the coordinates of the anchors are divided by the number from love.physics.getMeter. As a workaround, multiply your anchor coordinates with that number and it should work like expected.

Function

Synopsis

```
joint = love.physics.newRopeJoint( body1, body2, x1, y1, x2, y2, maxLength, collideConnected )
```

Arguments

`Body body1`
: The first body to attach to the joint.

`Body body2`
: The second body to attach to the joint.

`number x1`
: The x position of the first anchor point.

`number y1`
: The y position of the first anchor point.

`number x2`
: The x position of the second anchor point.

`number y2`
: The y position of the second anchor point.

`number maxLength`
: The maximum distance for the bodies.

`boolean collideConnected (false)`
: Specifies whether the two bodies should collide with each other.

Returns

`RopeJoint joint`
: The new RopeJoint.

See Also

- love.physics

- RopeJoint
- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newWeldJoint

love.physics.newWeldJoint · revision 21852 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Creates a constraint joint between two bodies. A WeldJoint essentially glues two bodies together. The constraint is a bit soft, however, due to Box2D's iterative solver.



Making changes to a World is not allowed inside of the beginContact, endContact, preSolve, and postSolve callback functions, as BOX2D locks the world during these callbacks.

Function

Synopsis

```
joint = love.physics.newWeldJoint( body1, body2, x, y, collideConnected )
```

Arguments

`Body body1`
: The first body to attach to the joint.

`Body body2`
: The second body to attach to the joint.

`number x`
: The x position of the anchor point (world space).

`number y`
: The y position of the anchor point (world space).

`boolean collideConnected (false)`
: Specifies whether the two bodies should collide with each other.

Returns

`WeldJoint joint`

The new WeldJoint.

Function

Synopsis

```
joint = love.physics.newWeldJoint( body1, body2, x1, y1, x2, y2, collideConnected )
```

Arguments

Body body1
The first body to attach to the joint.

Body body2
The second body to attach to the joint.

number x1
The x position of the first anchor point (world space).

number y1
The y position of the first anchor point (world space).

number x2
The x position of the second anchor point (world space).

number y2
The y position of the second anchor point (world space).

boolean collideConnected (false)
Specifies whether the two bodies should collide with each other.

Returns

WeldJoint joint
The new WeldJoint.

Function

Available since LÖVE 0.10.2
This variant is not supported in earlier versions.

Synopsis

```
joint = love.physics.newWeldJoint( body1, body2, x1, y1, x2, y2, collideConnected, referenceAngle )
```

Arguments

Body body1
The first body to attach to the joint.

Body body2
The second body to attach to the joint.

number x1
The x position of the first anchor point (world space).

number y1
The y position of the first anchor point (world space).

number x2

The x position of the second anchor point (world space).

number y2

The y position of the second anchor point (world space).

boolean collideConnected (false)

Specifies whether the two bodies should collide with each other.

number referenceAngle (0)

The reference angle between body1 and body2, in radians.

Returns

WeldJoint joint

The new WeldJoint.

See Also

- love.physics
- WeldJoint
- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newWheelJoint

love.physics.newWheelJoint · revision 16464 · last edited 2025-11-23

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Creates a wheel joint.

Function

Synopsis

```
joint = love.physics.newWheelJoint( body1, body2, x, y, ax, ay, collideConnected )
```

Arguments

`Body body1`
: The first body.

`Body body2`
: The second body.

`number x`
: The x position of the anchor point.

`number y`
: The y position of the anchor point.

`number ax`
: The x position of the axis unit vector.

`number ay`
: The y position of the axis unit vector.

`boolean collideConnected (false)`
: Specifies whether the two bodies should collide with each other.

Returns

`WheelJoint joint`

The new WheelJoint.

Function

Synopsis

```
joint = love.physics.newWheelJoint( body1, body2, x1, y1, x2, y2, ax, ay, collideConnected )
```

Arguments

Body body1
The first body.

Body body2
The second body.

number x1
The x position of the first anchor point.

number y1
The y position of the first anchor point.

number x2
The x position of the second anchor point.

number y2
The y position of the second anchor point.

number ax
The x position of the axis unit vector.

number ay
The y position of the axis unit vector.

boolean collideConnected (false)
Specifies whether the two bodies should collide with each other.

Returns

WheelJoint joint
The new WheelJoint.

See Also

- love.physics
- WheelJoint
- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.newWorld

love.physics.newWorld · revision 16135 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Creates a new World.

Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
world = love.physics.newWorld( xg, yg, sleep )
```

Arguments

`number xg (0)`
: The x component of gravity.

`number yg (0)`
: The y component of gravity.

`boolean sleep (true)`
: Whether the bodies in this world are allowed to sleep.

Returns

`World world`
: A brave new World.

Function

Removed in LÖVE 0.8.0

This variant is not supported in that and later versions.

This function creates a new World with the given size, no gravity and sleeping turned on.

Synopsis

```
world = love.physics.newWorld( x1, y1, x2, y2 )
```

Arguments

`number x1`
: The smallest x position in the world.

number y1
: The smallest y position in the world.

number x2
: The largest x position in the world.

number y2
: The largest y position in the world.

Returns

World world
: A World object.

Function

Removed in LÖVE 0.8.0
This variant is not supported in that and later versions.

Synopsis

```
world = love.physics.newWorld( x1, y1, x2, y2, xg, yg, sleep )
```

Arguments

number x1
: The smallest x position in the world.

number y1
: The smallest y position in the world.

number x2
: The largest x position in the world.

number y2
: The largest y position in the world.

number xg
: The x component of gravity.

number yg
: The y component of gravity.

boolean sleep (true)
: Whether sleep is possible in the world.

Returns

World world
: A brave new World.

See Also

- love.physics
- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.physics.setMeter

love.physics.setMeter · revision 22589 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Note that the body's coordinates in meters remain unchanged
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Sets the pixels to meter scale factor.

All coordinates in the physics module are divided by this number and converted to meters, and it creates a convenient way to draw the objects directly to the screen without the need for graphics transformations.

It is recommended to create shapes no larger than 10 times the scale. This is important because Box2D is tuned to work well with shape sizes from 0.1 to 10 meters. The default meter scale is 30.



love.physics.setMeter does not apply retroactively to created objects. Created objects retain their meter coordinates but the scale factor will affect their pixel coordinates. It is recommended to call this function before any further use of the love.physics module

Function

Synopsis

```
love.physics.setMeter( scale )
```

Arguments

number scale

The scale factor as an integer.

Returns

Nothing.

Examples

Note that the body's coordinates in meters remain unchanged

```
love.physics.setMeter(30) -- set 30 pixels/meter
body = love.physics.newBody(world, 300, 300, "dynamic") -- place the body at pixel coordinates (300,300) or in meter coordinates (10,10)
```

```
love.physics.setMeter(10) -- set 10 pixels/meter
body:getPosition() -- returns pixel coordinates (100,100)
```

See Also

- love.physics
- love.physics.getMeter

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.quit

love.quit · revision 29421 · last edited 2026-03-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
- See Also
- Other Languages

Available since LÖVE 0.7.0

This callback is not supported in earlier versions.

Callback function triggered by the default love.run when the game is closed.

Function

Synopsis

```
abort = love.quit( )
```

Arguments

None.

Returns

boolean abort

Abort quitting. If true, do not close the game.

Example

This example shows how the return value of love.quit can be used to stop the closing of the game.

```
local readyToQuit = false

function love.quit()
    if not readyToQuit then
        print("We are not ready to quit yet!")
        readyToQuit = true
        return true
    else
        print("Thanks for playing. Please play again soon!")
        return false
    end
end
```

See Also

- love
- love.event.quit

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.releaseerrhand

love.releaseerrhand · revision 11660 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - The default function used if you don't supply your own.
- See Also
- Other Languages

Removed in LÖVE 0.9.0

This callback is not supported in that and later versions.

Function

Synopsis

```
love.releaseerrhand( msg )
```

Arguments

string msg

The error message.

Returns

Nothing.

Examples

The default function used if you don't supply your own.

```
function love.releaseerrhand(msg)
    print("An error has occured, the game has been stopped.")

    if not love.graphics or not love.event or not love.graphics.isCreated() then
        return
    end

    love.graphics.setCanvas()
    love.graphics.setPixelEffect()

    -- Load.
    if love.audio then love.audio.stop() end
    love.graphics.reset()
    love.graphics.setBackgroundColor(89, 157, 220)
    local font = love.graphics.newFont(14)
    love.graphics.setFont(font)

    love.graphics.setColor(255, 255, 255, 255)

    love.graphics.clear()

    local err = {}

    p = string.format("An error has occured that caused %s to stop.\nYou can notify %s about this%s.",
```

```
love._release.title or "this game", love._release.author or "the author", love._release.url and " at " ..
love._release.url or "")

    local function draw()
        love.graphics.clear()
        love.graphics.printf(p, 70, 70, love.graphics.getWidth() - 70)
        love.graphics.present()
    end

    draw()

    local e, a, b, c
    while true do
        e, a, b, c = love.event.wait()

        if e == "quit" then
            return
        end
        if e == "keypressed" and a == "escape" then
            return
        end

        draw()

    end
end
```

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.resize

love.resize · revision 22506 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Example
- See Also
- Other Languages

Available since LÖVE 0.9.0

This callback is not supported in earlier versions.

Called when the window is resized, for example if the user resizes the window, or if `love.window.setMode` is called with an unsupported width or height in fullscreen and the window chooses the closest appropriate size.

Function

Synopsis

```
love.resize( w, h )
```

Arguments

number w
: The new width.

number h
: The new height.

Returns

Nothing.

Notes

Calls to `love.window.setMode` will **only** trigger this event if the width or height of the window after the call doesn't match the requested width and height. This can happen if a fullscreen mode is requested which doesn't match any supported mode, or if the fullscreen type is 'desktop' and the requested width or height don't match the desktop resolution.

Since 11.0, this function returns width and height in DPI-scaled units rather than pixels.

Example

```
function love.resize(w, h)
  print(("Window resized to width: %d and height: %d."):format(w, h))
end
```

See Also

- love
- love.window.setMode
- love.window.setFullscreen

- love.conf

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.run

love.run · revision 29427 · last edited 2025-09-30

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - The default function for 11.0, used if you don't supply your own.
  - The default function for 0.10.0, 0.10.1, and 0.10.2, used if you don't supply your own.
  - The default function for 0.9.0, 0.9.1, and 0.9.2, used if you don't supply your own.
  - The default function for 0.8.0, used if you don't supply your own.
  - The default function for 0.7.0, 0.7.1 and 0.7.2, used if you don't supply your own.
- Notes
  - Why is there a delay?
- See Also
- Other Languages

The main callback function, containing the main loop. A sensible default is used when left out.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
mainLoop = love.run ( )
```

Arguments

None.

Returns

`function mainLoop`

Function which handles one frame, including events and rendering, when called.

Function

Removed in LÖVE 11.0

This variant is not supported in that and later versions.

Synopsis

```
love.run( )
```

Arguments

None.

Returns

Nothing.

Examples

The default function for 11.0, used if you don't supply your own.

```
function love.run()
    if love.load then love.load(love.arg.parseGameArguments(arg), arg) end

    -- We don't want the first frame's dt to include time taken by love.load.
    if love.timer then love.timer.step() end

    local dt = 0

    -- Main loop time.
    return function()
        -- Process events.
        if love.event then
            love.event.pump()
            for name, a,b,c,d,e,f in love.event.poll() do
                if name == "quit" then
                    if not love.quit or not love.quit() then
                        return a or 0
                    end
                end
                love.handlers[name](a,b,c,d,e,f)
            end
        end

        -- Update dt, as we'll be passing it to update
        if love.timer then dt = love.timer.step() end

        -- Call update and draw
        if love.update then love.update(dt) end -- will pass 0 if love.timer is disabled

        if love.graphics and love.graphics.isActive() then
            love.graphics.origin()
            love.graphics.clear(love.graphics.getBackgroundColor())

            if love.draw then love.draw() end

            love.graphics.present()
        end

        if love.timer then love.timer.sleep(0.001) end
```

```
    end
end
```

The default function for 0.10.0, 0.10.1, and 0.10.2, used if you don't supply your own.

```
function love.run()

    if love.math then
        love.math.setRandomSeed(os.time())
    end

    if love.load then love.load(arg) end

    -- We don't want the first frame's dt to include time taken by love.load.
    if love.timer then love.timer.step() end

    local dt = 0

    -- Main loop time.
    while true do
        -- Process events.
        if love.event then
            love.event.pump()
            for name, a,b,c,d,e,f in love.event.poll() do
                if name == "quit" then
                    if not love.quit or not love.quit() then
                        return a
                    end
                end
                love.handlers[name](a,b,c,d,e,f)
            end
        end

        -- Update dt, as we'll be passing it to update
        if love.timer then
            love.timer.step()
            dt = love.timer.getDelta()
        end

        -- Call update and draw
        if love.update then love.update(dt) end -- will pass 0 if love.timer is disabled

        if love.graphics and love.graphics.isActive() then
            love.graphics.clear(love.graphics.getBackgroundColor())
            love.graphics.origin()
            if love.draw then love.draw() end
            love.graphics.present()
        end

        if love.timer then love.timer.sleep(0.001) end
    end

end
```

The default function for 0.9.0, 0.9.1, and 0.9.2, used if you don't supply your own.

```
function love.run()

    if love.math then
        love.math.setRandomSeed(os.time())
```

```
        for i=1,3 do love.math.random() end
    end

    if love.event then
        love.event.pump()
    end

    if love.load then love.load(arg) end

    -- We don't want the first frame's dt to include time taken by love.load.
    if love.timer then love.timer.step() end

    local dt = 0

    -- Main loop time.
    while true do
        -- Process events.
        if love.event then
            love.event.pump()
            for e,a,b,c,d in love.event.poll() do
                if e == "quit" then
                    if not love.quit or not love.quit() then
                        if love.audio then
                            love.audio.stop()
                        end
                        return
                    end
                end
                love.handlers[e](a,b,c,d)
            end
        end

        -- Update dt, as we'll be passing it to update
        if love.timer then
            love.timer.step()
            dt = love.timer.getDelta()
        end

        -- Call update and draw
        if love.update then love.update(dt) end -- will pass 0 if love.timer is disabled

        if love.window and love.graphics and love.window.isCreated() then
            love.graphics.clear()
            love.graphics.origin()
            if love.draw then love.draw() end
            love.graphics.present()
        end

        if love.timer then love.timer.sleep(0.001) end
    end

end
```

The default function for 0.8.0, used if you don't supply your own.

```
function love.run()

    math.randomseed(os.time())
    math.random() math.random()

    if love.load then love.load(arg) end
```

```
    local dt = 0

    -- Main loop time.
    while true do
        -- Process events.
        if love.event then
            love.event.pump()
            for e,a,b,c,d in love.event.poll() do
                if e == "quit" then
                    if not love.quit or not love.quit() then
                        if love.audio then
                            love.audio.stop()
                        end
                        return
                    end
                end
                love.handlers[e](a,b,c,d)
            end
        end

        -- Update dt, as we'll be passing it to update
        if love.timer then
            love.timer.step()
            dt = love.timer.getDelta()
        end

        -- Call update and draw
        if love.update then love.update(dt) end -- will pass 0 if love.timer is disabled
        if love.graphics then
            love.graphics.clear()
            if love.draw then love.draw() end
        end

        if love.timer then love.timer.sleep(0.001) end
        if love.graphics then love.graphics.present() end
    end

end
```

The default function for 0.7.0, 0.7.1 and 0.7.2, used if you don't supply your own.

```
function love.run()

    if love.load then love.load(arg) end

    local dt = 0

    -- Main loop time.
    while true do
        if love.timer then
            love.timer.step()
            dt = love.timer.getDelta()
        end
        if love.update then love.update(dt) end -- will pass 0 if love.timer is disabled
        if love.graphics then
            love.graphics.clear()
            if love.draw then love.draw() end
        end

        -- Process events.
```

```
        if love.event then
            for e,a,b,c in love.event.poll() do
                if e == "q" then
                    if not love.quit or not love.quit() then
                        if love.audio then
                            love.audio.stop()
                        end
                        return
                    end
                end
                love.handlers[e](a,b,c)
            end
        end

        if love.timer then love.timer.sleep(1) end
        if love.graphics then love.graphics.present() end
    end

end
```

Notes

Why is there a delay?

```
if love.timer then love.timer.sleep(0.001) end
```

It does a few useful things:

- Limits FPS to 1000 if vsync isn't enabled.
- Massively reduces CPU usage in many situations (especially with vsync disabled.)
- Gives control back to the OS for a bit.

For more information see https://love2d.org/forums/viewtopic.php?f=4&t=76998.

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.scene

love.scene · revision 30508 · last edited 2025-12-22

love.scene

love.screen

love.screen · revision 6374 · last edited 2023-07-05

A small lib to manage the löve screen easily.

Made for löve v0.7.1 but may run with older one like 0.5.x, 0.6.x 0.7.x (not checked).

You can find it here (http://w w w .w orldmaster.fr/files/love2d-release/love.screen-20110425.zip)

How to use:

- See samples on this post (http://love2d.org/forums/viewtopic.php?f=4&t=2888#p30128)

See also

- Original Thread (http://love2d.org/forums/viewtopic.php?f=4&t=2888#p30128)

love.sensor

love.sensor · revision 28284 · last edited 2024-04-13

Available since LÖVE 12.0

This module is not supported in earlier versions.

Provides functionality to access device sensors such as an accelerometer or gyroscope.

Functions

love.sensor.hasSensor	Check in-device sensor availability.	Added since 12.0

Enums

SensorType	Kinds of sensors.	Added since 12.0

See Also

- love
- love.sensorupdated
- Joystick:hasSensor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.sensor.hasSensor

love.sensor.hasSensor · revision 28973 · last edited 2023-10-01

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Check if the specified sensor exists in the device.

Function

Synopsis

```
available = love.sensor.hasSensor( sensorType )
```

Arguments

`SensorType sensorType`

Type of sensor to check.

Returns

`boolean available`

Sensor availability status.

See Also

- love.sensor
- SensorType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.sensorupdated

love.sensorupdated · revision 28143 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This callback is not supported in earlier versions.

Called when the in-device sensor is updated with new values.

Only sensors enabled with love.sensor.setEnabled will trigger this event.

Function

Synopsis

```
love.sensorupdated( sensorType, x, y, z )
```

Arguments

`SensorType sensorType`
: The type of sensor.

`number x`
: The new sensor 1st value.

`number y`
: The new sensor 2nd value.

`number z`
: The new sensor 3rd value.

Returns

Nothing

See Also

- love
- love.sensor.getData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.setDeprecationOutput

love.setDeprecationOutput · revision 22097 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Sets whether LÖVE displays warnings when using deprecated functionality. It is disabled by default in fused mode, and enabled by default otherwise.

When deprecation output is enabled, the first use of a formally deprecated LÖVE API will show a message at the bottom of the screen for a short time, and print the message to the console.

Function

Synopsis

```
love.setDeprecationOutput( enable )
```

Arguments

`boolean enable`

Whether to enable or disable deprecation output.

Returns

Nothing.

See Also

- love
- love.hasDeprecationOutput

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.sound

love.sound · revision 23037 · last edited 2023-07-05

This module is responsible for decoding sound files. It can't play the sounds, see love.audio for that.

Types

Decoder	An object which can gradually decode a sound file.
SoundData	Contains raw audio samples.

Functions

love.sound.newDecoder	Attempts to find a decoder for the encoded sound data in the specified file.
love.sound.newSoundData	Creates a new SoundData.

See Also

- love
- Audio Formats

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.sound.newDecoder

love.sound.newDecoder · revision 29498 · last edited 2025-03-23

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Attempts to find a decoder for the encoded sound data in the specified file. Throws an error if no suitable audio decoder is found.

Function

Synopsis

```
decoder = love.sound.newDecoder( file, buffer )
```

Arguments

`File file`
: The file with encoded sound data.

`number buffer (2048)`
: The size of each decoded chunk, in bytes.

Returns

`Decoder decoder`
: A new Decoder object.

Function

Synopsis

```
decoder = love.sound.newDecoder( filename, buffer )
```

Arguments

`string filename`
: The filename of the file with encoded sound data.

`number buffer (2048)`

The size of each decoded chunk, in bytes.

Returns

`Decoder decoder`
A new Decoder object.

Function

Synopsis

```
decoder = love.sound.newDecoder( filedata, buffer )
```

Arguments

`FileData filedata`
The encoded file data to decode into sound data.

`number buffer (2048)`
The size of each decoded chunk, in bytes.

Returns

`Decoder decoder`
A new Decoder object.

Function

Removed in LÖVE 0.7.1
This variant is not supported in that and later versions.

Synopsis

```
decoder = love.sound.newDecoder( file, buffer, rate )
```

Arguments

`File file`
The file with encoded sound data.

`number buffer (2048)`
The size of each decoded chunk, in bytes.

`number rate (44100)`
Samples per second, or quality of the audio. Use either 44100 (good), 22050 (medium) or 11025 (poor).

Returns

`Decoder decoder`
A new Decoder object.

Function

Removed in LÖVE 0.7.1

This variant is not supported in that and later versions.

Synopsis

```
decoder = love.sound.newDecoder( filename, buffer, rate )
```

Arguments

`string filename`
: The filename of the file with encoded sound data.

`number buffer (2048)`
: The size of each decoded chunk, in bytes.

`number rate (44100)`
: Samples per second, or quality of the audio. Use either 44100 (good), 22050 (medium) or 11025 (poor).

Returns

`Decoder decoder`
: A new Decoder object.

See Also

- love.sound
- Decoder

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.sound.newSoundData

love.sound.newSoundData · revision 27457 · last edited 2024-07-03

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Loading SoundData from files
  - Beep function
- See Also
- Other Languages

Creates new SoundData from a filepath, File, or Decoder. It's also possible to create SoundData with a custom sample rate, channel and bit depth.

The sound data will be decoded to the memory in a raw format. It is recommended to create only short sounds like effects, as a 3 minute song uses 30 MB of memory this way.

Function

Synopsis

```
soundData = love.sound.newSoundData( filename )
```

Arguments

`string filename`
: The file name of the file to load.

Returns

`SoundData soundData`
: A new SoundData object.

Function

Synopsis

```
soundData = love.sound.newSoundData( file )
```

Arguments

`File file`
: A File pointing to an audio file.

Returns

`SoundData soundData`

A new SoundData object.

Function

Synopsis

```
soundData = love.sound.newSoundData( decoder )
```

Arguments

Decoder decoder
Decode data from this Decoder until EOF.

Returns

SoundData soundData
A new SoundData object.

Function

Synopsis

```
soundData = love.sound.newSoundData( samples, rate, bits, channels )
```

Arguments

number samples
Total number of samples.

number rate (44100)
Number of samples per second

number bits (16)
Bits per sample (8 or 16).

number channels (2)
Either 1 for mono or 2 for stereo.

Returns

SoundData soundData
A new SoundData object.

Examples

Loading SoundData from files

```
wav = love.sound.newSoundData("doom.wav")
-- Beware: if doom.mp3 is a huge file, it will take
```

```
-- ages to decode.
mp3 = love.sound.newSoundData("doom.mp3")
```

Beep function

```
local rate      = 44100 -- samples per second
local length    = 1/32  -- 0.03125 seconds
local tone      = 440.0 -- Hz
local p         = math.floor(rate/tone) -- 100 (wave length in samples)
local soundData = love.sound.newSoundData(math.floor(length*rate), rate, 16, 1)
for i=0, soundData:getSampleCount() - 1 do
--  soundData:setSample(i, math.sin(2*math.pi*i/p)) -- sine wave.
    soundData:setSample(i, i%p<p/2 and 1 or -1)     -- square wave; the first half of the wave is 1, the second half is
-1.
end
local source = love.audio.newSource(soundData)
local function beep() source:play() end
```

Call it as:

```
beep()
```

See Also

- love.sound
- SoundData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.system

love.system · revision 31498 · last edited 2026-04-20

Available since LÖVE 0.9.0

This module is not supported in earlier versions.

Provides access to information about the user's system.

Functions

love.system.getClipboardText	Gets text from the clipboard.	Added since 0.9.0
love.system.getOS	Gets the current operating system.	Added since 0.9.0
love.system.getPowerInfo	Gets information about the system's power supply.	Added since 0.9.0
love.system.getProcessorCount	Gets the amount of logical processors in the system.	Added since 0.9.0
love.system.hasBackgroundMusic	Gets whether another application on the system is playing music in the background.	Added since 11.0
love.system.openURL	Opens a URL with the user's web or file browser.	Added since 0.9.1
love.system.setClipboardText	Puts text in the clipboard.	Added since 0.9.0
love.system.vibrate	Causes the device to vibrate, if possible.	Added since 0.10.0

Enums

PowerState	The basic state of the system's power supply.	Added since 0.9.0

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.system.getClipboardText

love.system.getClipboardText · revision 11223 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets text from the clipboard.

Function

Synopsis

```
text = love.system.getClipboardText( )
```

Arguments

None.

Returns

`string text`

The text currently held in the system's clipboard.

See Also

- love.system
- love.system.setClipboardText

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.system.getMemorySize

love.system.getMemorySize · revision 30590 · last edited 2026-01-29

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets the amount of RAM configured in the system in MiB.

Function

Synopsis

```
memorySize = love.system.getMemorySize( )
```

Arguments

None.

Returns

`number memorySize`

Amount of RAM configured in the system.

Notes

Returns total memory rather than currently available (not-in-use) memory

See Also

- love.system

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.system.getOS

love.system.getOS · revision 16125 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the current operating system. In general, LÖVE abstracts away the need to know the current operating system, but there are a few cases where it can be useful (especially in combination with os.execute (http://www.lua.org/manual/5.1/manual.html#pdf-os.execute).)

Function

Synopsis

```
osString = love.system.getOS( )
```

Arguments

None.

Returns

`string osString`

The current operating system. `"OS X"`, `"Windows"`, `"Linux"`, `"Android"` or `"iOS"`.

Notes

In LÖVE version 0.8.0, the **`love._os`** string contains the current operating system.

See Also

- love.system

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.system.getPowerInfo

love.system.getPowerInfo · revision 12167 · last edited 2026-04-24

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets information about the system's power supply.

Function

Synopsis

```
state, percent, seconds = love.system.getPowerInfo( )
```

Arguments

None.

Returns

`PowerState state`
: The basic state of the power supply.

`number percent (nil)`
: Percentage of battery life left, between 0 and 100. nil if the value can't be determined or there's no battery.

`number seconds (nil)`
: Seconds of battery life left. nil if the value can't be determined or there's no battery.

Examples

Print to console battery state, percent battery remaining, and seconds of battery life remaining. Note that percent and seconds remaining will be nil if no battery is present.

```
function love.load()
    print(love.system.getPowerInfo())
end
```

See Also

- love.system

- PowerState

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.system.getPreferredLocales

love.system.getPreferredLocales · revision 28365 · last edited 2025-12-01

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets preferred locales in order of user's preference.

Function

Synopsis

```
locales = love.system.getPreferredLocales( )
```

Arguments

None.

Returns

`table locales`

A sequence of strings in order of user's preference. Locales are in form xx_YY (or just xx when country is not available) where xx is an ISO-639 (https://en.w ikipedia.org/w iki/List_of_ISO_639-1_codes) language specifier and YY (if available) is an ISO-3166 (https://en.w ikipedia.org/w iki/List_of_ISO_3166_country_codes) country code

Notes

It is recommended to load preferred locales only once and if you want to listen to any changes (in case user changes system settings), use love.localechanged.

See Also

- love.system
- love.localechanged

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.system.getProcessorCount

love.system.getProcessorCount · revision 27292 · last edited 2026-04-24

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the amount of logical processors in the system.

Function

Synopsis

```
processorCount = love.system.getProcessorCount( )
```

Arguments

None.

Returns

number processorCount

Amount of logical processors.

Notes

The number includes the threads reported if technologies such as Intel's Hyper-threading (http://en.w ikipedia.org/w iki/Hyper-t hreading) are enabled. For example, on a 4-core CPU with Hyper-threading, this function will return 8.

See Also

- love.system
- love.thread.newThread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.system.hasBackgroundMusic

love.system.hasBackgroundMusic · revision 21299 · last edited 2026-04-24

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets whether another application on the system is playing music in the background.

Currently this is implemented on iOS and Android, and will always return false on other operating systems. The `t.audio.mixwithsystem` flag in love.conf can be used to configure whether background audio / music from other apps should play while LÖVE is open.

Function

Synopsis

```
backgroundmusic = love.system.hasBackgroundMusic( )
```

Arguments

None.

Returns

`boolean backgroundmusic`

True if the user is playing music in the background via another app, false otherwise.

See Also

- love.system

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.system.openURL

love.system.openURL · revision 23548 · last edited 2026-04-24

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
  - Open love2d.org when the game is loaded.
  - Open the game's save directory when "s" is pressed.
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Opens a URL with the user's web or file browser.

Function

Synopsis

```
success = love.system.openURL( url )
```

Arguments

`string url`

The URL to open. Must be formatted as a proper URL.

Returns

`boolean success`

Whether the URL was opened successfully.

Notes

Passing `file://` scheme in Android 7.0 (Nougat) and later always result in failure (https://developer.android.com/about/versions/nougat/android-7.0-changes.html#sharing-files). Prior to 11.2, this will crash LÖVE instead of returning false.

Examples

Open love2d.org when the game is loaded.

```
function love.load()
    love.system.openURL("http://love2d.org/")
end
```

Open the game's save directory when "s" is pressed.

```
function love.load()
    -- Make sure the save directory exists by writing an empty file.
    love.filesystem.write("test.txt", "")
end

function love.keypressed(key)
```

```
    if key == "s" then
        -- To open a file or folder, "file://" must be prepended to the path.
        love.system.openURL("file://"..love.filesystem.getSaveDirectory())
    end
end
```

See Also

- love.system

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.system.setClipboardText

love.system.setClipboardText · revision 22538 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Puts text in the clipboard.

Function

Synopsis

```
love.system.setClipboardText( text )
```

Arguments

string text

The new text to hold in the system's clipboard.

Returns

Nothing.

Examples

Set up OS agnostic keybindings for a copy paste buffer.

```
local buffer

function love.draw()

  love.graphics.print(
    "OS: "..love.system.getOS().."\n"..
    "Local buffer: "..tostring(buffer).."\n"..
    "System buffer: "..tostring(love.system.getClipboardText()))

end

function love.keypressed(key)

  local osString = love.system.getOS()

  local control

  if osString == "OS X" then
    control = love.keyboard.isDown("lgui","rgui")
  elseif osString == "Windows" or osString == "Linux" then
    control = love.keyboard.isDown("lctrl","rctrl")
  end
```

```
  if control then
    if key == "c" then
      if buffer then love.system.setClipboardText(buffer) end
    end
    if key == "v" then
      buffer = love.system.getClipboardText()
    end
  end

end
```

See Also

- love.system
- love.system.getClipboardText

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.system.vibrate

love.system.vibrate · revision 25888 · last edited 2026-04-24

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Causes the device to vibrate, if possible. Currently this will only work on Android and iOS devices that have a built-in vibration motor.

Function

Synopsis

```
love.system.vibrate( seconds )
```

Arguments

`number seconds (0.5)`

The duration to vibrate for. If called on an iOS device, it will always vibrate for 0.5 seconds due to limitations in the iOS system APIs.

Returns

Nothing.

Example

```
function love.touchpressed( id, x, y, dx, dy, pressure )
    love.system.vibrate(0.1)
end
```

See Also

- love.system

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.textedited

love.textedited · revision 31650 · last edited 2026-06-20

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Called when the candidate text for an IME (Input Method Editor) has changed.

The candidate text is not the final text that the user will eventually choose. Use love.textinput for that.

The IME candidate selection happens on OS level. During selection, certain keys like escape, arrow keys, return and backspace are consumed by IME interaction. Meaning love.keypressed will NOT fire while IME selection is active. In love.textedited check for text parameter to be empty string or for start parameter to be 0 to know that IME selelection was closed by user.

Example: user composes word in IME, then hits 'return' to select candidate. Now love.textedited fires with text parameter being empty string and start parameter being 0, after that love.textinput fires with the text parameter being the selected candidate. If the user now hits 'return' again love.keypressed will fire with key parameter being 'return', love.textedited will stay silent.

Function

Synopsis

```
love.textedited( text, start, length )
```

Arguments

`string text`
: The UTF-8 encoded unicode candidate text.

`number start`
: The start cursor of the selected candidate text.

`number length`
: The length of the selected candidate text. May be 0.

Returns

Nothing.

See Also

- love
- love.textinput
- love.keyboard.setTextInput
- love.keyboard.hasTextInput
- utf8

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.textinput

love.textinput · revision 30156 · last edited 2025-11-02

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Called when text has been entered by the user. For example if shift-2 is pressed on an American keyboard layout, the text "@" will be generated.

Function

Synopsis

```
love.textinput( text )
```

Arguments

`string text`
: The UTF-8 encoded unicode text.

Returns

Nothing.

Notes

Although Lua strings can store UTF-8 encoded unicode text just fine, many functions in Lua's string library will not treat the text as you might expect. For example, `#text` (and `string.len(text)`) will give the number of *bytes* in the string, rather than the number of unicode characters. The Lua wiki (http://lua-users.org/w iki/LuaUnicode) and a presentation by one of Lua's creators (http://w w w.lua.org/w shop12/lerusalimschy.pdf) give more in-depth explanations, with some tips.

The utf8 library can be used to operate on UTF-8 encoded unicode text (such as the text argument given in this function.)

On Android and iOS, textinput is disabled by default; call love.keyboard.setTextInput to enable it.

Examples

Record and print text the user writes.

```
function love.load()
    text = "Type away! -- "
end

function love.textinput(t)
    text = text .. t
end

function love.draw()
```

```
    love.graphics.printf(text, 0, 0, love.graphics.getWidth())
end
```

Print text the user writes, and erase text when backspace is pressed.

```
local utf8 = require("utf8")

function love.load()
    text = "Type away! -- "

    -- enable key repeat so backspace can be held down to trigger love.keypressed multiple times.
    love.keyboard.setKeyRepeat(true)
end

function love.textinput(t)
    text = text .. t
end

function love.keypressed(key)
    if key == "backspace" then
        -- get the byte offset to the last UTF-8 character in the string.
        local byteoffset = utf8.offset(text, -1)

        if byteoffset then
            -- remove the last UTF-8 character.
            -- string.sub operates on bytes rather than UTF-8 characters, so we couldn't do string.sub(text, 1, -2).
            text = string.sub(text, 1, byteoffset - 1)
        end
    end
end

function love.draw()
    love.graphics.printf(text, 0, 0, love.graphics.getWidth())
end
```

See Also

- love
- love.keypressed
- love.keyboard.setTextInput
- love.keyboard.hasTextInput
- utf8
- TextInputField
- textinput vs keypressed (https://w iki.libsdl.org/SDL2/Tutorials-TextInput)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.thread

love.thread · revision 28268 · last edited 2023-07-05

Available since LÖVE 0.7.0

This module is not supported in earlier versions.

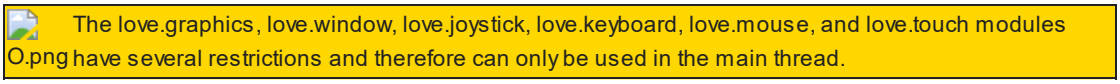
Allows you to work with threads.

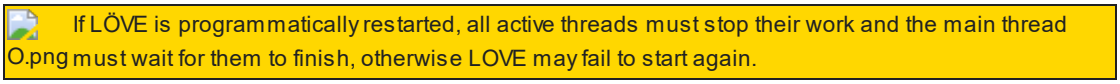
Threads are separate Lua environments, running in parallel to the main code. As their code runs separately, they can be used to compute complex operations without adversely affecting the frame rate of the main thread. However, as they are separate environments, they cannot access the variables and functions of the main thread, and communication between threads is limited.

All LÖVE objects (userdata) are shared among threads so you'll only have to send their references across threads. You may run into concurrency issues if you manipulate an object or global love state on multiple threads at the same time.

When a Thread is started, it only loads love.data, love.filesystem, and love.thread module. Every other module has to be loaded with require.

LÖVE provides a default implementation of the love.threaderror callback in order to see errors from threads. It can be overwritten and customized, and Thread:getError can also be used.





Types

Channel	An object which can be used to send and receive data between different threads.	Added since 0.9.0
Thread	A Thread represents a thread.	Added since 0.7.0

Functions

love.thread.getChannel	Creates or retrieves a named thread channel.	Added since 0.9.0	
love.thread.getThread	Look for a thread and get its object.	Added since 0.7.0	Removed in 0.9.0
love.thread.getThreads	Get all threads.	Added since 0.7.0	Removed in 0.9.0
love.thread.newChannel	Creates a new unnamed thread channel.	Added since 0.9.0	
love.thread.newThread	Creates a new Thread from a filename, string or FileData object containing Lua code.	Added since 0.7.0	

Examples

A simple example showing the general usage of a thread and using channels for communication.

```
-- This is the code that's going to run on the our thread. It should be moved
-- to its own dedicated Lua file, but for simplicity's sake we'll create it
-- here.
local threadCode = [[
-- Receive values sent via thread:start
local min, max = ...

for i = min, max do
    -- The Channel is used to handle communication between our main thread and
    -- this thread. On each iteration of the loop will push a message to it which
    -- we can then pop / receive in the main thread.
    love.thread.getChannel( 'info' ):push( i )
end
]]

local thread -- Our thread object.
local timer = 0  -- A timer used to animate our circle.

function love.load()
    thread = love.thread.newThread( threadCode )
    thread:start( 99, 1000 )
end

function love.update( dt )
    timer = timer + dt
end

function love.draw()
    -- Get the info channel and pop the next message from it.
    local info = love.thread.getChannel( 'info' ):pop()
    if info then
        love.graphics.print( info, 10, 10 )
    end

    -- We smoothly animate a circle to show that the thread isn't blocking our main thread.
    love.graphics.circle( 'line', 100 + math.sin( timer ) * 20, 100 + math.cos( timer ) * 20, 20 )
end
```

See Also

- love
- love.threaderror

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.thread.getChannel

love.thread.getChannel · revision 20570 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Communication betw een main/thread
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Creates or retrieves a named thread channel.

Function

Synopsis

```
channel = love.thread.getChannel( name )
```

Arguments

string name

The name of the channel you want to create or retrieve.

Returns

Channel channel

The Channel object associated with the name.

Examples

Communication between main/thread

```
-- main
thread      = love.thread.newThread ( "thread.lua" );
thread:start ();
channel     = {};
channel.a   = love.thread.getChannel ( "a" );
channel.b   = love.thread.getChannel ( "b" );
channel.b:push ( "foo" );

function love.update ( dt )
    local v = channel.a:pop ();
    if v then
        print ( tostring ( v ) );
        channel.b:push ( "foo" );
    end
end

-- thread
channel     = {};
channel.a   = love.thread.getChannel ( "a" );
channel.b   = love.thread.getChannel ( "b" );
```

```
while true do
    local v = channel.b:pop ();
    if v then
        print ( tostring ( v ) );
        channel.a:push ( "bar" );
    end
end
```

See Also

- love.thread
- Channel

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.thread.getThread

love.thread.getThread · revision 15950 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.9.0

This function is not supported in earlier or later versions.

Look for a thread and get its object.

Function

Synopsis

```
thread = love.thread.getThread( name )
```

Arguments

`string name`

The name of the thread to return.

Returns

`Thread thread`

The thread with that name.

Function

Synopsis

```
thread = love.thread.getThread( )
```

Arguments

None.

Returns

`Thread thread`

The current thread.

See Also

- love.thread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.thread.getThreads

love.thread.getThreads · revision 15951 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.9.0

This function is not supported in earlier or later versions.

Get all threads.

Function

Synopsis

```
threads = love.thread.getThreads( )
```

Arguments

None.

Returns

`table threads`

A table containing all threads indexed by their names.

See Also

- love.thread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.thread.newChannel

love.thread.newChannel · revision 12181 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Create a new unnamed thread channel.

One use for them is to pass new unnamed channels to other threads via Channel:push on a named channel.

Function

Synopsis

```
channel = love.thread.newChannel( )
```

Arguments

None.

Returns

`Channel channel`

The new Channel object.

See Also

- love.thread
- Channel

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.thread.newThread

love.thread.newThread · revision 27097 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Creates a new Thread from a filename, string or FileData object containing Lua code.

Function

Available since LÖVE 0.9.0

This variant is not supported in earlier versions.

Synopsis

```
thread = love.thread.newThread( filename )
```

Arguments

`string filename`

The name of the Lua file to use as the source.

Returns

`Thread thread`

A new Thread that has yet to be started.

Function

Available since LÖVE 0.9.0

This variant is not supported in earlier versions.

Synopsis

```
thread = love.thread.newThread( fileData )
```

Arguments

`FileData fileData`
The FileData containing the Lua code to use as the source.

Returns

`Thread thread`
A new Thread that has yet to be started.

Function

Available since LÖVE 0.9.2
This variant is not supported in earlier versions.

Synopsis

```
thread = love.thread.newThread( codestring )
```

Arguments

`string codestring`
A string containing the Lua code to use as the source. It needs to either be at least 1024 characters long, or contain at least one newline.

Returns

`Thread thread`
A new Thread that has yet to be started.

Function

Removed in LÖVE 0.9.0
This variant is not supported in that and later versions.

Synopsis

```
thread = love.thread.newThread( name, filename )
```

Arguments

`string name`
The name of the thread.

`string filename`

The name of the File to use as source.

Returns

`Thread thread`
A new Thread that has yet to be started.

Function

Removed in LÖVE 0.9.0
This variant is not supported in that and later versions.

Synopsis

```
thread = love.thread.newThread( name, file )
```

Arguments

`string name`
The name of the thread.

`File file`
The file to use as source.

Returns

`Thread thread`
A new Thread that has yet to be started.

Function

Removed in LÖVE 0.9.0
This variant is not supported in that and later versions.

Synopsis

```
thread = love.thread.newThread( name, data )
```

Arguments

`string name`
The name of the thread.

`Data data`
The data to use as source.

Returns

`Thread thread`

A new Thread that has yet to be started.

See Also

- love.thread
- Thread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.threaderror

love.threaderror · revision 29233 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
- See Also
- Other Languages

Available since LÖVE 0.9.0

This callback is not supported in earlier versions.

Callback function triggered when a Thread encounters an error. If not overwritten this callback will raise an error containing the error message of the threaderror.

Function

Synopsis

```
love.threaderror( thread, errorstr )
```

Arguments

`Thread thread`
: The thread which produced the error.

`string errorstr`
: The error message.

Returns

Nothing.

Example

```
function love.load()
  mythread = love.thread.newThread("thread.lua")
  mythread:start()
end

function love.threaderror(thread, errorstr)
  print("Thread error!\n"..errorstr)
  -- thread:getError() will return the same error string now.
end
```

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.timer

love.timer · revision 22296 · last edited 2024-05-08

Provides high-resolution timing functionality.

Functions

love.timer.getAverageDelta	Returns the average delta time over the last second.	Added since 0.9.0	
love.timer.getDelta	Returns the time between the last two frames.		
love.timer.getFPS	Returns the current frames per second.	Added since 0.2.1	
love.timer.getMicroTime	Returns the value of a timer with microsecond precision.		Removed in 0.9.0
love.timer.getTime	Returns the precise amount of time since some time in the past.	Added since 0.3.2	
love.timer.sleep	Pauses the current thread for the specified amount of time.	Added since 0.2.1	
love.timer.step	Measures the time between two frames.		

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.timer.getAverageDelta

love.timer.getAverageDelta · revision 13976 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Returns the average delta time (seconds per frame) over the last second.

Function

Synopsis

```
delta = love.timer.getAverageDelta( )
```

Arguments

None.

Returns

number delta

The average delta time over the last second.

Examples

Display text at the top left of the screen showing the average time taken to update and draw each frame.

```
function love.draw()
   local delta = love.timer.getAverageDelta()
   -- Display the frame time in milliseconds for convenience.
   -- A lower frame time means more frames per second.
   love.graphics.print(string.format("Average frame time: %.3f ms", 1000 * delta), 10, 10)
end
```

See Also

- love.timer
- love.timer.getDelta
- love.timer.getFPS

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.timer.getDelta

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Returns the time between the last two frames.

Function

Synopsis

```
dt = love.timer.getDelta( )
```

Arguments

None.

Returns

number dt

The time passed (in seconds).

See Also

- love.timer
- love.timer.getAverageDelta

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.timer.getFPS

love.timer.getFPS · revision 28087 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- See Also
- Other Languages

Available since LÖVE 0.2.1

This function is not supported in earlier versions.

Returns the current number of frames per second.

Function

Synopsis

```
fps = love.timer.getFPS( )
```

Arguments

None.

Returns

number fps

The current FPS.

Notes

The returned value is the number of frames rendered during the last second, rounded to the nearest integer value.

It is one divided by what love.timer.getAverageDelta returns, otherwise known as the reciprocal, or multiplicative inverse of it.

To get instantaneous frame rate values, use `1.0 / love.timer.getDelta()`, or `1.0 / dt` if in love.update, with `dt` given as the parameter.

Examples

Display text at the top left of the screen showing the current FPS.

```
function love.draw()
   love.graphics.print("Current FPS: "..tostring(love.timer.getFPS( )), 10, 10)
end
```

See Also

- love.timer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.timer.getMicroTime

love.timer.getMicroTime · revision 10682 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

love.timer.getTime is now microsecond-accurate.

Returns the value of a timer with an unspecified starting time. The time is accurate to the microsecond.

Function

Synopsis

```
t = love.timer.getMicroTime( )
```

Arguments

None.

Returns

number t

The time passed in seconds.

See Also

- love.timer
- love.timer.getTime

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.timer.getTime

love.timer.getTime · revision 27338 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Checking how long something takes
- See Also
- Other Languages

Available since LÖVE 0.3.2

This function is not supported in earlier versions.

Returns the value of a precise timer with an unspecified starting time.

This function should only be used to calculate differences between points in time, as the starting time of the timer is unknown prior to version 11.4. Beginning with 11.4, the timer is initialized to 0 when the love.timer module is first loaded (usually before main.lua is loaded).

Function

Synopsis

```
time = love.timer.getTime( )
```

Arguments

None.

Returns

number time

The time in seconds. Given as a decimal, accurate to the microsecond.

Examples

Checking how long something takes

```
local start = love.timer.getTime()

-- Concatenate "bar" 1000 times.
local foo = ""
for _ = 1, 1000 do
    foo = foo .. "bar"
end

-- Resulting time difference in seconds. Multiplying it by 1000 gives us the value in milliseconds.
local result = love.timer.getTime() - start
print( string.format( "It took %.3f milliseconds to concatenate 'bar' 1000 times!", result * 1000 ))
```

See Also

- love.timer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.timer.sleep

love.timer.sleep · revision 25513 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Use sleep to cap FPS at 30
  - More sophisticated way to cap 30 FPS
- See Also
- Other Languages

Available since LÖVE 0.2.1

This function is not supported in earlier versions.

Pauses the current thread for the specified amount of time.



This function causes the entire thread to pause for the duration of the sleep. Graphics will not draw, input events will not trigger, code will not run, and the window will be unresponsive if you use this as "wait()" in the main thread. Use love.update or a Timer library for that instead.

Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
love.timer.sleep( s )
```

Arguments

number s

Seconds to sleep for.

Returns

Nothing.

Function

Removed in LÖVE 0.8.0

This variant is not supported in that and later versions.

Synopsis

```
love.timer.sleep( ms )
```

Arguments

number ms

Milliseconds to sleep for.

Returns

Nothing.

Examples

Use sleep to cap FPS at 30

```
function love.update(dt)
   if dt < 1/30 then
      love.timer.sleep(1/30 - dt)
   end
end
```

More sophisticated way to cap 30 FPS

This takes into account the time spent updating and drawing each frame.

```
function love.load()
   min_dt = 1/30
   next_time = love.timer.getTime()
end

function love.update(dt)
   next_time = next_time + min_dt

   -- rest of function here
end

function love.draw()
   -- rest of function here

   local cur_time = love.timer.getTime()
   if next_time <= cur_time then
      next_time = cur_time
      return
   end
   love.timer.sleep(next_time - cur_time)
end
```

See Also

- love.timer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.timer.step

love.timer.step · revision 22332 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Measures the time between two frames. Calling this changes the return value of love.timer.getDelta.

Function

Synopsis

```
dt = love.timer.step( )
```

Arguments

None.

Returns

Available since LÖVE 11.0

`number dt`
: The time passed (in seconds).

See Also

- love.timer
- love.run

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.touch

love.touch · revision 31474 · last edited 2026-04-20

Available since LÖVE 0.10.0

This module is not supported in earlier versions.

Provides an interface to touch-screen presses.

Functions

love.touch.getPosition	Gets the current position of the specified touch-press.	Added since 0.10.0
love.touch.getPressure	Gets the current pressure of the specified touch-press.	Added since 0.10.0
love.touch.getTouches	Gets a list of all active touch-presses.	Added since 0.10.0

See Also

- love
- love.touchpressed
- love.touchreleased
- love.touchmoved

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.touch.getDeviceType

love.touch.getDeviceType · revision 30569 · last edited 2026-04-07

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets the type of touch device corresponding to the specified touch-press.

Function

Synopsis

```
devicetype = love.touch.getDeviceType( id )
```

Arguments

`light userdata id`

The identifier of the touch-press. Use love.touch.getTouches, love.touchpressed, or love.touchmoved to obtain touch id values.

Returns

`DeviceType devicetype`

The type of touch device corresponding to the given touch-press.

See Also

- love.touch
- DeviceType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.touch.getPosition

love.touch.getPosition · revision 15852 · last edited 2026-04-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
  - Draw a circle at each location in the window where a touch press is active.
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the current position of the specified touch-press, in pixels.

Function

Synopsis

```
x, y = love.touch.getPosition( id )
```

Arguments

light userdata id

The identifier of the touch-press. Use love.touch.getTouches, love.touchpressed, or love.touchmoved to obtain touch id values.

Returns

number x

The position along the x-axis of the touch-press inside the window, in pixels.

number y

The position along the y-axis of the touch-press inside the window, in pixels.

Notes

The unofficial Android and iOS ports of LÖVE 0.9.2 reported touch-press positions as normalized values in the range of [0, 1], whereas this API reports positions in pixels.

Examples

Draw a circle at each location in the window where a touch press is active.

```
function love.draw()
    local touches = love.touch.getTouches()

    for i, id in ipairs(touches) do
        local x, y = love.touch.getPosition(id)
        love.graphics.circle("fill", x, y, 20)
```

```
    end
end
```

See Also

- love.touch
- love.touch.getTouches
- love.touchpressed
- love.touchreleased

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.touch.getPressure

love.touch.getPressure · revision 16026 · last edited 2026-04-07

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the current pressure of the specified touch-press.

Function

Synopsis

```
pressure = love.touch.getPressure( id )
```

Arguments

light userdata id

The identifier of the touch-press. Use love.touch.getTouches, love.touchpressed, or love.touchmoved to obtain touch id values.

Returns

number pressure

The pressure of the touch-press. Most touch screens aren't pressure sensitive, in which case the pressure will be 1.

See Also

- love.touch
- love.touch.getTouches
- love.touchpressed
- love.touchmoved

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.touch.getTouches

love.touch.getTouches · revision 15141 · last edited 2026-04-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets a list of all active touch-presses.

Function

Synopsis

```
touches = love.touch.getTouches( )
```

Arguments

None.

Returns

`table touches`

A list of active touch-press id values, which can be used with love.touch.getPosition.

Notes

The id values are the same as those used as arguments to love.touchpressed, love.touchmoved, and love.touchreleased.

The id value of a specific touch-press is only guaranteed to be unique for the duration of that touch-press. As soon as love.touchreleased is called using that id, it may be reused for a new touch-press via love.touchpressed.

See Also

- love.touch
- love.touch.getPosition
- love.touchpressed
- love.touchreleased

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.touch.isMouse

love.touch.isMouse · revision 30601 · last edited 2026-04-07

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets whether a touch event was simulated from mouse input.

Function

Synopsis

```
ismouse = love.touch.isMouse( id )
```

Arguments

`light userdata id`

The identifier of the touch press.

Returns

`boolean ismouse`

True if the touch event was simulated from mouse input, false otherwise.

See Also

- love.touch

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.touchmoved

love.touchmoved · revision 22497 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Callback function triggered when a touch press moves inside the touch screen.

Function

Synopsis

```
love.touchmoved( id, x, y, dx, dy, pressure )
```

Arguments

light userdata id
: The identifier for the touch press.

number x
: The x-axis position of the touch inside the window, in pixels.

number y
: The y-axis position of the touch inside the window, in pixels.

number dx
: The x-axis movement of the touch inside the window, in pixels.

number dy
: The y-axis movement of the touch inside the window, in pixels.

number pressure
: The amount of pressure being applied. Most touch screens aren't pressure sensitive, in which case the pressure will be 1.

Returns

Nothing.

Notes

The identifier is only guaranteed to be unique for the specific touch press until love.touchreleased is called with that identifier, at which point it may be reused for new touch presses.

The unofficial Android and iOS ports of LÖVE 0.9.2 reported touch positions as normalized values in the range of [0, 1], whereas this API reports positions in pixels.

See Also

- love
- love.touchpressed

- love.touchreleased
- love.touch

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.touchpressed

love.touchpressed · revision 22498 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Callback function triggered when the touch screen is touched.

Function

Synopsis

```
love.touchpressed( id, x, y, dx, dy, pressure )
```

Arguments

light userdata id
: The identifier for the touch press.

number x
: The x-axis position of the touch press inside the window, in pixels.

number y
: The y-axis position of the touch press inside the window, in pixels.

number dx
: The x-axis movement of the touch press inside the window, in pixels. This should always be zero.

number dy
: The y-axis movement of the touch press inside the window, in pixels. This should always be zero.

number pressure
: The amount of pressure being applied. Most touch screens aren't pressure sensitive, in which case the pressure will be 1.

Returns

Nothing.

Notes

The identifier is only guaranteed to be unique for the specific touch press until love.touchreleased is called with that identifier, at which point it may be reused for new touch presses.

The unofficial Android and iOS ports of LÖVE 0.9.2 reported touch positions as normalized values in the range of [0, 1], whereas this API reports positions in pixels.

See Also

- love
- love.touchreleased

- love.touchmoved
- love.touch

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.touchreleased

love.touchreleased · revision 22499 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Callback function triggered when the touch screen stops being touched.

Function

Synopsis

```
love.touchreleased( id, x, y, dx, dy, pressure )
```

Arguments

light userdata id
: The identifier for the touch press.

number x
: The x-axis position of the touch inside the window, in pixels.

number y
: The y-axis position of the touch inside the window, in pixels.

number dx
: The x-axis movement of the touch inside the window, in pixels.

number dy
: The y-axis movement of the touch inside the window, in pixels.

number pressure
: The amount of pressure being applied. Most touch screens aren't pressure sensitive, in which case the pressure will be 1.

Returns

Nothing.

Notes

The identifier is only guaranteed to be unique for the specific touch press until love.touchreleased is called with that identifier, at which point it may be reused for new touch presses.

The unofficial Android and iOS ports of LÖVE 0.9.2 reported touch positions as normalized values in the range of [0, 1], whereas this API reports positions in pixels.

See Also

- love
- love.touchpressed

- love.touchmoved
- love.touch

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.update

love.update · revision 29419 · last edited 2025-09-30

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Callback function used by the default love.run to update the state of the game every frame.

Function

Synopsis

```
love.update( dt )
```

Arguments

number dt
: Time since the last update in seconds.

Returns

Nothing.

Examples

Run a function called *think* inside a table called *npc* once per second.

```
dtotal = 0   -- this keeps track of how much time has passed
function love.update(dt)
   dtotal = dtotal + dt   -- we add the time passed since the last update, probably a very small number like 0.01
   if dtotal >= 1 then
      dtotal = dtotal - 1   -- reduce our timer by a second, but don't discard the change... what if our framerate is
2/3 of a second?
      npc.think()
   end
end
```

Change a variable *var* at a constant rate (+/- 3 per second in this example).

```
var = 10   -- arbitrary starting value
rate = 3   -- change to change the rate at which the var is changed
function love.update(dt)
   if love.keyboard.isDown("down") then   -- reduce the value
      var = var - (dt * rate)
   end
   if love.keyboard.isDown("up") then   -- increase the value
      var = var + (dt * rate)
```

```
    end
end
```

See Also

- love
- World:update
- variable

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.video

love.video · revision 31476 · last edited 2026-04-20

Available since LÖVE 0.10.0

This module is not supported in earlier versions.

This module is responsible for decoding, controlling, and streaming video files.

It can't draw the videos, see love.graphics.newVideo and Video objects for that.

Types

VideoStream	An object which decodes, streams, and controls Videos.	

Functions

love.video.newVideoStream	Creates a new VideoStream.	

See Also

- love
- love.graphics.newVideo
- Video

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.video.newVideoStream

love.video.newVideoStream · revision 29129 · last edited 2026-04-06

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Creates a new VideoStream. Currently only Ogg Theora video files are supported. VideoStreams can't draw videos, see love.graphics.newVideo for that.



This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Synopsis

```
videostream = love.video.newVideoStream( filename )
```

Arguments

`string filename`

The file path to the Ogg Theora video file.

Returns

`VideoStream videostream`

A new VideoStream.

Function

Synopsis

```
videostream = love.video.newVideoStream( file )
```

Arguments

`File file`

The File object containing the Ogg Theora video.

Returns

`VideoStream videostream`

A new VideoStream.

Notes

Löve only provides the basic functionality provided by libtheora. If your video plays fine in a video player but not in Löve, try different encoding options.

See Also

- love.video
- VideoStream
- Video

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.visible

love.visible · revision 22507 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
- See Also
- Other Languages

Available since LÖVE 0.9.0

This callback is not supported in earlier versions.

Callback function triggered when window is minimized/hidden or unminimized by the user.

Function

Synopsis

```
love.visible( visible )
```

Arguments

boolean visible

True if the window is visible, false if it isn't.

Returns

Nothing.

Example

```
function love.visible(v)
    print(v and "Window is visible!" or "Window is not visible!");
end
```

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.wheelmoved

love.wheelmoved · revision 22495 · last edited 2026-04-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Smooth scrolling
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Callback function triggered when the mouse wheel is moved.

Function

Synopsis

```
love.wheelmoved( x, y )
```

Arguments

number x
: Amount of horizontal mouse wheel movement. Positive values indicate movement to the right.

number y
: Amount of vertical mouse wheel movement. Positive values indicate upward movement.

Returns

Nothing.

Examples

```
local text = ""

function love.wheelmoved(x, y)
    if y > 0 then
        text = "Mouse wheel moved up"
    elseif y < 0 then
        text = "Mouse wheel moved down"
    end
end

function love.draw()
    love.graphics.print(text, 10, 10)
end
```

Smooth scrolling

```
function love.load()
    posx, posy = love.graphics.getWidth() * 0.5, love.graphics.getHeight() * 0.5
    velx, vely = 0, 0 -- The scroll velocity
```

```
end

function love.draw()
    love.graphics.rectangle( 'line', posx, posy, 50, 50 )
end

function love.update( dt )
    posx = posx + velx * dt
    posy = posy + vely * dt

    -- Gradually reduce the velocity to create smooth scrolling effect.
    velx = velx - velx * math.min( dt * 10, 1 )
    vely = vely - vely * math.min( dt * 10, 1 )
end

function love.wheelmoved( dx, dy )
    velx = velx + dx * 20
    vely = vely + dy * 20
end
```

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window

love.window · revision 22219 · last edited 2025-02-21

Available since LÖVE 0.9.0

This module is not supported in earlier versions.

Provides an interface for modifying and retrieving information about the program's window.

Functions

love.window.close	Closes the window.	Added since 0.10.0	
love.window.fromPixels	Converts a number from pixels to density-independent units.	Added since 0.9.2	
love.window.getDPIScale	Gets the DPI scale factor associated with the window.	Added since 11.0	
love.window.getDesktopDimensions	Gets the width and height of the desktop.	Added since 0.9.0	
love.window.getDimensions	Gets the width and height of the window.	Added since 0.9.0	Removed in 0.10.0
love.window.getDisplayCount	Gets the number of connected monitors.	Added since 0.9.0	
love.window.getDisplayName	Gets the name of a display.	Added since 0.9.2	
love.window.getDisplayOrientation	Gets current device display orientation.	Added since 11.3	
love.window.getFullscreen	Gets whether the window is fullscreen.	Added since 0.9.0	
love.window.getFullscreenModes	Gets a list of supported fullscreen modes.	Added since 0.9.0	
love.window.getHeight	Gets the height of the window.	Added since 0.9.0	Removed in 0.10.0
love.window.getIcon	Gets the window icon.	Added since 0.9.0	
love.window.getMode	Gets the display mode and properties of the window.	Added since 0.9.0	
love.window.getPixelScale	Gets the DPI scale factor associated with the window.	Added since 0.9.1	Removed in 11.0

love.window.getPosition	Gets the position of the window on the screen.	Added since 0.9.2	
love.window.getSafeArea	Gets unobstructed area inside the window.	Added since 11.3	
love.window.getTitle	Gets the window title.	Added since 0.9.0	
love.window.getVSync	Gets current vsync value.	Added since 11.3	
love.window.getWidth	Gets the width of the window.	Added since 0.9.0	Removed in 0.10.0
love.window.hasFocus	Checks if the game window has keyboard focus.	Added since 0.9.0	
love.window.hasMouseFocus	Checks if the game window has mouse focus.	Added since 0.9.0	
love.window.isCreated	Checks if the window has been created.	Added since 0.9.0	Removed in 0.10.0
love.window.isDisplaySleepEnabled	Gets whether the display is allowed to sleep while the program is running.	Added since 0.10.0	
love.window.isMaximized	Gets whether the Window is currently maximized.	Added since 0.10.2	
love.window.isMinimized	Gets whether the Window is currently minimized.	Added since 11.0	
love.window.isOpen	Checks if the window is open.	Added since 0.10.0	
love.window.isVisible	Checks if the game window is visible.	Added since 0.9.0	
love.window.maximize	Makes the window as large as possible.	Added since 0.10.0	
love.window.minimize	Minimizes the window to the system's task bar / dock.	Added since 0.9.2	
love.window.requestAttention	Causes the window to request the attention of the user if it is not in the foreground.	Added since 0.10.0	
love.window.restore	Restores the size and position of the window if it was minimized or maximized.	Added since 11.0	
love.window.setDisplaySleepEnabled	Sets whether the display is allowed to sleep while the program is running.	Added since 0.10.0	
love.window.setFullscreen	Enters or exits fullscreen.	Added since 0.9.0	

love.window.setIcon	Sets the window icon.	Added since 0.9.0
love.window.setMode	Sets the display mode and properties of the window.	Added since 0.9.0
love.window.setPosition	Sets the position of the window on the screen.	Added since 0.9.2
love.window.setTitle	Sets the window title.	Added since 0.9.0
love.window.setVSync	Sets vertical synchronization mode.	Added since 11.3
love.window.showMessageBox	Displays a message box above the love window.	Added since 0.9.2
love.window.toPixels	Converts a number from density-independent units to pixels.	Added since 0.9.2
love.window.updateMode	Sets the display mode and properties of the window, without modifying unspecified properties.	Added since 11.0

Enums

DisplayOrientation	Types of device display orientation.	Added since 11.3
FullscreenType	Types of fullscreen modes.	Added since 0.9.0
MessageBoxType	Types of message box dialogs.	Added since 0.9.2

See Also

- love
- love.resize
- love.focus
- love.mousefocus
- love.visible

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.close

love.window.close · revision 28077 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Closes the window. It can be reopened with love.window.setMode.


love.graphics functions and objects will cause a hard crash of LÖVE if used while the window is closed.


This function does not quit the program. Use love.event.quit to do that.

Function

Synopsis

```
love.window.close( )
```

Arguments

None.

Returns

Nothing.

Notes

This only closes the window and leaves LÖVE running in the background. If you want to stop the program, consider love.event.quit.

See Also

- love.window
- love.window.setMode

- love.graphics.isActive

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.focus

love.window.focus · revision 29292 · last edited 2024-07-08

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Brings the window into the foreground above other windows and sets input focus.

This may not be functional on all platforms.

Function

Synopsis

```
love.window.focus( )
```

Arguments

None.

Returns

Nothing.

See Also

- love.window
- love.window.hasFocus

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.fromPixels

love.window.fromPixels · revision 15577 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Converts a number from pixels to density-independent units.

The pixel density inside the window might be greater (or smaller) than the "size" of the window. For example on a retina screen in Mac OS X with the `highdpi` window flag enabled, the window may take up the same physical size as an 800x600 window, but the area inside the window uses 1600x1200 pixels. `love.window.fromPixels(1600)` would return `800` in that case.

This function converts coordinates from pixels to the size users are expecting them to display at onscreen. love.window.toPixels does the opposite. The `highdpi` window flag must be enabled to use the full pixel density of a Retina screen on Mac OS X and iOS. The flag currently does nothing on Windows and Linux, and on Android it is effectively always enabled.

Most LÖVE functions return values and expect arguments in terms of pixels rather than density-independent units.

Function

Synopsis

```
value = love.window.fromPixels( pixelvalue )
```

Arguments

`number pixelvalue`

A number in pixels to convert to density-independent units.

Returns

`number value`

The converted number, in density-independent units.

Function

Synopsis

```
x, y = love.window.fromPixels( px, py )
```

Arguments

`number px`

The x-axis value of a coordinate in pixels.

number py
: The y-axis value of a coordinate in pixels.

Returns

number x
: The converted x-axis value of the coordinate, in density-independent units.

number y
: The converted y-axis value of the coordinate, in density-independent units.

Notes

The units of love.graphics.getWidth, love.graphics.getHeight, love.mouse.getPosition, mouse events, love.touch.getPosition, and touch events are always in terms of pixels.

See Also

- love.window
- love.window.toPixels
- love.window.getPixelScale
- love.window.setMode
- Config Files

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getDesktopDimensions

love.window.getDesktopDimensions · revision 28315 · last edited 2026-01-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Show the resolution of the monitor the window is currently in
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the width and height of the desktop.

Function

Synopsis

```
width, height = love.window.getDesktopDimensions( displayindex )
```

Arguments

number displayindex (1)

The index of the display, if multiple monitors are available. First is 1.

Returns

number width

The width of the desktop.

number height

The height of the desktop.

Examples

Show the resolution of the monitor the window is currently in

```
function love.draw()
    local _, _, flags = love.window.getMode()

    -- The window's flags contain the index of the monitor it's currently in.
    local width, height = love.window.getDesktopDimensions(flags.display)
```

```
    love.graphics.print(("display %d: %d x %d"):format(flags.display, width, height), 4, 10)
end
```

See Also

- love.window

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getDimensions

love.window.getDimensions · revision 15955 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0 and removed in LÖVE 0.10.0

Use love.graphics.getDimensions or love.window.getMode instead.

Gets the width and height of the window.

Function

Synopsis

```
width, height = love.window.getDimensions( )
```

Arguments

None.

Returns

number width
: The width of the window.

number height
: The height of the window.

See Also

- love.window
- love.window.getWidth
- love.window.getHeight
- love.window.setMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getDisplayCount

love.window.getDisplayCount · revision 13111 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the number of connected monitors.

Function

Synopsis

```
count = love.window.getDisplayCount( )
```

Arguments

None.

Returns

`number count`

The number of currently connected displays.

See Also

- love.window
- love.window.setMode
- love.window.getMode
- love.window.getFullscreenModes
- love.window.getDesktopDimensions

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getDisplayName

love.window.getDisplayName · revision 13863 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets the name of a display.

Function

Synopsis

```
name = love.window.getDisplayName( displayindex )
```

Arguments

`number displayindex (1)`

The index of the display to get the name of.

Returns

`string name`

The name of the specified display.

See Also

- love.window
- love.window.getDisplayCount

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getDisplayOrientation

love.window.getDisplayOrientation · revision 26750 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.3

This function is not supported in earlier versions.

Gets current device display orientation.

Function

Synopsis

```
orientation = love.window.getDisplayOrientation( displayindex )
```

Arguments

`number displayindex (nil)`

Display index to get its display orientation, or nil for default display index.

Returns

`DisplayOrientation orientation`

Current device display orientation.

See Also

- love.window
- DisplayOrientation
- love.displayrotated

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getDPIScale

love.window.getDPIScale · revision 25126 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function replaces love.window.getPixelScale.

Gets the DPI scale factor associated with the window.

The pixel density inside the window might be greater (or smaller) than the "size" of the window. For example on a retina screen in Mac OS X with the `highdpi` window flag enabled, the window may take up the same physical size as an 800x600 window, but the area inside the window uses 1600x1200 pixels. `love.window.getDPIScale()` would return `2.0` in that case.

The love.window.fromPixels and love.window.toPixels functions can also be used to convert between units.

The `highdpi` window flag must be enabled to use the full pixel density of a Retina screen on Mac OS X and iOS. The flag currently does nothing on Windows and Linux, and on Android it is effectively always enabled.

Function

Synopsis

```
scale = love.window.getDPIScale( )
```

Arguments

None.

Returns

`number scale`

The pixel scale factor associated with the window.

Notes

The units of love.graphics.getWidth, love.graphics.getHeight, love.mouse.getPosition, mouse events, love.touch.getPosition, and touch events are always in DPI-scaled units in LOVE 11.0+, unless the `usedpiscale` flag is set to false in love.window.setMode or love.conf.

See Also

- love.window
- love.window.toPixels
- love.window.fromPixels
- love.window.setMode

- Config Files

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getFullscreen

love.window.getFullscreen · revision 11354 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets whether the window is fullscreen.

Function

Synopsis

```
fullscreen, fstype = love.window.getFullscreen( )
```

Arguments

None.

Returns

`boolean fullscreen`

True if the window is fullscreen, false otherwise.

`FullscreenType fstype`

The type of fullscreen mode used.

See Also

- love.window
- love.window.setFullscreen
- love.window.setMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getFullscreenModes

love.window.getFullscreenModes · revision 26751 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Format of the returned table
  - Sort table to ensure it is in order
- See Also
- Other Languages

Available since LÖVE 0.9.0

Moved from love.graphics.getModes.

Gets a list of supported fullscreen modes.

Function

Synopsis

```
modes = love.window.getFullscreenModes( displayindex )
```

Arguments

number displayindex (1)

The index of the display, if multiple monitors are available.

Returns

table modes

A table of width/height pairs. (Note that this may not be in order.)

Examples

Format of the returned table

```
modes = love.window.getFullscreenModes()

-- modes = {
--   { width = 320, height = 240 },
--   { width = 640, height = 480 },
--   { width = 800, height = 600 },
--   { width = 1024, height = 768 },
```

```
--  ...
-- }
```

Sort table to ensure it is in order

```
modes = love.window.getFullscreenModes()
table.sort(modes, function(a, b) return a.width*a.height < b.width*b.height end)   -- sort from smallest to largest
```

See Also

- love.window
- love.window.setMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getHeight

love.window.getHeight · revision 15953 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0 and removed in LÖVE 0.10.0

Use love.graphics.getHeight or love.window.getMode instead.

Gets the height of the window.

Function

Synopsis

```
height = love.window.getHeight( )
```

Arguments

None.

Returns

number height

The height of the window.

See Also

- love.window
- love.window.getWidth
- love.window.getDimensions
- love.window.setMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getIcon

love.window.getIcon · revision 12427 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the window icon.

Function

Synopsis

```
imagedata = love.window.getIcon( )
```

Arguments

None.

Returns

`ImageData imagedata`

The window icon imagedata, or nil if no icon has been set with love.window.setIcon.

See Also

- love.window
- love.window.setIcon

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getMode

love.window.getMode · revision 28316 · last edited 2025-11-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

Moved from love.graphics.getMode.

Gets the display mode and properties of the window.

Function

Synopsis

```
width, height, flags = love.window.getMode( )
```

Arguments

None.

Returns

number width
: Window width.

number height
: Window height.

table flags
: Table with the window properties:

  boolean fullscreen
  : Fullscreen (true), or windowed (false).

  FullscreenType fullscreentype
  : The type of fullscreen mode used.

  number vsync
  : 1 if the graphics framerate is synchronized with the monitor's refresh rate, 0 otherwise.

  number msaa
  : The number of antialiasing samples used (0 if MSAA is disabled).

  boolean resizable
  : True if the window is resizable in windowed mode, false otherwise.

  boolean borderless
  : True if the window is borderless in windowed mode, false otherwise.

  boolean centered
  : True if the window is centered in windowed mode, false otherwise.

  number display
  : The index of the display the window is currently in, if multiple monitors are available. First is 1.

  number minwidth
  : The minimum width of the window, if it's resizable.

  number minheight
  : The minimum height of the window, if it's resizable.

Available since LÖVE 0.9.1

`boolean highdpi`
: True if high-dpi mode is allowed on Retina displays in OS X. Does nothing on non-Retina displays.

Available since LÖVE 0.9.2

`number refreshrate`
: The refresh rate of the screen's current display mode, in Hz. May be 0 if the value can't be determined.

`number x`
: The x-coordinate of the window's position in its current display.

`number y`
: The y-coordinate of the window's position in its current display.

Available since LÖVE 0.9.1 and removed in LÖVE 0.10.0

`boolean srgb`
: Removed in 0.10.0 (use love.graphics.isGammaCorrect instead). True if sRGB gamma correction is applied when drawing to the screen.

See Also

- love.window
- love.window.setMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getPixelScale

love.window.getPixelScale · revision 22247 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Removed in LÖVE 11.0

This function has been renamed to love.window.getDPIScale.

Gets the DPI scale factor associated with the window.

The pixel density inside the window might be greater (or smaller) than the "size" of the window. For example on a retina screen in Mac OS X with the `highdpi` window flag enabled, the window may take up the same physical size as an 800x600 window, but the area inside the window uses 1600x1200 pixels. `love.window.getPixelScale()` would return `2.0` in that case.

The love.window.fromPixels and love.window.toPixels functions can also be used to convert between units.

The `highdpi` window flag must be enabled to use the full pixel density of a Retina screen on Mac OS X and iOS. The flag currently does nothing on Windows and Linux, and on Android it is effectively always enabled.

Function

Synopsis

```
scale = love.window.getPixelScale( )
```

Arguments

None.

Returns

`number scale`
The pixel scale factor associated with the window.

Notes

The units of love.graphics.getWidth, love.graphics.getHeight, love.mouse.getPosition, mouse events, love.touch.getPosition, and touch events are always in terms of pixels.

See Also

- love.window
- love.window.toPixels
- love.window.fromPixels
- love.window.setMode

- Config Files

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getPointer

love.window.getPointer · revision 29433 · last edited 2025-01-16

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets a pointer to the window's low level internal object. Currently this is a SDL_Window (https://wiki.libsdl.org/SDL3/SDL_Window) pointer.

Function

Synopsis

```
pointer = love.window.getPointer( )
```

Arguments

None.

Returns

`light userdata pointer`

A pointer to the window's SDL_Window (https://wiki.libsdl.org/SDL3/SDL_Window) object.

See Also

- love.window

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getPosition

love.window.getPosition · revision 26754 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets the position of the window on the screen.

The window position is in the coordinate space of the display it is currently in.

Function

Synopsis

```
x, y, displayindex = love.window.getPosition( )
```

Arguments

None.

Returns

number x
: The x-coordinate of the window's position.

number y
: The y-coordinate of the window's position.

number displayindex
: The index of the display that the window is in.

See Also

- love.window
- love.window.setPosition

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getSafeArea

love.window.getSafeArea · revision 28205 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
  - Example
- See Also
- Other Languages

Available since LÖVE 11.3

This function is not supported in earlier versions.

Gets area inside the window which is known to be unobstructed by a system title bar, the iPhone X notch, etc. Useful for making sure UI elements can be seen by the user.

Function

Synopsis

```
x, y, w, h = love.window.getSafeArea( )
```

Arguments

None.

Returns

number x
: Starting position of safe area (x-axis).

number y
: Starting position of safe area (y-axis).

number w
: Width of safe area.

number h
: Height of safe area.

Notes

Values returned are in DPI-scaled units (the same coordinate system as most other window-related APIs), not in pixels.

Example

```
function love.draw()
    local safeX, safeY, safeW, safeH = love.window.getSafeArea()

    love.graphics.translate(safeX, safeY)
    love.graphics.rectangle("line", 0, 0, safeW, safeH)
    love.graphics.line(0, 0, safeW, safeH)
    love.graphics.line(0, safeH, safeW, 0)
    love.graphics.circle('line', safeW/2, safeH/2, safeW/2)
```

```
    love.graphics.circle('line', safeW/2, safeH/2, safeH/2)
end
```



See Also

- love.window
- love.graphics.translate

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Show safe area

love.window.getSystemTheme

love.window.getSystemTheme · revision 30716 · last edited 2026-04-04

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets the current system theme.

Function

Synopsis

```
theme = love.window.getSystemTheme( )
```

Arguments

None.

Returns

`SystemTheme theme`

The current system theme.

See Also

- love.window
- SystemTheme
- love.themechanged

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getTitle

love.window.getTitle · revision 11071 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the window title.

Function

Synopsis

```
title = love.window.getTitle( )
```

Arguments

None.

Returns

`string title`

The current window title.

See Also

- love.window
- love.window.setTitle

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getVSync

love.window.getVSync · revision 24031 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.3

This function is not supported in earlier versions.

Gets current vertical synchronization (vsync).

Function

Synopsis

```
vsync = love.window.getVSync( )
```

Arguments

None.

Returns

`number vsync`

Current vsync status. 1 if enabled, 0 if disabled, and -1 for adaptive vsync.

Notes

This can be less expensive alternative to love.window.getMode if you want to get current vsync status.

See Also

- love.window
- love.window.setVSync

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.getWidth

love.window.getWidth · revision 15954 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0 and removed in LÖVE 0.10.0

Use love.graphics.getWidth or love.window.getMode instead.

Gets the width of the window.

Function

Synopsis

```
width = love.window.getWidth( )
```

Arguments

None.

Returns

`number width`

The width of the window.

See Also

- love.window
- love.window.getHeight
- love.window.getDimensions
- love.window.setMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.hasFocus

love.window.hasFocus · revision 10721 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Checks if the game window has keyboard focus.

Function

Synopsis

```
focus = love.window.hasFocus( )
```

Arguments

None.

Returns

`boolean focus`

True if the window has the focus or false if not.

See Also

- love.window

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.hasMouseFocus

love.window.hasMouseFocus · revision 10747 · last edited 2026-01-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Checks if the game window has mouse focus.

Function

Synopsis

```
focus = love.window.hasMouseFocus( )
```

Arguments

None.

Returns

`boolean focus`

True if the window has mouse focus or false if not.

See Also

- love.window

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.isCreated

love.window.isCreated · revision 17488 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0 and removed in LÖVE 0.10.0

This function has been renamed to love.window.isOpen.

Checks if the window has been created.

Function

Synopsis

```
created = love.window.isCreated( )
```

Arguments

None.

Returns

`boolean created`

True if the window has been created, false otherwise.

See Also

- love.window

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.isDisplaySleepEnabled

love.window.isDisplaySleepEnabled · revision 15594 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets whether the display is allowed to sleep while the program is running.

Display sleep is disabled by default. Some types of input (e.g. joystick button presses) might not prevent the display from sleeping, if display sleep is allowed.

Function

Synopsis

```
enabled = love.window.isDisplaySleepEnabled( )
```

Arguments

None.

Returns

`boolean enabled`

True if system display sleep is enabled / allowed, false otherwise.

See Also

- love.window
- love.window.setDisplaySleepEnabled

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.isMaximized

love.window.isMaximized · revision 18091 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.2

This function is not supported in earlier versions.

Gets whether the Window is currently maximized.

The window can be maximized if it is not fullscreen and is resizable, and either the user has pressed the window's Maximize button or love.window.maximize has been called.

Function

Synopsis

```
maximized = love.window.isMaximized( )
```

Arguments

None.

Returns

`boolean maximized`

True if the window is currently maximized in windowed mode, false otherwise.

See Also

- love.window
- love.window.maximize

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.isMinimized

love.window.isMinimized · revision 22631 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets whether the Window is currently minimized.

Function

Synopsis

```
minimized = love.window.isMinimized( )
```

Arguments

None.

Returns

`boolean minimized`

True if the window is currently minimized, false otherwise.

See Also

- love.window
- love.window.minimize
- love.window.restore

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.isOccluded

love.window.isOccluded · revision 30607 · last edited 2026-02-13

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Checks if the window is completely covered by other windows.

An occluded window is distinct from a window which is not visible.

Function

Synopsis

```
occluded = love.window.isOccluded( )
```

Arguments

None.

Returns

`boolean occluded`

True if the window is completely covered by other windows, false otherwise.

See Also

- love.window
- love.occluded
- love.exposed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.isOpen

love.window.isOpen · revision 17489 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function has been renamed from love.window.isCreated.

Checks if the window is open.

Function

Synopsis

```
open = love.window.isOpen( )
```

Arguments

None.

Returns

`boolean open`

True if the window is open, false otherwise.

See Also

- love.window
- love.window.isCreated

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.isVisible

love.window.isVisible · revision 12581 · last edited 2026-01-07

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Checks if the game window is visible.

The window is considered visible if it's not minimized and the program isn't hidden.

Function

Synopsis

```
visible = love.window.isVisible( )
```

Arguments

None.

Returns

`boolean visible`

True if the window is visible or false if not.

See Also

- love.window
- love.visible

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.maximize

love.window.maximize · revision 15010 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Makes the window as large as possible.

This function has no effect if the window isn't resizable, since it essentially programmatically presses the window's "maximize" button.

Function

Synopsis

```
love.window.maximize( )
```

Arguments

None.

Returns

Nothing.

See Also

- love.window

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.minimize

love.window.minimize · revision 13699 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Minimizes the window to the system's task bar / dock.

Function

Synopsis

```
love.window.minimize( )
```

Arguments

None.

Returns

Nothing.

See Also

- love.window

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.requestAttention

love.window.requestAttention · revision 15359 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Causes the window to request the attention of the user if it is not in the foreground.

In Windows the taskbar icon will flash, and in OS X the dock icon will bounce.

Function

Synopsis

```
love.window.requestAttention( continuous )
```

Arguments

`boolean continuous (false)`

Whether to continuously request attention until the window becomes active, or to do it only once.

Returns

Nothing.

See Also

- love.window

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.restore

love.window.restore · revision 22641 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Restores the size and position of the window if it was minimized or maximized.

Function

Synopsis

```
love.window.restore( )
```

Arguments

None.

Returns

Nothing.

See Also

- love.window
- love.window.minimize
- love.window.maximize
- love.window.isMinimized
- love.window.isMaximized

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.setDisplaySleepEnabled

love.window.setDisplaySleepEnabled · revision 15593 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Sets whether the display is allowed to sleep while the program is running.

Display sleep is disabled by default. Some types of input (e.g. joystick button presses) might not prevent the display from sleeping, if display sleep is allowed.

Function

Synopsis

```
love.window.setDisplaySleepEnabled( enable )
```

Arguments

`boolean enable`

True to enable system display sleep, false to disable it.

Returns

Nothing.

See Also

- love.window
- love.window.isDisplaySleepEnabled

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.setFullscreen

love.window.setFullscreen · revision 27922 · last edited 2025-05-13

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Enters or exits fullscreen. The display to use when entering fullscreen is chosen based on which display the window is currently in, if multiple monitors are connected.

Function

Synopsis

```
success = love.window.setFullscreen( fullscreen )
```

Arguments

`boolean fullscreen`

Whether to enter or exit fullscreen mode.

Returns

`boolean success`

True if an attempt to enter fullscreen was successful, false otherwise.

Function

Synopsis

```
success = love.window.setFullscreen( fullscreen, fstype )
```

Arguments

`boolean fullscreen`

Whether to enter or exit fullscreen mode.

`FullscreenType fstype`

The type of fullscreen mode to use.

Returns

`boolean success`

True if an attempt to enter fullscreen was successful, false otherwise.

Notes

If fullscreen mode is entered and the window size doesn't match one of the monitor's display modes (in normal fullscreen mode) or the window size doesn't match the desktop size (in 'desktop' fullscreen mode), the window will be resized appropriately. The window will revert back to its original size again when fullscreen mode is exited using this function.

Examples

Make the window fullscreen in desktop mode.

```
function love.load()
    love.window.setFullscreen(true, "desktop")
end
```

Make the window fullscreen in exclusive mode and back on F11:

```
function love.keypressed(key, scancode, isrepeat)
    if key == "f11" then
        fullscreen = not fullscreen
        love.window.setFullscreen(fullscreen, "exclusive")
    end
end
```

See Also

- love.window
- love.window.getFullscreen
- love.window.setMode
- love.resize

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.setIcon

love.window.setIcon · revision 27809 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the window icon until the game is quit. Not all operating systems support very large icon images.

Like t.window.icon in conf.lua, love.window.setIcon allows you to set the executable icon at *runtime*. Any fused executables will have the default LÖVE application icon in a file explorer or launcher (start menu, dock, etc). See Game Distribution for steps to change the icon.

Function

Synopsis

```
success = love.window.setIcon( imagedata )
```

Arguments

`ImageData imagedata`

The window icon image.

Returns

`boolean success`

Whether the icon has been set successfully.

See Also

- love.window
- love.window.getIcon

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.setMode

love.window.setMode · revision 30511 · last edited 2025-12-24

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
  - Make the window resizable with vsync disabled and a minimum size
  - Set orientation to portrait on Android
- See Also
- Other Languages

Available since LÖVE 0.9.0

Moved from love.graphics.setMode.

Sets the display mode and properties of the window.

If width or height is 0, setMode will use the width and height of the desktop.

Changing the display mode may have side effects: for example, canvases will be cleared and values sent to shaders with Shader:send will be erased. Make sure to save the contents of canvases beforehand or re-draw to them afterward if you need to.

Using this method will clear any existing properties if they're not set i.e. 'fullscreen', 'resizable', 'usedpiscale' will revert to defaults. If you want to set the mode without resetting the window options already set, use love.window.updateMode instead.

Function

Synopsis

```
success = love.window.setMode( width, height, flags )
```

Arguments

`number width`
: Display width.

`number height`
: Display height.

`table flags`
: The flags table with the options:

  `boolean fullscreen (false)`
  : Fullscreen (true), or windowed (false).

  `FullscreenType fullscreentype ("desktop")`
  : The type of fullscreen to use. This defaults to "normal" in 0.9.0 through 0.9.2 and to "desktop" in 0.10.0 and older.

  `number vsync (1)`
  : Enables or disables vertical synchronisation ('vsync'): **1** to enable vsync, **0** to disable it, and **-1** to use adaptive vsync (where supported). Prior to 11.0 this was a boolean flag, rather than a number.

  `number msaa (0)`
  : The number of antialiasing samples.

  `boolean stencil (true)`
  : Whether a stencil buffer should be allocated. If true, the stencil buffer will have 8 bits.

  `number depth (0)`

The number of bits in the depth buffer.

`boolean resizable (false)`

True if the window should be resizable in windowed mode, false otherwise.

`boolean borderless (false)`

True if the window should be borderless in windowed mode, false otherwise.

`boolean centered (true)`

True if the window should be centered in windowed mode, false otherwise.

`number display (1)`

The index of the display to show the window in, if multiple monitors are available.

`number minwidth (1)`

The minimum width of the window, if it's resizable. Cannot be less than 1.

`number minheight (1)`

The minimum height of the window, if it's resizable. Cannot be less than 1.

Available since LÖVE 0.9.1

`boolean highdpi (false)`

True if high-dpi mode should be used on Retina displays in macOS and iOS. Does nothing on non-Retina displays. This is always enabled on Android.

Available since LÖVE 0.9.2

`number x (nil)`

The x-coordinate of the window's position in the specified display.

`number y (nil)`

The y-coordinate of the window's position in the specified display.

Available since LÖVE 11.3

`boolean usedpiscale (true)`

Disables LOVE's automatic DPI scaling on high-DPI displays when false. This only has an effect when the `highdpi` flag is set to true, since the OS (rather than LOVE) takes care of everything otherwise.

Available since LÖVE 0.9.1 and removed in LÖVE 0.10.0

`boolean srgb (false)`

Removed in 0.10.0 (set t.gammacorrect in conf.lua instead). True if sRGB gamma correction should be applied when drawing to the screen.

Returns

`boolean success`

True if successful, false otherwise.

Notes

love.conf may have more extensive documentation on the behavior of each window flag.

- If fullscreen is enabled and the width or height is not supported (see love.window.getFullscreenModes), the window may be resized to the closest available resolution and a resize event will be triggered.
- If the fullscreen type is "desktop", then the window will be automatically resized to the desktop resolution.
- If the width and height is bigger than or equal to the desktop dimensions (this includes setting both to 0) and fullscreen is set to false, it will appear "visually" fullscreen, but it's not true fullscreen and love.window.getFullscreen will return false in this case. This also applies to love.window.updateMode as well.
- If you have disabled the window in conf.lua (i.e. `t.window = false`) and use this function to manually create the window, then you must not call any other love.graphics.* function before this one. Doing so will result in undefined behavior and/or crashes because OpenGL cannot function properly without a window.
- On Android and iOS, the aspect ratio computed from the specified width and height determines whether the game will run in landscape or portrait. Furthermore it's asynchronous. Use love.resize to get the re-adjusted dimensions.
- Using this method in love.mousepressed can cause the next mouse press to be ignored, so it's recommended to call it in love.mousereleased instead if needed
- Transparent backgrounds are currently not supported.

Examples

Make the window resizable with vsync disabled and a minimum size

```
function love.load()
    love.window.setMode(800, 600, {resizable=true, vsync=0, minwidth=400, minheight=300})
end
```

Set orientation to portrait on Android

```
function love.load()
  -- set portrait mode
    love.window.setMode(1, 2) -- The actual resulting resolution will match the screen.
end

function love.resize(w, h)
-- get real dimensions
  WindowWidth = w
  WindowHeight = h
end
```

See Also

- love.window
- love.window.getMode
- love.window.getFullscreenModes

- love.window.setFullscreen
- love.window.setVSync
- love.resize

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.setPosition

love.window.setPosition · revision 26753 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Sets the position of the window on the screen.

The window position is in the coordinate space of the specified display.

Function

Synopsis

```
love.window.setPosition( x, y, displayindex )
```

Arguments

`number x`
: The x-coordinate of the window's position.

`number y`
: The y-coordinate of the window's position.

`number displayindex (1)`
: The index of the display that the new window position is relative to.

Returns

Nothing.

See Also

- love.window
- love.window.getPosition

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.setTitle

love.window.setTitle · revision 14011 · last edited 2025-02-21

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the window title.


Constantly updating the window title can lead to issues on some systems and therefore is discouraged.

Function

Synopsis

```
love.window.setTitle( title )
```

Arguments

`string` **`title`**
: The new window title.

Returns

Nothing.

See Also

- love.window
- love.window.getTitle

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.setVSync

love.window.setVSync · revision 27049 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.3

This function is not supported in earlier versions.

Sets vertical synchronization mode.

Function

Synopsis

```
love.window.setVSync( vsync )
```

Arguments

`number vsync`
: VSync number: 1 to enable, 0 to disable, -1 for adaptive vsync, 2 or larger will wait that many frames before syncing.

Returns

Nothing.

Notes

- Not all graphics drivers support adaptive vsync (-1 value) and values more than 1 (vsync at other intervals). In that case, it will be automatically set to 1.
- Having vsync be a number larger than one means that the game will sync to the nearest whole number of the screen's refresh rate divided by the given number; e.g. on a 60 Hz screen, vsync being 2 means it'll run at 30 FPS, 3 means 20 FPS, etc.
- It is recommended to keep vsync activated if you don't know about the possible implications of turning it off.
- This function doesn't recreate the window, unlike love.window.setMode and love.window.updateMode.

See Also

- love.window
- love.window.getVSync

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.showFileDialog

love.window.showFileDialog · revision 30120 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Open system file dialog capable of picking files and folder.



This documents a function for a future version of LÖVE and may be changed before final release.

Function

Synopsis

```
love.window.showFileDialog( type, callback, settings )
```

Arguments

`FileDialogType type`

File dialog type.

`function callback`

Function with 3 parameters: files (array of full platform-dependent paths to selected files, empty if the dialog is canceled by the user), filtername (nil if not set), and errorstring (nil if no error)

`table settings (nil)`

A table containing the given fields:

`string title (nil)`

Dialog window title.

`string acceptlabel (nil)`

Custom name for the Accept button.

`string cancellabel (nil)`

Custom name for the Cancel button.

`string defaultname (nil)`

Default folder/file name for the file text box.

`table filters (nil)`

Filter names as keys and filter pattern strings as values.

`boolean multiselect (false)`

Allow selecting multiple files.

`boolean attachtowindow (false)`

Attach to current game window.

Returns

Nothing.

Notes

- Not all platform supports this function or all dialog types. However "openfile" type is supported in most cases.
- This is an asynchronous function. You'll get your result in the callback specified when calling the function.
- Not all platform supports all settings. Some may support one subset of the settings, some may support the other.
- Use love.filesystem.openNativeFile instead of io.open to open files from paths given by a file dialog, especially on Windows and Android.

See Also

- love.window

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.showMessageBox

love.window.showMessageBox · revision 14278 · last edited 2026-01-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Display a simple message box if the user's system doesn't support shaders
  - Display a message box with customized buttons
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Displays a message box dialog above the love window. The message box contains a title, optional text, and buttons.



This function will pause all execution of the main thread until the user has clicked a button to exit the message box. Calling the function from a different thread may cause love to crash.

Function

Displays a simple message box with a single 'OK' button.

Synopsis

```
success = love.window.showMessageBox( title, message, type, attachtowindow )
```

Arguments

`string title`
: The title of the message box.

`string message`
: The text inside the message box.

`MessageBoxType type ("info")`
: The type of the message box.

`boolean attachtowindow (true)`
: Whether the message box should be attached to the love window or free-floating.

Returns

`boolean success`

Whether the message box was successfully displayed.

Function

Displays a message box with a customized list of buttons.

Synopsis

```
pressedbutton = love.window.showMessageBox( title, message, buttonlist, type, attachtowindow )
```

Arguments

string title
: The title of the message box.

string message
: The text inside the message box.

table buttonlist
: A table containing a list of button names to show. The table can also contain the fields `enterbutton` and `escapebutton`, which should be the index of the default button to use when the user presses 'enter' or 'escape', respectively.

MessageBoxType type ("info")
: The type of the message box.

boolean attachtowindow (true)
: Whether the message box should be attached to the love window or free-floating.

Returns

number pressedbutton
: The index of the button pressed by the user. May be 0 if the message box dialog was closed without pressing a button.

Examples

Display a simple message box if the user's system doesn't support shaders

```
local errortitle = "Shader support is required for the game to run"
local errormessage = "This system is below the minimum system requirements for the game.\
If your graphics drivers aren't up-to-date, try updating them and running the game again."

if not love.graphics.isSupported("shader") then
    love.window.showMessageBox(errortitle, errormessage, "error")
end
```

Display a message box with customized buttons

```
local title = "This is a title"
local message = "This is some text"
local buttons = {"OK", "No!", "Help", escapebutton = 2}

local pressedbutton = love.window.showMessageBox(title, message, buttons)
if pressedbutton == 1 then
    -- "OK" was pressed
elseif pressedbutton == 2 then
```

```
    -- etc.
end
```

See Also

- love.window

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.toPixels

love.window.toPixels · revision 21890 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Converts a number from density-independent units to pixels.

The pixel density inside the window might be greater (or smaller) than the "size" of the window. For example on a retina screen in Mac OS X with the `highdpi` window flag enabled, the window may take up the same physical size as an 800x600 window, but the area inside the window uses 1600x1200 pixels. `love.window.toPixels(800)` would return `1600` in that case.

This is used to convert coordinates from the size users are expecting them to display at onscreen to pixels. love.window.fromPixels does the opposite. The `highdpi` window flag must be enabled to use the full pixel density of a Retina screen on Mac OS X and iOS. The flag currently does nothing on Windows and Linux, and on Android it is effectively always enabled.

Most LÖVE functions return values and expect arguments in terms of pixels rather than density-independent units.

Function

Synopsis

```
pixelvalue = love.window.toPixels( value )
```

Arguments

`number value`

A number in density-independent units to convert to pixels.

Returns

`number pixelvalue`

The converted number, in pixels.

Function

Synopsis

```
px, py = love.window.toPixels( x, y )
```

Arguments

`number x`

The x-axis value of a coordinate in density-independent units to convert to pixels.

number y
: The y-axis value of a coordinate in density-independent units to convert to pixels.

Returns

number px
: The converted x-axis value of the coordinate, in pixels.

number py
: The converted y-axis value of the coordinate, in pixels.

Notes

The units of love.graphics.getWidth, love.graphics.getHeight, love.mouse.getPosition, mouse events, love.touch.getPosition, and touch events are always in terms of pixels.

See Also

- love.window
- love.window.fromPixels
- love.window.getDPIScale
- love.window.setMode
- Config Files

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.window.updateMode

love.window.updateMode · revision 25342 · last edited 2026-01-07

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Sets the display mode and properties of the window, without modifying unspecified properties.

If width or height is 0, updateMode will use the width and height of the desktop.

Changing the display mode may have side effects: for example, canvases will be cleared. Make sure to save the contents of canvases beforehand or re-draw to them afterward if you need to.

Function

Synopsis

```
success = love.window.updateMode( width, height, settings )
```

Arguments

`number width`
: Window width.

`number height`
: Window height.

`table settings`
: The settings table with the following optional fields. Any field not filled in will use the current value that would be returned by love.window.getMode.

  `boolean fullscreen`
  : Fullscreen (true), or windowed (false).

  `FullscreenType fullscreentype`
  : The type of fullscreen to use.

  `boolean vsync`
  : True if LÖVE should wait for vsync, false otherwise.

  `number msaa`
  : The number of antialiasing samples.

  `boolean resizable`
  : True if the window should be resizable in windowed mode, false otherwise.

  `boolean borderless`
  : True if the window should be borderless in windowed mode, false otherwise.

  `boolean centered`
  : True if the window should be centered in windowed mode, false otherwise.

  `number display`
  : The index of the display to show the window in, if multiple monitors are available.

  `number minwidth`

The minimum width of the window, if it's resizable. Cannot be less than 1.

`number minheight`
The minimum height of the window, if it's resizable. Cannot be less than 1.

`boolean highdpi`
True if high-dpi mode should be used on Retina displays in macOS and iOS. Does nothing on non-Retina displays.

`number x`
The x-coordinate of the window's position in the specified display.

`number y`
The y-coordinate of the window's position in the specified display.

Returns

`boolean success`
True if successful, false otherwise.

Notes

- If fullscreen is enabled and the width or height is not supported (see love.window.getFullscreenModes), the window may be resized to the closest available resolution and a resize event will be triggered.
- If the fullscreen type is "desktop", then the window will be automatically resized to the desktop resolution.
- In Android, the aspect ratio deducted from the width and the height is used to determine whetever the game run in landscape or portrait.
- Transparent backgrounds are currently not supported.

See Also

- love.window
- love.window.setMode
- love.window.getMode
- love.window.getFullscreenModes
- love.resize

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info